Evaluación Unidad 2

Diseño e implementación de un backend seguro completo
Pizzería La Fornace (Caso 19)

Integrantes:

Alejandro Suarez
Alvaro Asencios

Docente:

Ernesto Vivanco

Curso:

Desarrollo Web y Móvil (NRC: 8481)

Fecha:

09 de noviembre de 2026

Índice

Portada 6

Tecnologías (las del curso, con las versiones realmente usadas en la entrega) 6

Módulo 1 — Servicios derivados del frontend 6

1.1 Pantallas del frontend actual 6

1.2 Brechas detectadas y cómo se incorporan 7

1.3 Requisitos y casos de uso (del Caso 19) 7

1.4 Actores y roles 8

1.5 Convenciones del catálogo de servicios 8

1.6 Matriz de servicios 8

1.7 Ejemplos de contratos (datos ficticios) 14

1.8 Trazabilidad de pantallas 18

1.9 Cumplimiento de los pasos a seguir de la guía 18

1.10 Tipo de operación de cada servicio 19

1.11 Diccionario de campos 19

1.12 Qué envía el navegador y qué decide el servidor 20

1.13 Hallazgos del código existente que afectan el diseño 20

1.14 Control antes de avanzar 21

Módulo 2 — Clases y cardinalidades del dominio 21

2.1 Cumplimiento de los pasos a seguir de la guía 21

2.2 Decisiones de modelado 21

2.3 Leyenda de notación 22

2.4 Diagrama 2.1 — Dominio 22

2.5 Catálogo de clases 22

2.6 Tabla de relaciones 23

2.7 Diagrama 2.2 — Servicios públicos y DTO 28

2.8 Correspondencia entre operación UML e ID de servicio 29

2.9 Diagrama 2.3 — Servicios y entidades que usan 31

2.10 Modelo interno y DTO 32

2.11 Control antes de avanzar 34

Módulo 3 — Sistemas e integraciones 34

3.1 Cumplimiento de la guía 34

3.2 Decisiones de partición y límites 34

3.3 Matriz de sistemas 35

3.4 Autoridad de los datos....36
3.5 Estado oficial del pedido y del cobro....37
3.6 Servicios del Módulo 1 asignados a sistemas....37
3.7 Catálogo de intercambios....38
3.8 Identidad entre componentes....40
3.9 Pedidos ↔ inventario: disponibilidad, reserva y respuesta que no llega....41
3.10 Aviso de pago: verificación, duplicados y autoridad del cambio de estado....41
3.11 Fallos y consistencia del negocio....42
3.12 Sistemas complementarios: propios y externos....42
3.13 Diagrama 3.1 — Mapa de componentes....43
3.14 Diagramas de secuencia....43
3.15 Contratos de los intercambios principales (datos ficticios)....45
3.16 Control antes de avanzar....46
Módulo 4 — Stakeholders y permisos HTTP....46
4.1 Cumplimiento de los pasos a seguir de la guía....46
4.2 Registro de stakeholders....46
4.3 Stakeholders sin permisos operativos....47
4.4 Roles operativos y servicios del Módulo 1....47
4.5 Autenticación y responsable de verificar la autorización....48
4.6 Condiciones de acceso....48
4.7 Matriz resumen: rol por servicio....49
4.8 Matriz de permisos detallada....50
4.9 Verbos por recurso....55
4.10 Identidades técnicas: permisos entre sistemas....56
4.11 Anulación e historial....58
4.12 Cómo la matriz impide que un cliente consulte pedidos de otro....58
4.13 Pruebas de autorización previstas....58
4.14 Control antes de avanzar....59
Módulo 5 — Clases del API Gateway....59
5.1 Cumplimiento de los pasos a seguir de la guía....59
5.2 Cuántos gateways....60
5.3 Clasificación de rutas....60
5.4 Diagrama 5.1 — Clases del gateway, políticas, secretos y GraphQL....61
5.5 Tabla de relaciones....63
5.6 Políticas nombradas....64

5.7 Controles del gateway....65
5.8 Qué valida el gateway y qué valida cada sistema....65
5.9 Fronteras de confianza....66
5.10 Cómo se impide el acceso directo a los sistemas internos....67
5.11 Recorrido de una solicitud autorizada y de las denegadas....68
5.12 GraphQL Gateway en la red privada....69
5.13 Analítica y gRPC....69
5.14 Qué se aprovecha del gateway de ejemplo y qué se corrige....69
5.15 Control antes de avanzar....69
Módulo 6 — Rutas y secretos de integración....70
6.1 Cumplimiento de los pasos a seguir de la guía....70
6.2 Reglas de la lista de rutas....70
6.3 Matriz de rutas del API Gateway....70
6.4 Transformación que aplica el gateway....73
6.5 Lo que no se publica....73
6.6 GraphQL: operaciones, campos y subsistemas....73
6.7 Credencial del usuario y credencial entre sistemas....74
6.8 Mecanismo de cada integración....75
6.9 Inventario de secretos....75
6.10 Emisión, rotación y revocación....80
6.11 gRPC....81
6.12 Decisiones de seguridad....81
6.13 Diagrama 6.1 — Credenciales por tramo....82
6.14 Control antes de avanzar....82
Módulo 7 — Datos sensibles y secretos del cliente....82
7.1 Cumplimiento de los pasos a seguir de la guía....82
7.2 Qué se protege y cómo se clasifica....82
7.3 Matriz de datos y credenciales....83
7.4 Decisión: ¿se custodian secretos del cliente?....86
7.5 Qué devuelve cada servicio....86
7.6 Cómo se limita el acceso a lo propio....86
7.7 Almacenamiento y transporte....87
7.8 Qué no se registra....87
7.9 Retención y eliminación....87
7.10 Sesiones y tokens....88

7.11 Recorrido de un dato restringido: la dirección de entrega.....88
7.12 Casos de validación.....89
7.13 Riesgos aceptados.....91
7.14 Control antes de entregar.....91
Módulo 8 — Separación privada y agregaciones GraphQL.....91
8.1 Cumplimiento de los pasos a seguir de la guía.....91
8.2 Mapa de clasificación.....91
8.3 Límites de inventory, orders, clients y payments.....92
8.4 Separación privada.....93
8.5 Agregaciones y pantallas.....93
8.6 Schema GraphQL.....93
8.7 Resolvers.....95
8.8 Matriz de autorización.....95
8.9 Límites, plazos y errores.....96
8.10 Consultas de prueba y respuestas esperadas.....96
8.11 Integración con el frontend.....98
8.12 Diagrama 8.1 — Secuencia de resumenPedido.....99
8.13 Pruebas y evidencia.....99
8.14 Control antes de avanzar.....101
Módulo 9 — Analítica y comunicación gRPC cuando corresponden.....101
9.1 Cumplimiento de los pasos a seguir de la guía.....101
9.2 Caracterización de la carga.....102
9.3 Matriz de necesidades.....102
9.4 ADR-01 — No crear un servicio de analítica.....103
9.5 ADR-02 — No usar gRPC entre sistemas.....103
9.6 ADR-03 — Despacho por cola y sin streaming hacia el navegador.....103
9.7 Alternativa implementada: el reporte de ventas en payments-service.....104
9.8 Plan de la prueba de carga.....104
9.9 Resultados medidos.....105
9.10 Cuándo se revisan las decisiones.....107
9.11 Control antes de avanzar.....107
Módulo 10 — Backend completo e integración verificable.....107
10.1 Cumplimiento de los pasos a seguir de la guía.....107
10.2 Qué se entrega.....108
10.3 Persistencia, esquema y datos de prueba.....109

10.4 Integración de la arquitectura..... 109
10.5 Integración con el frontend..... 110
10.6 Cómo iniciar, detener, sembrar y probar..... 110
10.7 Demostración de los flujos principales..... 111
10.8 Matriz requisito, pantalla, servicio, prueba, resultado y evidencia..... 113
10.9 Resultados de las pruebas..... 116
10.10 Defectos que encontraron las pruebas y se corrigieron..... 128
10.11 Diferencias con el diseño y limitaciones..... 129
10.12 Control antes de cerrar..... 130

Portada

Campo	Detalle
Caso elegido	**Caso 19 "Sistema de Ventas On-line"**, adaptado al negocio Pizzería La Fornace (Maipú)
Integrantes	Alejandro Suarez, Alvaro Asencios
Versión del frontend	Evaluación 1 (`frontend-la-fornace`, 5 páginas HTML + Bootstrap 5.3). Esta entrega es el **backend**: se prueba por su API con los datos de ese frontend (los 12 productos de `js/data.js`). Como demostración opcional, el mismo frontend se extendió a 11 páginas conectadas al backend (`impl/frontend`)
Qué entregamos	El **backend**. El profesor indicó que *la entrega de backend es con datos de frontend, no con el frontend integrado*: probamos la API con los datos y los flujos de la Evaluación 1 y dejamos el frontend integrado solo como demostración opcional (Módulo 10, 10.5).
Repositorio	github.com/AlejandroSuarez105/unab-desarrollo-web-movil
Fecha de entrega	09 de noviembre de 2026

Tecnologías (las del curso, con las versiones realmente usadas en la entrega)

Componente	Tecnología	Origen en el repo
Servicios REST	Python 3.13 + FastAPI (uvicorn, httpx, motor, bcrypt)	`fastapi-items`, `lab-api-gateway`
API Gateway y Auth Service	Python + FastAPI + httpx	`lab-api-gateway/api-gateway`, `auth_service`
GraphQL Gateway	Node.js 22 + Apollo Server 5.5.1 + GraphQL 16.11 + DataLoader	`graphql-pizzeria`
Base de datos	MongoDB 7 (una base por sistema; controlador `motor` en Python)	`fastapi-items`, `graphql-pizzeria`
Administrador de secretos	HashiCorp Vault 1.18 (modo desarrollo), API KV v2	informe del lab de gateway
Mensajería de pedidos	Cola STOMP `/queue/cola_pedidos` en Apache ActiveMQ Classic 6.2.0 (persistente, confirmación manual)	`stomp/`
Entorno	Docker Compose con red privada interna y red pública (Módulo 6), o en local sin Docker con `scripts/stack.py` sobre MongoDB, Vault y ActiveMQ instalados en el equipo	informe del lab de gateway

> **Desviación declarada del caso:** el RNF 3 y 4 del Caso 19 piden Oracle, procedimientos almacenados y .NET o J2EE. Usamos el stack de los laboratorios del curso (FastAPI, Node, MongoDB). El resto de los RNF se cumple (capas separadas, servicios web, API de correo, cifrado de clave, control de sesiones, validaciones, mensajes de error claros, diseño responsivo). Las demás diferencias con el diseño y las limitaciones de la ejecución se declaran en el Módulo 10 (sección 10.11).

Módulo 1 — Servicios derivados del frontend

1.1 Pantallas del frontend actual

El frontend es HTML + JS plano. Hoy no tiene backend: productos, combos y sucursal viven en `js/data.js`, el carrito está en `localStorage` y el login y el pedido son simulados.

ID	Pantalla	Acciones y datos que maneja
P01	Inicio (`index.html`)	Carrusel (contenido fijo), grilla de **combos**, grilla de **más pedidas**, "¿Cómo pedir?" (fijo). Botones "Pedir combo" y "Agregar".
P02	Menú (`menu.html`)	Catálogo con filtro por categoría (clásica, premium, acompañamiento, bebida). Botón "Agregar".
P03	Carrito (`carrito.html`)	Lista de ítems, cambiar cantidad, quitar, resumen y botón "Confirmar pedido".
P04	Sucursales (`sucursales.html`)	Una sucursal: dirección, horario, mapa embebido, radio de despacho. Contenido informativo.
P05	Contacto (`contacto.html`)	Formulario: nombre, correo, teléfono, motivo, mensaje.
P06	Modal Iniciar sesión (todas las páginas)	Correo y contraseña. El enlace "Regístrate" no tiene destino (`href="#"`).
P07	Modal "Pedido confirmado" (en P03)	Mensaje de éxito tras confirmar.

Escritorio y móvil usan las mismas pantallas (Bootstrap responsivo). La diferencia es solo visual, así que **no genera servicios nuevos**.

1.2 Brechas detectadas y cómo se incorporan

El frontend actual no cubre varias funciones que exige el Caso 19. Las registramos como brechas y las resolvemos en la entrega final.

ID	Brecha	Propuesta
P08	No existe registro de clientes (RF5). Faltan RUN, provincia, región, fecha de nacimiento y sexo.	Página "Crear cuenta" (`registro.html`) con todos los datos del caso.
P09	No hay validación de correo (RF11).	Página "Confirmar correo" a la que llega el enlace del correo. Sirve también para definir la contraseña de cuentas creadas por el administrador.
P10	El carrito no pide dirección de entrega ni permite pagar (RF2).	En P03 mostrar la dirección de entrega (precargada del perfil). Crear la pantalla de **pago y retorno** con las opciones del caso (Servipag o depósito bancario).
P11	No hay historial de pedidos, estado, boleta ni anulación (RF3, RF8).	Pantalla "Mis pedidos".
P12	No hay perfil del cliente.	Pantalla "Mi cuenta".
P13–P18	No existe ninguna pantalla interna para el personal.	**Panel interno** (`panel.html`) con secciones por rol: P13 Productos (administrador, dueño), P14 Pedidos y ventas (cajero virtual, administrador), P15 Despacho (encargado de despachos), P16 Clientes (administrador), P17 Usuarios (administrador), P18 Reporte de ventas (dueño, administrador).
P03 / P04	El frontend cobra despacho ($1.500) y da gratis desde $15.000. El caso dice que la entrega es **siempre gratuita**, solo dentro de 3 km.	Quitar el cobro y el umbral del carrito y del texto de ayuda.

1.3 Requisitos y casos de uso (del Caso 19)

Mantenemos la numeración del caso para la trazabilidad.

ID	Requisito del caso
RF1	El cliente arma un pedido con los productos disponibles del menú.
RF2	La orden se prepara una vez confirmado el pago. El pago se hace por una plataforma externa (Servipag o depósito bancario).
RF3	Anulación de compra, indicando el motivo.
RF4	Se realizan ventas solo a pedidos de clientes registrados. Las ventas web se asocian a una caja que pertenece a un cajero virtual.
RF5	Registro de clientes con RUN, nombre completo, dirección, comuna, provincia, región, fecha de nacimiento, sexo, correo y teléfono. El cliente registrado puede comprar.
RF6	El registro lo puede hacer el cliente en la web o el administrador en el restaurante. En ambos casos se valida el correo.
RF7	Consultar y actualizar la disponibilidad de cada producto.
RF8	Para cada venta se genera la boleta digital y se envía por correo al cliente.
RF9	Reporte de ventas por período para el dueño.
RF10	Mantenedores de productos, usuarios y clientes.
RF11	Validar que el correo existe y está activo mediante una API de mensajería.
RF12	Autenticación de usuarios y acceso según el perfil asignado por el administrador.
RF13	El encargado de despacho obtiene e imprime las órdenes de despacho, que van a cocina en el orden en que llegan.

ID	Requisito del caso
RF14	*(Extra del frontend, no está en el caso)* Formulario de contacto de la página P05.

Caso de uso	Servicios que lo cubren
CU1 Autenticar usuarios	S01, S02
CU2 Administrar usuarios	S30–S33
CU3 Administrar clientes	S03–S05, S24–S29
CU4 Administrar productos	S07–S11
CU5 Genera pedido	S06, S12–S14, S20
CU6 Realizar venta	Operación interna tras S21; consulta en S22 y S23
CU7 Anular compra	S15, S17
CU8 Reporte de ventas	S34
CU9 Obtener orden de despacho	S18, S19

1.4 Actores y roles

Actor	Tipo	Rol en el sistema (`rol`)
Visitante	Sin sesión	— (anónimo)
Cliente	Usuario registrado	`cliente`
Administrador	Personal	`administrador`
Dueño	Personal	`dueno`
Cajero virtual	Personal	`cajero_virtual`
Encargado de despachos	Personal (el caso lo llama también "Encargado de Bodega" en CU9; los unificamos)	`encargado_despachos`
Plataforma de pago externa	Identidad externa	Notifica el resultado de un pago al backend
API de mensajería (correo)	Proveedor externo	Valida correos y envía correos

1.5 Convenciones del catálogo de servicios

- Base de ruta: `/api/v1`. Las rutas `/admin/...` son de uso interno del personal.
- Autenticación: cabecera `Authorization: Bearer <token>`.
- Formato de error: `{"error": {"codigo": "...", "mensaje": "..."}}`, con mensajes claros para el usuario (RNF 13).
- Clasificación de datos: **Público**, **Restringido** (datos personales, historial de pedidos), **Credencial** (contraseñas, tokens), **Operativo** (stock, estados internos, ventas).
- Lo que el servidor siempre decide: precios, total, cobertura de despacho, estado del pedido, estado del pago, rol del usuario, número de boleta. El navegador no los envía como dato confiable.
- Despacho: siempre gratuito, solo dentro del radio de 3 km. Una dirección fuera de cobertura se rechaza al crear el pedido.
- Ciclo de vida del pedido: `pendiente_pago → pagado → en_preparacion → en_despacho → entregado`, y `anulado`.

1.6 Matriz de servicios

Acceso: **A** = anónimo. **Au** = autenticado, con los roles indicados.

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasificación	Acceso
S01	RF12	P06 · "Entrar"	Iniciar sesión	POST `/api/v1/auth/login`	Cuerpo: `correo` (email), `password`	200 con `access_token`, `token_type`, `expires_in`, `rol`, `nombre`	400 datos inválidos. 401 credenciales inválidas (mensaje genérico). 403 cuenta no activa (correo sin confirmar o dada de baja). 429 muchos intentos.	Credencial · Restringido	A
S02	RF12	Navbar · "Cerrar sesión"	Cerrar sesión e invalidar el token	POST `/api/v1/auth/logout`	Cabecera Bearer. Sin cuerpo.	204	401 token inválido o vencido.	Credencial	Au (cualquier rol)

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasificación	Acceso
S03	RF5, RF11	P08 · "Crear cuenta"	Registrar un cliente desde la web	POST `/api/v1/auth/registe r`	Cuerpo: `run`, `nombreComp leto`, `correo`, `password` (mín. 8), `telefono`, `fechaNacim iento`, `sexo`, `direccion` con `calle`, `comuna`, `provincia`, `region`	201 con `id`, `correo`, `estado: "pendiente _verificac ion"`	400 validación (RUN con dígito verificador inválido, fecha futura). 409 correo o RUN ya registrado. 422 la API de mensajería indica que el correo no existe. 429. 503 API de mensajería no disponible. El rol siempre es `cliente`. Se envía un correo de confirmación.	Restringido · Credencial	A
S04	RF11, RF6	P09 · abrir el enlace del correo	Confirmar el correo y, si corresponde, definir la contraseña	POST `/api/v1/auth/verify-email`	Cuerpo: `token`, `password` (obligatoria solo si la cuenta fue creada por el administrador)	200 con `{estado: "activo"}`	400 token inválido. 410 token vencido o ya usado. El token es de un solo uso y vence en 24 h.	Credencial	A
S05	RF11	P09 · "Reenviar correo"	Reenviar el correo de confirmación	POST `/api/v1/auth/resend-verificati on`	Cuerpo: `correo`	202 siempre, exista o no la cuenta	429. No revela si el correo está registrado. Cada reenvío invalida el token anterior.	Restringido	A
S06	RF1, RF7	P01 combos y destacados · P02 cargar y filtrar · P03 mostrar ítems	Consultar el catálogo	GET `/api/v1/pr oducts`	Consulta opcional: `categoria`, `disponible`, `destacado`, `ids` (lista), `limit` (1–100, def. 50), `offset`	200 con lista de `{id, nombre, categoria, descripcio n, precio, imagen, destacado, disponible }`; `[]` si no hay	400 filtros inválidos. Solo campos públicos: **no** devuelve `stock`. Los combos son `categoria= combo`.	Público	A
S07	RF7, RF10	P13 · cargar lista	Listar productos con stock y retirados	GET `/api/v1/ad min/produc ts`	Consulta opcional: `categoria`, `incluirRet irados`, `limit`, `offset`	200 con lista de `{id, nombre, categoria, precio, stock, disponible , retirado}`	401. 403. 400 filtros inválidos.	Operativo	Au (administrador , dueno)
S08	RF10	P13 · "Nuevo producto"	Crear un producto	POST `/api/v1/pr oducts`	Cuerpo: `nombre`, `categoria`, `descripcio n`, `precio` (entero CLP > 0), `imagen` (URL), `destacado`, `stock` (≥ 0)	201 con el producto	400 validación. 401. 403. 409 nombre duplicado.	Público (catálogo) · Operativo (`stock`)	Au (administrador , dueno)
S09	RF10	P13 · "Guardar cambios"	Reemplazar los datos de un producto	PUT `/api/v1/pr oducts/{id }`	Ruta: `id`. Cuerpo igual a S08 sin `stock`.	200 con el producto	400. 401. 403. 404.	Público (catálogo)	Au (administrador , dueno)
S10	RF10	P13 · "Retirar"	Retirar un producto (baja lógica, conserva el historial de pedidos)	DELETE `/api/v1/pr oducts/{id }`	Ruta: `id`. Sin cuerpo.	204	401. 403. 404. Queda `retirado=t rue` y `disponible =false`.	Operativo	Au (administrador , dueno)

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasificación	Acceso
S11	RF7	P13 · "Disponibilidad y stock"	Actualizar stock y disponibilidad de un producto	PUT `/api/v1/products/{id}/stock`	Ruta: `id`. Cuerpo: `stock` (≥ 0), `disponible` (bool)	200 con `{id, stock, disponible}`	400. 401. 403. 404. Con `stock = 0` el producto queda no disponible aunque `disponible` sea verdadero.	Operativo	Au (administrador, dueno)
S12	RF1	P03 · "Confirmar pedido"	Crear un pedido	POST `/api/v1/orders`	Cabecera opcional `Idempotency-Key`. Cuerpo: `items[]` con `productoId` y `cantidad` (1–20); `direccionEntrega` opcional (`calle`, `comuna`, `referencia`). Si falta, se usa la del perfil.	201 con `id`, `estado`, `items[]` con precio unitario, `total`, `direccionEntrega`, `creadoEn`, `expiraEn`	400 validación. 401. 403 la cuenta no es de cliente activo. 404 producto inexistente. 409 producto no disponible o sin stock. 422 dirección fuera de la cobertura de 3 km. 503 inventario no responde. Se ignoran precios o totales enviados por el navegador. Si llega la misma `Idempotency-Key` del mismo cliente, devuelve el pedido ya creado en vez de crear otro. Estado inicial: `pendiente_pago`. Si no se paga antes de `expiraEn`, el pedido se anula y se devuelve el stock.	Restringido (dirección, pedido)	Au (cliente)
S13	RF1	P11 · cargar "Mis pedidos"	Listar los pedidos propios	GET `/api/v1/orders`	Consulta opcional: `estado`, `limit`, `offset`	200 con lista de `{id, estado, total, creadoEn}`; `[]` si no hay	401. 400 filtro inválido. Solo devuelve los del usuario.	Restringido	Au (cliente)
S14	RF1, RF2	P11 · "Ver detalle" · P10 retorno de pago	Consultar el detalle de un pedido	GET `/api/v1/orders/{id}`	Ruta: `id`	200 con `id`, `estado`, `estadoPago`, `items[]`, `total`, `direccionEntrega`, `historialEstados[]`, `boletaDisponible`, `creadoEn` y `expiraEn` (este último solo mientras está en `pendiente_pago`). Si lo pide el personal, además `clienteNombre`	401. 404 si no existe **o** no es del usuario (no se revela existencia). El personal autorizado puede consultar cualquier pedido.	Restringido	Au (cliente dueño; administrador, cajero_virtual, encargado_despachos)

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasificación	Acceso
S15	RF3	P11 · "Anular pedido"	Anular un pedido propio	POST `/api/v1/orders/{id}/cancel`	Ruta: `id`. Cuerpo: `motivo` (5–200)	200 con `{id, estado: "anulado", motivo, reembolsoPendiente}`	401. 404. 409 si el pedido ya está `en_preparacion` o más avanzado. Se devuelve el stock y el registro se conserva. Si ya estaba pagado, queda `reembolsoPendiente: true` (el reembolso es manual).	Restringido	Au (cliente dueño)
S16	RF2, RF4	P14 · cargar lista	Listar todos los pedidos para el personal	GET `/api/v1/admin/orders`	Consulta opcional: `estado`, `fecha`, `limit`, `offset`	200 con lista de `{id, estado, total, creadoEn, clienteNombre, direccionEntrega}`	401. 403. 400 filtro inválido.	Restringido	Au (administrador, cajero_virtual, encargado_despachos)
S17	RF2, RF3	P14 y P15 · "Avanzar estado" o "Anular"	Cambiar el estado de un pedido	PATCH `/api/v1/admin/orders/{id}/status`	Ruta: `id`. Cuerpo: `estado`, `motivo` (obligatorio si es `anulado`)	200 con el pedido actualizado	401. 403. 404. 409 transición inválida. 400 falta el motivo. Permitido: `pagado → en_preparacion → en_despacho → entregado` (encargado_despachos, administrador) . `anulado` solo lo hace el administrador, desde `pendiente_pago`, `pagado` o `en_preparacion`. **Nunca** permite pasar a `pagado`: eso solo lo hace el pago confirmado (S21).	Operativo	Au (encargado_despachos, administrador)
S18	RF13	P15 · cargar cola	Consultar la cola de órdenes de despacho en orden de llegada	GET `/api/v1/admin/dispatch-orders`	Consulta opcional: `impresa` (def. `false`), `limit`	200 con lista ordenada por `pagadoEn` ascendente de `{numeroOrden, pedidoId, pagadoEn, clienteNombre, telefono, direccionEntrega, items[], impresa}`	401. 403. 400. Una orden aparece en la cola solo después de que el pago está confirmado. Una orden anulada deja de aparecer.	Restringido	Au (encargado_despachos, administrador)
S19	RF13	P15 · "Imprimir orden"	Obtener la orden de despacho lista para imprimir	GET `/api/v1/admin/dispatch-orders/{pedidoId}/print`	Ruta: `pedidoId`	200 con documento HTML imprimible (`text/html`)	401. 403. 404 la orden no existe en la cola de despacho. Marca la orden como `impresa`; repetir la llamada no cambia el resultado.	Restringido	Au (encargado_despachos, administrador)

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasificación	Acceso
S20	RF2	P10 · "Pagar"	Iniciar el pago de un pedido en la plataforma externa	POST `/api/v1/orders/{id}/payments`	Ruta: `id`. Cuerpo: `medio` (`servipag` o `deposito`). El monto **no** se envía: lo toma el servidor.	201 con `pagoId`, `estado: "pendiente"`, `urlPago`, `expiraEn`	401. 404. 409 si el pedido no está en `pendiente_pago`, está vencido o ya tiene un pago aprobado. 502/503 plataforma de pago no disponible.	Restringido · Operativo	Au (cliente dueño)
S21	RF2, RF4, RF8	Sin pantalla: lo invoca la plataforma de pago	Recibir el resultado del pago	POST `/api/v1/payments/webhook`	Cuerpo definido por la plataforma: `eventoId`, `pagoId`, `estado`, `monto`; cabecera de firma	200 con `{recibido: true}`	401 firma inválida. 400 cuerpo inválido. Si el `monto` no coincide con el pedido no se aprueba. Un `eventoId` repetido responde 200 sin procesar otra vez. Es la **única** vía para pasar un pedido a `pagado`.	Operativo	Identidad de la plataforma (firma)
S22	RF4, RF8	P14 · cargar "Ventas"	Consultar las ventas de la caja virtual	GET `/api/v1/admin/sales`	Consulta opcional: `desde`, `hasta`, `limit`, `offset`	200 con lista de `{ventaId, pedidoId, numeroBoleta, total, fecha, correoEnviado}`	401. 403. 400 rango inválido.	Operativo	Au (cajero_virtual, administrador)
S23	RF8	P11 · "Ver boleta"	Obtener la boleta digital de un pedido pagado	GET `/api/v1/orders/{id}/receipt`	Ruta: `id`	200 con `numeroBoleta`, `fechaEmision`, `pedidoId`, `clienteNombre`, `items[]`, `total`, `ivaIncluido`	401. 404 (no existe o no es del usuario). 409 el pedido aún no está pagado.	Restringido	Au (cliente dueño; cajero_virtual, administrador)
S24	RF5	P12 · cargar perfil · P03 precarga dirección	Consultar mi perfil	GET `/api/v1/clients/me`	Sin cuerpo	200 con `id`, `run`, `nombreCompleto`, `correo`, `telefono`, `fechaNacimiento`, `sexo`, `direccion`	401. 403 si no es cliente.	Restringido	Au (cliente)
S25	RF5	P12 · "Guardar"	Reemplazar mis datos de perfil	PUT `/api/v1/clients/me`	Cuerpo: `nombreCompleto`, `telefono`, `fechaNacimiento`, `sexo`, `direccion`. `run` y `correo` no se cambian aquí.	200 con el perfil	400. 401. 403.	Restringido	Au (cliente)
S26	RF6, RF10, RF11	P16 · "Registrar cliente"	Registrar un cliente presencialmente	POST `/api/v1/admin/clients`	Cuerpo igual a S03 sin `password`	201 con `id`, `estado: "pendiente_verificacion"`	400. 401. 403. 409. 422 correo inexistente. 503. El cliente recibe un correo para confirmar y definir su contraseña (S04).	Restringido	Au (administrador)
S27	RF10	P16 · cargar lista	Listar clientes	GET `/api/v1/admin/clients`	Consulta opcional: `q` (nombre o RUN), `limit`, `offset`	200 con lista con los datos de perfil y `estado`	401. 403. 400.	Restringido	Au (administrador)

ID	Req.	Pantalla y acción	Propósito	Verbo y ruta	Entrada	Salida	Reglas y errores	Clasificación	Acceso
S28	RF10	P16 · "Guardar cambios"	Reemplazar los datos de un cliente	PUT `/api/v1/admin/clients/{id}`	Ruta: `id`. Cuerpo: `nombreCompleto`, `telefono`, `fechaNacimiento`, `sexo`, `direccion`	200 con el cliente	400. 401. 403. 404.	Restringido	Au (administrador)
S29	RF10	P16 · "Dar de baja"	Dar de baja a un cliente (baja lógica)	DELETE `/api/v1/admin/clients/{id}`	Ruta: `id`	204	401. 403. 404. Los pedidos y ventas se conservan. Los pedidos en curso continúan su flujo con los datos que ya guardan.	Restringido	Au (administrador)
S30	RF10, RF12	P17 · "Nuevo usuario"	Crear un usuario del personal	POST `/api/v1/admin/users`	Cuerpo: `nombreCompleto`, `correo`, `rol` (`administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`)	201 con `id`, `rol`, `estado: "pendiente_verificacion"`	400. 401. 403. 409 correo duplicado. 422 correo inexistente. Recibe un correo para definir su contraseña (S04).	Restringido · Credencial	Au (administrador)
S31	RF10	P17 · cargar lista	Listar usuarios del personal	GET `/api/v1/admin/users`	Consulta opcional: `rol`, `estado`, `limit`, `offset`	200 con lista de `{id, nombreCompleto, correo, rol, estado}`	401. 403. 400. Nunca devuelve contraseñas ni hashes.	Restringido	Au (administrador)
S32	RF10, RF12	P17 · "Guardar"	Cambiar nombre, rol o estado de un usuario	PUT `/api/v1/admin/users/{id}`	Ruta: `id`. Cuerpo: `nombreCompleto`, `rol`, `activo`	200 con el usuario	400. 401. 403. 404. 409 si deja al sistema sin administrador activo.	Restringido	Au (administrador)
S33	RF10	P17 · "Dar de baja"	Dar de baja a un usuario (baja lógica)	DELETE `/api/v1/admin/users/{id}`	Ruta: `id`	204	401. 403. 404. 409 si es el propio usuario o el último administrador. Sus sesiones se invalidan.	Restringido	Au (administrador)
S34	RF9	P18 · "Generar reporte"	Reporte de ventas por período	GET `/api/v1/admin/reports/sales`	Consulta: `desde`, `hasta` (rango máx. 92 días)	200 con `totalVentas`, `cantidadVentas`, `porDia[]`, `topProductos[]`	400 rango inválido. 401. 403. `totalVentas` es el monto recaudado en la caja virtual en el período. No descuenta reembolsos pendientes: la venta de un pedido anulado después de pagar sigue contando.	Operativo	Au (dueno, administrador)
S35	RF14	P05 · "Enviar mensaje"	Registrar un mensaje de contacto	POST `/api/v1/contact-messages`	Cuerpo: `nombre` (mín. 2), `correo`, `telefono` (8–15; dígitos, + y espacios), `motivo` (`consulta_pedido`, `reserva_evento`, `alianza_comercial` u `otro`), `mensaje` (10–1000)	201 con `{id, recibido: true}`	400 validación. 429 límite de solicitudes.	Restringido	A

Operaciones internas (sin ruta pública)

Las dispara el propio sistema, no un navegador. Se describen aquí porque el caso las exige.

Operación	Disparador	Qué hace	Req.
Realizar venta	S21 con pago aprobado	Registra la venta en la caja del cajero virtual, solo si el cliente está registrado. Genera el número de boleta.	RF4
Enviar boleta	Venta registrada	Pide a la API de mensajería enviar la boleta digital al correo del cliente. Si falla, se reintenta.	RF8
Publicar orden de despacho	Pedido pasa a `pagado`	Publica la orden de despacho (tipo `nueva`) en la cola ActiveMQ `cola_pedidos`. El sistema de despacho la guarda y S18 la muestra en orden de llegada. Si el pedido se anula, publica una orden de tipo `anulada`.	RF13
Expirar pedidos	Tarea periódica	Anula los pedidos `pendiente_pago` vencidos y devuelve el stock.	RF2
Validar y enviar correos	S03, S05, S26, S30	Llama a la API de mensajería para validar el correo y enviar el enlace de confirmación.	RF11

Acciones sin servicio (con justificación)

Pantalla y acción	Justificación
P01 carrusel y "¿Cómo pedir?"	Contenido fijo del sitio, sin datos del negocio.
P01 y P02 "Agregar" y "Pedir combo"; P03 cambiar cantidad y "Quitar"	El carrito vive en `localStorage` y guarda solo `id` y `cantidad`. El precio real se calcula al crear el pedido (S12).
P04 Sucursales	Hay una sola sucursal con datos informativos. La regla de cobertura de 3 km se aplica en S12.
P07 Modal "Pedido confirmado"	Se muestra con la respuesta de S12.

Servicios que no creamos a propósito

- No hay `DELETE` de pedidos, ventas ni boletas. Se anulan o se conservan para mantener el historial.
- No existe una operación para marcar un pedido como pagado desde un navegador. La mutation `confirmarPago` del repo actual se elimina.

1.7 Ejemplos de contratos (datos ficticios)

S01 Iniciar sesión

```
// POST /api/v1/auth/login
{ "correo": "camila@ejemplo.cl", "password": "ClaveFicticia123" }
// 200
{ "access_token": "tok_ficticio_abc123", "token_type": "bearer", "expires_in": 900, "rol": "cliente",
"nombre": "Camila Rojas" }
// 403
{ "error": { "codigo": "CUENTA_NO_ACTIVA", "mensaje": "Confirma tu correo para activar la cuenta." } }
```

S02 Cerrar sesión

```
POST /api/v1/auth/logout   (Authorization: Bearer tok_ficticio_abc123)
// 204 sin cuerpo
```

S03 Registrar cliente (web)

```
// POST /api/v1/auth/register
{ "run": "12.345.678-5", "nombreCompleto": "Camila Rojas Pérez", "correo": "camila@ejemplo.cl",
  "password": "ClaveFicticia123", "telefono": "+56911112222", "fechaNacimiento": "1995-04-20", "sexo":
"F",
  "direccion": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "provincia": "Santiago", "region":
"Metropolitana" } }
// 201
{ "id": "cli_0001", "correo": "camila@ejemplo.cl", "estado": "pendiente_verificacion" }
// 422
{ "error": { "codigo": "CORREO_NO_VALIDO", "mensaje": "No pudimos comprobar que ese correo exista." } }
```

S04 Confirmar correo

```
// POST /api/v1/auth/verify-email
{ "token": "tok_verificacion_ficticio_xyz", "password": "ClaveFicticia123" }
// 200
{ "estado": "activo" }
// 410
{ "error": { "codigo": "TOKEN_VENCIDO", "mensaje": "El enlace venció o ya fue usado. Pide uno nuevo." } }
```

S05 Reenviar confirmación

```
// POST /api/v1/auth/resend-verification
{ "correo": "camila@ejemplo.cl" }
// 202 sin cuerpo (siempre, exista o no la cuenta)
```

S06 Consultar catálogo

```
// GET /api/v1/products?categoria=clasica&disponible=true
// 200
[ { "id": "prod_001", "nombre": "Napolitana", "categoria": "clasica",
    "descripcion": "Salsa de tomate, mozzarella, jamón, orégano fresco.",
    "precio": 8990, "imagen": "https://ejemplo.cl/napolitana.jpg", "destacado": true, "disponible": true }
]
// sin resultados -> []
```

S07 Listar productos (administración)

```
// GET /api/v1/admin/products?incluirRetirados=true
// 200
[ { "id": "prod_001", "nombre": "Napolitana", "categoria": "clasica", "precio": 8990,
    "stock": 40, "disponible": true, "retirado": false },
  { "id": "prod_009", "nombre": "Pizza Hawaiana", "categoria": "clasica", "precio": 9490,
    "stock": 0, "disponible": false, "retirado": true } ]
```

S08 Crear producto

```
// POST /api/v1/products
{ "nombre": "Pizza Hawaiana", "categoria": "clasica", "descripcion": "Jamón y piña.",
  "precio": 9290, "imagen": "https://ejemplo.cl/hawaiana.jpg", "destacado": false, "stock": 25 }
// 201
{ "id": "prod_009", "nombre": "Pizza Hawaiana", "categoria": "clasica", "descripcion": "Jamón y piña.",
  "precio": 9290, "imagen": "https://ejemplo.cl/hawaiana.jpg", "destacado": false, "disponible": true }
```

S09 Modificar producto

```
// PUT /api/v1/products/prod_009
{ "nombre": "Pizza Hawaiana", "categoria": "clasica", "descripcion": "Jamón, piña y mozzarella.",
  "precio": 9490, "imagen": "https://ejemplo.cl/hawaiana.jpg", "destacado": true }
// 200 -> el producto actualizado
```

S10 Retirar producto

```
DELETE /api/v1/products/prod_009   // 204 sin cuerpo
```

S11 Actualizar disponibilidad y stock

```
// PUT /api/v1/products/prod_009/stock
{ "stock": 12, "disponible": true }
// 200
{ "id": "prod_009", "stock": 12, "disponible": true }
```

S12 Crear pedido

```
// POST /api/v1/orders   (Idempotency-Key: 7f3c-ficticia)
{ "items": [ { "productoId": "prod_001", "cantidad": 2 }, { "productoId": "prod_007", "cantidad": 1 } ],
  "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "referencia": "Casa azul" } }
// 201
{ "id": "ped_0001", "estado": "pendiente_pago",
  "items": [ { "productoId": "prod_001", "nombre": "Napolitana", "cantidad": 2, "precioUnitario": 8990 },
             { "productoId": "prod_007", "nombre": "Bebida 500ml", "cantidad": 1, "precioUnitario": 1990 }
],
  "total": 19970,
  "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "referencia": "Casa azul" },
  "creadoEn": "2026-11-07T18:30:00Z", "expiraEn": "2026-11-07T18:45:00Z" }
// 409
{ "error": { "codigo": "SIN_STOCK", "mensaje": "La Napolitana no tiene stock suficiente." } }
// 422
{ "error": { "codigo": "FUERA_DE_COBERTURA", "mensaje": "Solo despachamos dentro de 3 km del local." } }
```

S13 Listar mis pedidos

```
// GET /api/v1/orders?estado=pagado&limit=10
// 200
[ { "id": "ped_0001", "estado": "pagado", "total": 19970, "creadoEn": "2026-11-07T18:30:00Z" } ]
```

S14 Detalle de pedido

```
// GET /api/v1/orders/ped_0001
// 200
{ "id": "ped_0001", "estado": "pagado", "estadoPago": "aprobado",
  "items": [ { "productoId": "prod_001", "nombre": "Napolitana", "cantidad": 2, "precioUnitario": 8990 },
             { "productoId": "prod_007", "nombre": "Bebida 500ml", "cantidad": 1, "precioUnitario": 1990 }
],
  "total": 19970,
  "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "referencia": "Casa azul" },
```

```
  "historialEstados": [ { "estado": "pendiente_pago", "fecha": "2026-11-07T18:30:00Z" },
                        { "estado": "pagado", "fecha": "2026-11-07T18:33:00Z" } ],
  "boletaDisponible": true }
// 404 (pedido ajeno o inexistente)
{ "error": { "codigo": "NO_ENCONTRADO", "mensaje": "Pedido no encontrado." } }
```

S15 Anular pedido (cliente)

```
// POST /api/v1/orders/ped_0002/cancel
{ "motivo": "Me equivoqué de dirección" }
// 200
{ "id": "ped_0002", "estado": "anulado", "motivo": "Me equivoqué de dirección", "reembolsoPendiente":
false }
```

S16 Listar pedidos (personal)

```
// GET /api/v1/admin/orders?estado=pagado
// 200
[ { "id": "ped_0001", "estado": "pagado", "total": 19970, "creadoEn": "2026-11-07T18:30:00Z",
    "clienteNombre": "Camila Rojas Pérez", "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna":
"Maipú" } } ]
```

S17 Cambiar estado de pedido

```
// PATCH /api/v1/admin/orders/ped_0001/status
{ "estado": "en_preparacion" }
// 200 -> el pedido con estado "en_preparacion"
// 409
{ "error": { "codigo": "TRANSICION_INVALIDA", "mensaje": "No se puede pasar de pagado a entregado." } }
```

S18 Cola de órdenes de despacho

```
// GET /api/v1/admin/dispatch-orders
// 200 (más antigua primero)
[ { "numeroOrden": "OD-000045", "pedidoId": "ped_0001", "pagadoEn": "2026-11-07T18:33:00Z",
    "clienteNombre": "Camila Rojas Pérez", "telefono": "+56911112222",
    "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "referencia": "Casa azul" },
    "items": [ { "nombre": "Napolitana", "cantidad": 2 }, { "nombre": "Bebida 500ml", "cantidad": 1 } ] }
]
```

S19 Imprimir orden de despacho

```
GET /api/v1/admin/dispatch-orders/ped_0001/print
// 200 Content-Type: text/html (documento con número de orden, ítems y dirección)
```

S20 Iniciar pago

```
// POST /api/v1/orders/ped_0001/payments
{ "medio": "servipag" }
// 201
{ "pagoId": "pag_0001", "estado": "pendiente", "urlPago":
"https://pagos-prueba.ejemplo.cl/pagar/ficticio123",
  "expiraEn": "2026-11-07T18:45:00Z" }
```

S21 Notificación de la plataforma de pago

```
// POST /api/v1/payments/webhook   (cabecera de firma de la plataforma)
{ "eventoId": "evt_ficticio_001", "pagoId": "pag_0001", "estado": "aprobado", "monto": 19970 }
// 200
{ "recibido": true }
```

S22 Ventas de la caja virtual

```
// GET /api/v1/admin/sales?desde=2026-11-07&hasta=2026-11-07
// 200
[ { "ventaId": "ven_0001", "pedidoId": "ped_0001", "numeroBoleta": "B-000123", "total": 19970,
    "fecha": "2026-11-07T18:33:10Z", "correoEnviado": true } ]
```

S23 Obtener boleta

```
// GET /api/v1/orders/ped_0001/receipt
// 200
{ "numeroBoleta": "B-000123", "fechaEmision": "2026-11-07T18:33:10Z", "pedidoId": "ped_0001",
  "clienteNombre": "Camila Rojas Pérez",
  "items": [ { "nombre": "Napolitana", "cantidad": 2, "precioUnitario": 8990 },
             { "nombre": "Bebida 500ml", "cantidad": 1, "precioUnitario": 1990 } ],
  "total": 19970, "ivaIncluido": 3188 }
```

S24 Consultar mi perfil

```
// GET /api/v1/clients/me
// 200
{ "id": "cli_0001", "run": "12.345.678-5", "nombreCompleto": "Camila Rojas Pérez", "correo":
"camila@ejemplo.cl",
  "telefono": "+56911112222", "fechaNacimiento": "1995-04-20", "sexo": "F",
```

```
  "direccion": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "provincia": "Santiago", "region":
"Metropolitana" } }
```

S25 Actualizar mi perfil

```
// PUT /api/v1/clients/me
{ "nombreCompleto": "Camila Rojas Pérez", "telefono": "+56933334444", "fechaNacimiento": "1995-04-20",
"sexo": "F",
  "direccion": { "calle": "Calle Falsa 456", "comuna": "Maipú", "provincia": "Santiago", "region":
"Metropolitana" } }
// 200 -> el perfil actualizado
```

S26 Registrar cliente (administrador)

```
// POST /api/v1/admin/clients
{ "run": "11.111.111-1", "nombreCompleto": "Pedro Soto Díaz", "correo": "pedro@ejemplo.cl", "telefono":
"+56955556666",
  "fechaNacimiento": "1988-09-02", "sexo": "M",
  "direccion": { "calle": "Pasaje Ficticio 78", "comuna": "Maipú", "provincia": "Santiago", "region":
"Metropolitana" } }
// 201
{ "id": "cli_0002", "estado": "pendiente_verificacion" }
```

S27 Listar clientes

```
// GET /api/v1/admin/clients?q=soto
// 200
[ { "id": "cli_0002", "run": "11.111.111-1", "nombreCompleto": "Pedro Soto Díaz", "correo":
"pedro@ejemplo.cl",
    "telefono": "+56955556666", "fechaNacimiento": "1988-09-02", "sexo": "M",
    "direccion": { "calle": "Pasaje Ficticio 78", "comuna": "Maipú", "provincia": "Santiago", "region":
"Metropolitana" },
    "estado": "activo" } ]
```

S28 Modificar cliente

```
// PUT /api/v1/admin/clients/cli_0002
{ "nombreCompleto": "Pedro Soto Díaz", "telefono": "+56977778888", "fechaNacimiento": "1988-09-02",
"sexo": "M",
  "direccion": { "calle": "Pasaje Ficticio 80", "comuna": "Maipú", "provincia": "Santiago", "region":
"Metropolitana" } }
// 200 -> el cliente actualizado
```

S29 Dar de baja cliente

```
DELETE /api/v1/admin/clients/cli_0002   // 204 sin cuerpo
```

S30 Crear usuario del personal

```
// POST /api/v1/admin/users
{ "nombreCompleto": "Luis Mena", "correo": "luis@ejemplo.cl", "rol": "cajero_virtual" }
// 201
{ "id": "usr_0003", "rol": "cajero_virtual", "estado": "pendiente_verificacion" }
```

S31 Listar usuarios

```
// GET /api/v1/admin/users?rol=cajero_virtual
// 200
[ { "id": "usr_0003", "nombreCompleto": "Luis Mena", "correo": "luis@ejemplo.cl", "rol": "cajero_virtual",
"estado": "activo" } ]
```

S32 Modificar usuario

```
// PUT /api/v1/admin/users/usr_0003
{ "nombreCompleto": "Luis Mena", "rol": "encargado_despachos", "activo": true }
// 200 -> el usuario actualizado
// 409
{ "error": { "codigo": "ULTIMO_ADMINISTRADOR", "mensaje": "Debe quedar al menos un administrador activo."
} }
```

S33 Dar de baja usuario

```
DELETE /api/v1/admin/users/usr_0003   // 204 sin cuerpo
```

S34 Reporte de ventas

```
// GET /api/v1/admin/reports/sales?desde=2026-11-01&hasta=2026-11-07
// 200
{ "desde": "2026-11-01", "hasta": "2026-11-07", "totalVentas": 1250000, "cantidadVentas": 82,
  "porDia": [ { "fecha": "2026-11-07", "ventas": 14, "total": 210000 } ],
  "topProductos": [ { "productoId": "prod_001", "nombre": "Napolitana", "unidades": 40 } ] }
```

S35 Mensaje de contacto

```
// POST /api/v1/contact-messages
{ "nombre": "Pedro Soto", "correo": "pedro@ejemplo.cl", "telefono": "+56955556666",
  "motivo": "reserva_evento", "mensaje": "Quisiera cotizar 20 pizzas para un cumpleaños." }
```

```
// 201
{ "id": "msg_0001", "recibido": true }
```

Los agregados GraphQL del Módulo 8 (por ejemplo `resumenPedido` para P11) usarán estos mismos servicios como base.

1.8 Trazabilidad de pantallas

Cada acción de cada pantalla, con el servicio que la atiende y el requisito del Caso 19.

Pantalla	Acción	Servicio	Req.
P01 Inicio	Cargar combos	S06 (`categoria=combo`)	RF1, RF7
P01 Inicio	Cargar "más pedidas"	S06 (`destacado=true`)	RF1, RF7
P01 Inicio	"Pedir combo" y "Agregar"; carrusel; "¿Cómo pedir?"	Sin servicio (carrito local o contenido fijo)	—
P02 Menú	Cargar el catálogo y filtrar por categoría	S06	RF1, RF7
P02 Menú	"Agregar"	Sin servicio (carrito local)	—
P03 Carrito	Mostrar el detalle de los ítems	S06 (`ids`)	RF1
P03 Carrito	Precargar la dirección de entrega	S24	RF5
P03 Carrito	"Confirmar pedido"	S12	RF1
P03 Carrito	Cambiar cantidad y "Quitar"	Sin servicio (carrito local)	—
P04 Sucursales	Ver ubicación, horario y mapa	Sin servicio (contenido informativo)	—
P05 Contacto	"Enviar mensaje"	S35	RF14
P06 Modal de sesión (y navbar)	"Entrar"	S01	RF12
P06 Modal de sesión (y navbar)	"Cerrar sesión"	S02	RF12
P07 Modal "Pedido confirmado"	Mostrar el resultado	Respuesta de S12	RF1
P08 Crear cuenta	"Crear cuenta"	S03	RF5, RF11
P09 Confirmar correo	Abrir el enlace y confirmar (o definir contraseña)	S04	RF11, RF6
P09 Confirmar correo	"Reenviar correo"	S05	RF11
P10 Pago y retorno	"Pagar"	S20	RF2
P10 Pago y retorno	Consultar el estado tras volver del proveedor	S14	RF2
P11 Mis pedidos	Cargar la lista	S13	RF1
P11 Mis pedidos	"Ver detalle"	S14	RF1, RF2
P11 Mis pedidos	"Anular pedido"	S15	RF3
P11 Mis pedidos	"Ver boleta"	S23	RF8
P12 Mi cuenta	Cargar y "Guardar" el perfil	S24, S25	RF5
P13 Panel · Productos	Listar con stock	S07	RF7, RF10
P13 Panel · Productos	"Nuevo producto", "Guardar cambios", "Retirar"	S08, S09, S10	RF10
P13 Panel · Productos	"Disponibilidad y stock"	S11	RF7
P14 Panel · Pedidos y ventas	Listar pedidos y ver detalle	S16, S14	RF2, RF4
P14 Panel · Pedidos y ventas	"Anular" (administrador)	S17	RF3
P14 Panel · Pedidos y ventas	Listar ventas de la caja virtual y ver boleta	S22, S23	RF4, RF8
P15 Panel · Despacho	Ver la cola de órdenes de despacho	S18	RF13
P15 Panel · Despacho	"Imprimir orden"	S19	RF13
P15 Panel · Despacho	"Avanzar estado"	S17	RF2
P16 Panel · Clientes	"Registrar cliente"	S26	RF6, RF10, RF11
P16 Panel · Clientes	Listar, "Guardar cambios" y "Dar de baja"	S27, S28, S29	RF10
P17 Panel · Usuarios	"Nuevo usuario", listar, "Guardar" y "Dar de baja"	S30, S31, S32, S33	RF10, RF12
P18 Panel · Reporte	"Generar reporte"	S34	RF9
Sin pantalla	La plataforma de pago avisa el resultado	S21	RF2, RF4, RF8

Los 35 servicios aparecen en esta tabla. Los servicios sin pantalla propia (S21) y las operaciones internas (venta, boleta, orden de despacho, expiración y correos) están en 1.6.

1.9 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está en este documento
1. Recorrer cada pantalla y anotar acciones, formularios, listados y mensajes. Comparar escritorio y móvil.	1.1 (pantallas P01–P07 y la nota de escritorio y móvil), 1.2 (brechas P08–P18) y 1.8 (trazabilidad por pantalla).

Paso de la guía	Dónde está en este documento
2. Asignar un identificador a cada servicio y vincularlo con una pantalla y un requisito. Reutilizar servicios.	1.6, columnas **ID**, **Req.** y **Pantalla y acción**. Reutilización: S06 lo usan P01, P02 y P03; S14 lo usan P10 y P11; S17 lo usan P14 y P15.
3. Clasificar la operación (consultar, agregar, modificar, eliminar) y proponer el verbo. No inventar operaciones destructivas.	1.10 (tipo de operación) y columna **Verbo y ruta** de 1.6. Los tres `DELETE` son bajas lógicas.
4. Definir parámetros de ruta, consulta, cabeceras y cuerpo. Indicar campos, tipos, obligatoriedad, validación y clasificación de cada dato.	Columna **Entrada** de 1.6 y el diccionario de campos de 1.11.
5. Definir respuesta, códigos de estado y errores. Diferenciar lo que envía el navegador de lo que valida el servidor.	Columnas **Salida** y **Reglas y errores** de 1.6, ejemplos de 1.7 y tabla de 1.12.

1.10 Tipo de operación de cada servicio

Tipo	Verbo	Servicios
Consultar	GET	S06, S07, S13, S14, S16, S18, S19, S22, S23, S24, S27, S31, S34
Agregar o ejecutar una operación justificada	POST	S01, S02, S03, S04, S05, S08, S12, S15, S20, S21, S26, S30, S35
Modificar	PUT (reemplaza el recurso) o PATCH (solo el estado, S17)	S09, S11, S17, S25, S28, S32
Eliminar	DELETE (baja lógica; conserva el historial)	S10, S29, S33

Los POST que no crean un recurso (S01, S02, S04, S05, S15, S20, S21) ejecutan una operación: iniciar o cerrar sesión, confirmar un correo, reenviar un correo, anular un pedido, iniciar un pago y recibir un aviso del proveedor.

1.11 Diccionario de campos

Campo	Tipo	Obligatorio	Validación	Clasificación	Servicios
`correo`	Texto (email)	Sí	Formato válido, máx. 120. Debe existir según la API de mensajería. Único.	Restringido	S01, S03, S05, S26, S30, S35
`password`	Texto	Sí en S01 y S03. En S04 solo si la cuenta no tiene contraseña.	Mín. 8, máx. 72. Nunca se devuelve.	Credencial	S01, S03, S04
`token` (verificación)	Texto	Sí	De un solo uso. Vence en 24 h.	Credencial	S04
`access_token`	Texto	Salida de S01. Va en la cabecera `Authorization`.	Vence a los 900 s. Se invalida en S02.	Credencial	S01, S02 y toda ruta autenticada
`run`	Texto (`12.345.678-5`)	Sí	Formato y dígito verificador. Único. No se puede modificar después.	Restringido	S03, S24, S26, S27
`nombreCompleto`	Texto	Sí	2–100 caracteres	Restringido	S03, S24–S28, S30–S32
`telefono`	Texto	Sí (clientes)	8–15; solo dígitos, + y espacios	Restringido	S03, S25, S26, S28, S35
`fechaNacimiento`	Fecha ISO	Sí	No futura y posterior a 1900	Restringido	S03, S25, S26, S28
`sexo`	Enumerado	Sí	`F`, `M` u `O`	Restringido	S03, S25, S26, S28
`direccion.calle`	Texto	Sí	3–120 caracteres	Restringido	S03, S12, S25, S26, S28
`direccion.comuna`	Texto	Sí	De una lista de comunas. Para entregar debe estar dentro de los 3 km.	Restringido	S03, S12, S25, S26, S28
`direccion.provincia`, `direccion.region`	Texto	Sí en el registro	De una lista	Restringido	S03, S25, S26, S28
`direccionEntrega.referencia`	Texto	No	Máx. 120	Restringido	S12
`rol`	Enumerado	Sí en S30 y S32	`administrador`, `dueno`, `cajero_virtual` o `encargado_despachos`. El rol `cliente` solo se asigna al registrarse.	Operativo	S01 (salida), S30–S32
`activo`	Booleano	Sí en S32	—	Operativo	S32
`categoria`	Enumerado	Sí al crear o editar	`clasica`, `premium`, `acompanamiento`, `bebida` o `combo`	Público	S06–S09
`nombre` (producto)	Texto	Sí	2–80 caracteres. Único.	Público	S08, S09
`descripcion`	Texto	Sí	Máx. 300	Público	S08, S09
`precio`	Entero (CLP)	Sí	Mayor que 0	Público	S08, S09
`imagen`	URL	No	`https`, máx. 300	Público	S08, S09
`destacado`	Booleano	No (por defecto falso)	—	Público	S06, S08, S09
`stock`	Entero	Sí en S08 y S11	Mayor o igual que 0	Operativo	S08, S11
`disponible`	Booleano	Sí en S11	—	Público (en el catálogo)	S06, S11

Campo	Tipo	Obligatorio	Validación	Clasificación	Servicios
`productoId`	Texto	Sí	Existe, no está retirado y está disponible	Público	S12
`cantidad`	Entero	Sí	1–20 por ítem	Operativo	S12
`items`	Lista	Sí	1–30 ítems	Operativo	S12
`Idempotency-Key` (cabecera)	Texto	No	8–64 caracteres	Operativo	S12
`motivo` (anulación)	Texto	Sí en S15. En S17 solo si `estado` es `anulado`.	5–200 caracteres	Restringido	S15, S17
`estado` (pedido)	Enumerado	Sí en S17	Solo transiciones permitidas	Operativo	S13, S14, S16, S17
`impresa`	Booleano	No	Lo fija S19. Valor inicial `false`.	Operativo	S18, S19
`medio`	Enumerado	Sí	`servipag` o `deposito`	Operativo	S20
`eventoId`, `pagoId`, `monto`	Texto, texto, entero	Sí	Firma válida. `monto` igual al total del pedido. `eventoId` no repetido.	Operativo	S21
`desde`, `hasta`	Fecha ISO	Sí en S34. Opcionales en S22.	`desde` menor o igual que `hasta`. Máx. 92 días.	Operativo	S22, S34
`limit`, `offset`	Enteros	No	`limit` 1–100. `offset` mayor o igual que 0.	Público	Todas las listas
`q` (búsqueda)	Texto	No	Máx. 60 caracteres	Restringido	S27
`motivo` (contacto)	Enumerado	Sí	`consulta_pedido`, `reserva_evento`, `alianza_comercial` u `otro` (el formulario muestra el texto y envía el valor)	Restringido	S35
`mensaje`	Texto	Sí	10–1000 caracteres	Restringido	S35

1.12 Qué envía el navegador y qué decide el servidor

Dato	El navegador envía	El servidor decide
Precios y total del pedido	Solo `productoId` y `cantidad`	Toma el precio vigente del catálogo y calcula el total. Ignora cualquier precio o total recibido.
Despacho	La dirección de entrega	Comprueba que esté dentro de los 3 km. La entrega es siempre gratuita.
Estado del pedido	La acción solicitada (anular, avanzar)	Valida la transición y quién la pide.
Estado del pago	Solo el `medio` de pago	Lo fija únicamente el aviso firmado del proveedor (S21), con el monto verificado.
Identidad y rol	El token en la cabecera	Verifica el token y obtiene usuario y rol. No acepta `clienteId` ni `rol` en el cuerpo.
Stock y disponibilidad	Nada (en el catálogo público)	El sistema de inventario descuenta, devuelve y calcula `disponible`.
Número de boleta, ids, fechas, vencimientos	Nada	Los genera el servidor.
Venta y caja virtual	Nada	Se registra automáticamente al aprobarse el pago.

1.13 Hallazgos del código existente que afectan el diseño

Los encontramos al leer el repo. Se corrigen en la implementación final.

Dónde	Hallazgo	Qué haremos
`lab-api-gateway/auth_service`	Usuarios con contraseñas en texto plano (`"123"`, `"456"`, `"789"`) y secreto `PizzeriaSecret123` como valor por defecto en el código. Roles `cliente`, `cajero`, `admin`.	Contraseñas con hash adaptativo y sal. Secretos solo desde Vault o variables de entorno, sin valor por defecto. Roles del caso (5 roles).
`graphql-pizzeria/server.js`	Sin autenticación. `confirmarPago`, `delPedido`, `delCliente` y `delUsuario` están abiertas. `addPedido` no valida cantidades ni existencia del producto. El tipo `Usuario` expone `pass`. El modelo de cliente no tiene provincia, región, fecha de nacimiento ni sexo.	Se elimina `confirmarPago` y las mutations destructivas. El GraphQL queda privado, con identidad verificada. Se completa el modelo de cliente según RF5.
`apigateway/gateway.py`	Reenvía sin autenticación y con destinos fijos.	Se reemplaza por el gateway con autenticación y permisos del lab.
`frontend-la-fornace/js/data.js` y `carrito.html`	Precios y productos en el navegador. Cobro de despacho de $1.500 y umbral de $15.000.	El catálogo viene de S06. El despacho es gratis dentro de 3 km, como dice el caso.
`frontend-la-fornace/js/main.js`	Login y confirmación de pedido simulados, ids numéricos.	Se conectan a S01 y S12, con ids reales del backend.
`stomp/productor.py`	Ya publica pedidos en `/queue/cola_pedidos` con credenciales `demo`/`demo123` por defecto.	Se reutiliza la cola para las órdenes de despacho, con credenciales desde Vault.

1.14 Control antes de avanzar

Pregunta	Respuesta
¿Cada acción tiene un servicio o una justificación?	Sí. Las acciones sin servicio están justificadas en 1.6. Las operaciones que exige el caso y no tienen pantalla están en "Operaciones internas".
¿La matriz distingue campos públicos de datos restringidos?	Sí. Columna "Clasificación". El catálogo público no incluye `stock`. RUN, fecha de nacimiento, sexo, dirección, teléfono y correo son restringidos.
¿Los contratos explican qué se transporta sin copiar el modelo de base de datos?	Sí. Solo viajan los campos que usa cada pantalla.
¿Cada RF y CU del caso tiene servicios?	Sí. Ver 1.3.

Módulo 2 — Clases y cardinalidades del dominio

Los diagramas se entregan en formato editable (PlantUML, archivos `.puml`) y legible (`.svg` y `.png`). Están en la carpeta `diagramas/`.

2.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está en este documento
1. Identificar las clases del dominio, con atributos, tipos e identificadores.	Diagrama 2.1 (2.4) y catálogo de clases (2.5).
2. Representar los servicios públicos como «service», con las operaciones de los IDs del Módulo 1.	Diagrama 2.2 (2.7) y correspondencia operación–ID (2.8).
3. Relacionar entidades con asociaciones o composiciones, con multiplicidades en ambos extremos.	Diagrama 2.1 (2.4) y tabla de relaciones R01–R17 (2.6).
4. Relacionar servicios y entidades con dependencias UML, sin interpretarlas como relaciones entre registros.	Diagrama 2.3 (2.9).
5. Explicar las reglas de cada cardinalidad y separar el modelo interno de los DTO.	Columna "Regla" de 2.6 y tabla de DTO (2.10).

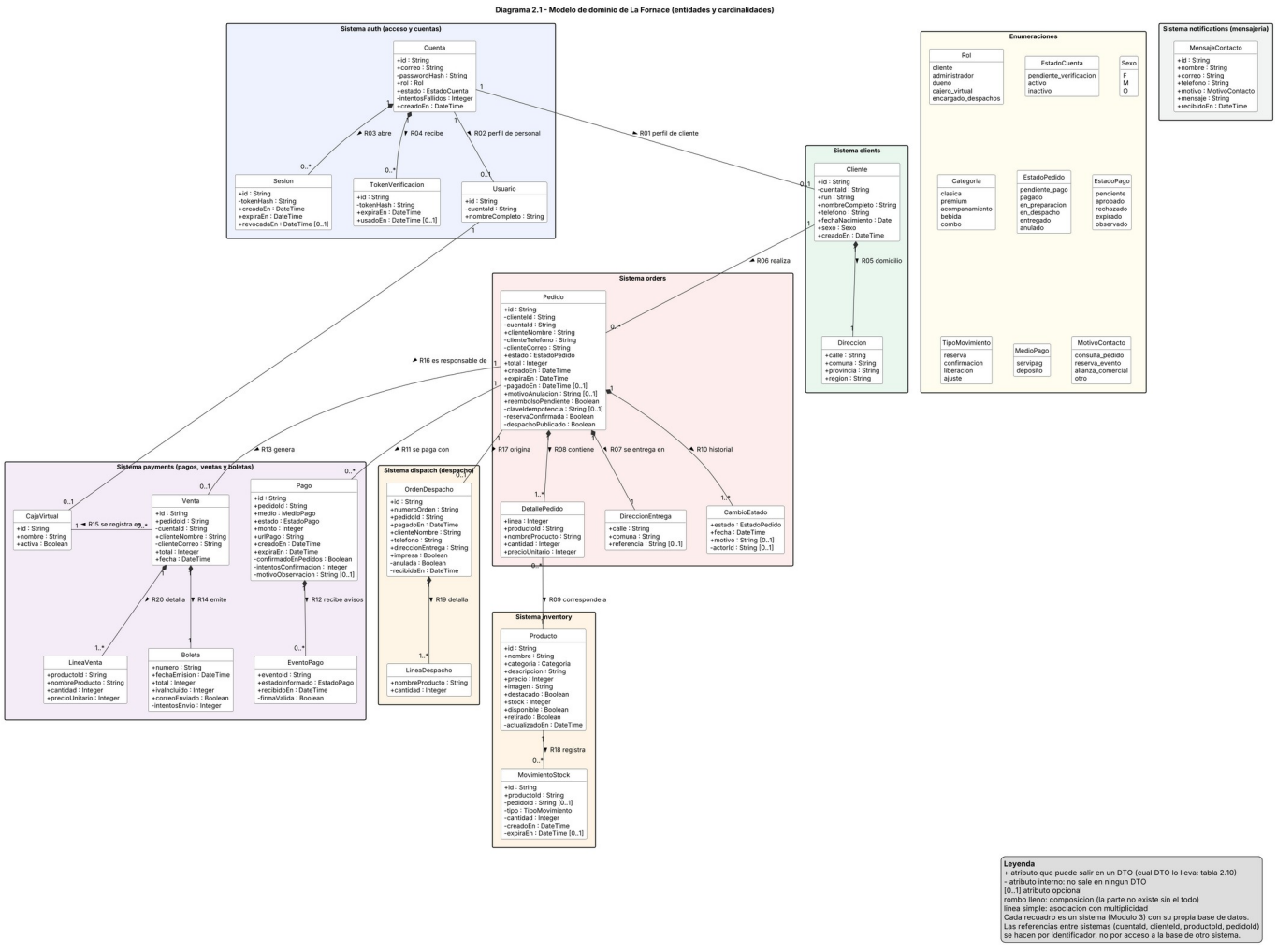
2.2 Decisiones de modelado

Decisión pendiente de la guía	Resolución
¿El carrito y el pedido son clases distintas?	**Sí.** El carrito existe solo en el navegador (`localStorage`, con `id` y `cantidad`). El backend no tiene clase `Carrito`: el `Pedido` nace al confirmar (S12) y se crea siempre con al menos un ítem. Por eso no hay borrador vacío y la relación `Pedido` 1 – 1..* `DetallePedido` no necesita excepciones.
¿Qué ocurre con el historial si un producto deja de estar disponible?	El producto no se borra: S10 lo marca `retirado` (baja lógica). Además, cada `DetallePedido` guarda una copia del nombre y del precio al momento de la compra. Los pedidos antiguos, las boletas y los reportes siguen mostrando los mismos datos aunque el producto cambie de precio o se retire.
¿Qué atributos se conservan internamente y cuáles salen en cada respuesta?	En el diagrama 2.1 los atributos con `-` son internos y no salen en ningún DTO (`passwordHash`, `tokenHash`, `intentosFallidos`, `actorId`, `firmaValida`, `intentosEnvio`, `actualizadoEn`, `claveIdempotencia`, `cuentaId`, `clienteId` del pedido, `anulada`, `recibidaEn`, y las banderas de reintento `reservaConfirmada`, `despachoPublicado`, `confirmadoEnPedidos`, `intentosConfirmacion`). Los atributos con `+` pueden salir, pero solo en el DTO del rol que los necesita. Por ejemplo, `stock` y `retirado` salen en `ProductoAdminDTO` y `StockDTO` (administración) y nunca en `ProductoPublicoDTO` (catálogo público). La tabla 2.10 indica, por DTO, qué sale y qué queda dentro.
¿Dónde viven el correo, el rol y la contraseña?	En `Cuenta` (acceso). `Cliente` y `Usuario` guardan solo los datos personales. Las listas de administración (S27 y S31) combinan perfil y cuenta. Por esta razón la búsqueda `q` de S27 es por nombre o RUN.
¿Por qué hay dos clases de dirección?	`Direccion` es el domicilio del cliente (cambia con el perfil). `DireccionEntrega` es una copia fija en el pedido (no cambia aunque el perfil cambie). Cada una es composición de un solo dueño.
¿`MensajeContacto` se relaciona con `Cliente`?	No. Lo envía un visitante anónimo y no se vincula con ninguna cuenta.
¿Cómo se relacionan clases de sistemas distintos?	Por identificador (`cuentaId`, `clienteId`, `productoId`, `pedidoId`), nunca por acceso a la base de otro sistema. Las asociaciones del diagrama expresan la relación de negocio; la columna "Alcance" de la tabla 2.6 indica cuáles cruzan sistemas, y la forma de integrarlas se define en el Módulo 3.
¿Por qué `Pedido`, `Venta` y `OrdenDespacho` guardan nombre del cliente y otros datos que ya existen en `Cliente`?	Son copias congeladas al momento del hecho. La autoridad de esos datos sigue siendo `Cliente` (clients). La copia evita que un pedido, una boleta o una orden de despacho dependan de otro sistema para mostrarse, y conserva lo que el cliente tenía al comprar. Nunca se usa para modificar al cliente.
¿Dónde se guarda que un pedido está pagado?	Solo en `Pedido.estado` (orders). `Pago` (payments) guarda el resultado del cobro. `estadoPago` y `boletaDisponible` se deducen del estado del pedido, para que no existan dos fuentes de la verdad sobre lo mismo.
¿Qué hace la clase `CajaVirtual`?	El caso asocia las ventas web a una caja de un cajero virtual y el dueño consulta lo recaudado. Se modela una caja con un cajero responsable.
¿Y la cobertura de 3 km?	Es una regla de configuración que valida S12 sobre `DireccionEntrega.comuna`. No necesita una clase propia.

2.3 Leyenda de notación

Símbolo	Significado
`+ atributo`	Atributo que puede salir en un DTO. Qué DTO lo lleva está en la tabla 2.10.
`- atributo`	Atributo interno. No sale en ningún DTO.
`[0..1]` junto a un atributo	Atributo opcional.
Línea simple con números en los extremos	Asociación. Los números son las multiplicidades: `1`, `0..1`, `0..*` o `1..*`.
Línea con rombo lleno	Composición. La parte no existe sin el todo.
Flecha punteada	Dependencia UML: el servicio usa esa clase o ese DTO. **No** es una relación entre registros del negocio.
`«service»`	Servicio público. Cada operación lleva su ID del Módulo 1 entre corchetes.
`«DTO»`	Contrato que viaja entre el navegador y el backend.
`«enumeration»`	Lista fija de valores. Se usa como tipo de atributo y no se dibujan líneas hacia ella.
Recuadros de colores	Cada recuadro es un sistema con su propia base de datos (Módulo 3).

2.4 Diagrama 2.1 — Dominio



Fuente editable: `diagramas/diagrama-2-1-dominio.puml`. Versión vectorial: `diagramas/diagrama-2-1-dominio.svg`.

2.5 Catálogo de clases

Clase	Sistema	Qué representa	Identificador	Origen (requisito · pantalla)
`Cuenta`	auth	Credencial de acceso de una persona: correo, contraseña con hash, rol y estado.	`id`	RF5, RF11, RF12 · P06, P08, P09
`Sesion`	auth	Sesión activa asociada a un token. Permite el control y la revocación de sesiones.	`id`	RF12 (RNF 7) · S01, S02
`TokenVerificacion`	auth	Token de un solo uso para confirmar el correo o definir la contraseña.	`id`	RF11 · P09

Clase	Sistema	Qué representa	Identificador	Origen (requisito · pantalla)
`Usuario`	auth	Perfil del personal del restaurante (administrador, dueño, cajero virtual, encargado de despachos).	`id`	RF10, RF12 · P17
`Cliente`	clients	Datos personales del cliente registrado.	`id`	RF5, RF6 · P08, P12, P16
`Direccion`	clients	Domicilio del cliente (calle, comuna, provincia, región). Objeto de valor.	Sin id propio (parte del Cliente)	RF5 · P08, P12
`Producto`	inventory	Ítem del menú, incluidos los combos, con precio, stock y disponibilidad.	`id`	RF1, RF7, RF10 · P01, P02, P13
`MovimientoStock`	inventory	Registro de cada reserva, confirmación, liberación o ajuste de stock. Permite repetir una reserva sin descontar dos veces.	`id`	RF1, RF7 · P13
`Pedido`	orders	Pedido de un cliente. Su estado sigue el ciclo de vida del Módulo 1.	`id`	RF1, RF2, RF3 · P03, P11
`DetallePedido`	orders	Línea del pedido. Congela nombre y precio del producto al momento de la compra.	`pedido` + `linea`	RF1 · P03, P11
`DireccionEntrega`	orders	Copia de la dirección de entrega al crear el pedido, con referencia opcional.	Sin id propio (parte del Pedido)	RF1 (radio de 3 km) · P03
`CambioEstado`	orders	Registro de cada cambio de estado del pedido (historial).	`pedido` + `fecha`	RF2, RF3 · P11, P14
`OrdenDespacho`	dispatch	Orden que se imprime y va a cocina en orden de llegada. Es una copia que recibe el sistema de despacho cuando el pago se confirma.	`id`, `numeroOrden`	RF13 · P15
`LineaDespacho`	dispatch	Producto y cantidad de una orden de despacho. Sin precios.	`orden` + `linea`	RF13 · P15
`Pago`	payments	Intento de pago de un pedido en la plataforma externa.	`id`	RF2 · P10
`EventoPago`	payments	Aviso recibido de la plataforma de pago. Su `eventoId` evita procesar dos veces el mismo aviso.	`eventoId`	RF2 · S21
`Venta`	payments	Venta registrada en la caja virtual cuando el pago se aprueba. Guarda copia del nombre y el correo del cliente.	`id`	RF4 · P14
`LineaVenta`	payments	Producto, cantidad y precio de una venta. Copia congelada del pedido pagado.	`venta` + `producto`	RF4, RF8, RF9 · P14, P18
`Boleta`	payments	Boleta digital de una venta, enviada por correo.	`numero`	RF8 · P11, P14
`CajaVirtual`	payments	Caja de las ventas web. Pertenece a un cajero virtual.	`id`	RF4 · P14
`MensajeContacto`	notifications	Mensaje enviado por un visitante desde la página de contacto.	`id`	RF14 · P05

Enumeraciones: `Rol` (cliente, administrador, dueno, cajero_virtual, encargado_despachos), `EstadoCuenta` (pendiente_verificacion, activo, inactivo), `Sexo`, `Categoria`, `EstadoPedido`, `EstadoPago`, `MedioPago` y `MotivoContacto`. Sus valores están en el diagrama.

2.6 Tabla de relaciones

La multiplicidad de origen es la que está junto a la clase de origen. La de destino, junto a la clase de destino.

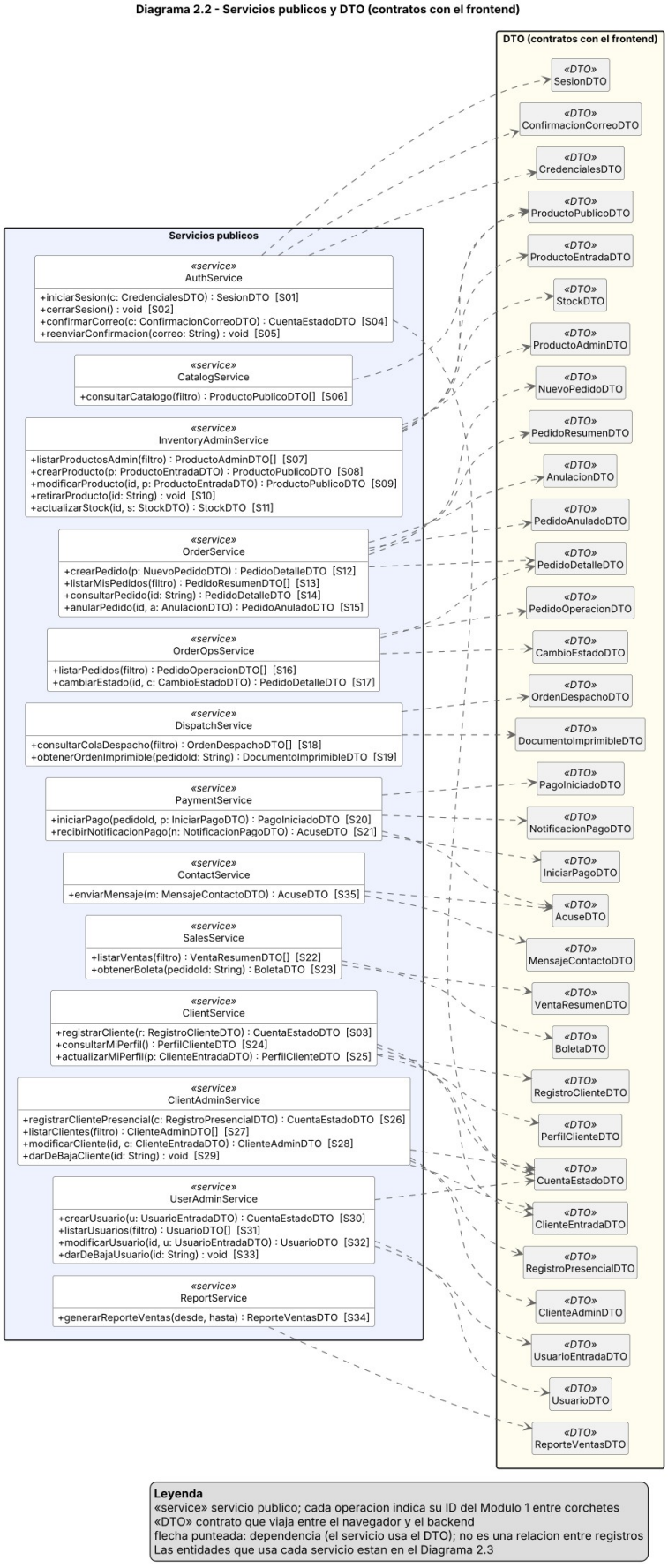
ID	Clase origen	Clase destino	Mult. origen	Mult. destino	Tipo	Nombre	Alcance	Regla de negocio que la origina
R01	`Cuenta`	`Cliente`	1	0..1	Asociación	perfil de cliente	auth → clients (por `cuentaId`)	Toda cuenta con rol `cliente` tiene exactamente un perfil de cliente (RF5). Las cuentas del personal no tienen perfil de cliente, por eso el extremo del cliente es 0..1. Un cliente pertenece a una sola cuenta: el correo es único.
R02	`Cuenta`	`Usuario`	1	0..1	Asociación	perfil de personal	dentro de auth	Toda cuenta con rol distinto de `cliente` tiene un perfil de usuario (RF10, RF12). Las cuentas de clientes no lo tienen. Una cuenta tiene un perfil de cliente **o** uno de personal, nunca ambos.
R03	`Cuenta`	`Sesion`	1	0..*	Composición	abre	dentro de auth	Una cuenta puede tener varias sesiones a la vez (varios dispositivos) o ninguna. Una sesión pertenece a una sola cuenta y se invalida si la cuenta se da de baja (RNF 7).
R04	`Cuenta`	`TokenVerificacion`	1	0..*	Composición	recibe	dentro de auth	Cada reenvío del correo (S05) genera un token nuevo y deja inválido el anterior. Una cuenta recién creada ya tiene uno; después puede tener varios o ninguno vigente. Un token pertenece a una sola cuenta.
R05	`Cliente`	`Direccion`	1	1	Composición	domicilio	dentro de clients	El caso exige dirección, comuna, provincia y región al registrar a un cliente (RF5), así que siempre hay exactamente un domicilio. No se comparte entre clientes.

ID	Clase origen	Clase destino	Mult. origen	Mult. destino	Tipo	Nombre	Alcance	Regla de negocio que la origina
R06	`Cliente`	`Pedido`	1	0..*	Asociación	realiza	clients → orders (por `clienteId`)	Las ventas se hacen solo a pedidos de clientes registrados (RF4), por eso un pedido tiene exactamente un cliente. Un cliente recién registrado aún no tiene pedidos. Con la baja lógica del cliente (S29) sus pedidos se conservan.
R07	`Pedido`	`DireccionEntrega`	1	1	Composición	se entrega en	dentro de orders	Cada pedido guarda una copia de la dirección donde se entrega, validada dentro de los 3 km. Si el cliente cambia su domicilio después, el pedido conserva la dirección original.
R08	`Pedido`	`DetallePedido`	1	1..*	Composición	contiene	dentro de orders	Un pedido se crea con al menos un ítem (RF1) y no existe en el servidor un pedido vacío. Cada línea pertenece a un solo pedido.
R09	`DetallePedido`	`Producto`	0..*	1	Asociación	corresponde a	orders → inventory (por `productoId`)	Cada línea corresponde a exactamente un producto del menú. Un producto puede estar en muchas líneas o en ninguna (por ejemplo, uno recién creado). La línea guarda nombre y precio congelados, y el producto retirado (S10) no se borra, así que el historial queda intacto.
R10	`Pedido`	`CambioEstado`	1	1..*	Composición	historial	dentro de orders	Al crear un pedido se registra el estado inicial `pendiente_pago`, así que siempre hay al menos un cambio. Cada transición agrega uno más. El historial permite conservar el registro de una anulación (RF3).

ID	Clase origen	Clase destino	Mult. origen	Mult. destino	Tipo	Nombre	Alcance	Regla de negocio que la origina
R11	`Pedido`	`Pago`	1	0..*	Asociación	se paga con	orders → payments (por `pedidoId`)	Un pedido recién creado aún no tiene pagos. Puede acumular varios intentos (rechazado, expirado), pero solo uno puede quedar aprobado. Cada pago pertenece a un solo pedido.
R12	`Pago`	`EventoPago`	1	0..*	Composición	recibe avisos	dentro de payments	La plataforma puede avisar más de una vez el resultado de un mismo pago. Se guarda cada `eventoId` para no procesar dos veces el mismo aviso. Un pago sin avisos aún está pendiente.
R13	`Pedido`	`Venta`	1	0..1	Asociación	genera	orders → payments (por `pedidoId`)	La venta existe solo cuando el pago está aprobado (RF2, RF4). Un pedido no pagado o anulado no tiene venta. Un pedido genera a lo más una venta, y toda venta corresponde a un pedido.
R14	`Venta`	`Boleta`	1	1	Composición	emite	dentro de payments	Para cada venta se genera su boleta (RF8). La boleta no existe sin su venta y cada venta tiene una sola.
R15	`Venta`	`CajaVirtual`	0..*	1	Asociación	se registra en	dentro de payments	Las ventas web se asocian a una caja (RF4). Cada venta se registra en exactamente una caja. Una caja acumula muchas ventas, y el dueño consulta el monto recaudado (RF9). Una caja nueva aún no tiene ventas.
R16	`Usuario`	`CajaVirtual`	1	0..1	Asociación	es responsable de	auth → payments (por `usuarioId`)	La caja pertenece a un cajero virtual (RF4). Un usuario cajero virtual tiene a lo más una caja, y toda caja tiene exactamente un cajero responsable. Otros usuarios (por ejemplo, el administrador) no tienen caja.

ID	Clase origen	Clase destino	Mult. origen	Mult. destino	Tipo	Nombre	Alcance	Regla de negocio que la origina
R17	`Pedido`	`OrdenDespacho`	1	0..1	Asociación	origina	orders → dispatch (por `pedidoId`)	La orden de despacho se genera solo cuando el pago está confirmado (RF2, RF13), por eso un pedido pendiente o anulado no la tiene. Cada pedido pagado genera una sola orden.
R18	`Producto`	`MovimientoStock`	1	0..*	Asociación	registra	dentro de inventory	Cada cambio de stock queda como un movimiento (reserva, confirmación, liberación o ajuste). Un producto nuevo aún no tiene movimientos. Un movimiento pertenece a un solo producto. Como el movimiento de un pedido se identifica por `pedidoId` y tipo, repetir una reserva no descuenta dos veces.
R19	`OrdenDespacho`	`LineaDespacho`	1	1..*	Composición	detalla	dentro de dispatch	Una orden de despacho lleva al menos un producto (viene de un pedido que tiene al menos un ítem, R08). Cada línea pertenece a una sola orden y no existe sin ella.
R20	`Venta`	`LineaVenta`	1	1..*	Composición	detalla	dentro de payments	La venta guarda una copia congelada de las líneas del pedido pagado, así la boleta y el reporte no dependen del pedido ni del precio actual del producto. Al menos una línea por venta; cada línea pertenece a una sola venta.

2.7 Diagrama 2.2 — Servicios públicos y DTO



Fuente editable: `diagramas/diagrama-2-2-servicios.puml`. Versión vectorial: `diagramas/diagrama-2-2-servicios.svg`.

Los 13 servicios agrupan las operaciones por tema. En el Módulo 3 cada servicio se asigna a un sistema. Una excepción a tener presente: S03 (registrar cliente) se agrupa con los servicios de clientes porque crea el perfil del cliente y pide la cuenta al sistema de autenticación.

2.8 Correspondencia entre operación UML e ID de servicio

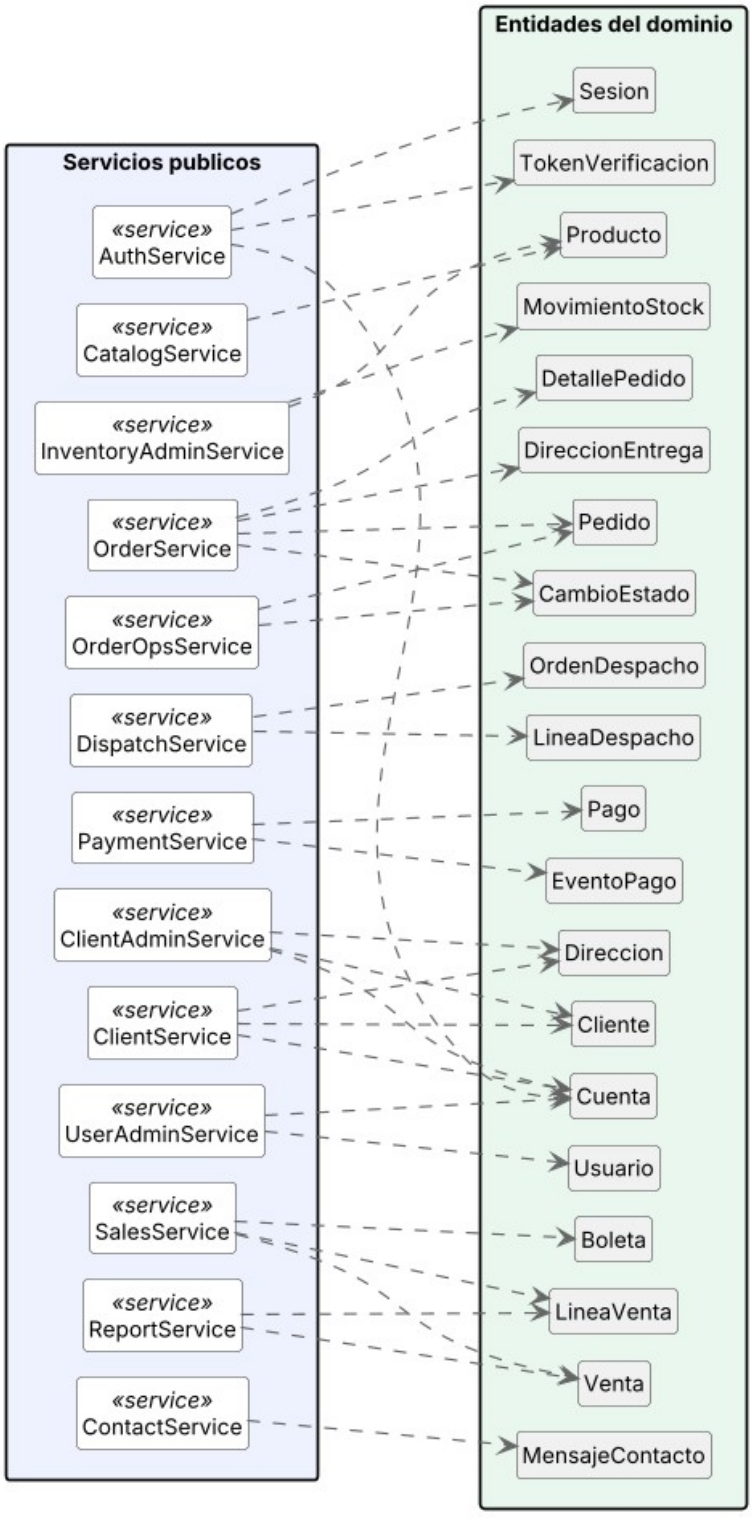
ID	Servicio «service»	Operación UML	Verbo y ruta (Módulo 1)
S01	AuthService	iniciarSesion(c: CredencialesDTO) : SesionDTO	POST /api/v1/auth/login
S02	AuthService	cerrarSesion() : void	POST /api/v1/auth/logout
S03	ClientService	registrarCliente(r: RegistroClienteDTO) : CuentaEstadoDTO	POST /api/v1/auth/register
S04	AuthService	confirmarCorreo(c: ConfirmacionCorreoDTO) : CuentaEstadoDTO	POST /api/v1/auth/verify-email
S05	AuthService	reenviarConfirmacion(correo: String) : void	POST /api/v1/auth/resend-verification
S06	CatalogService	consultarCatalogo(filtro) : ProductoPublicoDTO[]	GET /api/v1/products
S07	InventoryAdminService	listarProductosAdmin(filtro) : ProductoAdminDTO[]	GET /api/v1/admin/products
S08	InventoryAdminService	crearProducto(p: ProductoEntradaDTO) : ProductoPublicoDTO	POST /api/v1/products
S09	InventoryAdminService	modificarProducto(id, p: ProductoEntradaDTO) : ProductoPublicoDTO	PUT /api/v1/products/{id}
S10	InventoryAdminService	retirarProducto(id: String) : void	DELETE /api/v1/products/{id}
S11	InventoryAdminService	actualizarStock(id, s: StockDTO) : StockDTO	PUT /api/v1/products/{id}/stock
S12	OrderService	crearPedido(p: NuevoPedidoDTO) : PedidoDetalleDTO	POST /api/v1/orders
S13	OrderService	listarMisPedidos(filtro) : PedidoResumenDTO[]	GET /api/v1/orders
S14	OrderService	consultarPedido(id: String) : PedidoDetalleDTO	GET /api/v1/orders/{id}
S15	OrderService	anularPedido(id, a: AnulacionDTO) : PedidoAnuladoDTO	POST /api/v1/orders/{id}/cancel
S16	OrderOpsService	listarPedidos(filtro) : PedidoOperacionDTO[]	GET /api/v1/admin/orders
S17	OrderOpsService	cambiarEstado(id, c: CambioEstadoDTO) : PedidoDetalleDTO	PATCH /api/v1/admin/orders/{id}/status
S18	DispatchService	consultarColaDespacho(filtro) : OrdenDespachoDTO[]	GET /api/v1/admin/dispatch-orders
S19	DispatchService	obtenerOrdenImprimible(pedidoId: String) : DocumentoImprimibleDTO	GET /api/v1/admin/dispatch-orders/{pedidoId}/print
S20	PaymentService	iniciarPago(pedidoId, p: IniciarPagoDTO) : PagoIniciadoDTO	POST /api/v1/orders/{id}/payments
S21	PaymentService	recibirNotificacionPago(n: NotificacionPagoDTO) : AcuseDTO	POST /api/v1/payments/webhook
S22	SalesService	listarVentas(filtro) : VentaResumenDTO[]	GET /api/v1/admin/sales
S23	SalesService	obtenerBoleta(pedidoId: String) : BoletaDTO	GET /api/v1/orders/{id}/receipt
S24	ClientService	consultarMiPerfil() : PerfilClienteDTO	GET /api/v1/clients/me
S25	ClientService	actualizarMiPerfil(p: ClienteEntradaDTO) : PerfilClienteDTO	PUT /api/v1/clients/me
S26	ClientAdminService	registrarClientePresencial(c: RegistroPresencialDTO) : CuentaEstadoDTO	POST /api/v1/admin/clients
S27	ClientAdminService	listarClientes(filtro) : ClienteAdminDTO[]	GET /api/v1/admin/clients
S28	ClientAdminService	modificarCliente(id, c: ClienteEntradaDTO) : ClienteAdminDTO	PUT /api/v1/admin/clients/{id}

ID	Servicio «service»	Operación UML	Verbo y ruta (Módulo 1)
S29	ClientAdminService	darDeBajaCliente(id: String) : void	DELETE /api/v1/admin/clients/{id}
S30	UserAdminService	crearUsuario(u: UsuarioEntradaDTO) : CuentaEstadoDTO	POST /api/v1/admin/users
S31	UserAdminService	listarUsuarios(filtro) : UsuarioDTO[]	GET /api/v1/admin/users
S32	UserAdminService	modificarUsuario(id, u: UsuarioEntradaDTO) : UsuarioDTO	PUT /api/v1/admin/users/{id}
S33	UserAdminService	darDeBajaUsuario(id: String) : void	DELETE /api/v1/admin/users/{id}
S34	ReportService	generarReporteVentas(desde, hasta) : ReporteVentasDTO	GET /api/v1/admin/reports/sales
S35	ContactService	enviarMensaje(m: MensajeContactoDTO) : AcuseDTO	POST /api/v1/contact-messages

Los 35 servicios del Módulo 1 están representados. La operación interna "realizar venta" y las demás internas no se muestran porque no son servicios públicos.

2.9 Diagrama 2.3 — Servicios y entidades que usan

Diagrama 2.3 - Servicios publicos y entidades del dominio que usan



Leyenda
«service» servicio publico (operaciones y IDs en el Diagrama 2.2)
flecha punteada: dependencia (el servicio usa la clase); no significa un registro por servicio
Las relaciones entre entidades y sus multiplicidades estan en el Diagrama 2.1

Fuente editable: `diagramas/diagrama-2-3-servicios-entidades.puml`. Versión vectorial: `diagramas/diagrama-2-3-servicios-entidades.svg`.

Servicio	Entidades que usa	IDs
AuthService	Cuenta, Sesion, TokenVerificacion	S01, S02, S04, S05
CatalogService	Producto	S06
InventoryAdminService	Producto, MovimientoStock	S07–S11
OrderService	Pedido, DetallePedido, DireccionEntrega, CambioEstado	S12–S15
OrderOpsService	Pedido, CambioEstado	S16, S17
DispatchService	OrdenDespacho, LineaDespacho	S18, S19
PaymentService	Pago, EventoPago	S20, S21
SalesService	Venta, LineaVenta, Boleta	S22, S23
ClientService	Cliente, Direccion, Cuenta	S03, S24, S25
ClientAdminService	Cliente, Direccion, Cuenta	S26–S29
UserAdminService	Usuario, Cuenta	S30–S33
ReportService	Venta, LineaVenta	S34
ContactService	MensajeContacto	S35

Estas dependencias indican que el servicio usa la clase para cumplir sus operaciones. No significan que exista un registro de esa clase por cada instancia del servicio.

2.10 Modelo interno y DTO

El modelo interno (diagrama 2.1) nunca se envía completo. Cada servicio recibe y devuelve un DTO con los campos mínimos que necesita su pantalla.

DTO	Sentido	Servicios	Entidades de origen	Campos que viajan	Campos que quedan dentro
CredencialesDTO	Entrada	S01	Cuenta	correo, password	—
SesionDTO	Salida	S01	Cuenta, Sesion	access_token, token_type, expires_in, rol, nombre	tokenHash, intentosFallidos, passwordHash
RegistroClienteDTO	Entrada	S03	Cuenta, Cliente, Direccion	run, nombreCompleto, correo, password, telefono, fechaNacimiento, sexo, direccion	id, rol, estado (los asigna el servidor)
RegistroPresencialDTO	Entrada	S26	Cuenta, Cliente, Direccion	Igual a RegistroClienteDTO sin password	id, rol, estado
CuentaEstadoDTO	Salida	S03, S04, S26, S30	Cuenta	id, correo o rol (según el servicio), estado	passwordHash, intentosFallidos
ConfirmacionCorreoDTO	Entrada	S04	TokenVerificacion, Cuenta	token, password (solo si la cuenta aún no tiene)	tokenHash, expiraEn, usadoEn
ProductoPublicoDTO	Salida	S06, S08, S09	Producto	id, nombre, categoria, descripcion, precio, imagen, destacado, disponible	stock, retirado, actualizadoEn
ProductoAdminDTO	Salida	S07	Producto	id, nombre, categoria, precio, stock, disponible, retirado	actualizadoEn
ProductoEntradaDTO	Entrada	S08, S09	Producto	nombre, categoria, descripcion, precio, imagen, destacado (y stock solo en S08)	id, disponible, retirado (los calcula el servidor)
StockDTO	Entrada y salida	S11	Producto, MovimientoStock	stock, disponible (y id en la salida)	retirado; el cambio queda como MovimientoStock de tipo ajuste
NuevoPedidoDTO	Entrada	S12	Pedido, DetallePedido, DireccionEntrega	items[productoId, cantidad], direccionEntrega (opcional)	Precios, total, estado, ids y fechas: los calcula el servidor
PedidoResumenDTO	Salida	S13	Pedido	id, estado, total, creadoEn	clienteId, reembolsoPendiente

DTO	Sentido	Servicios	Entidades de origen	Campos que viajan	Campos que quedan dentro
PedidoDetalleDTO	Salida	S12, S14, S17	Pedido, DetallePedido, DireccionEntrega, CambioEstado	id, estado, estadoPago, items[productoId, nombre, cantidad, precioUnitario], total, direccionEntrega, historialEstados[estado, fecha], boletaDisponible, creadoEn, expiraEn. Solo si lo pide el personal (S14): también clienteNombre	clienteId, cuentaId, actorId de cada cambio, banderas de reintento. estadoPago y boletaDisponible se deducen del estado del pedido; el pedido no guarda datos del pago
AnulacionDTO	Entrada	S15	Pedido	motivo	—
PedidoAnuladoDTO	Salida	S15	Pedido	id, estado, motivo, reembolsoPendiente	clienteId, historial
PedidoOperacionDTO	Salida	S16	Pedido, DireccionEntrega	id, estado, total, creadoEn, clienteNombre, direccionEntrega	clienteId, cuentaId, clienteTelefono, clienteCorreo
CambioEstadoDTO	Entrada	S17	CambioEstado	estado, motivo (obligatorio si es anulado)	actorId (lo toma del token)
OrdenDespachoDTO	Salida	S18	OrdenDespacho, LineaDespacho	numeroOrden, pedidoId, pagadoEn, clienteNombre, telefono, direccionEntrega, items[nombre, cantidad], impresa	Precios, total, run, correo (la orden de despacho nunca los recibe), anulada, recibidaEn
DocumentoImprimibleDTO	Salida	S19	OrdenDespacho, LineaDespacho	Documento HTML con número de orden, ítems y dirección	Todo lo que no necesita cocina
IniciarPagoDTO	Entrada	S20	Pago	medio	monto (lo toma el servidor del pedido)
PagoIniciadoDTO	Salida	S20	Pago	pagoId, estado, urlPago, expiraEn	pedidoId, monto, banderas de confirmación
NotificacionPagoDTO	Entrada	S21	Pago, EventoPago	eventoId, pagoId, estado, monto (la firma va en una cabecera)	firmaValida (la calcula el servidor)
VentaResumenDTO	Salida	S22	Venta, Boleta	ventaId, pedidoId, numeroBoleta, total, fecha, correoEnviado	intentosEnvio, cuentaId, clienteCorreo, caja interna
BoletaDTO	Salida	S23	Boleta, Venta, LineaVenta	numeroBoleta, fechaEmision, pedidoId, clienteNombre, items, total, ivaIncluido	cuentaId, clienteCorreo, intentosEnvio
PerfilClienteDTO	Salida	S24, S25	Cliente, Direccion, Cuenta	id, run, nombreCompleto, correo, telefono, fechaNacimiento, sexo, direccion	cuentaId, passwordHash, rol, intentosFallidos
ClienteEntradaDTO	Entrada	S25, S28	Cliente, Direccion	nombreCompleto, telefono, fechaNacimiento, sexo, direccion	run, correo, id (no se modifican por aquí)
ClienteAdminDTO	Salida	S27, S28	Cliente, Direccion, Cuenta	Los de PerfilClienteDTO más estado	cuentaId, passwordHash, intentosFallidos
UsuarioEntradaDTO	Entrada	S30, S32	Usuario, Cuenta	nombreCompleto, rol; correo solo en S30; activo solo en S32	id, passwordHash
UsuarioDTO	Salida	S31, S32	Usuario, Cuenta	id, nombreCompleto, correo, rol, estado	passwordHash, cuentaId, intentosFallidos
ReporteVentasDTO	Salida	S34	Venta, LineaVenta	desde, hasta, totalVentas, cantidadVentas, porDia[], topProductos[]	Datos de clientes individuales (cuentaId, clienteNombre, clienteCorreo)
MensajeContactoDTO	Entrada	S35	MensajeContacto	nombre, correo, telefono, motivo, mensaje	id, recibidoEn
AcuseDTO	Salida	S21, S35	—	recibido (y id en S35)	—

Notas de lectura:

- `PedidoDetalleDTO` es el contrato completo de un pedido. S12 solo llena los campos que existen al crear el pedido (sin `estadoPago` ni `boletaDisponible`, que aparecen cuando hay pago o venta), tal como indica el ejemplo del Módulo 1.
- `Pedido.claveIdempotencia` guarda la cabecera `Idempotency-Key` de S12. Si llega la misma clave del mismo cliente, se devuelve el pedido ya creado. Nunca sale en un DTO.

2.11 Control antes de avanzar

Pregunta	Respuesta
¿Todas las operaciones del Módulo 1 están representadas?	Sí. Los 35 IDs (S01 a S35) aparecen como operaciones en el diagrama 2.2 y en la tabla 2.8.
¿Cada asociación tiene ambas multiplicidades y una regla que la respalda?	Sí. Las 20 relaciones (R01 a R20) tienen ambos extremos en el diagrama 2.1 y su regla en la tabla 2.6.
¿El diagrama evita exponer datos internos solo porque forman parte de una clase?	Sí. Los atributos internos llevan `-` y cada DTO lista qué campos salen y cuáles no (2.10).

Módulo 3 — Sistemas e integraciones

Los diagramas se entregan en formato editable (PlantUML, `.puml`) y legible (`.svg` y `.png`). Están en la carpeta `diagramas/`.

3.1 Cumplimiento de la guía

Qué pide la guía	Dónde está
Sistemas con responsabilidad propia, incluidos inventory, orders, clients y payments como microservicios privados.	Matriz 3.3 y diagrama 3.1 (3.13).
Justificar los límites con cohesión, acoplamiento y propiedad de datos, sin dividir por tabla ni por pantalla.	3.2.
Asignar a cada sistema su responsabilidad, la autoridad sobre sus datos y el dueño del estado oficial.	3.3, 3.4 y 3.5.
Asignar los servicios del Módulo 1 a los sistemas, con sus dependencias.	3.6.
Definir los intercambios: operación o evento, datos mínimos, autenticación entre componentes y manejo de errores.	3.7 y 3.8.
Sistemas complementarios (despacho, boletas, mensajería), separando los propios de los proveedores externos.	3.12.
Matriz de sistemas, mapa de componentes y secuencia de un flujo representativo.	3.3, 3.13 y 3.14.
Resolver el caso pedidos ↔ inventario (disponibilidad, reserva y respuesta que no llega).	3.9.
Resolver el caso del aviso de pago (verificación, duplicados y quién autoriza el cambio de estado).	3.10.

3.2 Decisiones de partición y límites

La regla fue dividir por **quién es dueño de un dato y de sus reglas**, no por tabla ni por pantalla. Cada sistema tiene su propia base de datos (una sola instancia de MongoDB, con una base y un usuario por sistema) y ningún sistema lee la base de otro.

Sistema	Cohesión: qué reúne	Acoplamiento: de quién depende	Propiedad: de qué datos es dueño
`auth-service`	Todo lo que prueba quién es una persona: credenciales, sesiones, verificación de correo y el personal. Cambia por razones de seguridad.	Solo del servicio de mensajería para enviar el enlace.	Cuenta, Sesion, TokenVerificacion, Usuario.
`clients-service`	Datos personales y domicilio del cliente, y su registro. Cambia por razones comerciales.	De auth, porque cada cliente tiene una cuenta.	Cliente, Direccion.
`inventory-service`	Producto, precio, disponibilidad y stock con sus reservas. Se consulta mucho y se modifica poco.	De nadie. Es el sistema más independiente.	Producto, MovimientoStock.
`orders-service`	El ciclo de vida del pedido y la regla de que solo un pago confirmado lo pasa a `pagado`.	De clients (datos del cliente), inventory (reservas) y, por cola, de dispatch.	Pedido, DetallePedido, DireccionEntrega, CambioEstado.
`payments-service`	Todo lo que depende de la plataforma externa de pago: cobro, aviso firmado, venta en caja, boleta y reporte de ventas.	De orders (confirmar el pago) y de mensajería (enviar la boleta).	Pago, EventoPago, Venta, LineaVenta, Boleta, CajaVirtual.
`dispatch-service`	La cola que usa la cocina: orden de llegada e impresión. No necesita saber precios.	De nadie (solo consume la cola).	OrdenDespacho, LineaDespacho.

Sistema	Cohesión: qué reúne	Acoplamiento: de quién depende	Propiedad: de qué datos es dueño
`notifications-service`	Toda la salida de correo y los mensajes de contacto. Aísla al proveedor externo.	Del proveedor externo de correo.	MensajeContacto.

Alternativas que se descartaron:

Alternativa	Por qué no
Un servicio por clase del Módulo 2 (21 servicios).	Cada pedido tocaría seis servicios y cada cambio exigiría coordinar a todos. Es dividir por tabla.
Un servicio por pantalla.	Las pantallas comparten datos (P03, P10 y P11 usan el mismo pedido). Se duplicaría la lógica.
Unir orders con payments.	El pago depende de un tercero (firma, avisos repetidos, reintentos) y tiene otro ritmo de cambio. Separarlos deja que la autoridad del estado del pedido sea una sola.
Unir inventory con orders.	El precio y el stock tienen otro dueño y otra frecuencia de cambio que el estado de un pedido. Además el catálogo público se consulta sin sesión.
Unir auth con clients.	Las credenciales necesitan más aislamiento que los datos personales, y el personal también usa auth sin tener perfil de cliente.
Separar boletas y caja virtual de payments.	No tienen datos ni reglas propias: nacen 1 a 1 de una venta, que ya es de payments. Separarlas agregaría una llamada sin ganar independencia. Se explican como sistema complementario en 3.12.

Reparto de los 35 servicios del Módulo 1:

Sistema	Cantidad	Servicios
`auth-service`	8	S01, S02, S04, S05, S30, S31, S32, S33
`clients-service`	7	S03, S24, S25, S26, S27, S28, S29
`inventory-service`	6	S06, S07, S08, S09, S10, S11
`orders-service`	6	S12, S13, S14, S15, S16, S17
`payments-service`	5	S20, S21, S22, S23, S34
`dispatch-service`	2	S18, S19
`notifications-service`	1	S35

La agrupación UML del Módulo 2 se conserva, con un ajuste: S03 (registrar cliente) se agrupa con los servicios de clients porque crea el perfil del cliente y pide la cuenta a auth. Así auth no depende de clients y no hay ciclo.

3.3 Matriz de sistemas

Sistema	Responsabilidad	Datos que administra	Servicios asociados	Dependencias	Contrato de integración	Credencial o identidad requerida	Respuesta ante fallos
`api-gateway` (público)	Único punto de entrada. Valida el token, quita cualquier identidad enviada por el navegador, agrega la identidad verificada, limita la tasa y enruta por prefijo.	Ninguno de negocio. No tiene base de datos.	Los 35 pasan por él. Sin token: S01, S03, S04, S05, S06, S21 y S35.	auth (I01) y todos para enrutar (I02). Vault (I18).	Hacia afuera: HTTPS y `/api/v1/...`. Hacia adentro: JSON y cabeceras `X-Authenticated-*`.	`X-Service-Id` y `X-Service-Secret` hacia auth. `X-Gateway-Secret` hacia los demás. Ambos en Vault.	Auth caído: 503 en rutas con token (falla cerrado). Las rutas públicas siguen. Sistema caído: 502 o 503 sin reintentar escrituras.
`auth-service` (privado)	Identidad: cuentas, contraseñas con hash, sesiones y tokens, verificación de correo y personal. **Dueño del estado oficial de la cuenta** (`pendiente_verificacion`, `activo`, `inactivo`).	Cuenta, Sesion, TokenVerificacion, Usuario.	S01, S02, S04, S05, S30, S31, S32, S33.	notifications (I06). Vault.	Internos: `POST /internal/introspect` (I01), `POST` y `DELETE /internal/accounts` (I03), `GET /internal/accounts` (I04), `PATCH /internal/accounts/{id}` (I05).	Acepta a api-gateway (I01) y a clients-service (I03, I04, I05), cada uno con su par de credenciales. Usuario `auth_user` solo sobre `auth_db`.	Mensajería caída: no crea cuenta ni cambia token. Base de datos caída: 503. Nunca devuelve hashes en un error.
`clients-service` (privado)	Perfil y domicilio del cliente y su alta (web o presencial). **Autoridad de los datos personales** (RUN, nombre, teléfono, nacimiento, sexo, domicilio).	Cliente, Direccion.	S03, S24, S25, S26, S27, S28, S29.	auth (I03, I04, I05). Vault.	Interno: `GET /internal/clients/by-account/{cuentaId}` (I07).	Acepta a orders-service (I07). Usa su par de credenciales hacia auth. Usuario `clients_user` solo sobre `clients_db`.	Auth caído: S03, S26, S27, S28 y S29 responden 503 y no dejan datos a medias (compensación en I03). S24 y S25 no dependen de auth y siguen funcionando.

Sistema	Responsabilidad	Datos que administra	Servicios asociados	Dependencias	Contrato de integración	Credencial o identidad requerida	Respuesta ante fallos
`inventory-service` (privado)	Productos, precio, disponibilidad, stock y reservas. **Autoridad de precio y stock.**	Producto, MovimientoStock.	S06, S07, S08, S09, S10, S11.	Ninguna. Vault.	Internos: `POST /internal/stock/reservations` (I08), `.../confirm` (I09) y `.../release` (I10).	Solo acepta a orders-service en las rutas internas. Usuario `inventory_user` solo sobre `inventory_db`.	Su caída detiene el catálogo y la creación de pedidos, pero no el pago de pedidos ya creados. Libera solas las reservas vencidas sin confirmar.
`orders-service` (privado)	Crea el pedido, congela precios y dirección, lleva el historial, vence los pedidos y publica la orden de despacho. **Dueño del estado oficial del pedido.**	Pedido, DetallePedido, DireccionEntrega, CambioEstado.	S12, S13, S14, S15, S16, S17.	clients (I07), inventory (I08, I09, I10), ActiveMQ (I14). Vault.	Internos: `GET /internal/orders/{id}/payable` (I11) y `POST /internal/orders/{id}/payment-confirmation` (I12).	Acepta a payments-service en I11 e I12. Usa sus pares hacia clients e inventory y un usuario de ActiveMQ de solo escritura. Usuario `orders_user` solo sobre `orders_db`.	Inventory caído: S12 responde 503 sin crear el pedido. ActiveMQ caído: el pedido igual queda `pagado` y la orden se publica después. Tareas de reintento cada 30 s.
`payments-service` (privado)	Inicia el cobro, valida el aviso, registra la venta en la caja virtual, genera la boleta y calcula el reporte de ventas. **Dueño del estado del cobro** (`pendiente`, `aprobado`, `rechazado`, `expirado`, `observado`).	Pago, EventoPago, Venta, LineaVenta, Boleta, CajaVirtual.	S20, S21, S22, S23, S34.	orders (I11, I12), notifications (I13), plataforma de pago (I16). Vault.	Público con firma: `POST /api/v1/payments/webhook` (I15).	Secreto HMAC del webhook y credencial de la plataforma (Vault). Pares hacia orders y notifications. Usuario `payments_user` solo sobre `payments_db`.	Orders caído: el pago queda `aprobado` y se reintenta. Pedido no pagable: pago `observado`. Correo caído: la venta existe y se reintenta el envío.
`dispatch-service` (privado)	Guarda las órdenes de despacho que llegan por la cola, las ordena por llegada y las entrega para imprimir. **Dueño de la orden de despacho y de su impresión.**	OrdenDespacho, LineaDespacho.	S18, S19.	ActiveMQ (consumidor). Vault.	Consume `cola_pedidos` (I14).	Usuario de ActiveMQ de solo lectura. Usuario `dispatch_user` solo sobre `dispatch_db`.	Broker caído: sigue mostrando lo ya guardado y reconecta. Mensaje repetido: se ignora. Mensaje inválido: va al registro de errores sin detener el consumo.
`notifications-service` (privado)	Valida correos, envía los correos del sistema (enlaces y boletas) y guarda los mensajes de contacto. **Dueño del mensaje de contacto.**	MensajeContacto.	S35.	Proveedor externo de correo (I17). Vault.	Interno: `POST /internal/emails` (I06 e I13).	Acepta a auth-service y a payments-service. Clave del proveedor en Vault. Usuario `notifications_user` solo sobre `notifications_db`.	Proveedor caído: 503 a quien llama. S35 siempre guarda el mensaje, porque no usa al proveedor.

3.4 Autoridad de los datos

Cada dato tiene un solo sistema que lo puede cambiar. Los demás solo guardan una copia congelada cuando necesitan mostrarlo después sin volver a preguntar.

Dato	Sistema autoridad	Copias y por qué existen	Quién lo cambia
Correo, contraseña (hash), rol y estado de la cuenta	auth	Orders guarda `clienteCorreo` en el pedido y payments `clienteCorreo` en la venta, solo para enviar la boleta. El correo llega en `X-Authenticated-Email`, ya verificado por el gateway.	Solo auth (S04, S30, S32, S33 e I05).
Sesión y token	auth	Ninguna.	Solo auth (S01, S02).
RUN, nombre, teléfono, nacimiento, sexo y domicilio del cliente	clients	`clienteNombre` y `clienteTelefono` en el pedido, `clienteNombre` en la venta y en la orden de despacho. Son copias del momento de la compra y nunca modifican al cliente.	Solo clients (S03, S25, S26, S28).
Nombre, precio, categoría, disponibilidad y stock del producto	inventory	Nombre y precio unitario en cada línea del pedido, de la venta y de la orden de despacho (sin precio). Congelan lo que se compró.	Solo inventory (S08 a S11 e I08 a I10).

Dato	Sistema autoridad	Copias y por qué existen	Quién lo cambia
Reservas de stock	inventory	Orders guarda `reservaConfirmada` solo como bandera de reintento.	Solo inventory, a pedido de orders.
Estado del pedido, ítems, total, dirección de entrega, historial, motivo de anulación y `reembolsoPendiente`	orders	`estadoPago` y `boletaDisponible` se deducen del estado, no se guardan en otro lado.	Solo orders (S12, S15, S17, I12 y el vencimiento).
Pago (estado, monto, medio, `urlPago`) y avisos recibidos	payments	Ninguna. Orders no guarda datos del pago.	Solo payments (S20, S21).
Venta, líneas, boleta y caja virtual	payments	Ninguna. S23 y S34 se responden con estos datos, sin llamar a orders.	Solo payments (S21).
Orden de despacho y si fue impresa	dispatch	Ninguna.	Solo dispatch (I14 y S19).
Mensaje de contacto	notifications	Ninguna.	Solo notifications (S35).
Credenciales entre sistemas y secretos	Vault	Cada sistema las lee al arrancar.	Solo el administrador de la infraestructura.

Regla de lectura: el sistema autoridad se consulta por un servicio o una integración (3.7), nunca por la base. Las copias se usan para mostrar, nunca para decidir. Por ejemplo, orders decide el precio de un pedido nuevo con el precio que devuelve inventory en I08, no con una copia.

3.5 Estado oficial del pedido y del cobro

El estado del pedido tiene un solo dueño (orders) y el del cobro otro (payments). Se relacionan por una sola regla: un pedido está `pagado` si y solo si payments confirmó un pago `aprobado` por I12.

Transición	Quién la ejecuta	Cómo	Quién la autoriza
(nuevo) → `pendiente_pago`	orders	S12	Cliente con token y rol `cliente`.
`pendiente_pago` → `pagado`	orders	Solo al recibir I12	La credencial `payments-service`, después de que orders compruebe que el pedido sigue `pendiente_pago`, no venció y el monto es igual al total. **No** S17, **ni** el frontend.
`pagado` → `en_preparacion` → `en_despacho` → `entregado`	orders	S17	`encargado_despachos` o `administrador`.
`pendiente_pago`, `pagado` o `en_preparacion` → `anulado`	orders	S15 (cliente, mientras no esté `en_preparacion`), S17 (administrador) o el vencimiento automático	Cliente dueño, administrador o el propio sistema. Al anular se libera el stock (I10) y, si estaba pagado, se publica una orden `anulada` (I14) y queda `reembolsoPendiente`.

Estado del cobro	Quién lo fija	Cuándo
`pendiente`	payments	S20 crea el cobro.
`aprobado`, `rechazado`, `expirado`	payments	Un aviso válido (S21). Solo `aprobado` lleva a I12.
`observado`	payments	Monto distinto, pago que no estaba pendiente o pedido que ya no se puede pagar. Queda para revisión y reembolso manual.

Conciliación: una tarea revisa cada 10 minutos los pagos `aprobado` con `confirmadoEnPedidos` en falso y repite I12. Si pasan más de 10 minutos, queda un registro de alerta para el administrador.

3.6 Servicios del Módulo 1 asignados a sistemas

ID	Servicio	Sistema	Acceso (Módulo 1)	Integraciones que dispara	Datos que toca
S01	Iniciar sesión	`auth-service`	A	—	Lee Cuenta; escribe Sesion
S02	Cerrar sesión e invalidar el token	`auth-service`	Au (cualquier rol)	—	Escribe Sesion
S03	Registrar un cliente desde la web	`clients-service`	A	I03 (y por auth, I06)	Escribe Cliente y Direccion; Cuenta por I03
S04	Confirmar el correo y, si corresponde, definir la contraseña	`auth-service`	A	—	Lee TokenVerificacion; escribe Cuenta
S05	Reenviar el correo de confirmación	`auth-service`	A	I06	Escribe TokenVerificacion
S06	Consultar el catálogo	`inventory-service`	A	—	Lee Producto
S07	Listar productos con stock y retirados	`inventory-service`	Au (administrador, dueno)	—	Lee Producto
S08	Crear un producto	`inventory-service`	Au (administrador, dueno)	—	Escribe Producto
S09	Reemplazar los datos de un producto	`inventory-service`	Au (administrador, dueno)	—	Escribe Producto

ID	Servicio	Sistema	Acceso (Módulo 1)	Integraciones que dispara	Datos que toca
S10	Retirar un producto (baja lógica, conserva el historial de pedidos)	`inventory-service`	Au (administrador, dueno)	—	Escribe Producto
S11	Actualizar stock y disponibilidad de un producto	`inventory-service`	Au (administrador, dueno)	—	Escribe Producto y MovimientoStock
S12	Crear un pedido	`orders-service`	Au (cliente)	I07, I08 (I10 si falla)	Escribe Pedido, DetallePedido, DireccionEntrega, CambioEstado
S13	Listar los pedidos propios	`orders-service`	Au (cliente)	—	Lee Pedido
S14	Consultar el detalle de un pedido	`orders-service`	Au (cliente dueño; administrador, cajero_virtual, encargado_despachos)	—	Lee Pedido y sus partes
S15	Anular un pedido propio	`orders-service`	Au (cliente dueño)	I10, I14 si estaba pagado	Escribe Pedido y CambioEstado
S16	Listar todos los pedidos para el personal	`orders-service`	Au (administrador, cajero_virtual, encargado_despachos)	—	Lee Pedido
S17	Cambiar el estado de un pedido	`orders-service`	Au (encargado_despachos, administrador)	I10 e I14 si anula	Escribe Pedido y CambioEstado
S18	Consultar la cola de órdenes de despacho en orden de llegada	`dispatch-service`	Au (encargado_despachos, administrador)	—	Lee OrdenDespacho
S19	Obtener la orden de despacho lista para imprimir	`dispatch-service`	Au (encargado_despachos, administrador)	—	Lee OrdenDespacho; escribe `impresa`
S20	Iniciar el pago de un pedido en la plataforma externa	`payments-service`	Au (cliente dueño)	I11, I16	Escribe Pago
S21	Recibir el resultado del pago	`payments-service`	Identidad de la plataforma (firma)	I12, I13	Escribe EventoPago, Pago, Venta, LineaVenta, Boleta
S22	Consultar las ventas de la caja virtual	`payments-service`	Au (cajero_virtual, administrador)	—	Lee Venta y Boleta
S23	Obtener la boleta digital de un pedido pagado	`payments-service`	Au (cliente dueño; cajero_virtual, administrador)	—	Lee Boleta, Venta y LineaVenta
S24	Consultar mi perfil	`clients-service`	Au (cliente)	—	Lee Cliente y Direccion
S25	Reemplazar mis datos de perfil	`clients-service`	Au (cliente)	—	Escribe Cliente y Direccion
S26	Registrar un cliente presencialmente	`clients-service`	Au (administrador)	I03 (y por auth, I06)	Escribe Cliente y Direccion; Cuenta por I03
S27	Listar clientes	`clients-service`	Au (administrador)	I04	Lee Cliente y Direccion; estado por I04
S28	Reemplazar los datos de un cliente	`clients-service`	Au (administrador)	I04	Escribe Cliente y Direccion
S29	Dar de baja a un cliente (baja lógica)	`clients-service`	Au (administrador)	I05	Cambia el estado de la Cuenta por I05
S30	Crear un usuario del personal	`auth-service`	Au (administrador)	I06	Escribe Usuario y Cuenta
S31	Listar usuarios del personal	`auth-service`	Au (administrador)	—	Lee Usuario y Cuenta
S32	Cambiar nombre, rol o estado de un usuario	`auth-service`	Au (administrador)	—	Escribe Usuario y Cuenta
S33	Dar de baja a un usuario (baja lógica)	`auth-service`	Au (administrador)	—	Escribe Cuenta y Sesion
S34	Reporte de ventas por período	`payments-service`	Au (dueno, administrador)	—	Lee Venta y LineaVenta
S35	Registrar un mensaje de contacto	`notifications-service`	A	—	Escribe MensajeContacto

Operaciones internas del Módulo 1: "Realizar venta" y "Enviar boleta" pertenecen a payments (I12 e I13). "Publicar orden de despacho" y "Expirar pedidos" pertenecen a orders (I14 e I10). "Validar y enviar correos" pertenece a auth (I06).

3.7 Catálogo de intercambios

Las rutas `/internal/...` no las publica el gateway: solo existen dentro de la red privada. Todas las respuestas de error usan el formato del Módulo 1.

ID	Origen → destino	Operación o evento	Tipo	Datos mínimos	Autenticación	Si falla
I01	`api-gateway` → `auth-service`	`POST /internal/introspect`	Síncrono	Entrada: `token`. Salida: `active`, `cuentaId`, `correo`, `roles`	`X-Service-Id: api-gateway` y `X-Service-Secret` (par propio)	Auth no responde: el gateway devuelve 503 y no deja pasar la solicitud (falla cerrado). Token inactivo: 401.
I02	`api-gateway` → todos los sistemas de negocio	Reenvío de `/api/v1/...` a la ruta interna del sistema	Síncrono	La solicitud original más `X-Authenticated-User` (cuentaId), `X-Authenticated-Email`, `X-Authenticated-Roles` y `X-Request-Id`	`X-Gateway-Secret`: un secreto distinto por sistema destino (`gateway-to-<sistema>`), guardado en Vault	Sistema sin respuesta en 10 s: 502 o 503 con el formato de error del Módulo 1. No reintenta POST, PUT, PATCH ni DELETE para no duplicar efectos.
I03	`clients-service` → `auth-service`	`POST /internal/accounts` (crear cuenta `pendiente_verificacion`). Compensación: `DELETE /internal/accounts/{cuentaId}` solo si sigue pendiente	Síncrono	`correo`, `rol` (siempre `cliente`), `password` (solo en S03)	`X-Service-Id: clients-service` + `X-Service-Secret`	409 correo duplicado, 422 correo inexistente, 503 sin respuesta: S03 y S26 devuelven el mismo código y no crean perfil. Si la cuenta se creó y el perfil no se pudo guardar, clients llama a la compensación.
I04	`clients-service` → `auth-service`	`GET /internal/accounts?ids=a,b,c`	Síncrono	Entrada: lista de `cuentaId`. Salida: `cuentaId`, `correo`, `estado`	`X-Service-Id: clients-service` + `X-Service-Secret`	Sin respuesta: S27 y S28 devuelven 503. No se muestran clientes con `estado` inventado.
I05	`clients-service` → `auth-service`	`PATCH /internal/accounts/{cuentaId}` con `estado: inactivo`	Síncrono	`cuentaId`, `estado`. Auth invalida las sesiones de esa cuenta	`X-Service-Id: clients-service` + `X-Service-Secret`	Sin respuesta: S29 devuelve 503 y no queda nada a medias, porque la baja es solo el cambio de estado de la cuenta. Repetir la llamada no cambia el resultado.
I06	`auth-service` → `notifications-service`	`POST /internal/emails` con plantilla `verificacion`	Síncrono	`correo`, `enlace` (con el token de un solo uso)	`X-Service-Id: auth-service` + `X-Service-Secret`	422 si el proveedor dice que el correo no existe. 503 si el proveedor no responde: auth no crea la cuenta ni cambia el token (S03, S05, S26, S30).
I07	`orders-service` → `clients-service`	`GET /internal/clients/by-account/{cuentaId}`	Síncrono	Salida: `clienteId`, `nombreCompleto`, `telefono`, `direccion` (calle, comuna, provincia, región)	`X-Service-Id: orders-service` + `X-Service-Secret`	404 sin perfil: S12 responde 403. Sin respuesta en 3 s: S12 responde 503 y no reserva stock, porque esta llamada se hace antes que I08.
I08	`orders-service` → `inventory-service`	`POST /internal/stock/reservations`	Síncrono	Entrada: `pedidoId`, `expiraEn`, `items[productoId, cantidad]`. Salida: `items[productoId, nombre, precioUnitario, cantidad]`	`X-Service-Id: orders-service` + `X-Service-Secret`	409 `SIN_STOCK` con los productos que faltan. Sin respuesta en 3 s: un reintento con el mismo `pedidoId`; luego 503 y liberación (I10). Detalle en 3.9.
I09	`orders-service` → `inventory-service`	`POST /internal/stock/reservations/{pedidoId}/confirm`	Síncrono (con reintento)	`pedidoId`	`X-Service-Id: orders-service` + `X-Service-Secret`	Si falla, `Pedido.reservaConfirmada` queda en falso y una tarea cada 30 s repite la llamada. La reserva tiene 5 minutos de margen sobre el vencimiento del pedido.
I10	`orders-service` → `inventory-service`	`POST /internal/stock/reservations/{pedidoId}/release`	Síncrono (mejor esfuerzo)	`pedidoId`	`X-Service-Id: orders-service` + `X-Service-Secret`	Si falla, la reserva sin confirmar vence sola. Para un pedido ya confirmado que se anula, orders repite la llamada hasta que responda.

ID	Origen → destino	Operación o evento	Tipo	Datos mínimos	Autenticación	Si falla
I11	payments-service → orders-service	GET /internal/orders/{id}/payable?cuentaId=...	Síncrono	Salida: pedidoId, total, estado, expiraEn	X-Service-Id: payments-service + X-Service-Secret	404 si el pedido no es del usuario, 409 si no está pendiente_pago o venció. Sin respuesta en 5 s: S20 responde 503 y no crea el pago.
I12	payments-service → orders-service	POST /internal/orders/{id}/payment-confirmation	Síncrono (con reintento)	Entrada: pagoId, monto. Salida: pedidoId, total, cuentaId, clienteNombre, clienteCorreo, items[productoId, nombre, cantidad, precioUnitario], pagadoEn	X-Service-Id: payments-service + X-Service-Secret. Es la única credencial que orders acepta para pasar a pagado	409 PEDIDO_NO_PAGABLE (anulado, vencido o monto distinto): el pago queda observado. Sin respuesta: el pago queda aprobado con confirmadoEnPedidos en falso y una tarea cada 30 s repite la llamada. Orders la trata como idempotente por pagoId.
I13	payments-service → notifications-service	POST /internal/emails con plantilla boleta	Síncrono (con reintento)	correo, numeroBoleta, total, items	X-Service-Id: payments-service + X-Service-Secret	Si falla, la venta y la boleta ya existen: Boleta.correoEnviado queda en falso y una tarea cada 5 minutos reintenta hasta 5 veces. Se ve en S22.
I14	orders-service → dispatch-service	Mensaje en la cola ActiveMQ /queue/cola_pedidos	Asíncrono (STOMP)	tipo (nueva o anulada), pedidoId, pagadoEn, clienteNombre, telefono, direccionEntrega, items[nombre, cantidad]. Sin precios, RUN ni correo	Usuario de ActiveMQ solo de escritura para orders y solo de lectura para dispatch (credenciales en Vault)	Broker caído: el pedido queda pagado con despachoPublicado en falso y una tarea cada 30 s publica. Dispatch ignora un mensaje repetido (mismo pedidoId y tipo) y manda al registro de errores el mensaje con formato inválido.
I15	Plataforma de pago (externa) → payments-service (por el gateway)	POST /api/v1/payments/webhook (S21)	Síncrono	eventoId, pagoId, estado, monto; cabeceras X-Timestamp y X-Signature	Firma HMAC-SHA256 con el secreto del webhook (Vault). La ruta no lleva token de usuario	401 firma inválida. 400 cuerpo inválido. Todo lo demás responde 200 para que la plataforma no reintente sin necesidad. Reglas en 3.10.
I16	payments-service → Plataforma de pago (externa)	Crear cobro (adaptador de prueba propio)	Síncrono	pagoId, monto, medio. Salida: urlPago	Credencial de la plataforma (Vault)	Sin respuesta: S20 responde 502 o 503 y no guarda un pago pendiente.
I17	notifications-service → Proveedor de correo (externo)	Validar un correo y enviar un mensaje	Síncrono	correo, asunto y cuerpo de la plantilla	Clave del proveedor (Vault)	El proveedor no responde: 503 a quien llamó (I06 o I13). S35 no usa esta llamada.
I18	todos los sistemas → Vault	GET /v1/secret/data/<sistema>	Síncrono, al arrancar	Credenciales de su base de datos, secretos de integración y de ActiveMQ	Token de Vault de cada sistema, con política de solo lectura sobre su ruta	Si Vault no responde o falta un secreto, el sistema no arranca. No hay valores por defecto en el código.

Orden de dependencias entre sistemas de negocio: dispatch-service, inventory-service, notifications-service, auth-service, clients-service, orders-service, payments-service. Cada sistema solo llama a sistemas que aparecen antes en esta lista, así que no hay ciclos. Auth no llama a clients, y payments llama a orders pero orders nunca llama a payments.

3.8 Identidad entre componentes

Camino	Qué se envía	Qué verifica el que recibe
Navegador → gateway	Authorization: Bearer <token>	El gateway pregunta a auth (I01). Sin token válido, 401.
Gateway → sistema	X-Gateway-Secret (uno por sistema destino), X-Authenticated-User (cuentaId), X-Authenticated-Email, X-Authenticated-Roles, X-Request-Id	El sistema compara X-Gateway-Secret en tiempo constante, con el secreto que le corresponde solo a él, y confía en las demás cabeceras solo si coincide. El gateway **borra** cualquier cabecera X-Authenticated-* que traiga el navegador.

Camino	Qué se envía	Qué verifica el que recibe
Sistema → sistema	`X-Service-Id`, `X-Service-Secret` y, si actúa en nombre de un usuario, `X-Authenticated-User`	El receptor revisa que el par exista en su lista de llamadores y que esa ruta esté permitida para ese llamador. Por ejemplo, solo `payments-service` puede llamar a `payment-confirmation`.
Plataforma de pago → payments	Firma HMAC-SHA256 y marca de tiempo	Payments recalcula la firma con el secreto de Vault y rechaza marcas con más de 5 minutos.
Orders y dispatch ↔ ActiveMQ	Usuario y contraseña de STOMP	El broker permite escribir solo a orders y leer solo a dispatch.

Reglas comunes:

- Cada par llamador–llamado tiene su propio secreto, guardado en Vault bajo `secret/data/services/<llamador>-to-<llamado>`. Si se filtra uno, solo se rota ese.
- El servicio que recibe nunca usa la identidad de un usuario para decidir si otro sistema puede llamarlo. Son dos comprobaciones distintas: quién es el sistema que llama y qué usuario lo originó.
- La red privada de Docker no publica ningún puerto salvo el del gateway. Las bases de datos, Vault y ActiveMQ solo se ven desde dentro de esa red.
- Se sigue el patrón del laboratorio (`X-Gateway-Secret`, `/introspect`, secretos en Vault) y se corrige lo que el laboratorio dejaba débil: no hay secretos por defecto en el código (ver 1.13).

3.9 Pedidos ↔ inventario: disponibilidad, reserva y respuesta que no llega

Pregunta	Regla
¿Quién decide si hay stock?	Inventory. Orders nunca lee ni cambia el stock. Tampoco confía en el precio que el navegador mande: toma nombre y precio de la respuesta de I08.
¿Cuándo se reserva?	En S12, después de validar el cliente (I07) y la cobertura de 3 km, y antes de guardar el pedido.
¿Es todo o nada?	Sí. Para cada producto, inventory descuenta con una actualización condicional (`stock >= cantidad`, producto disponible y no retirado). Si alguno falla, devuelve lo que ya descontó y responde 409 con los productos que faltan. Los movimientos de tipo `reserva` quedan registrados en MovimientoStock.
¿Qué pasa si dos clientes compran la última unidad?	La actualización condicional es atómica por producto. Gana quien llega primero. El otro recibe 409.
¿Se puede repetir la misma solicitud?	Sí. La reserva se identifica por `pedidoId`. Si llega otra vez con el mismo `pedidoId`, inventory devuelve la respuesta anterior sin descontar de nuevo.
¿Qué pasa si la respuesta no llega?	Orders espera 3 s y repite **una vez** con el mismo `pedidoId` (seguro, por lo anterior). Si sigue sin respuesta, intenta liberar (I10), responde 503 `INVENTARIO_NO_DISPONIBLE` y **no crea el pedido**. El cliente reintenta S12 con la misma `Idempotency-Key`.
¿Y si la reserva sí se hizo pero orders no lo supo?	Cada reserva lleva `expiraEn` (vencimiento del pedido más 5 minutos). Inventory libera las reservas vencidas sin confirmar con una tarea cada minuto. El stock nunca queda retenido más de 20 minutos.
¿Cuándo se confirma?	Cuando orders pasa el pedido a `pagado` (I09). Una reserva confirmada ya no vence.
¿Cuándo se libera?	Al anular el pedido (S15 o S17) o al vencer sin pago (I10). Se devuelve el stock y queda un movimiento `liberacion`.
¿Y el ajuste manual de stock (S11)?	Se registra como movimiento `ajuste` sobre el stock disponible. No toca las reservas existentes.
Caso residual	Si la confirmación (I09) llegara después de que la reserva venció y el stock ya se vendió, inventory responde 409. Orders deja el pedido marcado para revisión en S16 y el administrador lo resuelve a mano. El margen de 5 minutos y el reintento cada 30 s lo vuelven muy poco probable.

3.10 Aviso de pago: verificación, duplicados y autoridad del cambio de estado

El navegador no decide que un pedido está pagado. Al volver de la plataforma de pago solo consulta S14 y muestra el estado que dice orders. El único camino a `pagado` es: aviso firmado → payments → I12 → orders.

Paso	Verificación en payments	Si no se cumple
1	Firma HMAC-SHA256 sobre la marca de tiempo y el cuerpo, con el secreto de Vault. Marca de tiempo de máximo 5 minutos.	401. No se guarda nada.
2	Cuerpo válido (`eventoId`, `pagoId`, `estado`, `monto`) y `pagoId` existente.	400.
3	`eventoId` no registrado antes. Hay un índice único sobre `eventoId`.	Si ya existe: 200 `{recibido: true}` y no se procesa otra vez.
4	Se registra el EventoPago.	—
5	El pago estaba `pendiente` y el `monto` informado es igual al `monto` guardado al iniciar el pago (S20), que viene del total del pedido.	El pago pasa a `observado`, el pedido no cambia y se responde 200.

Paso	Verificación en payments	Si no se cumple
6	`estado` informado: `rechazado` o `expirado` actualizan el pago y el pedido sigue `pendiente_pago`. `aprobado` pasa el pago a `aprobado` y llama a I12.	—
7	Orders comprueba que el pedido está `pendiente_pago`, no venció, el `monto` es igual al total y el `pagoId` no se usó antes. Solo entonces lo pasa a `pagado`, registra el cambio con actor "sistema", confirma la reserva (I09) y publica la orden de despacho (I14).	409 `PEDIDO_NO_PAGABLE`: el pago queda `observado` y se reembolsa a mano.
8	Con la respuesta de orders, payments crea la Venta con sus líneas en la caja virtual, genera el número de boleta y la Boleta, y llama a I13.	Si falla el correo, la venta ya existe y el envío se reintenta.

Por qué no hay duplicados: el índice único en `eventoId` evita procesar dos veces el mismo aviso, el índice único en `Venta.pedidoId` evita dos ventas por pedido, y orders trata I12 como idempotente por `pagoId` (si el pedido ya está `pagado` con ese mismo pago, devuelve la misma respuesta). Si payments se cae entre I12 y la creación de la venta, el reintento repite I12, recibe la misma respuesta y completa la venta.

Quién autoriza el cambio de estado: orders, que es el dueño del estado del pedido. Payments solo lo solicita con la credencial `payments-service`, y orders vuelve a comprobar las reglas con sus propios datos. Si el aviso llega con el monto alterado o por otro camino, no hay forma de llegar a `pagado`.

3.11 Fallos y consistencia del negocio

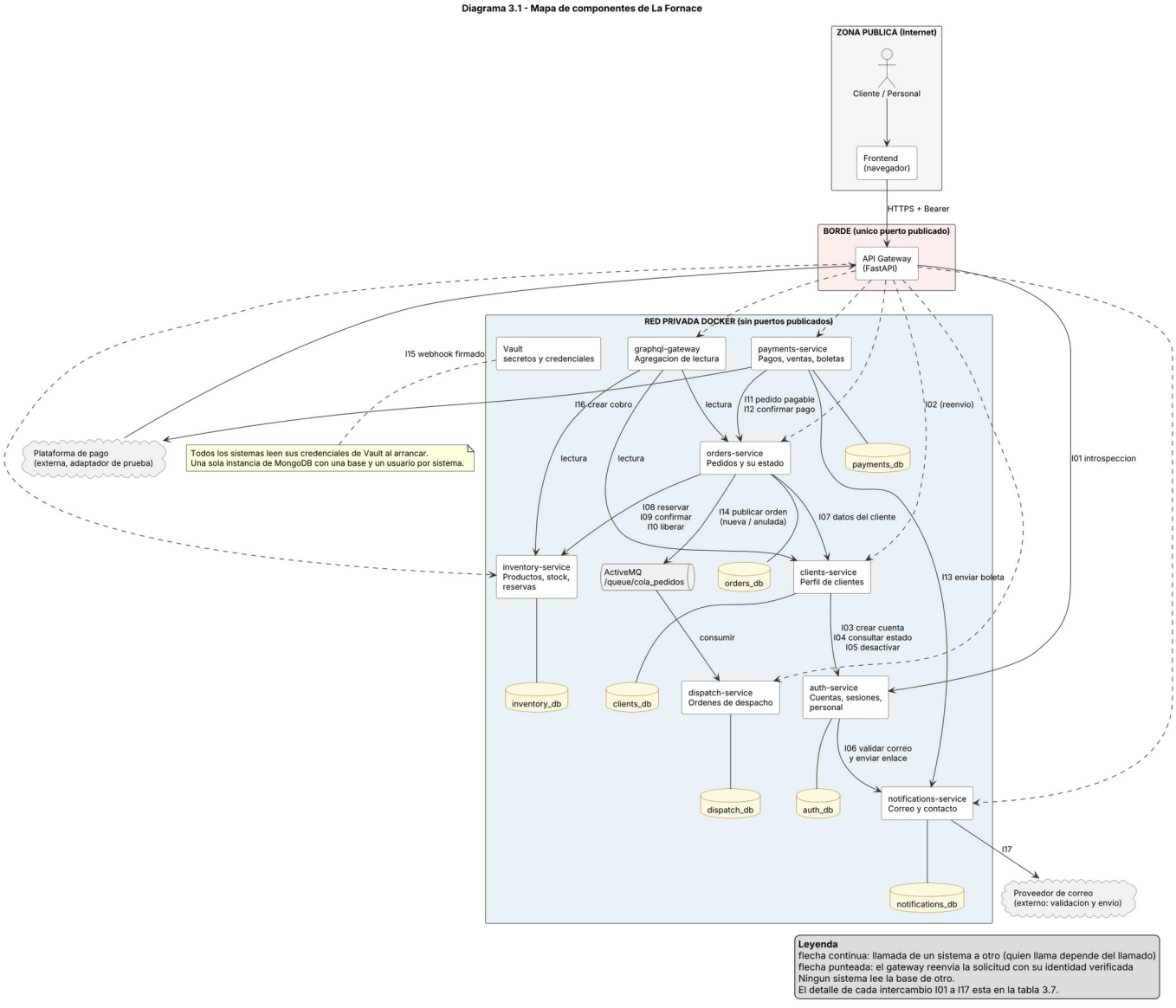
Situación	Qué ocurre	Estado final
Falta stock en S12	Inventory revierte y responde 409.	Sin pedido y sin stock retenido.
Inventory no responde en S12	Un reintento, luego liberación y 503.	Sin pedido. Si quedó una reserva, vence sola.
Orders no puede guardar el pedido después de reservar	Orders libera (I10) y responde 500.	Sin pedido y sin stock retenido.
El cliente hace doble clic en "Confirmar"	La misma `Idempotency-Key` devuelve el pedido ya creado.	Un solo pedido y una sola reserva.
Aviso de pago repetido	Se responde 200 sin procesar.	Un solo pago, una sola venta.
Aviso con firma inválida o antigua	401.	Nada cambia.
Aviso con monto distinto	Pago `observado`.	Pedido `pendiente_pago`. Revisión manual.
Aviso aprobado y orders no responde	Pago `aprobado` con `confirmadoEnPedidos` en falso. Se repite I12 cada 30 s.	Al volver orders, el pedido pasa a `pagado` y se completa la venta.
Aviso aprobado de un pedido anulado o vencido	Orders responde 409.	Pago `observado` y reembolso manual.
Pedido sin pago a los 15 minutos	Orders lo anula (motivo "vencido") y libera el stock.	`anulado`, stock devuelto.
Cliente anula un pedido ya pagado (S15)	Se anula, se libera el stock, se publica una orden `anulada` y queda `reembolsoPendiente`.	`anulado`. La venta y la boleta se conservan. El reporte S34 muestra lo recaudado y no descuenta reembolsos.
ActiveMQ caído al pagar	El pedido pasa a `pagado` igual. `despachoPublicado` queda en falso y se publica después.	La orden llega a cocina con retraso, no se pierde.
Proveedor de correo caído al enviar la boleta	La venta y la boleta existen. `correoEnviado` en falso. Reintento cada 5 minutos, hasta 5 veces.	Visible en S22.
Proveedor de correo caído en el registro (S03)	Auth responde 503 a clients y clients responde 503 sin crear perfil.	Sin cuenta y sin perfil.
Falla el guardado del perfil después de crear la cuenta	Clients borra la cuenta pendiente (compensación de I03).	Sin cuenta y sin perfil.
Auth caído	Las rutas con token responden 503. El catálogo (S06) y el aviso de pago (S21) siguen, porque no usan token.	No se acepta ninguna solicitud sin verificar.
Baja de un cliente (S29)	Solo cambia el estado de la cuenta a `inactivo` y se invalidan sus sesiones. Los pedidos en curso siguen su flujo con los datos que ya guardan.	Pedidos y ventas intactos.

3.12 Sistemas complementarios: propios y externos

Sistema	Propio o externo	Qué hace	Cómo se integra
GraphQL Gateway (`graphql-gateway`)	Propio, privado. Se detalla en los Módulos 5 y 8	Agrega consultas de solo lectura que combinan orders, inventory y clients para una pantalla. No tiene datos propios ni hace escrituras.	`POST /api/v1/graphql` por el API Gateway. Llama a los sistemas con su propio par de credenciales y el contexto verificado del usuario.
Despacho (`dispatch-service`)	Propio	Recibe las órdenes pagadas, las ordena por llegada y las entrega para imprimir. Se separa porque su único dato es una copia sin precios y se alimenta por cola.	Cola ActiveMQ `cola_pedidos` (I14).
Boletas	Propio, dentro de `payments-service`	Genera la boleta digital de cada venta con IVA incluido. Se simplifica: es un documento interno y no se envía a una entidad tributaria.	Se genera en el mismo paso que la venta (3.10, paso 8).

Sistema	Propio o externo	Qué hace	Cómo se integra
Mensajería (`notifications-service`)	Propio, sobre un proveedor externo	Valida correos, envía los enlaces y las boletas, y guarda los mensajes de contacto.	I06 e I13 hacia notifications. I17 hacia el proveedor.
Proveedor de correo	Externo	Valida que un correo exista y lo envía.	I17, con clave en Vault. Se puede reemplazar sin cambiar a los demás sistemas.
Plataforma de pago	Externa. En la entrega se usa un adaptador de prueba propio que imita a Servipag, porque Servipag no tiene un entorno de prueba público (ver 10.11)	Cobra y avisa el resultado.	I16 y I15.
ActiveMQ	Infraestructura propia	Transporta las órdenes de despacho.	STOMP, con usuarios de solo lectura o solo escritura.
Vault	Infraestructura propia	Guarda los secretos y las credenciales entre sistemas.	I18, al arrancar cada sistema.
MongoDB	Infraestructura propia	Una instancia, una base y un usuario por sistema.	Cada sistema solo ve su base.

3.13 Diagrama 3.1 — Mapa de componentes

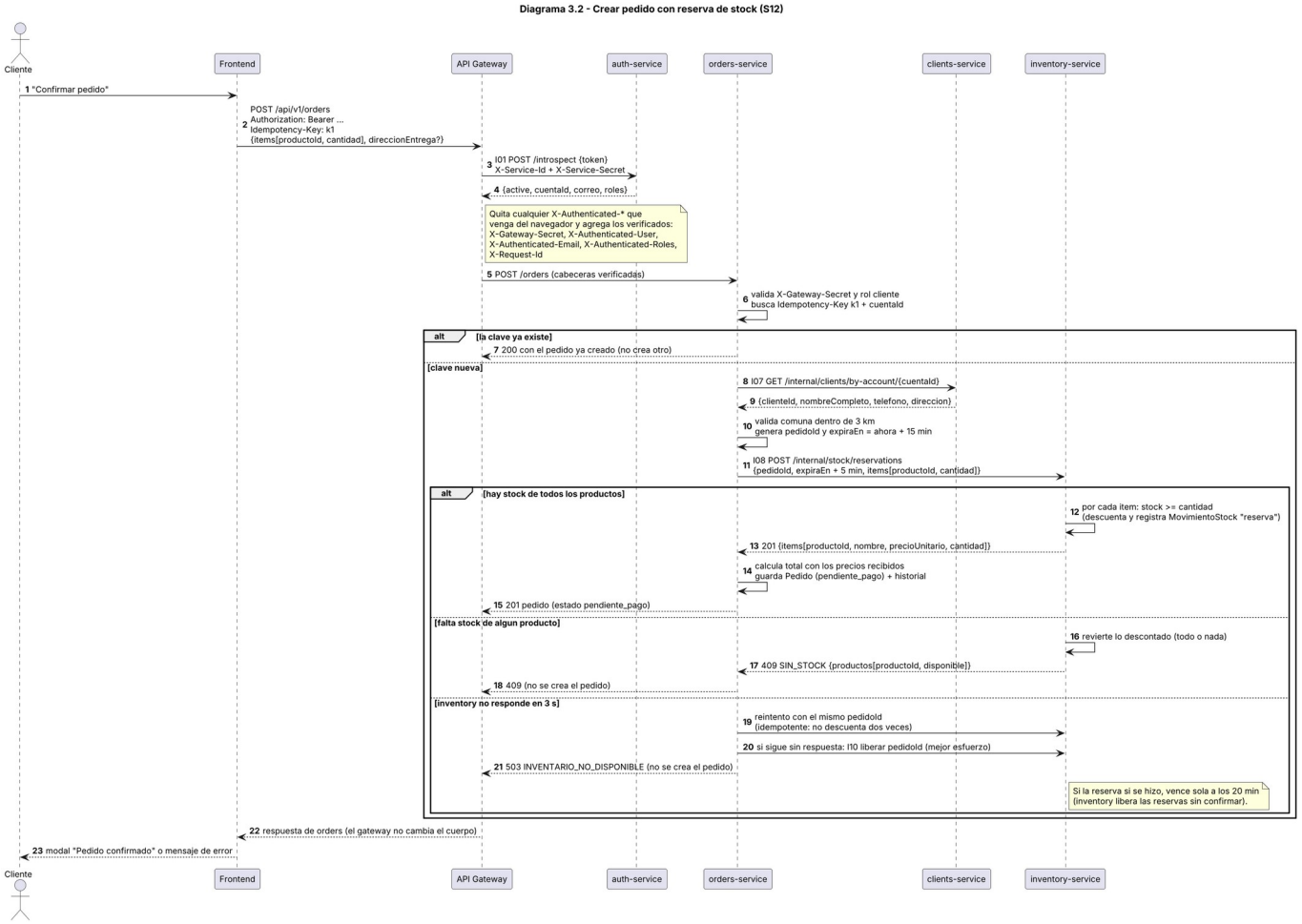


Fuente editable: `diagramas/diagrama-3-1-componentes.puml`. Versión vectorial: `diagramas/diagrama-3-1-componentes.svg`.

3.14 Diagramas de secuencia

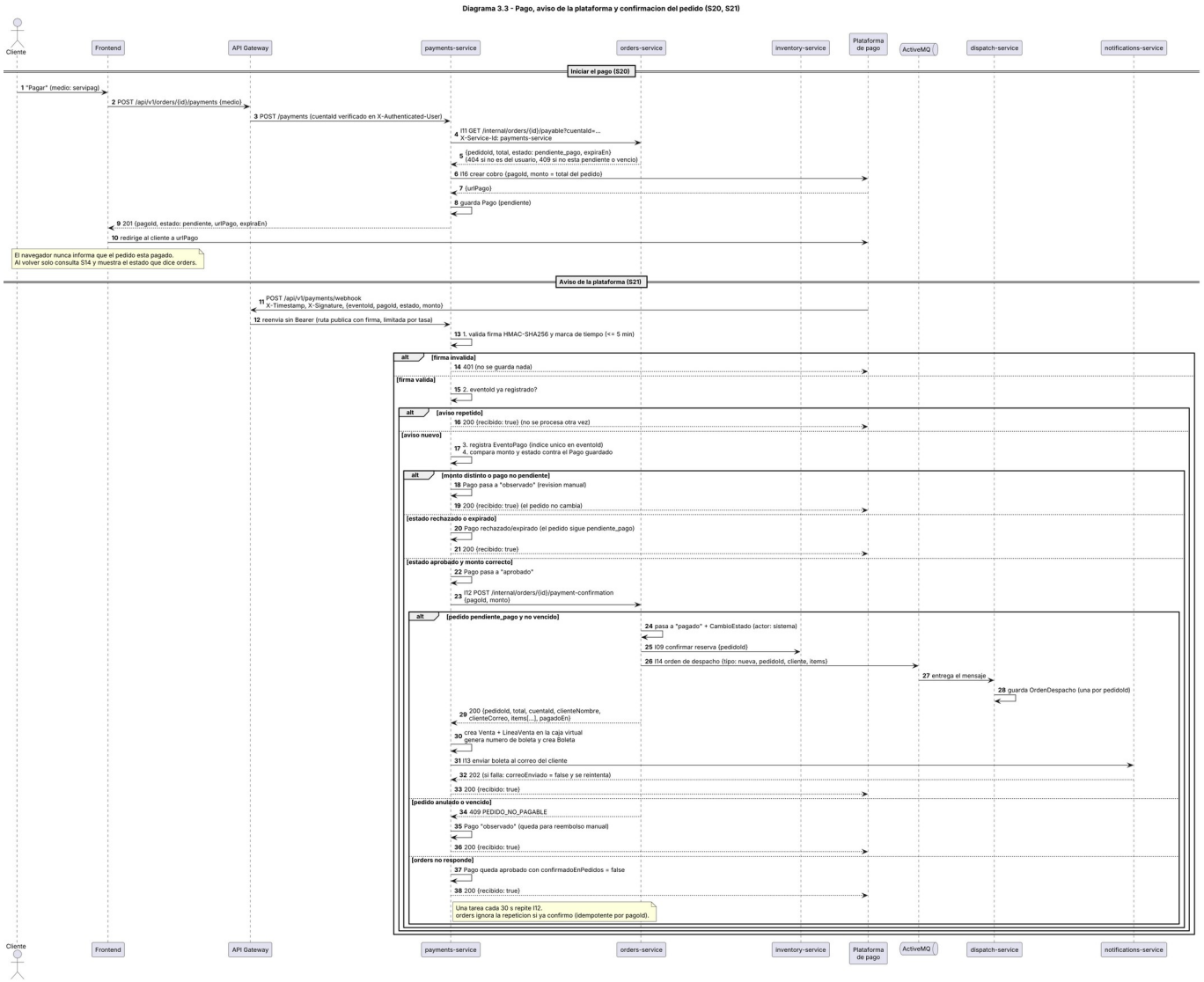
Se muestran los dos tramos del flujo de compra: crear el pedido con reserva de stock, y pagarlo con el aviso de la plataforma. Los dos incluyen las alternativas de fallo de 3.9 y 3.10.

Diagrama 3.2 — Crear pedido con reserva de stock (S12)



Fuente editable: `diagramas/diagrama-3-2-secuencia-pedido.puml`. Versión vectorial: `diagramas/diagrama-3-2-secuencia-pedido.svg`.

Diagrama 3.3 — Pago, aviso y confirmación (S20, S21)



Fuente editable: `diagramas/diagrama-3-3-secuencia-pago.puml`. Versión vectorial: `diagramas/diagrama-3-3-secuencia-pago.svg`.

3.15 Contratos de los intercambios principales (datos ficticios)

I08 Reservar stock (orders → inventory)

```
// POST /internal/stock/reservations
// X-Service-Id: orders-service   X-Service-Secret: <desde Vault>
{ "pedidoId": "ped_0001", "expiraEn": "2026-11-07T18:50:00Z",
  "items": [ { "productoId": "prod_001", "cantidad": 2 }, { "productoId": "prod_007", "cantidad": 1 } ] }
// 201
{ "pedidoId": "ped_0001",
  "items": [ { "productoId": "prod_001", "nombre": "Napolitana", "precioUnitario": 8990, "cantidad": 2 },
             { "productoId": "prod_007", "nombre": "Bebida 500ml", "precioUnitario": 1990, "cantidad": 1 }
] }
// 409
{ "error": { "codigo": "SIN_STOCK", "mensaje": "La Napolitana no tiene stock suficiente.",
             "productos": [ { "productoId": "prod_001", "disponible": 1 } ] } }
```

I12 Confirmar el pago (payments → orders)

```
// POST /internal/orders/ped_0001/payment-confirmation
// X-Service-Id: payments-service   X-Service-Secret: <desde Vault>
{ "pagoId": "pag_0001", "monto": 19970 }
// 200
{ "pedidoId": "ped_0001", "total": 19970, "cuentaId": "cta_0007", "clienteNombre": "Camila Rojas",
  "clienteCorreo": "camila@ejemplo.cl", "pagadoEn": "2026-11-07T18:34:12Z",
  "items": [ { "productoId": "prod_001", "nombre": "Napolitana", "cantidad": 2, "precioUnitario": 8990 },
             { "productoId": "prod_007", "nombre": "Bebida 500ml", "cantidad": 1, "precioUnitario": 1990 }
] }
```

```
// 409
{ "error": { "codigo": "PEDIDO_NO_PAGABLE", "mensaje": "El pedido está anulado o vencido." } }
```

I15 Aviso de la plataforma de pago (S21)

```
// POST /api/v1/payments/webhook
// X-Timestamp: 1794161261   X-Signature: sha256=<HMAC del timestamp y el cuerpo>
{ "eventoId": "evt_ficticio_001", "pagoId": "pag_0001", "estado": "aprobado", "monto": 19970 }
// 200
{ "recibido": true }
```

I14 Orden de despacho (mensaje en `/queue/cola_pedidos`)

```
{ "tipo": "nueva", "pedidoId": "ped_0001", "pagadoEn": "2026-11-07T18:34:12Z",
  "clienteNombre": "Camila Rojas", "telefono": "+56911112222",
  "direccionEntrega": "Av. Ejemplo 1234, Maipú (Casa azul)",
  "items": [ { "nombre": "Napolitana", "cantidad": 2 }, { "nombre": "Bebida 500ml", "cantidad": 1 } ] }
```

3.16 Control antes de avanzar

Pregunta	Respuesta
¿Hay al menos cuatro sistemas con funciones reales?	Sí. Hay siete sistemas de negocio privados (auth, clients, inventory, orders, payments, dispatch, notifications), cada uno con su base y sus servicios (3.2 y 3.3), más el gateway.
¿Cada dato tiene una autoridad?	Sí. La tabla 3.4 asigna un sistema a cada dato y explica cada copia.
¿Los 35 servicios del Módulo 1 están asignados?	Sí. La tabla 3.6 tiene los 35 IDs, cada uno en un solo sistema.
¿Hay ciclos de dependencia?	No. El orden de 3.7 lo muestra y el diagrama 3.1 no tiene flechas de vuelta.
¿Se consideraron los fallos y la consistencia del negocio?	Sí. 3.9, 3.10 y 3.11 cubren stock, respuesta perdida, aviso repetido, aviso alterado, sistemas caídos y compensaciones.
¿Quién autoriza que un pedido pase a pagado?	Solo orders, a petición de payments con su credencial, y nunca el frontend (3.5 y 3.10).

Módulo 4 — Stakeholders y permisos HTTP

Este módulo define quién necesita cada servicio, para qué lo usa y sobre qué recursos puede actuar. Usa los IDs S01 a S35 del Módulo 1 y los sistemas del Módulo 3. El detalle de cómo el gateway aplica estas reglas se diseña en el Módulo 5.

4.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Identificar stakeholders y su interés; separar partes interesadas, roles de usuario e identidades técnicas.	Registro 4.2 (columna "Clasificación").
2. Relacionar cada rol operativo con servicios concretos del Módulo 1.	Tabla 4.4 y matriz resumen 4.7.
3. Indicar GET, POST, PUT y DELETE como permitido, denegado o no aplicable. Un verbo permitido no autoriza todas las rutas.	Tabla 4.9 (por recurso y verbo) y matriz detallada 4.8.
4. Agregar condiciones (propiedad, ámbito, estado) y definir autenticación y responsable de verificar la autorización.	Condiciones 4.6, responsables 4.5 y columna "Se verifica en" de 4.8.
5. Registrar los stakeholders sin acceso directo como "sin permisos operativos" y explicar cómo reciben su información.	Tabla 4.3.
Fundamentar los permisos de administración y las identidades de proveedores o sistemas.	Tablas 4.4 y 4.10.
Conservar el historial en las anulaciones.	4.11.

4.2 Registro de stakeholders

Se separan tres clases: **partes interesadas** (personas o grupos con interés en el sistema, tengan o no acceso), **roles de usuario** (los que tienen cuenta e invocan servicios) e **identidades técnicas** (sistemas que invocan servicios por sí mismos).

Nombre o categoría	Clasificación	Interés	Rol asociado	Uso de información
Visitante	Parte interesada sin sesión	Conocer la carta, contactar al local y registrarse.	Ninguno (anónimo)	Solo datos públicos del catálogo (S06). Entrega su nombre, correo, teléfono y mensaje al contactar (S35) y sus datos al registrarse (S03).
Cliente	Rol de usuario	Pedir, pagar, anular y recibir su boleta.	`cliente`	Lee y cambia solo sus datos y pedidos. Su RUN, dirección, correo e historial son restringidos.

Nombre o categoría	Clasificación	Interés	Rol asociado	Uso de información
Dueño del restaurante	Rol de usuario	Saber cuánto se vende y mantener la carta.	`dueno`	Cifras agregadas de ventas (S34) y administración del catálogo. No ve datos de clientes individuales ni pedidos.
Administrador	Rol de usuario	Administrar usuarios, clientes y productos, y atender pedidos.	`administrador`	Datos restringidos de clientes y pedidos para operar. No ve contraseñas ni hashes.
Cajero virtual	Rol de usuario	Controlar las ventas de la caja web y sus boletas.	`cajero_virtual`	Ventas, boletas y pedidos (lectura).
Encargado de despachos	Rol de usuario	Pasar las órdenes a cocina en orden de llegada y registrar el avance.	`encargado_despachos`	Cola de órdenes de despacho (sin precios) y estado del pedido.
Cocina	Parte interesada sin acceso	Preparar los pedidos en el orden en que llegan (RF13).	Sin rol. **Sin permisos operativos**	Recibe la orden impresa.
Repartidor	Parte interesada sin acceso (derivada del despacho a domicilio, RF1)	Saber qué entregar y a qué dirección.	Sin rol. **Sin permisos operativos**	Recibe la orden impresa con nombre, teléfono y dirección.
Contabilidad del local	Parte interesada sin acceso (derivada del reporte, RF9)	Revisar lo vendido en un período.	Sin rol. **Sin permisos operativos**	Recibe el reporte que el dueño o el administrador generan e imprimen.
Responsable de la infraestructura (el equipo)	Parte interesada sin acceso a la API	Arrancar los sistemas y custodiar los secretos.	Sin rol en la API. **Sin permisos operativos**	Administra Vault y los contenedores fuera de la API. No lee datos de negocio por la API.
Plataforma de pago	Identidad técnica externa	Informar el resultado de un cobro.	`plataforma-de-pago` (firma HMAC)	Recibe `pagoId` y monto al crear el cobro. Envía `eventoId`, `pagoId`, `estado` y `monto`.
Proveedor de correo	Identidad técnica externa	Validar correos y enviarlos.	Clave de API	Recibe correo de destino y el contenido del mensaje.
`api-gateway`, `graphql-gateway` y los siete sistemas de negocio	Identidades técnicas internas	Cumplir las integraciones del Módulo 3.	Par `X-Service-Id` y `X-Service-Secret` por cada camino	Reciben solo los datos mínimos de cada integración (3.7).

4.3 Stakeholders sin permisos operativos

Stakeholder	Cómo recibe la información que necesita	Por qué no tiene acceso directo
Cocina	La orden impresa que entrega el encargado de despachos (S19). Trae número de orden, productos, cantidades, nombre, teléfono y dirección. No lleva precios, RUN ni correo.	Solo cumple el pedido. Cada acceso extra sería un dato restringido más expuesto.
Repartidor	La misma orden impresa.	Necesita la dirección del momento y no un acceso al sistema.
Contabilidad del local	El reporte de ventas del período (S34) que genera el dueño o el administrador.	Necesita cifras, no datos de clientes. Las boletas individuales llegan por correo al cliente.
Responsable de la infraestructura	Registros de los contenedores y estado de salud de cada sistema. Los secretos los gestiona en Vault.	Las operaciones de administración de la plataforma se hacen sobre la infraestructura y no con una cuenta de negocio.

4.4 Roles operativos y servicios del Módulo 1

Rol	Servicios	Detalle de los servicios	Fundamento de sus permisos
Visitante (sin rol)	6	S01, S03, S04, S05, S06, S35	Solo necesita ver la carta, contactar y registrarse. No toca datos restringidos.
`cliente`	15	S01, S02, S03, S04, S05, S06, S12, S13, S14, S15, S20, S23, S24, S25, S35	Opera únicamente sobre lo suyo: crea y paga sus pedidos, los anula mientras no estén en preparación, ve su boleta y mantiene su perfil.
`administrador`	28	S01, S02, S03, S04, S05, S06, S07, S08, S09, S10, S11, S14, S16, S17, S18, S19, S22, S23, S26, S27, S28, S29, S30, S31, S32, S33, S34, S35	Es quien asigna perfiles (RF12) y administra usuarios, clientes y productos (RF10, RF7). Atiende pedidos y puede anular (RF3). No confirma pagos, no ve contraseñas y no puede dejar al sistema sin administrador.
`dueno`	13	S01, S02, S03, S04, S05, S06, S07, S08, S09, S10, S11, S34, S35	Administra la carta (RF7, CU4) y consulta el reporte de ventas (RF9, CU8). No opera pedidos ni ve datos de clientes individuales, porque supervisa con cifras agregadas.
`cajero_virtual`	11	S01, S02, S03, S04, S05, S06, S14, S16, S22, S23, S35	Controla las ventas de la caja web (RF4, CU6) y las boletas (RF8). Consulta pedidos para cotejar, pero no cambia su estado ni administra catálogo o usuarios.

Rol	Servicios	Detalle de los servicios	Fundamento de sus permisos
`encargado_despachos`	12	S01, S02, S03, S04, S05, S06, S14, S16, S17, S18, S19, S35	Prepara y despacha (RF13, CU9): ve la cola de órdenes, la imprime y avanza el estado un paso a la vez. No anula, no ve ventas y no ve precios en la orden.

Los permisos de administración se justifican por el requisito que los origina: RF10 y RF12 (usuarios y perfiles), RF7 (catálogo), RF3 (anulación), RF4 y RF8 (ventas y boletas), RF9 (reporte) y RF13 (despacho). Ningún rol tiene más servicios de los que su función exige, y ninguno puede pasar un pedido a `pagado`.

4.5 Autenticación y responsable de verificar la autorización

La **identidad** responde quién es quien llama. La **autorización** responde qué puede hacer sobre ese recurso. Son dos comprobaciones distintas y las hace quien tiene el dato para decidirla.

Pregunta	Quién la responde	Cómo
¿Quién es el usuario? (autenticación)	`auth-service`, consultado por `api-gateway`	Token Bearer (S01). El gateway lo valida en cada solicitud con I01. Si no está activo o venció, 401.
¿Quién es el sistema que llama?	El sistema llamado	`X-Gateway-Secret` para el gateway y `X-Service-Id` con `X-Service-Secret` para los sistemas (3.8).
¿Quién es la plataforma de pago?	`payments-service`	Firma HMAC y marca de tiempo (C8).
¿El rol puede usar esta ruta y este verbo? (autorización gruesa)	`api-gateway` y luego el sistema destino	Primera barrera en el gateway, que corta antes de molestar a un sistema. Segunda barrera en el sistema, con `X-Authenticated-Roles`, por si se llama sin pasar por el gateway.
¿El recurso es de esa persona o está en un estado que lo permite? (autorización fina)	Solo el sistema dueño del dato	Propiedad (C1), estado (C2) y ámbito (C3) las comprueba orders, payments o clients. El gateway no tiene esos datos y no puede decidirlas.

Reglas comunes:

- El navegador no manda `cuentaId`, `rol` ni `estado` en el cuerpo. El gateway los toma del token y los entrega por cabeceras que él mismo escribe.
- Un 404 en lugar de 403 cuando el recurso es de otro cliente evita revelar que existe.
- Ocultar un botón en el frontend no sustituye ninguna de estas comprobaciones. Cada servicio se verifica en el backend aunque la pantalla no muestre la acción.

4.6 Condiciones de acceso

ID	Condición	Regla	Quién la verifica
C1	Propiedad del recurso	El pedido, la boleta o el perfil pertenece a la cuenta autenticada: `cuentaId` del recurso igual a `X-Authenticated-User`. La consulta siempre filtra por ese valor. Si no coincide, 404 sin revelar si el recurso existe.	orders, payments y clients
C2	Estado del pedido	S15 solo desde `pendiente_pago` o `pagado` (409 desde `en_preparacion`). S20 solo si el pedido está `pendiente_pago`, no venció y no tiene pago aprobado. S23 solo desde `pagado`.	orders y payments
C3	Ámbito de los datos	El personal recibe el DTO operativo (sin RUN, correo ni fecha de nacimiento). El cliente recibe solo su propio DTO. El dueño nunca recibe datos de pedidos ni de clientes individuales.	orders, payments y clients
C4	Cuenta activa	La cuenta debe estar `activo`. Una cuenta `pendiente_verificacion` o `inactivo` no opera (403 `CUENTA_NO_ACTIVA`).	auth y orders
C5	Integridad de la administración	No se puede dejar al sistema sin un administrador activo ni darse de baja a uno mismo (409).	auth
C6	Período de consulta	Rango máximo de 92 días y `desde` menor o igual que `hasta` (400).	payments

ID	Condición	Regla	Quién la verifica
C7	Transición permitida según el rol	`encargado_despachos`: `pagado` → `en_preparacion` → `en_despacho` → `entregado`, de a un paso. `administrador`: las mismas más `anulado` desde `pendiente_pago`, `pagado` o `en_preparacion`, con motivo. Nadie pasa un pedido a `pagado` por esta ruta: solo lo hace el pago confirmado (409).	orders
C8	Firma del proveedor	Firma HMAC-SHA256 y marca de tiempo de máximo 5 minutos. No se acepta token de usuario en esta ruta.	payments
C9	Límite de solicitudes	Las rutas públicas con escritura tienen límite por origen (429).	api-gateway
C10	Token de un solo uso	El token de verificación vence, solo se usa una vez y el reenvío invalida el anterior.	auth
C11	Identidad del sistema llamador	`X-Service-Id` y `X-Service-Secret` del par llamador–llamado, y que esa ruta esté permitida para ese llamador.	sistema llamado
C12	Solo campos públicos	El catálogo público devuelve únicamente nombre, categoría, descripción, precio, imagen, destacado y disponibilidad. Nunca `stock` ni productos retirados.	inventory
C13	Baja lógica	DELETE no borra el registro: lo marca como retirado o inactivo y conserva el historial.	inventory, clients y auth

4.7 Matriz resumen: rol por servicio

"Sí" significa permitido y "No" denegado. Entre paréntesis va la condición que además se debe cumplir. S21 no la usa ningún rol: solo la plataforma de pago con firma.

ID	Servicio	Visitante	`cliente`	`administrador`	`dueno`	`cajero_virtual`	`encargado_despachos`
S01	Iniciar sesión	Sí	Sí	Sí	Sí	Sí	Sí
S02	Cerrar sesión e invalidar el token	No	Sí	Sí	Sí	Sí	Sí
S03	Registrar un cliente desde la web	Sí	Sí	Sí	Sí	Sí	Sí
S04	Confirmar el correo y, si corresponde, definir la contraseña	Sí	Sí	Sí	Sí	Sí	Sí
S05	Reenviar el correo de confirmación	Sí	Sí	Sí	Sí	Sí	Sí
S06	Consultar el catálogo	Sí	Sí	Sí	Sí	Sí	Sí
S07	Listar productos con stock y retirados	No	No	Sí	Sí	No	No
S08	Crear un producto	No	No	Sí	Sí	No	No
S09	Reemplazar los datos de un producto	No	No	Sí	Sí	No	No
S10	Retirar un producto (baja lógica, conserva el historial de pedidos)	No	No	Sí	Sí	No	No
S11	Actualizar stock y disponibilidad de un producto	No	No	Sí	Sí	No	No
S12	Crear un pedido	No	Sí	No	No	No	No
S13	Listar los pedidos propios	No	Sí (C1)	No	No	No	No
S14	Consultar el detalle de un pedido	No	Sí (C1)	Sí	No	Sí	Sí
S15	Anular un pedido propio	No	Sí (C1, C2)	No	No	No	No

ID	Servicio	Visitante	cliente	administrador	dueno	cajero_virtual	encargado_despachos
S16	Listar todos los pedidos para el personal	No	No	Sí	No	Sí	Sí
S17	Cambiar el estado de un pedido	No	No	Sí (C7)	No	No	Sí (C7)
S18	Consultar la cola de órdenes de despacho en orden de llegada	No	No	Sí	No	No	Sí
S19	Obtener la orden de despacho lista para imprimir	No	No	Sí	No	No	Sí
S20	Iniciar el pago de un pedido en la plataforma externa	No	Sí (C1, C2)	No	No	No	No
S21	Recibir el resultado del pago	No	No	No	No	No	No
S22	Consultar las ventas de la caja virtual	No	No	Sí	No	Sí	No
S23	Obtener la boleta digital de un pedido pagado	No	Sí (C1, C2)	Sí	No	Sí	No
S24	Consultar mi perfil	No	Sí (C1)	No	No	No	No
S25	Reemplazar mis datos de perfil	No	Sí (C1)	No	No	No	No
S26	Registrar un cliente presencialmente	No	No	Sí	No	No	No
S27	Listar clientes	No	No	Sí	No	No	No
S28	Reemplazar los datos de un cliente	No	No	Sí	No	No	No
S29	Dar de baja a un cliente (baja lógica)	No	No	Sí	No	No	No
S30	Crear un usuario del personal	No	No	Sí	No	No	No
S31	Listar usuarios del personal	No	No	Sí	No	No	No
S32	Cambiar nombre, rol o estado de un usuario	No	No	Sí	No	No	No
S33	Dar de baja a un usuario (baja lógica)	No	No	Sí	No	No	No
S34	Reporte de ventas por período	No	No	Sí	Sí	No	No
S35	Registrar un mensaje de contacto	Sí	Sí	Sí	Sí	Sí	Sí

4.8 Matriz de permisos detallada

Cada servicio tiene una fila por regla de decisión. Las filas "Denegado" agrupan a los demás roles y al visitante. Hay 72 filas.

Rol o identidad	ID	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
visitante, cualquier rol	S01	`/api/v1/auth/login`	POST	Permitido	Con correo y contraseña válidos y cuenta `activo` (C4). Límite de intentos y de tasa (C9).	`api-gateway`: ruta pública y límite de tasa. `auth-service`: credenciales y estado de la cuenta
`cliente`, `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`	S02	`/api/v1/auth/logout`	POST	Permitido	Cierra solo la sesión del token enviado. No recibe `id` de otra sesión.	`api-gateway`: token (I01) y rol por ruta. `auth-service`: rol y sesión del token
Visitante (sin token)	S02	`/api/v1/auth/logout`	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `auth-service` (segunda barrera)

Rol o identidad	ID	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
visitante, cualquier rol	S03	`/api/v1/auth/register`	POST	Permitido	El `rol` es siempre `cliente` y el `estado` `pendiente_verificacion`; el cuerpo no puede traerlos. Límite de tasa (C9).	`api-gateway`: ruta pública y límite de tasa. `clients-service` y `auth-service`: validación y rol fijo
visitante, cualquier rol	S04	`/api/v1/auth/verify-email`	POST	Permitido	Token de un solo uso y vigente (C10).	`api-gateway`: ruta pública. `auth-service`: token
visitante, cualquier rol	S05	`/api/v1/auth/resend-verification`	POST	Permitido	Límite de tasa (C9). Cada reenvío invalida el token anterior (C10).	`api-gateway`: ruta pública y límite de tasa. `auth-service`: token
visitante, cualquier rol	S06	`/api/v1/products`	GET	Permitido	Solo campos públicos (C12).	`api-gateway`: ruta pública. `inventory-service`: DTO público
`administrador`, `dueno`	S07	`/api/v1/admin/products`	GET	Permitido	Ve `stock` y productos retirados.	`api-gateway`: token (I01) y rol por ruta. `inventory-service`: rol y ruta administrativa
Visitante (sin token), `cliente`, `cajero_virtual`, `encargado_despachos`	S07	`/api/v1/admin/products`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `inventory-service` (segunda barrera)
`administrador`, `dueno`	S08	`/api/v1/products`	POST	Permitido	Precio mayor que 0. `stock` inicial solo lo fija este rol.	`api-gateway`: token (I01) y rol por ruta. `inventory-service`: rol y validación
Visitante (sin token), `cliente`, `cajero_virtual`, `encargado_despachos`	S08	`/api/v1/products`	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `inventory-service` (segunda barrera)
`administrador`, `dueno`	S09	`/api/v1/products/{id}`	PUT	Permitido	Reemplaza los datos del catálogo. No cambia `stock` (se hace con S11).	`api-gateway`: token (I01) y rol por ruta. `inventory-service`: rol y validación
Visitante (sin token), `cliente`, `cajero_virtual`, `encargado_despachos`	S09	`/api/v1/products/{id}`	PUT	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `inventory-service` (segunda barrera)
`administrador`, `dueno`	S10	`/api/v1/products/{id}`	DELETE	Permitido	Baja lógica (C13): el producto retirado sigue en los pedidos antiguos.	`api-gateway`: token (I01) y rol por ruta. `inventory-service`: rol y baja lógica
Visitante (sin token), `cliente`, `cajero_virtual`, `encargado_despachos`	S10	`/api/v1/products/{id}`	DELETE	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `inventory-service` (segunda barrera)
`administrador`, `dueno`	S11	`/api/v1/products/{id}/stock`	PUT	Permitido	Cambia `stock` y `disponible`. Queda un movimiento `ajuste`.	`api-gateway`: token (I01) y rol por ruta. `inventory-service`: rol y validación
Visitante (sin token), `cliente`, `cajero_virtual`, `encargado_despachos`	S11	`/api/v1/products/{id}/stock`	PUT	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `inventory-service` (segunda barrera)
`cliente`	S12	`/api/v1/orders`	POST	Permitido	Cuenta activa con perfil (C4). El `cuentaId` sale del token, nunca del cuerpo. Precios y total los calcula el servidor.	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y cuenta activa (C4); `cuentaId` tomado de `X-Authenticated-User`

Rol o identidad	ID	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
Visitante (sin token), `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`	S12	`/api/v1/orders`	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `orders-service` (segunda barrera)
`cliente`	S13	`/api/v1/orders`	GET	Permitido	Solo los pedidos de su cuenta (C1). La consulta se filtra por el token y no acepta el id de otro cliente.	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y filtro por `cuentaId` (C1)
Visitante (sin token), `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`	S13	`/api/v1/orders`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `orders-service` (segunda barrera)
`cliente`	S14	`/api/v1/orders/{id}`	GET	Permitido	Solo si el pedido es suyo (C1).	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y propiedad (C1)
`administrador`, `cajero_virtual`, `encargado_despachos`	S14	`/api/v1/orders/{id}`	GET	Permitido	Cualquier pedido, con el DTO operativo (C3).	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y ámbito (C3)
cliente (de otra cuenta)	S14	`/api/v1/orders/{id}`	GET	Denegado	404, sin datos y sin revelar si el pedido existe.	`orders-service`: propiedad (C1)
Visitante (sin token), `dueno`	S14	`/api/v1/orders/{id}`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `orders-service` (segunda barrera)
`cliente`	S15	`/api/v1/orders/{id}/cancel`	POST	Permitido	Solo su pedido (C1) y solo desde `pendiente_pago` o `pagado` (C2). Motivo de 5 a 200 caracteres. Anular no borra el registro.	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y propiedad (C1) y estado (C2)
cliente (de otra cuenta)	S15	`/api/v1/orders/{id}/cancel`	POST	Denegado	404, sin datos y sin revelar si el pedido existe.	`orders-service`: propiedad (C1)
Visitante (sin token), `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`	S15	`/api/v1/orders/{id}/cancel`	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `orders-service` (segunda barrera)
`administrador`, `cajero_virtual`, `encargado_despachos`	S16	`/api/v1/admin/orders`	GET	Permitido	Todos los pedidos, con el DTO operativo (C3).	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y ámbito (C3)
Visitante (sin token), `cliente`, `dueno`	S16	`/api/v1/admin/orders`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `orders-service` (segunda barrera)
`encargado_despachos`	S17	`/api/v1/admin/orders/{id}/status`	PATCH	Permitido	Solo avanzar un paso (C7). No puede anular.	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y transición según rol (C7)
`administrador`	S17	`/api/v1/admin/orders/{id}/status`	PATCH	Permitido	Avanzar y anular con motivo (C7).	`api-gateway`: token (I01) y rol por ruta. `orders-service`: rol y transición según rol (C7)
encargado_despachos (anular), cualquier rol (pasar a `pagado`)	S17	`/api/v1/admin/orders/{id}/status`	PATCH	Denegado	Pasar a `pagado` por esta ruta: 409. Solo lo hace el pago confirmado. `encargado_despachos` que intenta `anulado`: 403.	`orders-service`: transición (C7)
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`	S17	`/api/v1/admin/orders/{id}/status`	PATCH	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `orders-service` (segunda barrera)

Rol o identidad	ID	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
encargado_despachos, administrador	S18	/api/v1/admin/dispatch-orders	GET	Permitido	Cola en orden de llegada, sin precios, RUN ni correo.	api-gateway: token (I01) y rol por ruta. dispatch-service: rol y ruta administrativa
Visitante (sin token), cliente, dueno, cajero_virtual	S18	/api/v1/admin/dispatch-orders	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	api-gateway (primera barrera) y dispatch-service (segunda barrera)
encargado_despachos, administrador	S19	/api/v1/admin/dispatch-orders/{pedidoId}/print	GET	Permitido	Marca la orden como impresa. Repetir no cambia el resultado.	api-gateway: token (I01) y rol por ruta. dispatch-service: rol y ruta administrativa
Visitante (sin token), cliente, dueno, cajero_virtual	S19	/api/v1/admin/dispatch-orders/{pedidoId}/print	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	api-gateway (primera barrera) y dispatch-service (segunda barrera)
cliente	S20	/api/v1/orders/{id}/payments	POST	Permitido	Solo su pedido (C1), en pendiente_pago y no vencido (C2). El monto lo toma el servidor del pedido.	api-gateway: token (I01) y rol por ruta. orders-service: rol y propiedad y estado (C1, C2), consultados por payments-service en I11
cliente (de otra cuenta)	S20	/api/v1/orders/{id}/payments	POST	Denegado	404, sin datos y sin revelar si el pedido existe.	orders-service: propiedad (C1), a pedido de payments-service
Visitante (sin token), administrador, dueno, cajero_virtual, encargado_despachos	S20	/api/v1/orders/{id}/payments	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	api-gateway (primera barrera) y payments-service (segunda barrera)
plataforma-de-pago	S21	/api/v1/payments/webhook	POST	Permitido	Firma y marca de tiempo válidas (C8). Un eventoId repetido responde 200 sin procesar. Con monto distinto no se aprueba.	api-gateway: ruta pública con límite de tasa. payments-service: firma (C8), eventoId y monto
cualquier otro llamador, incluso con token de usuario	S21	/api/v1/payments/webhook	POST	Denegado	401. Un usuario no puede informar un pago.	payments-service: firma (C8)
cajero_virtual, administrador	S22	/api/v1/admin/sales	GET	Permitido	Ventas de la caja virtual en un período (C6).	api-gateway: token (I01) y rol por ruta. payments-service: rol y período (C6)
Visitante (sin token), cliente, dueno, encargado_despachos	S22	/api/v1/admin/sales	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	api-gateway (primera barrera) y payments-service (segunda barrera)
cliente	S23	/api/v1/orders/{id}/receipt	GET	Permitido	Solo su boleta (C1) y con el pedido pagado o posterior (C2).	api-gateway: token (I01) y rol por ruta. payments-service: rol y propiedad y estado (C1, C2)
cajero_virtual, administrador	S23	/api/v1/orders/{id}/receipt	GET	Permitido	Cualquier boleta.	api-gateway: token (I01) y rol por ruta. payments-service: rol y ámbito
cliente (de otra cuenta)	S23	/api/v1/orders/{id}/receipt	GET	Denegado	404, sin datos y sin revelar si la boleta existe.	payments-service: propiedad (C1)
Visitante (sin token), dueno, encargado_despachos	S23	/api/v1/orders/{id}/receipt	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	api-gateway (primera barrera) y payments-service (segunda barrera)
cliente	S24	/api/v1/clients/me	GET	Permitido	Solo su propio perfil (C1). La ruta no lleva id: la cuenta sale del token.	api-gateway: token (I01) y rol por ruta. clients-service: rol y cuenta del token (C1)

Rol o identidad	ID	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
Visitante (sin token), `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`	S24	`/api/v1/clients/me`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `clients-service` (segunda barrera)
`cliente`	S25	`/api/v1/clients/me`	PUT	Permitido	Solo su propio perfil (C1). `run` y `correo` no se cambian.	`api-gateway`: token (I01) y rol por ruta. `clients-service`: rol y cuenta del token (C1)
Visitante (sin token), `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`	S25	`/api/v1/clients/me`	PUT	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `clients-service` (segunda barrera)
`administrador`	S26	`/api/v1/admin/clients`	POST	Permitido	El rol del nuevo usuario es siempre `cliente`.	`api-gateway`: token (I01) y rol por ruta. `clients-service`: rol y rol fijo; `auth-service` rechaza otro rol en I03
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S26	`/api/v1/admin/clients`	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `clients-service` (segunda barrera)
`administrador`	S27	`/api/v1/admin/clients`	GET	Permitido	Lista de clientes con perfil y `estado` (C3).	`api-gateway`: token (I01) y rol por ruta. `clients-service`: rol y ruta administrativa
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S27	`/api/v1/admin/clients`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `clients-service` (segunda barrera)
`administrador`	S28	`/api/v1/admin/clients/{id}`	PUT	Permitido	Reemplaza los datos personales. No cambia RUN ni correo.	`api-gateway`: token (I01) y rol por ruta. `clients-service`: rol y validación
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S28	`/api/v1/admin/clients/{id}`	PUT	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `clients-service` (segunda barrera)
`administrador`	S29	`/api/v1/admin/clients/{id}`	DELETE	Permitido	Baja lógica (C13): pedidos y ventas se conservan.	`api-gateway`: token (I01) y rol por ruta. `clients-service`: rol y baja lógica
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S29	`/api/v1/admin/clients/{id}`	DELETE	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `clients-service` (segunda barrera)
`administrador`	S30	`/api/v1/admin/users`	POST	Permitido	Crea personal con rol distinto de `cliente`.	`api-gateway`: token (I01) y rol por ruta. `auth-service`: rol y validación de rol
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S30	`/api/v1/admin/users`	POST	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `auth-service` (segunda barrera)
`administrador`	S31	`/api/v1/admin/users`	GET	Permitido	Nunca devuelve contraseñas ni hashes.	`api-gateway`: token (I01) y rol por ruta. `auth-service`: rol y ruta administrativa
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S31	`/api/v1/admin/users`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `auth-service` (segunda barrera)
`administrador`	S32	`/api/v1/admin/users/{id}`	PUT	Permitido	No puede dejar al sistema sin administrador activo (C5).	`api-gateway`: token (I01) y rol por ruta. `auth-service`: rol y integridad (C5)

Rol o identidad	ID	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S32	`/api/v1/admin/users/{id}`	PUT	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `auth-service` (segunda barrera)
`administrador`	S33	`/api/v1/admin/users/{id}`	DELETE	Permitido	Baja lógica (C13). No a sí mismo ni al último administrador (C5). Invalida sus sesiones.	`api-gateway`: token (I01) y rol por ruta. `auth-service`: rol y integridad (C5)
Visitante (sin token), `cliente`, `dueno`, `cajero_virtual`, `encargado_despachos`	S33	`/api/v1/admin/users/{id}`	DELETE	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `auth-service` (segunda barrera)
`dueno`, `administrador`	S34	`/api/v1/admin/reports/sales`	GET	Permitido	Reporte agregado, sin datos de clientes individuales (C3). Rango de 92 días (C6).	`api-gateway`: token (I01) y rol por ruta. `payments-service`: rol y período (C6)
Visitante (sin token), `cliente`, `cajero_virtual`, `encargado_despachos`	S34	`/api/v1/admin/reports/sales`	GET	Denegado	401 sin token. 403 con un rol no permitido. Que el botón esté oculto no cambia esta regla.	`api-gateway` (primera barrera) y `payments-service` (segunda barrera)
visitante, cualquier rol	S35	`/api/v1/contact-messages`	POST	Permitido	Límite de tasa (C9). Solo guarda el mensaje; no devuelve datos del negocio.	`api-gateway`: ruta pública y límite de tasa. `notifications-service`: validación

4.9 Verbos por recurso

Cada recurso se muestra con sus cinco verbos. "Permitido" indica los roles que pueden usarlo, "Denegado" que los demás reciben 401 o 403, y "No aplicable" que la ruta no define ese verbo (405). Un verbo permitido en un recurso no autoriza otro: por ejemplo, GET es permitido para un cliente en `/api/v1/orders/{id}` solo sobre sus pedidos y denegado en `/api/v1/admin/orders`, y POST es permitido en `/api/v1/orders` para un cliente y denegado en `/api/v1/products`.

Recurso	GET	POST	PUT	PATCH	DELETE
`/api/v1/auth/login`	No aplicable (405)	S01 · Permitido: todos	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/auth/logout`	No aplicable (405)	S02 · Permitido: `cliente`, `administrador`, `dueno`, `cajero_virtual`, `encargado_despachos`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/auth/register`	No aplicable (405)	S03 · Permitido: todos	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/auth/verify-email`	No aplicable (405)	S04 · Permitido: todos	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/auth/resend-verification`	No aplicable (405)	S05 · Permitido: todos	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/products`	S06 · Permitido: todos	S08 · Permitido: `administrador`, `dueno`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/products`	S07 · Permitido: `administrador`, `dueno`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/products/{id}`	No aplicable (405)	No aplicable (405)	S09 · Permitido: `administrador`, `dueno`. Denegado: los demás	No aplicable (405)	S10 · Permitido: `administrador`, `dueno`. Denegado: los demás
`/api/v1/products/{id}/stock`	No aplicable (405)	No aplicable (405)	S11 · Permitido: `administrador`, `dueno`. Denegado: los demás	No aplicable (405)	No aplicable (405)
`/api/v1/orders`	S13 · Permitido: `cliente` (solo lo propio, C1). Denegado: los demás	S12 · Permitido: `cliente`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)

Recurso	GET	POST	PUT	PATCH	DELETE
`/api/v1/orders/{id}`	S14 · Permitido: `cliente` (solo lo propio, C1), `administrador`, `cajero_virtual`, `encargado_despachos`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/orders/{id}/cancel`	No aplicable (405)	S15 · Permitido: `cliente` (solo lo propio, C1). Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/orders`	S16 · Permitido: `administrador`, `cajero_virtual`, `encargado_despachos`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/orders/{id}/status`	No aplicable (405)	No aplicable (405)	No aplicable (405)	S17 · Permitido: `encargado_despachos`, `administrador`. Denegado: los demás	No aplicable (405)
`/api/v1/admin/dispatch-orders`	S18 · Permitido: `encargado_despachos`, `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/dispatch-orders/{pedidoId}/print`	S19 · Permitido: `encargado_despachos`, `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/orders/{id}/payments`	No aplicable (405)	S20 · Permitido: `cliente` (solo lo propio, C1). Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/payments/webhook`	No aplicable (405)	S21 · Permitido: la plataforma de pago (firma). Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/sales`	S22 · Permitido: `cajero_virtual`, `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/orders/{id}/receipt`	S23 · Permitido: `cliente` (solo lo propio, C1), `cajero_virtual`, `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/clients/me`	S24 · Permitido: `cliente` (solo lo propio, C1). Denegado: los demás	No aplicable (405)	S25 · Permitido: `cliente` (solo lo propio, C1). Denegado: los demás	No aplicable (405)	No aplicable (405)
`/api/v1/admin/clients`	S27 · Permitido: `administrador`. Denegado: los demás	S26 · Permitido: `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/clients/{id}`	No aplicable (405)	No aplicable (405)	S28 · Permitido: `administrador`. Denegado: los demás	No aplicable (405)	S29 · Permitido: `administrador`. Denegado: los demás
`/api/v1/admin/users`	S31 · Permitido: `administrador`. Denegado: los demás	S30 · Permitido: `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/admin/users/{id}`	No aplicable (405)	No aplicable (405)	S32 · Permitido: `administrador`. Denegado: los demás	No aplicable (405)	S33 · Permitido: `administrador`. Denegado: los demás
`/api/v1/admin/reports/sales`	S34 · Permitido: `dueno`, `administrador`. Denegado: los demás	No aplicable (405)	No aplicable (405)	No aplicable (405)	No aplicable (405)
`/api/v1/contact-messages`	No aplicable (405)	S35 · Permitido: todos	No aplicable (405)	No aplicable (405)	No aplicable (405)

Semántica de los verbos: GET consulta y no cambia nada (S19 solo marca la orden como impresa y es idempotente). POST crea un recurso o ejecuta una operación justificada (iniciar sesión, anular, pagar). PUT reemplaza la representación del recurso indicado. DELETE solicita una baja lógica. Se agrega PATCH solo en S17, y su semántica es cambiar únicamente el campo `estado` de un pedido sin reemplazar el resto.

4.10 Identidades técnicas: permisos entre sistemas

Las rutas `/internal/...` no se publican. Cada fila indica qué identidad puede llamar qué ruta y con qué límite. Lo que no aparece como permitido está denegado.

Identidad	Integración	Destino	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
`api-gateway`	I01	`auth-service`	`/internal/introspect`	POST	Permitido	Solo para validar el token recibido.	`auth-service`: C11
`api-gateway`	I02	Cualquier sistema de negocio	`/internal/...`	Todos	Denegado	El gateway solo reenvía `/api/v1/...`. Las rutas internas no se publican.	`api-gateway`: tabla de rutas
`clients-service`	I03	`auth-service`	`/internal/accounts`	POST	Permitido	Solo con `rol: cliente`. Si pide otro rol, 403.	`auth-service`: C11 y rol
`clients-service`	I03	`auth-service`	`/internal/accounts/{cuentaId}`	DELETE	Permitido	Solo compensación: la cuenta debe seguir `pendiente_verificacion` y ser de rol `cliente`.	`auth-service`: C11 y estado
`clients-service`	I04	`auth-service`	`/internal/accounts?ids=`	GET	Permitido	Devuelve solo `cuentaId`, `correo` y `estado`.	`auth-service`: C11
`clients-service`	I05	`auth-service`	`/internal/accounts/{cuentaId}`	PATCH	Permitido	Solo `estado: inactivo`, y solo sobre cuentas de rol `cliente`.	`auth-service`: C11 y rol de la cuenta
`clients-service`	—	`auth-service`	`/internal/accounts/{cuentaId}`	PUT	Denegado	No puede cambiar contraseñas ni roles de otra cuenta.	`auth-service`: C11
`auth-service`	I06	`notifications-service`	`/internal/emails`	POST	Permitido	Solo con plantilla `verificacion`.	`notifications-service`: C11 y plantilla
`orders-service`	I07	`clients-service`	`/internal/clients/by-account/{cuentaId}`	GET	Permitido	Devuelve nombre, teléfono y domicilio. No devuelve RUN, fecha de nacimiento ni sexo.	`clients-service`: C11
`orders-service`	I08, I09, I10	`inventory-service`	`/internal/stock/reservations...`	POST	Permitido	Reservar, confirmar y liberar. No puede cambiar precio ni stock por otra vía.	`inventory-service`: C11
`payments-service`	I11	`orders-service`	`/internal/orders/{id}/payable`	GET	Permitido	Solo `pedidoId`, `total`, `estado` y `expiraEn`, para la cuenta indicada.	`orders-service`: C11 y propiedad (C1)
`payments-service`	I12	`orders-service`	`/internal/orders/{id}/payment-confirmation`	POST	Permitido	Única identidad que puede llevar un pedido a `pagado`. Orders vuelve a comprobar estado, vencimiento y monto.	`orders-service`: C11 y reglas del pedido
`payments-service`	I13	`notifications-service`	`/internal/emails`	POST	Permitido	Solo con plantilla `boleta`.	`notifications-service`: C11 y plantilla
`plataforma-de-pago` (externa)	I15	`payments-service`	`/api/v1/payments/webhook`	POST	Permitido	Firma y marca de tiempo (C8). No tiene acceso a ninguna otra ruta.	`payments-service`: C8
`notifications-service`	I17	Proveedor de correo (externo)	API del proveedor	POST	Permitido	Clave del proveedor guardada en Vault, solo para enviar y validar correos.	Proveedor: clave de API
`orders-service`	I14	ActiveMQ `/queue/cola_pedidos`	STOMP	SEND	Permitido	Usuario del broker solo de escritura.	ActiveMQ: usuario y permisos
`dispatch-service`	I14	ActiveMQ `/queue/cola_pedidos`	STOMP	SUBSCRIBE	Permitido	Usuario del broker solo de lectura.	ActiveMQ: usuario y permisos

Identidad	Integración	Destino	Ruta	Verbo	Decisión	Alcance y condición	Se verifica en
`inventory-service`, `dispatch-service`, `notifications-service`	—	Cualquier otro sistema	`/internal/...`	Todos	Denegado	No hacen llamadas hacia otros sistemas de negocio. No existe par de credenciales para ellos.	Sistema llamado: C11
`clients-service` u otro sistema distinto de `payments-service`	—	`orders-service`	`/internal/orders/{id}/payment-confirmation`	POST	Denegado	El par de credenciales no está en la lista permitida de esa ruta.	`orders-service`: C11
`orders-service`	—	`payments-service`	Cualquier ruta	Todos	Denegado	Orders nunca llama a payments. Evita el ciclo de dependencias (Módulo 3).	`payments-service`: C11
`graphql-gateway`	—	`orders-service`	`/orders` y `/orders/{id}`	GET	Permitido	Solo lectura y con el contexto del usuario propagado. Orders sigue aplicando la propiedad (C1).	`orders-service`: C11 y C1
`graphql-gateway`	—	`inventory-service`	`/products`	GET	Permitido	Solo campos públicos del catálogo (C12).	`inventory-service`: C11
`graphql-gateway`	—	`clients-service`	`/clients/me`	GET	Permitido	Solo el perfil de la cuenta propagada (C1).	`clients-service`: C11 y C1
`graphql-gateway`	—	Cualquier sistema	Cualquier ruta	POST, PUT, PATCH, DELETE	Denegado	No hay mutaciones: las reglas de negocio y las escrituras quedan en cada sistema. Tampoco puede llamar rutas `/internal/....`	Sistema llamado: C11
Navegador (cualquier rol)	—	Cualquier sistema	`/internal/...`	Todos	Denegado	No hay ruta publicada. El gateway responde 404.	`api-gateway`: tabla de rutas

4.11 Anulación e historial

Anular un pedido no lo borra. No existe DELETE sobre pedidos, ventas ni boletas. La anulación (S15 o S17) cambia `estado` a `anulado`, agrega un registro `CambioEstado` con el motivo y quién lo pidió, y conserva el pedido, sus líneas y la dirección. Si ya estaba pagado, queda `reembolsoPendiente` y se conservan la venta y la boleta.

Las bajas de producto (S10), cliente (S29) y usuario (S33) también son lógicas: el registro queda marcado y los pedidos antiguos siguen mostrando los mismos datos porque cada línea guarda el nombre y el precio del momento de la compra.

4.12 Cómo la matriz impide que un cliente consulte pedidos de otro

Se comprueba con tres filas de la matriz 4.8:

1. S13: el cliente solo lista los pedidos de su `cuentaId`. La consulta se filtra con el valor de `X-Authenticated-User` y no existe parámetro para indicar otro cliente.
2. S14: el cliente puede leer un pedido solo si `Pedido.cuentaId` coincide con el suyo (C1). La búsqueda es `{id, cuentaId: <del token>}`. Si no coincide, devuelve 404, igual que un pedido inexistente.
3. S15, S20 y S23: aplican la misma regla C1 antes de mirar el estado, así que un cliente tampoco puede anular, pagar ni pedir la boleta de otro.

Regla aplicada por: `orders-service` (S13 a S15 y S20 a través de I11) y `payments-service` (S23). Las pruebas T-AUT-01 a T-AUT-04 (4.13) muestran la ejecución autorizada y la denegada de la misma regla.

4.13 Pruebas de autorización previstas

Se ejecutan en el Módulo 10, con la evidencia de cada resultado.

ID	Regla	Caso	Resultado esperado	Servicio que la aplica
T-AUT-01	C1	El cliente A consulta con su token un pedido de la cuenta B (S14).	404, sin datos de B y sin diferencia visible con un pedido inexistente.	`orders-service`

ID	Regla	Caso	Resultado esperado	Servicio que la aplica
T-AUT-02	C1	El cliente A consulta su propio pedido (S14).	200 con el pedido.	`orders-service`
T-AUT-03	C1, C2	El cliente A anula un pedido de B (S15). Luego anula uno propio en `en_preparacion`.	404. Luego 409.	`orders-service`
T-AUT-04	C1	El cliente A pide la boleta de un pedido de B (S23).	404.	`payments-service`
T-AUT-05	Rol	Un cliente intenta DELETE de un producto (S10) y POST de un producto (S08).	403 en ambos.	`api-gateway` e `inventory-service`
T-AUT-06	C12	Un visitante consulta el catálogo (S06) y luego un pedido (S14).	200 sin `stock`. Luego 401.	`inventory-service`, `api-gateway`
T-AUT-07	C7	`encargado_despachos` intenta anular (S17). Cualquier rol intenta pasar a `pagado`.	403. 409.	`orders-service`
T-AUT-08	C8	Webhook con firma inválida y luego con firma válida (S21).	401 y no cambia nada. Luego 200 y el pedido pasa a `pagado`.	`payments-service`
T-AUT-09	Rol, C3	El dueño pide la lista de pedidos (S16) y el reporte (S34).	403 y 200.	`api-gateway` y `orders-service`
T-AUT-10	Identidad	Un cliente envía `X-Authenticated-Roles: administrador` en su solicitud a S16.	El gateway borra la cabecera y la solicitud se evalúa con el rol real: 403.	`api-gateway`
T-AUT-11	C11	Llamada directa a `orders-service` sin `X-Gateway-Secret`. Luego con el secreto pero sin identidad.	401 en ambas.	`orders-service`
T-AUT-12	C11	`clients-service` llama a `payment-confirmation` de orders.	403.	`orders-service`
T-AUT-13	Red	Se pide `/internal/introspect` por el gateway.	404.	`api-gateway`
T-AUT-14	C5	El administrador se da de baja a sí mismo (S33). Se cambia el rol del último administrador (S32).	409 en ambos.	`auth-service`

4.14 Control antes de avanzar

Pregunta	Respuesta
¿Los permisos se expresan por servicio y recurso?	Sí. La matriz 4.8 tiene una fila por servicio y regla, con ruta y verbo, y la tabla 4.9 muestra cada recurso con sus verbos.
¿Se distinguen identidad y autorización?	Sí. La tabla 4.5 separa quién es quien llama (token, secreto de sistema, firma) de lo que puede hacer (rol, propiedad, estado).
¿La matriz permite comprobar que un usuario no consulta pedidos de otro?	Sí. La condición C1 se aplica en S13, S14, S15, S20 y S23, y las pruebas T-AUT-01 a T-AUT-04 la ejercitan.
¿Hay stakeholders sin permisos operativos?	Sí. Cocina, repartidor, contabilidad y el responsable de la infraestructura figuran sin permisos y la tabla 4.3 explica cómo reciben su información.
¿Se fundamentaron los permisos de administración y las identidades de sistemas?	Sí. Los permisos por rol se justifican en 4.4 y los de cada sistema en 4.10.

Módulo 5 — Clases del API Gateway

Los diagramas se entregan en formato editable (PlantUML, `.puml`) y legible (`.svg` y `.png`). Están en la carpeta `diagramas/`. Este módulo complementa el mapa de componentes del Módulo 3: no lo reemplaza.

5.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Decidir cuántos gateways y por qué.	5.2.
2. Clasificar rutas anónimas, autenticadas, administrativas e internas.	5.3.
3. Modelar ApiGateway, RouteDefinition, AuthenticationPolicy, AuthorizationPolicy, ServiceTarget, SecretProvider, GraphQLGateway, Schema, Resolver y adaptadores.	Diagrama 5.1 y catálogo de clases (5.4).
4. Atributos, operaciones, multiplicidades y dependencias hacia autenticación, sistemas y secretos.	Diagrama 5.1 y tabla de relaciones (5.5).
5. Qué valida el gateway y qué valida cada sistema.	5.8.
Fronteras de confianza y recorrido de una solicitud autorizada y una denegada.	5.9 (diagrama 5.2) y 5.11 (diagrama 5.3).

Paso de la guía	Dónde está
Validación de identidad, autorización por ruta, límites, tiempos, errores y registros.	5.6 y 5.7.
Cómo se impide el acceso directo a los sistemas internos.	5.10.

5.2 Cuántos gateways

Se usan **dos**, de naturaleza distinta:

Gateway	Rol	Dónde está
API Gateway	Borde: recibe todo lo que viene de Internet. Autentica, autoriza por ruta, limita y reenvía.	Es el único con un puerto publicado.
GraphQL Gateway	Agregación: combina consultas de lectura de varios sistemas para una pantalla. No autentica a usuarios y no publica nada hacia Internet.	Dentro de la red privada. Se llega a él solo por el API Gateway (`POST /api/v1/graphql`).

Se descartaron otras opciones:

Alternativa	Por qué no
Un API Gateway por tipo de público (uno para clientes y otro para personal).	Las diferencias entre públicos se resuelven con políticas de autorización por ruta. Dos gateways duplicarían la validación de tokens, los secretos y la configuración, sin que cambien la red, el protocolo ni la escala.
Un API Gateway por sistema.	Habría seis puertos publicados y seis lugares donde fallar al validar identidad. Es lo contrario de un punto de entrada único.
Un solo gateway que también agregue datos (sin GraphQL Gateway).	Mezclaría el borde público con lógica de composición. El borde debe ser fino, y la agregación debe vivir en la red privada.
Sin GraphQL Gateway.	La guía exige al menos uno privado con una agregación útil de dos o más sistemas (Módulo 8).

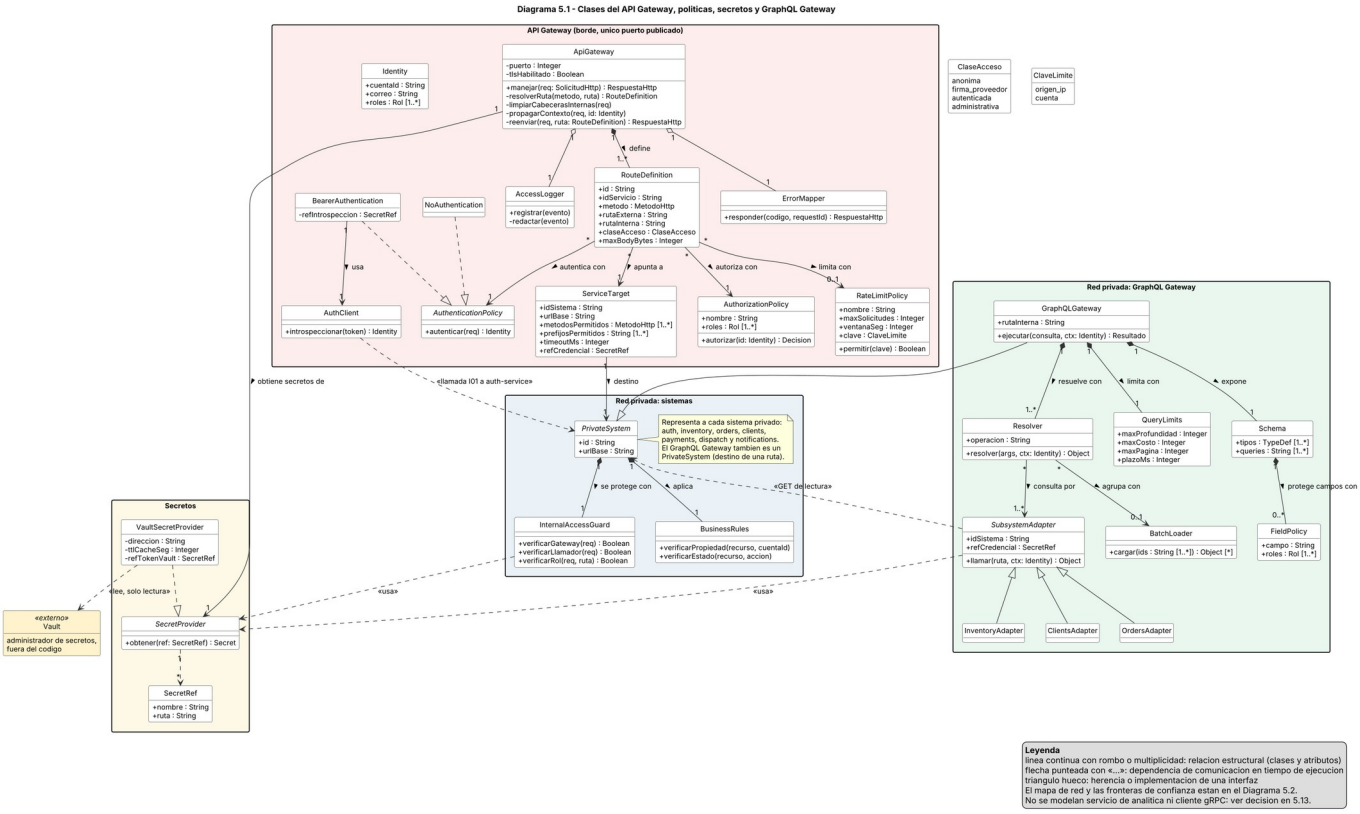
Un segundo gateway de borde solo se justificaría si hubiera una exposición, un protocolo de autenticación o una escala distintos. En La Fornace no ocurre ninguna de las tres.

5.3 Clasificación de rutas

Clase de ruta	Servicios	Quién llama	Autenticación en el gateway	Política de autorización	Datos que puede transportar
Externa anónima	S01, S03, S04, S05, S06, S35	Cualquier visitante	Ninguna. Límite de solicitudes por IP	Ninguna	S06 solo datos públicos. S01, S03, S04, S05 y S35 reciben credenciales o datos personales de quien las envía, pero no devuelven datos de otras personas.
Externa con firma del proveedor	S21	Plataforma de pago	Ninguna del usuario. El gateway solo limita tasa, verbo y tamaño	Ninguna en el gateway. La firma la verifica payments	Resultado de un pago.
Externa autenticada	S02, S12, S13, S14, S15, S20, S23, S24, S25	Cliente (y personal en S14, S23 y S02)	Token Bearer (I01)	`ROLES-TODOS`, `ROLES-CLIENTE`, `ROLES-PEDIDO-LECTURA` o `ROLES-BOLETA`	Datos restringidos del propio cliente: pedidos, dirección, boleta y perfil. La propiedad la comprueba el sistema dueño.
Administrativa	S07, S08, S09, S10, S11, S16, S17, S18, S19, S22, S26, S27, S28, S29, S30, S31, S32, S33, S34	Personal	Token Bearer (I01)	`ROLES-CATALOGO`, `ROLES-PEDIDOS-PERSONAL`, `ROLES-DESPACHO`, `ROLES-VENTAS`, `ROLES-REPORTE` o `ROLES-ADMIN`	Datos restringidos de clientes y pedidos, y datos operativos. Incluye rutas bajo `/api/v1/products` (S08 a S11), que no tienen el prefijo `admin` pero son administrativas por su rol.
Externa autenticada: GraphQL	`POST /api/v1/graphql` (nueva, Módulo 8)	Cliente y personal	Token Bearer (I01)	`ROLES-TODOS` en la ruta y política por campo dentro del gateway GraphQL	Agregaciones de lectura. Cada campo se autoriza por separado.
Interna	Todo `/internal/...` y el gateway GraphQL	Solo sistemas y el API Gateway	No se publica	No se publica	No aplica desde Internet. Ver 5.10.

Una ruta pública de red puede transportar datos restringidos para un usuario autorizado. Por eso la clase no depende del prefijo de la ruta, sino de quién puede llamarla y qué devuelve. La lista completa de rutas, con su destino y sus controles, es el contenido del Módulo 6.

5.4 Diagrama 5.1 — Clases del gateway, políticas, secretos y GraphQL



Fuente editable: `diagramas/diagrama-5-1-clases-gateway.puml`. Versión vectorial: `diagramas/diagrama-5-1-clases-gateway.svg`.

El diagrama separa tres cosas: la **configuración de rutas** (`RouteDefinition` y `ServiceTarget`), las **políticas** (autenticación, autorización y límites) y los **secretos** (`SecretProvider`). Las líneas con rombo y multiplicidad son relaciones estructurales. Las flechas punteadas con estereotipo son dependencias de comunicación en tiempo de ejecución; el mapa de red está en el diagrama 5.2.

Clase	Tipo	Responsabilidad	Atributos relevantes	Operaciones	Depende de
`ApiGateway`	Clase	Punto de entrada único. Recibe la solicitud, la valida con las políticas de su ruta, descarta la identidad que traiga el navegador, agrega el contexto verificado y la reenvía.	`puerto`, `tlsHabilitado`	`manejar`, `resolverRuta`, `limpiarCabecerasInternas`, `propagarContexto`, `reenviar`	SecretProvider, AccessLogger, ErrorMapper
`RouteDefinition`	Clase de configuración	Una fila de la lista de rutas permitidas: verbo, ruta externa, ruta interna, clase de acceso y límites. Es la única forma de llegar a un sistema.	`id`, `idServicio`, `metodo`, `rutaExterna`, `rutaInterna`, `claseAcceso`, `maxBodyBytes`	—	AuthenticationPolicy, AuthorizationPolicy, RateLimitPolicy, ServiceTarget
`AuthenticationPolicy`	Interfaz	Cómo se identifica quien llama. Se implementa con `NoAuthentication` (rutas anónimas) y `BearerAuthentication` (token de usuario).	—	`autenticar`	AuthClient (solo Bearer)
`AuthorizationPolicy`	Clase de política	Conjunto de roles que pueden usar un grupo de rutas. Es la primera barrera: corta antes de molestar a un sistema.	`nombre`, `roles`	`autorizar`	Identity
`RateLimitPolicy`	Clase de política	Límite de solicitudes por ventana de tiempo y por origen o por cuenta.	`nombre`, `maxSolicitudes`, `ventanaSeg`, `clave`	`permitir`	—

Clase	Tipo	Responsabilidad	Atributos relevantes	Operaciones	Depende de
`ServiceTarget`	Clase de configuración	Un sistema interno al que se puede reenviar. Fija la URL, los verbos y los prefijos de ruta admitidos, el plazo y la referencia de la credencial.	`idSistema`, `urlBase`, `metodosPermitidos`, `prefijosPermitidos`, `timeoutMs`, `refCredencial`	—	PrivateSystem, SecretProvider
`Identity`	Objeto de valor	Identidad verificada del usuario: la que devuelve Auth Service y que el gateway propaga.	`cuentaId`, `correo`, `roles`	—	—
`AuthClient`	Clase	Cliente de Auth Service para la introspección del token (I01).	—	`introspeccionar`	Auth Service, SecretProvider
`ErrorMapper`	Clase	Convierte cualquier falla en un error controlado con `codigo`, `mensaje` y `requestId`, sin detalles internos.	—	`responder`	—
`AccessLogger`	Clase	Registra cada solicitud sin secretos, sin tokens y sin cuerpo.	—	`registrar`, `redactar`	—
`SecretProvider`	Interfaz	Entrega un secreto a partir de una referencia simbólica. Nadie lee secretos desde el código ni desde variables fijas.	—	`obtener`	SecretRef
`VaultSecretProvider`	Clase	Implementación contra Vault con un token de solo lectura y un caché corto.	`direccion`, `ttlCacheSeg`, `refTokenVault`	—	Vault
`SecretRef`	Objeto de valor	Referencia simbólica (por ejemplo, `gateway-to-orders`). Nunca contiene el valor.	`nombre`, `ruta`	—	—
`PrivateSystem`	Clase abstracta	Cualquier sistema de la red privada. Cada uno tiene su guardia de acceso y sus reglas de negocio.	`id`, `urlBase`	—	InternalAccessGuard, BusinessRules
`InternalAccessGuard`	Clase	Lo que cada sistema comprueba antes de atender: que lo llame el gateway o un sistema permitido, y que el rol sirva para la ruta.	—	`verificarGateway`, `verificarLlamador`, `verificarRol`	SecretProvider
`BusinessRules`	Clase	Comprobaciones que solo el dueño del dato puede hacer: propiedad, estado y reglas del negocio.	—	`verificarPropiedad`, `verificarEstado`	—
`GraphQLGateway`	Clase	Gateway privado de agregación. Recibe consultas de lectura ya autenticadas por el API Gateway y las resuelve llamando a los sistemas.	`rutaInterna`	`ejecutar`	Schema, Resolver, QueryLimits
`Schema`	Clase	Tipos y consultas expuestos, con las políticas por campo.	`tipos`, `queries`	—	FieldPolicy
`FieldPolicy`	Clase de política	Roles que pueden leer un campo. Evita filtrar datos restringidos al elegir campos.	`campo`, `roles`	—	—
`Resolver`	Clase	Resuelve una consulta combinando datos de uno o más sistemas.	`operacion`	`resolver`	SubsystemAdapter, BatchLoader
`BatchLoader`	Clase	Agrupa en una sola llamada los ids repetidos (por ejemplo, los productos de varios pedidos) para evitar una llamada por ítem.	—	`cargar`	SubsystemAdapter
`SubsystemAdapter`	Clase abstracta	Cliente de lectura hacia un sistema. Lleva su propia credencial y el contexto del usuario.	`idSistema`, `refCredencial`	`llamar`	PrivateSystem, SecretProvider
`OrdersAdapter`, `InventoryAdapter`, `ClientsAdapter`	Clases	Adaptadores hacia orders, inventory y clients. Solo hacen GET.	—	—	OrdersService, InventoryService, ClientsService

Clase	Tipo	Responsabilidad	Atributos relevantes	Operaciones	Depende de
`QueryLimits`	Clase de política	Profundidad, costo, tamaño de página y plazo máximo de una consulta.	`maxProfundidad`, `maxCosto`, `maxPagina`, `plazoMs`	—	—
`ClaseAcceso`, `ClaveLimite`	Enumeraciones	`ClaseAcceso`: anonima, firma_proveedor, autenticada, administrativa. `ClaveLimite`: origen_ip, cuenta.	—	—	—

5.5 Tabla de relaciones

ID	Origen	Destino	Mult. origen	Mult. destino	Tipo	Regla
R5-01	`ApiGateway`	`RouteDefinition`	1	1..*	Composición	Cada definición pertenece a un solo gateway y no existe fuera de él. Un gateway sin rutas no deja pasar nada, así que tiene al menos una.
R5-02	`RouteDefinition`	`AuthenticationPolicy`	*	1	Asociación	Toda ruta tiene exactamente una forma de autenticarse (anónima o con token). La política se **comparte**: la misma `BearerAuthentication` sirve a todas las rutas autenticadas, y así hay una sola implementación que auditar.
R5-03	`RouteDefinition`	`AuthorizationPolicy`	*	1	Asociación	Toda ruta tiene un conjunto de roles. Varias rutas comparten el mismo (por ejemplo, `ROLES-ADMIN`), lo que evita copiar listas de roles y que se desalineen.
R5-04	`RouteDefinition`	`RateLimitPolicy`	*	0..1	Asociación	Una ruta puede tener un límite compartido o ninguno. Las rutas administrativas usan el límite por cuenta.
R5-05	`RouteDefinition`	`ServiceTarget`	*	1	Asociación	Toda ruta apunta a exactamente un destino. Un destino atiende a muchas rutas. El destino limita los verbos y prefijos, así que una ruta mal escrita no puede alcanzar `/internal/...`.
R5-06	`ServiceTarget`	`PrivateSystem`	1	1	Asociación	Cada destino corresponde a un sistema de la red privada y cada sistema tiene un solo destino registrado en el gateway.
R5-07	`BearerAuthentication`	`AuthClient`	1	1	Asociación	La autenticación con token necesita un cliente de Auth Service para la introspección.
R5-08	`ApiGateway`	`SecretProvider`	1	1	Asociación	Un gateway usa un solo proveedor de secretos, que entrega `gateway-introspect` (para I01) y un `gateway-to-<sistema>` por cada destino.
R5-09	`PrivateSystem`	`InternalAccessGuard`	1	1	Composición	Cada sistema trae su propia guardia. No hay un sistema que dependa de que el gateway haya filtrado antes.

ID	Origen	Destino	Mult. origen	Mult. destino	Tipo	Regla
R5-10	PrivateSystem	BusinessRules	1	1	Composición	Las reglas de propiedad y estado pertenecen al dueño del dato y no se delegan al gateway.
R5-11	GraphQLGateway	Schema	1	1	Composición	Un solo esquema por gateway GraphQL.
R5-12	GraphQLGateway	Resolver	1	1..*	Composición	Cada consulta expuesta tiene un resolver.
R5-13	GraphQLGateway	QueryLimits	1	1	Composición	Los límites son parte de la configuración del gateway GraphQL.
R5-14	Schema	FieldPolicy	1	0..*	Composición	Los campos restringidos tienen una política y los públicos no necesitan ninguna.
R5-15	Resolver	SubsystemAdapter	*	1..*	Asociación	Un resolver consulta a uno o más sistemas (al menos uno). Un adaptador lo pueden usar varios resolvers.
R5-16	Resolver	BatchLoader	*	0..1	Asociación	Solo los resolvers que repiten ids (productos de varios pedidos) usan agrupación.

Políticas compartidas: las políticas de autenticación, autorización y límites no pertenecen a una sola ruta. Muchas rutas apuntan a la misma política (R5-02 a R5-04). Se comparten porque la regla es la misma (por ejemplo, "solo el administrador") y mantenerla en un solo lugar impide que dos rutas del mismo grupo queden con reglas distintas.

5.6 Políticas nombradas

Autorización por rol

Salen de la columna "Acceso" del Módulo 1 y de la matriz del Módulo 4.

Política	Roles	Descripción	Servicios
ROLES-TODOS	administrador, cajero_virtual, cliente, dueno, encargado_despachos	Cualquier usuario con sesión	S02
ROLES-CLIENTE	cliente	Solo el cliente	S12, S13, S15, S20, S24, S25
ROLES-PEDIDO-LECTURA	administrador, cajero_virtual, cliente, encargado_despachos	Cliente (lo suyo) y personal que atiende pedidos	S14
ROLES-BOLETA	administrador, cajero_virtual, cliente	Cliente (lo suyo), cajero virtual y administrador	S23
ROLES-CATALOGO	administrador, dueno	Administrador y dueño	S07, S08, S09, S10, S11
ROLES-REPORTE	administrador, dueno	Dueño y administrador, para el reporte	S34
ROLES-PEDIDOS-PERSONAL	administrador, cajero_virtual, encargado_despachos	Personal que atiende pedidos	S16
ROLES-DESPACHO	administrador, encargado_despachos	Encargado de despachos y administrador	S17, S18, S19
ROLES-VENTAS	administrador, cajero_virtual	Cajero virtual y administrador	S22
ROLES-ADMIN	administrador	Solo el administrador	S26, S27, S28, S29, S30, S31, S32, S33

Las rutas anónimas usan `NoAuthentication` y no tienen política de autorización. S21 tampoco la tiene en el gateway: su control es la firma, que verifica payments.

Límites de solicitudes

Política	Clave	Límite	Servicios	Motivo
RL-SENSIBLE	Por IP	10 por minuto	S01, S03, S04, S05, S35	Frena la fuerza bruta de contraseñas y el envío masivo de correos o mensajes.
RL-CATALOGO	Por IP	120 por minuto	S06	Consulta pública frecuente, con tope para evitar abusos.
RL-WEBHOOK	Por IP de origen	60 por minuto	S21	El volumen esperado de avisos es bajo.

Política	Clave	Límite	Servicios	Motivo
RL-USUARIO	Por cuenta	120 por minuto	S02, S12 a S15, S20, S23 a S25 y las rutas administrativas	Evita que una sola cuenta sature a los sistemas. Si se excede, 429.

5.7 Controles del gateway

Control	Cómo se aplica
Validación de identidad	El gateway pregunta a Auth Service por el token en cada solicitud (I01). No decodifica ni confía en el token por sí mismo. Si Auth Service no responde, 503: no deja pasar sin verificar (falla cerrado).
Autorización por ruta	Cada `RouteDefinition` apunta a una `AuthorizationPolicy` con los roles de la tabla 5.6. Lo no listado, denegado.
Límites de solicitudes	Tabla 5.6. Se responde 429 con `Retry-After`.
Tiempos de espera	10 segundos por destino. 5 segundos para el aviso de pago. 15 segundos para GraphQL. El gateway no reintenta POST, PUT, PATCH ni DELETE para no duplicar efectos. Si el destino no responde, 502 o 503.
Tamaño del cuerpo	64 KB por defecto y 16 KB para el aviso de pago. Más que eso, 413.
Mensajes de error controlados	Siempre `{ error: { codigo, mensaje, requestId } }` (formato del Módulo 1). Nunca trazas, URLs internas ni nombres de campos de la base de datos.
Registros sin secretos	Se registra `requestId`, ruta, verbo, estado, duración, `cuentaId` y decisión. Nunca se registran tokens, contraseñas, cuerpos, cabeceras de secretos ni correos.
Redirección restringida	No existe un reenvío genérico del tipo `/api/{path}`. Cada fila de la lista fija verbo, ruta externa, destino y ruta interna. Además, cada `ServiceTarget` limita los verbos y prefijos que acepta, así que una fila mal escrita no puede llegar a `/internal/...`.
Cabeceras	Descarta `X-Authenticated-*`, `X-Gateway-Secret`, `X-Service-*` y `X-Request-Id` que traiga el navegador. Escribe las suyas.

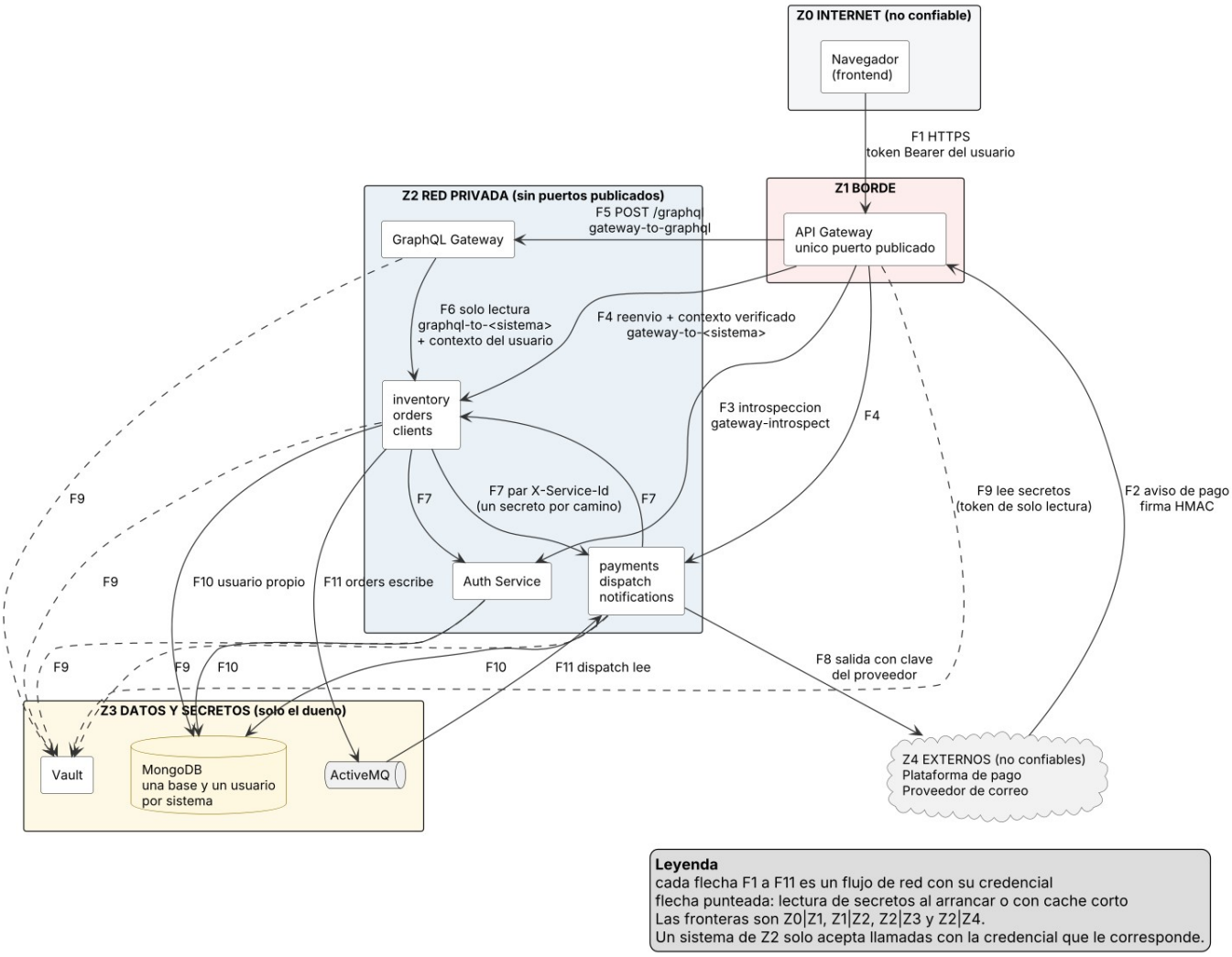
5.8 Qué valida el gateway y qué valida cada sistema

Comprobación	API Gateway	Cada sistema
Que la ruta y el verbo estén en la lista permitida	Sí. Lo que no está en la lista responde 404 (ruta) o 405 (verbo). No hay reenvío genérico.	No lo necesita, pero rechaza rutas internas que no sean para quien llama.
TLS, tamaño del cuerpo y límite de solicitudes	Sí. TLS en el borde y también hacia cada destino (6.12).	Los sistemas atienden solo por TLS. No repiten el límite ni el tamaño.
Identidad del usuario (token)	Sí, con I01 a Auth Service. Sin token válido, 401.	No ven el token. Confían en las cabeceras `X-Authenticated-*` solo si el secreto del gateway coincide.
Rol permitido para la ruta	Sí (primera barrera). Si no está, 403.	Sí (segunda barrera) con `X-Authenticated-Roles`, por si alguien llama sin pasar por el gateway.
Cabeceras de identidad que manda el navegador	Las descarta todas y escribe solo las suyas.	Reciben solo las del gateway.
Que la llamada venga del gateway o de un sistema permitido	Agrega `X-Gateway-Secret` propio del destino.	Sí. `InternalAccessGuard` compara el secreto en tiempo constante y revisa el par llamador–ruta. Si falla, 401 o 403.
Propiedad del recurso (C1)	No. No tiene el dato.	Sí. El sistema filtra por `cuentaId` y responde 404 si no coincide.
Estado del recurso y reglas del negocio (C2, C5, C7)	No.	Sí. Cada sistema aplica las suyas.
Validación del cuerpo y de los campos	Solo tamaño máximo y tipo de contenido.	Sí. Esquema, rangos y formatos.
Firma del aviso de pago (C8)	No. Solo limita tasa, verbo y tamaño.	Sí. payments verifica firma, marca de tiempo, `eventoId` y monto.
Precios, totales, estado de pago e identidad enviados por el navegador	No los interpreta.	Los ignoran: el servidor calcula el precio, el estado de pago lo fija el aviso firmado y la identidad sale del token.
Tiempos de espera y errores controlados	Sí. Plazo por destino y mensajes sin detalles internos.	Sí en sus propias llamadas (por ejemplo, orders a inventory en 3 s).
Registros sin secretos	Sí.	Sí.

Regla general: el gateway comprueba lo que se puede decidir sin conocer el negocio (quién eres, qué rol tienes, si la ruta existe y cuánto llamas). Cada sistema comprueba lo que solo él puede decidir (si el recurso es tuyo, si su estado lo permite y si los datos cumplen las reglas). Ninguna de las dos capas reemplaza a la otra.

5.9 Fronteras de confianza

Diagrama 5.2 - Fronteras de confianza y flujos de red



Fuente editable: `diagramas/diagrama-5-2-fronteras.puml`. Versión vectorial: `diagramas/diagrama-5-2-fronteras.svg`.

Este diagrama muestra **flujos de red**, a diferencia del 5.1, que muestra la estructura de clases. Cada flecha F1 a F11 lleva su credencial.

Frontera	Qué cruza	Controles en la frontera	Qué pasa si falla
Z0 ↔ Z1 Internet y borde	Navegador → API Gateway (F1). Plataforma de pago → API Gateway (F2).	TLS. Solo el puerto del gateway está publicado. Lista de rutas permitidas, límite de solicitudes, tamaño máximo y token.	Una solicitud mal formada o sin credencial se corta en el borde con 400, 401, 404, 405 o 429.
Z1 ↔ Z2 Borde y red privada	API Gateway → Auth Service (F3), sistemas (F4) y GraphQL Gateway (F5).	El gateway agrega el secreto del destino y el contexto verificado. Cada sistema vuelve a comprobar secreto, rol, propiedad y estado.	Si el gateway fuera vulnerado, un sistema igual rechaza lo que no cumpla sus reglas. Los secretos son por destino: el robo de uno no abre los demás.
Dentro de Z2 Sistema a sistema	F6 (GraphQL → sistemas) y F7.	Par `X-Service-Id` y secreto por camino. Cada ruta interna acepta solo a su llamador. GraphQL solo hace GET.	Un sistema comprometido solo alcanza las rutas de sus propias integraciones.
Z2 ↔ Z3 Sistemas y datos o secretos	Sistemas → Vault (F9), MongoDB (F10) y ActiveMQ (F11).	Cada sistema tiene su usuario de base de datos, su política de solo lectura en Vault y su usuario del broker. Ni el gateway ni el navegador tienen credenciales de base de datos.	Un sistema solo lee su base. Si Vault no responde, ver 5.10.
Z2 ↔ Z4 Sistemas y externos	payments → plataforma de pago y notifications → proveedor de correo (F8). La plataforma entrega avisos por Z0 ↔ Z1 (F2).	Clave del proveedor guardada en Vault y usada solo por el sistema dueño de esa integración. El aviso se acepta solo con firma.	Un aviso falsificado no pasa la firma. La clave del proveedor nunca llega al navegador.

Identificación de componentes: API Gateway (Z1), Auth Service y GraphQL Gateway (Z2), administrador de secretos Vault (Z3), sistemas privados (Z2) con sus bases de datos (Z3). No hay servicios de analítica ni de alta demanda (5.13).

5.10 Cómo se impide el acceso directo a los sistemas internos

Hay cuatro capas, y cada una por sí sola ya impide un acceso directo desde Internet:

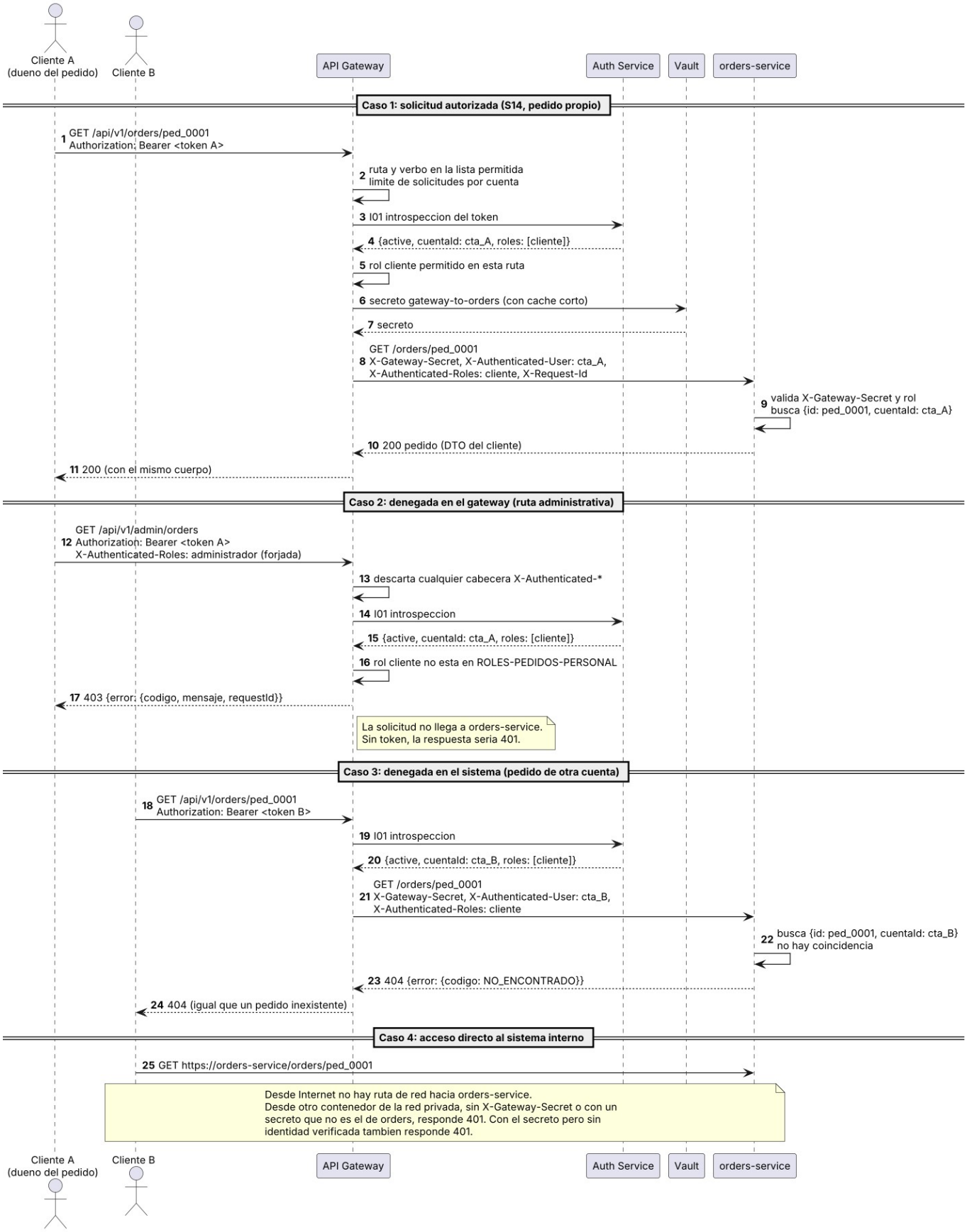
1. **Red.** Los sistemas, Vault, MongoDB y ActiveMQ no publican puertos. Solo el API Gateway tiene uno. Desde Internet no hay ruta de red hacia ellos.
2. **Lista de rutas.** El gateway solo reenvía las filas de su lista y nunca `/internal/...`. Lo que no está listado, 404.
3. **Secreto por destino.** Cada sistema exige `X-Gateway-Secret` con el secreto que le corresponde solo a él (`gateway-to-<sistema>`), o un par de servicio permitido para esa ruta. Alguien dentro de la red privada sin el secreto recibe 401.
4. **Identidad y negocio.** Aunque tuviera el secreto, el sistema revisa el rol y la propiedad con la identidad propagada. Sin identidad verificada, 401.

Si Vault no responde: un sistema en ejecución sigue con los secretos que cargó al arrancar, y el gateway mantiene el último valor hasta 15 minutos. Pasado ese plazo el gateway responde 503 y no deja pasar nada. Un sistema que no puede leer sus secretos al arrancar no arranca (detalle en 6.10). No hay valores por defecto.

Si una clave se filtra: se revoca y reemplaza solo esa referencia (por ejemplo, `gateway-to-orders`) y los demás caminos siguen intactos. El detalle de rotación y revocación está en el inventario de secretos del Módulo 6.

5.11 Recorrido de una solicitud autorizada y de las denegadas

Diagrama 5.3 - Solicitud autorizada y solicitudes denegadas



Fuente editable: `diagramas/diagrama-5-3-secuencia-acceso.puml`. Versión vectorial: `diagramas/diagrama-5-3-secuencia-acceso.svg`.

Caso	Qué ocurre	Dónde se decide	Resultado
1. Cliente A consulta su pedido (S14).	El gateway encuentra la ruta, valida el token con Auth Service, comprueba el rol y reenvía con el secreto de orders y el contexto verificado. Orders busca el pedido por `id` y `cuentaId`.	Gateway: token y rol. Orders: secreto, rol y propiedad.	200
2. Cliente A pide `/api/v1/admin/orders` y envía una cabecera de rol falsa.	El gateway descarta la cabecera, obtiene el rol real y ve que `cliente` no está en `ROLES-PEDIDOS-PERSONAL`.	Gateway: autorización por ruta.	403. No llega a orders.
3. Cliente B consulta el pedido de A.	El rol sirve para la ruta, así que el gateway reenvía. Orders no encuentra un pedido con ese `id` y la cuenta de B.	Orders: propiedad (C1).	404, igual que un pedido inexistente.
4. Alguien intenta llamar a orders directamente.	Desde Internet no hay ruta. Dentro de la red privada, sin el secreto o con el de otro sistema, 401.	Red y `InternalAccessGuard`.	Sin acceso.

5.12 GraphQL Gateway en la red privada

- **Para qué:** entregar al frontend consultas que combinan sistemas sin exponerlos. Por ejemplo, `resumenPedido` une el estado y los ítems de orders con el nombre, la imagen y la descripción vigentes de inventory, y `miCuenta` une el perfil de clients con los últimos pedidos de orders. El esquema, los resolvers y la matriz de autorización final son el Módulo 8.
- **Cómo se llega:** el navegador llama a `POST /api/v1/graphql` en el API Gateway. El gateway autentica, aplica `ROLES-TODOS` y reenvía con el secreto `gateway-to-graphql` y el contexto verificado. El GraphQL Gateway no vuelve a autenticar al usuario.
- **Qué hace con cada sistema:** llama a los sistemas por sus APIs de lectura con su propio par de credenciales (`graphql-to-orders`, `graphql-to-inventory` y `graphql-to-clients`) y con el contexto del usuario. Los sistemas siguen aplicando la propiedad: si pide un pedido ajeno, orders responde 404 aunque el que llama sea el GraphQL Gateway.
- **Qué no hace:** no tiene mutaciones, no accede a bases de datos y no decide reglas de negocio. Las escrituras siguen siendo los servicios REST.
- **Seguridad por campo:** cada campo restringido tiene una `FieldPolicy`. Elegir un campo en la consulta no lo autoriza.
- **Límites:** `QueryLimits` fija la profundidad, el costo, el tamaño de página y el plazo. `BatchLoader` evita una llamada por ítem.
- **Si un sistema no responde:** la consulta devuelve lo que sí pudo resolver y marca los campos que faltan en `errors`, sin inventar valores (Módulo 8).

5.13 Analítica y gRPC

No se modelan un servicio de analítica ni un cliente gRPC en esta etapa. Es una decisión preliminar, basada en la carga del caso: un solo local, el reporte de ventas (S34) cubre como máximo 92 días con un volumen de ventas diario de orden bajo, y no hay un flujo que requiera streaming (el despacho usa una cola ActiveMQ). La decisión se formaliza con un ADR y una medición de carga en el Módulo 9. Si la medición mostrara lo contrario, se agregarían un servicio privado de analítica y su adaptador, que el GraphQL Gateway consultaría como cualquier otro subsistema.

5.14 Qué se aprovecha del gateway de ejemplo y qué se corrige

El gateway del curso muestra el flujo correcto: validar el token con Auth Service y reenviar con un secreto de Vault. Para esta entrega se corrige lo que quedaba abierto:

En el ejemplo	En esta propuesta
Reenvío genérico `/api/{path}` a un solo destino fijo.	Lista de rutas con verbo, ruta externa, destino y ruta interna, y destinos con verbos y prefijos limitados.
Un solo secreto de backend para todo.	Un secreto por destino y por camino.
Consulta de Vault en cada solicitud.	Caché corto del secreto, con falla cerrada si no hay valor.
Pasa `X-Authenticated-Client` leyendo un campo que la identidad no trae (falla con error interno), y escribe el tipo de contenido con un nombre de cabecera incorrecto.	Contexto de identidad definido (`cuentaId`, correo, roles) y cabeceras con sus nombres correctos.
Mensajes de error con detalle libre.	`ErrorMapper` con código, mensaje y `requestId`.
Sin límites de solicitudes ni de tamaño.	Políticas de límite y tamaño máximo por ruta.

5.15 Control antes de avanzar

Pregunta	Respuesta
¿Se entiende cómo se impide el acceso directo no autorizado a los sistemas internos?	Sí. 5.10 explica las cuatro capas (red, lista de rutas, secreto por destino, identidad y negocio) y el caso 4 de 5.11 lo ejemplifica.

Pregunta	Respuesta
¿El diagrama distingue configuración de rutas, políticas y secretos?	Sí. En el 5.1, `RouteDefinition` y `ServiceTarget` son configuración; las políticas son clases aparte compartidas; y `SecretProvider` con `SecretRef` está en su propio paquete.
¿La elección de uno o varios gateways tiene fundamento?	Sí. 5.2 justifica un API Gateway de borde y un GraphQL Gateway privado, y descarta las alternativas con motivos concretos.

Módulo 6 — Rutas y secretos de integración

Todo lo que aparece como credencial en este módulo es una **referencia simbólica**. No hay valores reales en el informe, ni en el repositorio, ni en las capturas.

6.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Lista explícita de rutas y verbos, vinculada con un servicio y una política del Módulo 4.	6.3 (36 filas: 35 servicios y la ruta de GraphQL).
2. Ruta externa, transformación y ruta interna. Sin endpoints internos publicados por omisión.	6.3, 6.4 y 6.5.
3. Credencial del usuario distinta de la credencial entre sistemas. Cómo se conserva una identidad confiable para comprobar propiedad.	6.7.
4. Secreto, certificado o identidad de servicio por integración, con justificación.	6.8.
5. Responsable, almacenamiento, permisos mínimos, rotación y revocación.	6.9 y 6.10.
Matriz de rutas, operaciones GraphQL y gRPC.	6.3, 6.6 y 6.11.
Inventario de secretos.	6.9.
Decisiones de seguridad (un secreto por camino, credencial de payments mínima, base de datos propia, cabeceras, TLS).	6.12.
Qué pasa si una clave se filtra o Vault no responde.	6.10.

6.2 Reglas de la lista de rutas

1. **Lista cerrada.** El gateway reenvía solo lo que está en la tabla 6.3 (verbo y ruta exactos). Otra ruta responde 404, y otro verbo en una ruta existente, 405.
2. **Un destino por ruta.** Cada fila fija el sistema y la ruta interna. No hay reenvío genérico por prefijo.
3. **Nada interno por omisión.** El gateway no publica `/internal/...`, la documentación automática de los sistemas (`/docs`, `/openapi.json`), `/metrics` ni los puertos de Vault, MongoDB y la consola de ActiveMQ (6.5).
4. **Política vinculada.** Cada fila indica el nombre de la política de roles (M5 5.6) y las condiciones del Módulo 4 que verifica el sistema dueño del dato.
5. **Credencial por destino.** La referencia de la última columna es la que el gateway usa para ese destino. Ninguna fila usa la credencial de otro.

Controles comunes

Control	Regla
Plazo de espera	`T10`: 10 s. `T5`: 5 s (aviso de pago). `T15`: 15 s (GraphQL). Sin respuesta: 502 o 503. El gateway no reintenta ninguna solicitud.
Tamaño	`B64`: cuerpo máximo de 64 KB. `B16`: 16 KB (aviso de pago y GraphQL). URL máxima de 2 KB. Más que eso, 413 o 414.
Límite de solicitudes	`RL-SENSIBLE`, `RL-CATALOGO`, `RL-WEBHOOK` y `RL-USUARIO` (M5 5.6). Excedido: 429.
CORS	Solo se acepta el origen del frontend. Las rutas con firma del proveedor (R21) no responden CORS.
Errores	`{ error: { codigo, mensaje, requestId } }` (M1 y M5 5.7). Sin trazas ni URLs internas.

6.3 Matriz de rutas del API Gateway

Los números de fila coinciden con los de los servicios del Módulo 1 (R01 es S01). Los roles se abrevian con el nombre de la política (tabla de M5 5.6).

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Referencia de credencial
R01	API Gateway	POST	`/api/v1/auth/login`	Anónimo (`NoAuthentication`). M4: C4, C9	`auth-service`: POST `/auth/login`	`RL-SENSIBLE`; plazo 10 s; cuerpo máx. 64 KB; límite de intentos fallidos por cuenta en auth (C9)	`gateway-to-auth`

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Referencia de credencial
R02	API Gateway	POST	/api/v1/auth/logout	Autenticado (I01). ROLES-TODOS: administrador, cajero_virtual, cliente, dueno, encargado_despachos	auth-service: POST /auth/logout	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-auth
R03	API Gateway	POST	/api/v1/auth/register	Anónimo (NoAuthentication). M4: C9	clients-service: POST /clients/register	RL-SENSIBLE; plazo 10 s; cuerpo máx. 64 KB	gateway-to-clients
R04	API Gateway	POST	/api/v1/auth/verify-email	Anónimo (NoAuthentication). M4: C10	auth-service: POST /auth/verify-email	RL-SENSIBLE; plazo 10 s; cuerpo máx. 64 KB	gateway-to-auth
R05	API Gateway	POST	/api/v1/auth/resend-verification	Anónimo (NoAuthentication). M4: C9, C10	auth-service: POST /auth/resend-verification	RL-SENSIBLE; plazo 10 s; cuerpo máx. 64 KB	gateway-to-auth
R06	API Gateway	GET	/api/v1/products	Anónimo (NoAuthentication). M4: C12	inventory-service: GET /products	RL-CATALOGO; plazo 10 s; cuerpo máx. 64 KB; respuesta solo con campos públicos (C12)	gateway-to-inventory
R07	API Gateway	GET	/api/v1/admin/products	Autenticado (I01). ROLES-CATALOGO: administrador, dueno	inventory-service: GET /admin/products	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-inventory
R08	API Gateway	POST	/api/v1/products	Autenticado (I01). ROLES-CATALOGO: administrador, dueno	inventory-service: POST /products	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-inventory
R09	API Gateway	PUT	/api/v1/products/{id}	Autenticado (I01). ROLES-CATALOGO: administrador, dueno	inventory-service: PUT /products/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-inventory
R10	API Gateway	DELETE	/api/v1/products/{id}	Autenticado (I01). ROLES-CATALOGO: administrador, dueno. M4: C13	inventory-service: DELETE /products/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB; Baja lógica (C13)	gateway-to-inventory
R11	API Gateway	PUT	/api/v1/products/{id}/stock	Autenticado (I01). ROLES-CATALOGO: administrador, dueno	inventory-service: PUT /products/{id}/stock	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-inventory
R12	API Gateway	POST	/api/v1/orders	Autenticado (I01). ROLES-CLIENTE: cliente. M4: C4	orders-service: POST /orders	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB; Pasa Idempotency-Key	gateway-to-orders
R13	API Gateway	GET	/api/v1/orders	Autenticado (I01). ROLES-CLIENTE: cliente. M4: C1	orders-service: GET /orders	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-orders
R14	API Gateway	GET	/api/v1/orders/{id}	Autenticado (I01). ROLES-PEDIDO-LECTURA: administrador, cajero_virtual, cliente, encargado_despachos. M4: C1, C3	orders-service: GET /orders/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-orders
R15	API Gateway	POST	/api/v1/orders/{id}/cancel	Autenticado (I01). ROLES-CLIENTE: cliente. M4: C1, C2	orders-service: POST /orders/{id}/cancel	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-orders
R16	API Gateway	GET	/api/v1/admin/orders	Autenticado (I01). ROLES-PEDIDOS-PERSONAL: administrador, cajero_virtual, encargado_despachos. M4: C3	orders-service: GET /admin/orders	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-orders

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Referencia de credencial
R17	API Gateway	PATCH	/api/v1/admin/orders/{id}/status	Autenticado (I01). ROLES-DESPACHO: administrador, encargado_despachos. M4: C7	orders-service: PATCH /admin/orders/{id}/status	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB; Nunca pasa a pagado (C7)	gateway-to-orders
R18	API Gateway	GET	/api/v1/admin/dispatch-orders	Autenticado (I01). ROLES-DESPACHO: administrador, encargado_despachos	dispatch-service: GET /admin/dispatch-orders	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-dispatch
R19	API Gateway	GET	/api/v1/admin/dispatch-orders/{pedidoId}/print	Autenticado (I01). ROLES-DESPACHO: administrador, encargado_despachos	dispatch-service: GET /admin/dispatch-orders/{pedidoId}/print	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-dispatch
R20	API Gateway	POST	/api/v1/orders/{id}/payments	Autenticado (I01). ROLES-CLIENTE: cliente. M4: C1, C2	payments-service: POST /orders/{id}/payments	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-payments
R21	API Gateway	POST	/api/v1/payments/webhook	Firma del proveedor (C8). Sin token de usuario. M4: C8	payments-service: POST /payments/webhook	RL-WEBHOOK; plazo 5 s; cuerpo máx. 16 KB; Pasa X-Timestamp, X-Signature y el cuerpo sin modificar; sin CORS ni token de usuario	gateway-to-payments
R22	API Gateway	GET	/api/v1/admin/sales	Autenticado (I01). ROLES-VENTAS: administrador, cajero_virtual. M4: C6	payments-service: GET /admin/sales	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-payments
R23	API Gateway	GET	/api/v1/orders/{id}/receipt	Autenticado (I01). ROLES-BOLETA: administrador, cajero_virtual, cliente. M4: C1, C2	payments-service: GET /orders/{id}/receipt	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-payments
R24	API Gateway	GET	/api/v1/clients/me	Autenticado (I01). ROLES-CLIENTE: cliente. M4: C1	clients-service: GET /clients/me	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-clients
R25	API Gateway	PUT	/api/v1/clients/me	Autenticado (I01). ROLES-CLIENTE: cliente. M4: C1	clients-service: PUT /clients/me	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-clients
R26	API Gateway	POST	/api/v1/admin/clients	Autenticado (I01). ROLES-ADMIN: administrador	clients-service: POST /admin/clients	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-clients
R27	API Gateway	GET	/api/v1/admin/clients	Autenticado (I01). ROLES-ADMIN: administrador. M4: C3	clients-service: GET /admin/clients	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-clients
R28	API Gateway	PUT	/api/v1/admin/clients/{id}	Autenticado (I01). ROLES-ADMIN: administrador	clients-service: PUT /admin/clients/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-clients
R29	API Gateway	DELETE	/api/v1/admin/clients/{id}	Autenticado (I01). ROLES-ADMIN: administrador. M4: C13	clients-service: DELETE /admin/clients/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB; Baja lógica (C13)	gateway-to-clients
R30	API Gateway	POST	/api/v1/admin/users	Autenticado (I01). ROLES-ADMIN: administrador	auth-service: POST /admin/users	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-auth
R31	API Gateway	GET	/api/v1/admin/users	Autenticado (I01). ROLES-ADMIN: administrador	auth-service: GET /admin/users	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-auth
R32	API Gateway	PUT	/api/v1/admin/users/{id}	Autenticado (I01). ROLES-ADMIN: administrador. M4: C5	auth-service: PUT /admin/users/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB	gateway-to-auth
R33	API Gateway	DELETE	/api/v1/admin/users/{id}	Autenticado (I01). ROLES-ADMIN: administrador. M4: C5, C13	auth-service: DELETE /admin/users/{id}	RL-USUARIO; plazo 10 s; cuerpo máx. 64 KB; Baja lógica (C13)	gateway-to-auth

ID	Gateway	Verbo	Ruta externa	Acceso y rol	Sistema y ruta destino	Controles	Referencia de credencial
R34	API Gateway	GET	`/api/v1/admin/reports/sales`	Autenticado (I01). `ROLES-REPORTE`: `administrador`, `dueno`. M4: C3, C6	`payments-service`: GET `/admin/reports/sales`	`RL-USUARIO`; plazo 10 s; cuerpo máx. 64 KB	`gateway-to-payments`
R35	API Gateway	POST	`/api/v1/contact-messages`	Anónimo (`NoAuthentication`). M4: C9	`notifications-service`: POST `/contact-messages`	`RL-SENSIBLE`; plazo 10 s; cuerpo máx. 64 KB	`gateway-to-notifications`
R36	API Gateway → GraphQL Gateway	POST	`/api/v1/graphql`	Autenticado (I01). `ROLES-TODOS`. Cada operación y campo se autoriza dentro del GraphQL Gateway (6.6)	`graphql-gateway`: POST `/graphql`	`RL-USUARIO`; plazo 15 s; cuerpo máx. 16 KB; límites de consulta (M8)	`gateway-to-graphql`

Rutas por sistema destino (suman 35 más la ruta de GraphQL):

Sistema	Rutas
`auth-service`	8
`clients-service`	7
`dispatch-service`	2
`inventory-service`	6
`notifications-service`	1
`orders-service`	6
`payments-service`	5

6.4 Transformación que aplica el gateway

Elemento	Regla
Prefijo	Se quita `/api/v1`. Por ejemplo, `GET /api/v1/orders/{id}` pasa a `GET /orders/{id}`.
Excepción de ruta	R03 (registro desde la web) es `POST /api/v1/auth/register` para el navegador, pero su sistema es clients, que expone `POST /clients/register`. Es la única ruta renombrada.
Cabeceras del navegador	Pasan solo `Content-Type` y `Accept`. Además, `Idempotency-Key` en R12 y `X-Timestamp` y `X-Signature` en R21. Todo lo demás se descarta, en especial `Authorization`, `Cookie`, `X-Authenticated-*`, `X-Gateway-Secret`, `X-Service-*` y `X-Request-Id`.
Cabeceras que agrega	`X-Gateway-Secret` (el del destino), y en rutas autenticadas `X-Authenticated-User` (cuentaId), `X-Authenticated-Email`, `X-Authenticated-Roles` y `X-Authenticated-Session` (id de la sesión, para cerrarla en S02), tomados de la respuesta de auth (I01). Siempre `X-Request-Id`.
Token del usuario	No se reenvía a los sistemas. Los sistemas nunca ven el token ni necesitan validarlo.
Cuerpo y consulta	Se reenvían sin cambios. Cada sistema valida campos, rangos y filtros. El gateway solo limita el tamaño. En R21 el cuerpo pasa exacto, porque la firma se calcula sobre él.
Respuesta	Se devuelve el estado y el cuerpo del sistema. Los errores de red del gateway usan el formato de error del Módulo 1.

6.5 Lo que no se publica

Elemento	Por qué no se publica	Qué ocurre si alguien lo pide
`/internal/...` de cualquier sistema	Solo lo usan otros sistemas con su par de credenciales	Por el gateway, 404. Dentro de la red, 401 sin el par correcto.
`/docs` y `/openapi.json`	Revelan el contrato interno y no son parte de los servicios	404. Se desactivan en cada sistema y en el gateway.
Salud y métricas	Las usa Docker dentro de la red	404 por el gateway.
Vault, MongoDB, ActiveMQ y su consola	Infraestructura de datos y secretos	Sin puerto publicado.
El GraphQL Gateway directamente	Es privado (M5 5.12)	Solo se llega por R36.

6.6 GraphQL: operaciones, campos y subsistemas

R36 no autoriza a todas las operaciones: el API Gateway solo comprueba que haya sesión (`ROLES-TODOS`). Cada operación, recurso y campo se autoriza dentro del GraphQL Gateway con `FieldPolicy`, y los sistemas siguen comprobando la propiedad. Todas son consultas de lectura y **no hay mutaciones** (las escrituras siguen siendo REST).

ID	Operación	Tipo	Quién puede	Campos autorizados	Resolver	Subsistemas consultados
G01	`resumenPedido(id: ID!)`	Query	`ROLES-PEDIDO-LECTURA`. El cliente solo si el pedido es suyo (C1). El dueño no tiene acceso (C3).	Cliente dueño: `id`, `estado`, `estadoPago`, `total`, `creadoEn`, `direccionEntrega`, `historialEstados`, `items { cantidad precioUnitario producto { nombre categoria descripcion precio imagen disponible } }`. Personal: además `clienteNombre`. Nadie: RUN, correo, teléfono, fecha de nacimiento, sexo, stock.	`resolverResumenPedido` con `BatchLoader` de productos	`orders-service`: GET `/orders/{id}` (`graphql-to-orders`, con el contexto del usuario). `inventory-service`: GET `/products?ids=...` (`graphql-to-inventory`), una sola llamada para todos los ítems
G02	`miCuenta(limitePedidos: Int = 5)`	Query	`ROLES-CLIENTE`. Solo la cuenta propagada (C1).	`perfil { nombreCompleto correo telefono direccion { calle comuna provincia region } }` y `pedidos { id estado total creadoEn }`. El esquema no incluye `run`, `fechaNacimiento` ni `sexo`.	`resolverMiCuenta`	`clients-service`: GET `/clients/me` (`graphql-to-clients`). `orders-service`: GET `/orders?limit=` (`graphql-to-orders`)

Reglas comunes: la introspección del esquema se desactiva fuera del desarrollo, una operación no listada no existe en el esquema, y elegir un campo no lo autoriza (M5 5.12). El esquema completo, los límites de profundidad y costo, y la matriz final de autorización están en el Módulo 8.

6.7 Credencial del usuario y credencial entre sistemas

	Credencial del usuario	Credencial entre sistemas
Qué es	Token Bearer opaco de 256 bits, emitido por auth al iniciar sesión (S01), con vigencia de 900 s (M1)	`X-Gateway-Secret` por destino, o el par `X-Service-Id` y `X-Service-Secret` por camino
Quién la tiene	El navegador	Los componentes que la guardan en memoria desde Vault
Qué prueba	Qué usuario actúa y con qué roles	Qué componente llama
Quién la valida	Solo el API Gateway, preguntando a auth (I01)	El sistema que recibe
Cuánto dura	900 s o hasta cerrar sesión (S02)	Hasta la próxima rotación
Dónde viaja	`Authorization` hacia el gateway. No pasa a ningún sistema	Cabeceras dentro de la red privada, siempre por TLS. Nunca en la ruta ni en parámetros de consulta

Cómo se conserva la identidad confiable. El gateway deja en la solicitud el `X-Authenticated-User` que auth confirmó, y los sistemas lo creen solo si el secreto del gateway coincide. Cuando un sistema llama a otro en nombre del usuario, reenvía esa misma identidad, y el sistema que tiene el dato la vuelve a verificar contra el recurso:

Flujo	Cómo llega la identidad	Quién comprueba la propiedad
S14 consultar mi pedido	Gateway a orders con `X-Authenticated-User`	Orders filtra por `cuentaId` (C1).
S12 crear pedido	Gateway a orders. Orders pide a clients el perfil de esa cuenta (I07)	Clients entrega solo el de la cuenta indicada.
S20 iniciar pago	Gateway a payments con `X-Authenticated-User`. Payments consulta a orders con `cuentaId` (I11)	Orders responde 404 si el pedido no es de esa cuenta (C1).
S21 aviso de pago	No hay usuario. El actor es la plataforma, identificada por la firma	Payments valida firma, marca de tiempo y monto. Orders valida estado, vencimiento y monto en I12.
R36 consulta GraphQL	Gateway al GraphQL Gateway con el contexto. GraphQL lo reenvía a cada sistema	Cada sistema aplica C1, aunque llame GraphQL.
Tarea interna (por ejemplo, el vencimiento de pedidos)	No hay usuario	Corre dentro del propio sistema, sin llamada externa.

6.8 Mecanismo de cada integración

Se usa **secreto estático por par** sobre **TLS**, y no certificados de cliente (mTLS) ni tokens de corta vida entre sistemas. Razón: el alcance del caso son nueve componentes en una sola red privada, y gestionar certificados de cliente por par y un emisor de tokens añade piezas que hay que operar y que no cambian a quién se puede llamar. El riesgo de un secreto estático se reduce con: un valor distinto por camino, TLS con comprobación del nombre del servidor, lista de rutas permitidas por llamador (M4 4.10), rotación con doble valor y revocación. Si en el futuro hubiera más sistemas o equipos, el siguiente paso es mTLS.

Integración	Camino	Mecanismo	Referencia	Justificación
I01	`api-gateway` → `auth-service`	Identidad de servicio con secreto de par	`gateway-introspect`	Es una llamada de sistema a sistema con una ruta única y fija.
I02	`api-gateway` → sistemas y GraphQL Gateway	Secreto compartido por destino	`gateway-to-<destino>`	Un secreto por destino. El robo de uno no abre los demás.
I03 a I05	`clients-service` → `auth-service`	Identidad de servicio con secreto de par	`clients-to-auth`	Par exclusivo. Auth limita las rutas y los roles permitidos.
I06	`auth-service` → `notifications-service`	Identidad de servicio con secreto de par	`auth-to-notifications`	Notifications permite solo la plantilla `verificacion` para este par.
I07	`orders-service` → `clients-service`	Identidad de servicio con secreto de par	`orders-to-clients`	Solo lectura y sin datos sensibles.
I08 a I10	`orders-service` → `inventory-service`	Identidad de servicio con secreto de par	`orders-to-inventory`	Solo reservas. No cambia precio ni stock por otra vía.
I11 e I12	`payments-service` → `orders-service`	Identidad de servicio con secreto de par	`payments-to-orders`	Es el único par aceptado para confirmar un pago. Dos rutas en una sola credencial, porque ambas pertenecen al mismo flujo y orders ya limita cada ruta por llamador.
I13	`payments-service` → `notifications-service`	Identidad de servicio con secreto de par	`payments-to-notifications`	Notifications permite solo la plantilla `boleta` para este par.
I14	`orders-service` y `dispatch-service` ↔ ActiveMQ	Usuario y contraseña de STOMP, uno por rol	`activemq-orders-writer` y `activemq-dispatch-reader`	El broker no tiene un mecanismo más simple de identidad. Los permisos de escritura y lectura quedan separados.
I15	Plataforma de pago → `payments-service`	Firma HMAC-SHA256 con marca de tiempo	`payments-webhook-secret`	No hay token de usuario. La firma identifica al emisor y la marca de tiempo (5 minutos) y el `eventoId` impiden repeticiones.
I16	`payments-service` → plataforma de pago	Clave de API del proveedor	`payments-provider-key`	Lo exige el proveedor.
I17	`notifications-service` → proveedor de correo	Clave de API del proveedor	`mail-provider-key`	Lo exige el proveedor.
I18	Todos los componentes → Vault	Token de Vault con política de solo lectura	`vault-token-<componente>`	Cada componente lee únicamente lo suyo (tabla 6.9).
Consultas GraphQL	`graphql-gateway` → orders, inventory y clients	Identidad de servicio con secreto de par	`graphql-to-<sistema>`	Solo GET. El contexto del usuario viaja en cabeceras y cada sistema vuelve a comprobar la propiedad.
Base de datos	Cada sistema → su base de MongoDB	Usuario y contraseña de la base	`db-<sistema>`	Permisos solo sobre la base propia. Ni el navegador ni el gateway la conocen.

6.9 Inventario de secretos

Hay 44 referencias. Ninguna tiene valor en este documento.

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación segura	Alcance (permisos)	Rotación	Revocación
`gateway-to-auth`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, auth-service	El gateway se identifica ante auth. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-auth`	Solo sirve en rutas de negocio de auth. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-to-clients`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, clients-service	El gateway se identifica ante clients. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-clients`	Solo sirve en rutas de negocio de clients. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación segura	Alcance (permisos)	Rotación	Revocación
`gateway-to-inventory`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, inventory-service	El gateway se identifica ante inventory. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-inventory`	Solo sirve en rutas de negocio de inventory. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-to-orders`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, orders-service	El gateway se identifica ante orders. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-orders`	Solo sirve en rutas de negocio de orders. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-to-payments`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, payments-service	El gateway se identifica ante payments. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-payments`	Solo sirve en rutas de negocio de payments. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-to-dispatch`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, dispatch-service	El gateway se identifica ante dispatch. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-dispatch`	Solo sirve en rutas de negocio de dispatch. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-to-notifications`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, notifications-service	El gateway se identifica ante notifications. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-notifications`	Solo sirve en rutas de negocio de notifications. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-to-graphql`	Secreto compartido estático (`X-Gateway-Secret`)	Responsable de infraestructura	api-gateway, graphql-gateway	El gateway se identifica ante graphql. Cada destino tiene un valor distinto	Vault `secret/data/services/gateway-to-graphql`	Solo sirve en rutas de negocio de graphql. No abre `/internal/...` ni otro sistema	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`gateway-introspect`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	api-gateway, auth-service	Validar tokens de usuario (I01)	Vault `secret/data/services/gateway-introspect`	Solo `POST /internal/introspect`. No crea cuentas ni sesiones	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`clients-to-auth`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	clients-service, auth-service	Crear, consultar y dar de baja cuentas (I03, I04, I05)	Vault `secret/data/services/clients-to-auth`	Solo las rutas de I03 a I05 y solo sobre cuentas de rol `cliente` (M4 4.10)	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`auth-to-notifications`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	auth-service, notifications-service	Enviar el correo de verificación (I06)	Vault `secret/data/services/auth-to-notifications`	Solo `POST /internal/emails` con plantilla `verificacion`	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`orders-to-clients`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	orders-service, clients-service	Leer nombre, teléfono y domicilio de una cuenta (I07)	Vault `secret/data/services/orders-to-clients`	Solo `GET /internal/clients/by-account/{cuentaId}`. Sin RUN ni fecha de nacimiento	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación segura	Alcance (permisos)	Rotación	Revocación
`orders-to-inventory`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	orders-service, inventory-service	Reservar, confirmar y liberar stock (I08, I09, I10)	Vault `secret/data/services/orders-to-inventory`	Solo `/internal/stock/reservations...`. No cambia precio ni stock por otra vía	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`payments-to-orders`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	payments-service, orders-service	Consultar si un pedido es pagable y confirmar el pago (I11, I12)	Vault `secret/data/services/payments-to-orders`	Solo `GET .../payable` y `POST .../payment-confirmation`. Es la única credencial que puede llevar un pedido a `pagado`	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`payments-to-notifications`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	payments-service, notifications-service	Enviar la boleta por correo (I13)	Vault `secret/data/services/payments-to-notifications`	Solo `POST /internal/emails` con plantilla `boleta`	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`graphql-to-orders`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	graphql-gateway, orders-service	Leer pedidos para las agregaciones	Vault `secret/data/services/graphql-to-orders`	Solo `GET` de `/orders` y `/orders/{id}`, con el contexto del usuario	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`graphql-to-inventory`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	graphql-gateway, inventory-service	Leer el catálogo público para las agregaciones	Vault `secret/data/services/graphql-to-inventory`	Solo `GET /products`, campos públicos (C12)	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`graphql-to-clients`	Par de servicio (`X-Service-Id` y `X-Service-Secret`)	Responsable de infraestructura	graphql-gateway, clients-service	Leer el perfil propio para las agregaciones	Vault `secret/data/services/graphql-to-clients`	Solo `GET /clients/me` de la cuenta propagada	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se borra el valor en Vault y se emite uno nuevo. Los componentes lo toman en 5 minutos o al reiniciar
`payments-webhook-secret`	Clave HMAC-SHA256 compartida	Plataforma de pago (en el laboratorio, su adaptador propio, con valor generado por el responsable de infraestructura)	payments-service, plataforma de pago	Firmar y verificar el aviso de pago (I15, C8)	Vault `secret/data/payments/webhook`	Solo verifica avisos. No permite crear cobros ni llamar a otras rutas	Cada 90 días. Doble valor durante 10 minutos	Se borra y se reemplaza. Los avisos firmados con el valor anterior se rechazan al terminar la ventana
`payments-provider-key`	Clave de API del proveedor	Plataforma de pago	payments-service	Crear cobros (I16)	Vault `secret/data/payments/provider`	Solo crear cobros. Sin permiso para reembolsos ni para consultar otras cuentas	Cada 180 días o según el proveedor	Se revoca en el panel del proveedor y se guarda la nueva en Vault
`mail-provider-key`	Clave de API del proveedor	Proveedor de correo	notifications-service	Validar y enviar correos (I17)	Vault `secret/data/notifications/mail`	Solo envío y validación desde el dominio de la pizzería	Cada 180 días o según el proveedor	Se revoca en el panel del proveedor y se guarda la nueva en Vault
`activemq-orders-writer`	Usuario y contraseña de STOMP	Responsable de infraestructura	orders-service	Publicar órdenes de despacho (I14)	Vault `secret/data/orders/activemq`	Solo SEND sobre `/queue/cola_pedidos`	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña en el broker y en Vault
`activemq-dispatch-reader`	Usuario y contraseña de STOMP	Responsable de infraestructura	dispatch-service	Recibir órdenes de despacho (I14)	Vault `secret/data/dispatch/activemq`	Solo SUBSCRIBE sobre `/queue/cola_pedidos`	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña en el broker y en Vault

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación segura	Alcance (permisos)	Rotación	Revocación
`activemq-admin`	Usuario y contraseña de la consola del broker	Responsable de infraestructura	script de despliegue	Crear la cola y los usuarios del broker	Archivo de despliegue fuera del repositorio. No se monta en ningún contenedor de negocio	Administración del broker. La consola web no se publica	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia en el broker y en el archivo de despliegue
`db-auth`	Usuario y contraseña de MongoDB	Responsable de infraestructura	auth-service	Acceder a la base de auth	Vault `secret/data/auth/db`	`readWrite` solo sobre la base de auth. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`db-clients`	Usuario y contraseña de MongoDB	Responsable de infraestructura	clients-service	Acceder a la base de clients	Vault `secret/data/clients/db`	`readWrite` solo sobre la base de clients. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`db-inventory`	Usuario y contraseña de MongoDB	Responsable de infraestructura	inventory-service	Acceder a la base de inventory	Vault `secret/data/inventory/db`	`readWrite` solo sobre la base de inventory. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`db-orders`	Usuario y contraseña de MongoDB	Responsable de infraestructura	orders-service	Acceder a la base de orders	Vault `secret/data/orders/db`	`readWrite` solo sobre la base de orders. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`db-payments`	Usuario y contraseña de MongoDB	Responsable de infraestructura	payments-service	Acceder a la base de payments	Vault `secret/data/payments/db`	`readWrite` solo sobre la base de payments. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`db-dispatch`	Usuario y contraseña de MongoDB	Responsable de infraestructura	dispatch-service	Acceder a la base de dispatch	Vault `secret/data/dispatch/db`	`readWrite` solo sobre la base de dispatch. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`db-notifications`	Usuario y contraseña de MongoDB	Responsable de infraestructura	notifications-service	Acceder a la base de notifications	Vault `secret/data/notifications/db`	`readWrite` solo sobre la base de notifications. Sin acceso a las demás ni a tareas de administración	Cada 90 días y ante sospecha. Con doble valor (6.10)	Se cambia la contraseña del usuario en MongoDB y en Vault, y se reinicia el sistema
`mongo-root`	Usuario administrador de MongoDB	Responsable de infraestructura	script de despliegue	Crear las bases y los usuarios `db-<sistema>` una sola vez	Archivo de despliegue fuera del repositorio. No va a Vault ni a ningún contenedor de negocio	Solo en la inicialización	Al terminar cada inicialización	Se cambia en el servidor de base de datos
`vault-token-api-gateway`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	api-gateway	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-graphql-gateway`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	graphql-gateway	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos

Referencia	Tipo	Emisor	Consumidor	Finalidad	Ubicación segura	Alcance (permisos)	Rotación	Revocación
`vault-token-auth-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	auth-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-clients-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	clients-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-inventory-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	inventory-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-orders-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	orders-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-payments-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	payments-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-dispatch-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	dispatch-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-token-notifications-service`	Token de Vault	Vault (lo emite el script de inicialización con la política del componente)	notifications-service	Leer sus secretos al arrancar o al renovarlos (I18)	Archivo montado en el contenedor, fuera del repositorio	Política de solo lectura sobre las rutas que le corresponden (6.9)	Vigencia de 30 días, renovable por el responsable	Se revoca en Vault y se emite uno nuevo. El componente deja de leer secretos
`vault-root-token`	Token raíz y llaves de desbloqueo de Vault	Responsable de infraestructura	script de inicialización	Crear políticas y tokens, y cargar los secretos iniciales	Archivo local fuera del repositorio y de las capturas. En el laboratorio, Vault en modo servidor con almacenamiento persistente	Administración total. No se entrega a ningún componente	Se regenera en cada reinstalación	Se revoca desde Vault y se genera uno nuevo
`tls-ca`	Clave privada de la CA del proyecto	Responsable de infraestructura	script de despliegue	Firmar los certificados de los componentes	Archivo local fuera del repositorio. Solo el certificado público (sin clave) se distribuye	Solo firma. Los componentes confían en su certificado público	Cada 3 años	Se genera una CA nueva y se reemplazan todos los certificados
`tls-<componente>`	Certificado y clave privada TLS	Responsable de infraestructura	api-gateway, graphql-gateway, auth-service, clients-service, inventory-service, orders-service, payments-service, dispatch-service, notifications-service, Vault	Cifrar los tramos HTTP y comprobar el nombre del servidor	Archivos montados en cada contenedor, fuera del repositorio. Un par por componente	Solo el nombre de su componente	Cada año	Se emite uno nuevo con la CA. El anterior se elimina del contenedor

Qué lee cada componente de Vault

Cada componente usa su propio token y su política de solo lectura solo cubre estas rutas. Es el resultado de combinar la columna "Consumidor" del inventario.

Componente	Token	Referencias que puede leer
api-gateway	vault-token-api-gateway	gateway-to-auth, gateway-to-clients, gateway-to-inventory, gateway-to-orders, gateway-to-payments, gateway-to-dispatch, gateway-to-notifications, gateway-to-graphql, gateway-introspect, tls-api-gateway
graphql-gateway	vault-token-graphql-gateway	gateway-to-graphql, graphql-to-orders, graphql-to-inventory, graphql-to-clients, tls-graphql-gateway
auth-service	vault-token-auth-service	gateway-to-auth, gateway-introspect, clients-to-auth, auth-to-notifications, db-auth, tls-auth
clients-service	vault-token-clients-service	gateway-to-clients, clients-to-auth, orders-to-clients, graphql-to-clients, db-clients, tls-clients
inventory-service	vault-token-inventory-service	gateway-to-inventory, orders-to-inventory, graphql-to-inventory, db-inventory, tls-inventory
orders-service	vault-token-orders-service	gateway-to-orders, orders-to-clients, orders-to-inventory, payments-to-orders, graphql-to-orders, activemq-orders-writer, db-orders, tls-orders
payments-service	vault-token-payments-service	gateway-to-payments, payments-to-orders, payments-to-notifications, payments-webhook-secret, payments-provider-key, db-payments, tls-payments
dispatch-service	vault-token-dispatch-service	gateway-to-dispatch, activemq-dispatch-reader, db-dispatch, tls-dispatch
notifications-service	vault-token-notifications-service	gateway-to-notifications, auth-to-notifications, payments-to-notifications, mail-provider-key, db-notifications, tls-notifications

Qué se verificó de las credenciales de base de datos. En Docker Compose, MongoDB corre con autenticación: `mongo-root` se genera al azar en el `.env` local, el contenedor `preparacion` crea un usuario `<sistema>_user` por sistema con el rol `readWrite` solo sobre su base y guarda la clave en Vault (`secret/<sistema>/db`), y las pruebas CMP-DB-01 a CMP-DB-04 comprueban que una conexión sin credenciales es rechazada y que cada sistema es rechazado en las otras seis bases (10.9.11). En el modo local de pruebas MongoDB corre sin usuarios (`MONGO_AUTH=0`); se declara en 10.11.

Un componente solo lee los secretos de los caminos en que participa, como emisor o como receptor. El gateway no lee credenciales de bases de datos ni claves de proveedores. El GraphQL Gateway no lee ninguna base de datos. Cada sistema solo lee la suya.

6.10 Emisión, rotación y revocación

Etapa	Cómo se hace
Emisión	El responsable de infraestructura ejecuta un script de despliegue que genera valores aleatorios de 256 bits, crea las bases y los usuarios, las políticas y los tokens de Vault, y escribe los secretos. El script no imprime valores y no guarda nada en el repositorio.
Entrega a los componentes	Cada componente recibe solo su archivo de token de Vault y lee el resto con él. No hay secretos en variables de entorno del repositorio, en el código ni en imágenes. El repositorio lleva un `.env.example` con nombres de referencia y sin valores.
Rotación	En Vault cada secreto tiene `actual` y `anterior`. Se escribe el valor nuevo como `actual`, el viejo pasa a `anterior`, y quien verifica acepta los dos durante 10 minutos (más que los 5 minutos de caché). Luego se borra `anterior`.
Revocación inmediata	Se borra `anterior` y se escribe un valor nuevo sin ventana. Los componentes lo toman al renovar su caché (hasta 5 minutos) o al reiniciar. Para bases de datos y brokers se cambia primero la contraseña en el servidor.
Responsable	El responsable de infraestructura (M4 4.2). No tiene permisos operativos sobre pedidos, clientes ni ventas.
Registro	Cada rotación y revocación queda anotada con fecha y referencia, nunca con el valor.

Si una clave se filtra

Credencial	Qué permite al atacante	Qué se hace
`gateway-to-<destino>`	Quien lo tenga y esté dentro de la red privada puede hacerse pasar por el gateway ante ese destino, pero el destino aún exige identidad verificada, rol y propiedad. No sirve contra otros destinos.	Rotar solo ese secreto (6.10). Revisar los registros del destino por llamadas sin `X-Request-Id` del gateway.
`gateway-introspect`	Quien lo tenga puede preguntar a auth si un token es válido (y obtener la cuenta y sus roles). No puede crear sesiones.	Rotar. Revisar consultas de introspección fuera del patrón normal.

Credencial	Qué permite al atacante	Qué se hace
Par `<a>-to-<b>`	Quien lo tenga puede llamar las rutas permitidas para ese par y nada más. En el par `payments-to-orders` incluye confirmar pagos, por lo que es el de mayor cuidado.	Rotar el par. En `payments-to-orders`, además revisar los pagos confirmados desde la última rotación contra los avisos firmados registrados (`EventoPago`).
`payments-webhook-secret`	Quien lo tenga puede firmar avisos de pago falsos.	Rotar de inmediato **sin** ventana de doble valor. Revisar los pagos aprobados contra la plataforma de pago real.
`payments-provider-key`, `mail-provider-key`	Uso indebido de la cuenta del proveedor.	Revocar en el panel del proveedor, guardar la nueva y revisar el uso.
`db-<sistema>`	Acceso de lectura y escritura a la base de ese sistema, pero solo desde la red privada.	Cambiar la contraseña en MongoDB y en Vault. Revisar la base.
`activemq-*`	Con el de escritura se pueden publicar órdenes de despacho falsas. Con el de lectura se leen las órdenes (nombre, teléfono y dirección).	Cambiar la contraseña en el broker y en Vault. Revisar la cola.
`vault-token-<componente>`	Lectura de los secretos de ese componente.	Revocar el token y rotar los secretos que ese componente podía leer.
`vault-root-token`	Control total de los secretos.	Rotar **todos** los secretos y regenerar Vault.
`tls-<componente>` o `tls-ca`	Hacerse pasar por ese componente (la CA, por cualquiera).	Reemplazar el certificado (o la CA y todos los certificados).
Token Bearer de un usuario	Actuar como ese usuario hasta que venza o se cierre la sesión.	El usuario cierra sesión (S02) o el administrador da de baja la cuenta. Auth invalida las sesiones.

Si Vault no responde

Situación	Qué pasa	Efecto
Un sistema que arranca	No arranca. Falta un secreto y no hay valores por defecto (I18).	No acepta tráfico hasta que Vault responda.
Un sistema ya en ejecución	Sigue con los secretos que cargó al arrancar. No toma rotaciones hasta que Vault vuelva.	Sin cambios para el usuario.
`api-gateway` en ejecución	Renueva sus secretos cada 5 minutos. Si Vault no responde, mantiene el último valor hasta 15 minutos.	Pasado ese plazo responde 503 y no deja pasar nada (falla cerrado).
`graphql-gateway` en ejecución	Igual que un sistema en ejecución.	Sin cambios para el usuario.
Durante una rotación	Si Vault cae antes de que todos lean el valor nuevo, vale la ventana de doble valor: el receptor acepta el valor anterior 10 minutos.	Si la caída dura más, se restablece Vault primero y se repite la rotación.

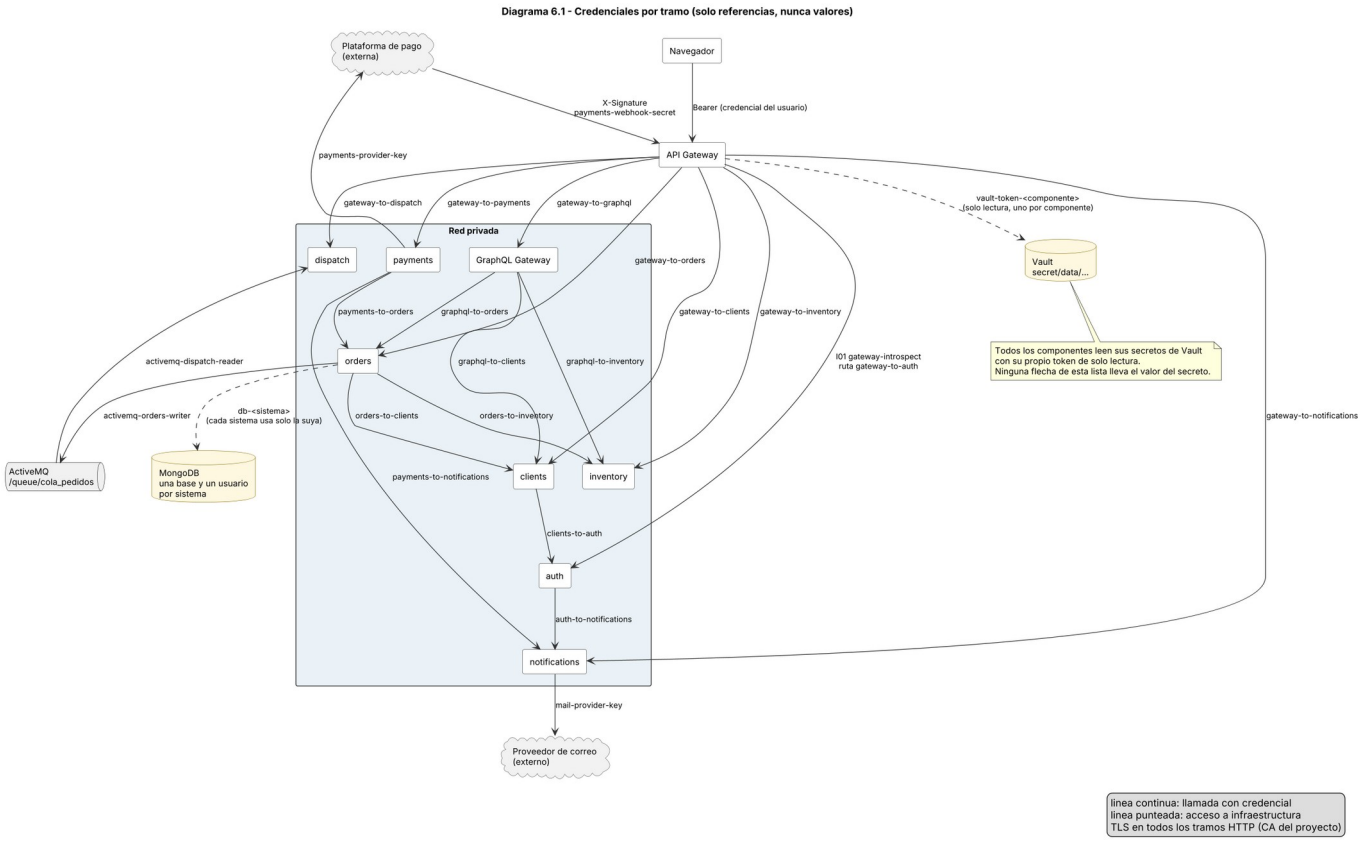
6.11 gRPC

No hay servicio ni método gRPC en esta propuesta (M5 5.13). Por eso la matriz no tiene filas con servicio, mensaje, modalidad, identidad permitida y plazo. Si el ADR del Módulo 9 elige gRPC para una operación, se agregarán sus filas aquí, con la identidad de servicio ya definida y un plazo máximo por método.

6.12 Decisiones de seguridad

Regla de la guía	Cómo se cumple
No asignar el mismo secreto a todos los sistemas.	Hay un secreto distinto por camino: 8 de gateway, 1 de introspección y 9 de sistema a sistema (6.9).
La credencial de payments solo con los permisos de su integración.	`payments-to-orders` solo abre dos rutas de orders y `payments-to-notifications` solo la plantilla `boleta`. `payments-provider-key` solo crea cobros. El gateway no tiene la clave del proveedor.
La credencial de base de datos pertenece al sistema, no al navegador ni al gateway.	`db-<sistema>` la lee únicamente ese sistema, con permisos sobre su base.
El gateway descarta cabeceras de identidad internas y transmite solo contexto verificado.	6.4: descarta `X-Authenticated-*`, `X-Gateway-Secret`, `X-Service-*` y `X-Request-Id` del navegador, y escribe las suyas a partir de I01.
Sin secretos en rutas, parámetros, repositorios, capturas y registros.	Los secretos van solo en cabeceras o en el cuerpo de la conexión. Los registros no guardan cabeceras de credenciales ni cuerpos (M5 5.7). Las capturas del informe se toman con las cabeceras de credenciales ocultas. El repositorio ignora `secrets/`, `.env` y los archivos de claves.
TLS donde viajan credenciales o datos restringidos.	TLS en navegador → gateway, gateway → sistemas, sistema → sistema, sistemas → Vault y hacia los proveedores externos. Los certificados los firma la CA del proyecto y cada cliente comprueba el nombre del servidor. **Excepción:** MongoDB y ActiveMQ (STOMP) quedan dentro de la red privada sin TLS en esta entrega. Se compensa con una red aislada sin puertos publicados, usuarios de mínimo permiso (`db-<sistema>` y `activemq-*`) y consola del broker sin publicar. Se declara como mejora pendiente.

6.13 Diagrama 6.1 — Credenciales por tramo



Fuente editable: `diagramas/diagrama-6-1-credenciales.puml`. Versión vectorial: `diagramas/diagrama-6-1-credenciales.svg`.

Cada flecha lleva la referencia de la credencial que usa. La flecha punteada de Vault representa el acceso de todos los componentes con su propio token. La de MongoDB, el acceso de cada sistema a su base.

6.14 Control antes de avanzar

Pregunta	Respuesta
¿Cada ruta tiene destino, autorización y controles definidos?	Sí. Las 36 filas de 6.3 tienen sistema, ruta interna, política, condiciones del Módulo 4, controles y credencial.
¿Cada credencial tiene un consumidor y una finalidad?	Sí. 6.9 los define para las 44 referencias y la tabla "Qué lee cada componente" los deriva.
¿Está previsto qué hacer si una clave se filtra o Vault no responde?	Sí. 6.10 trae las acciones por tipo de credencial y por situación de Vault.

Módulo 7 — Datos sensibles y secretos del cliente

7.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Inventario de campos públicos, personales, sensibles según el contexto y credenciales, con su motivo.	7.2 y 7.3.
2. Decidir si hay que custodiar secretos del cliente, con fundamento.	7.4.
3. Quién consulta o modifica cada dato, qué devuelve cada servicio y cómo se limita el acceso a lo propio.	7.3 (roles), 7.5 y 7.6.
4. Almacenamiento, transporte, retención, eliminación y exclusión de logs. Hash adaptativo con sal.	7.7, 7.8 y 7.9.
5. Sesiones y tokens, expiración y revocación. Ninguna clave compartida en el navegador.	7.10.
Matriz de datos y credenciales.	7.3.
Recorrido de un dato restringido desde su captura hasta su eliminación.	7.11.
Casos de validación.	7.12.

7.2 Qué se protege y cómo se clasifica

Se distinguen tres cosas que se suelen confundir:

Tipo	Qué es	Ejemplos en La Fornace	Cómo se protege
Dato personal	Información sobre una persona identificable	RUN, nombre, correo, teléfono, fecha de nacimiento, sexo, dirección, historial de pedidos	Se limita quién lo ve, qué servicio lo devuelve y cuánto tiempo se guarda. No es una clave.
Credencial del usuario	Lo que prueba quién es el usuario	Contraseña, token de sesión, token de verificación	Se guarda solo como hash, vence y se revoca. Nunca se devuelve.
Secreto de integración	Lo que prueba quién es un sistema	Secretos del gateway, pares entre sistemas, clave del proveedor de pagos, credenciales de base de datos	Vault, permisos mínimos, rotación (Módulo 6). El navegador nunca lo recibe.

La clasificación del Módulo 1 se mantiene: **Público**, **Operativo**, **Restringido** y **Credencial**. Se agrega **Secreto de integración** (Módulo 6). El rótulo "sensible según el contexto" no es una categoría legal nueva. Marca los datos restringidos que, por lo que revelan en esta aplicación, merecen cuidado extra:

- **Dirección de entrega y teléfono**: juntos, indican dónde vive una persona y cuándo pide comida a su casa.
- **RUN**: es el identificador que se usa para suplantar a alguien.
- **Historial de pedidos**: muestra hábitos y horarios.

No se recogen datos de salud, origen, creencias ni datos biométricos. `fechaNacimiento` y `sexo` son personales, y se piden porque el Caso 19 los incluye en el registro (RF5).

7.3 Matriz de datos y credenciales

Cada fila indica el sistema que custodia el dato y que puede cambiarlo. Las copias congeladas (M3 3.4) están en la columna del sistema.

ID	Dato o credencial	Finalidad	Clasificación	Sistema custodio	Roles autorizados	Representación en el frontend	Protección	Retención
D01	`run`	Identificar de forma única al cliente (RF5). No se puede cambiar después del alta.	Restringido. Sensible según el contexto: se usa para suplantar identidades.	`clients-service`	Lee: `cliente` (el propio) y `administrador`. Modifica: nadie tras el alta.	Se muestra en "Mi perfil" (P12) y en administración de clientes (P16). No aparece en pedidos, boletas, despacho ni listas del personal. No se guarda en el navegador.	TLS. Solo viaja en S03, S24, S26 y S27. No sale de clients: l07 no lo incluye. Nunca en registros ni en mensajes de error.	Mientras exista el cliente. Se anonimiza 24 meses después de la baja.
D02	`nombreCompleto`	Nombrar al cliente en el pedido, el despacho y la boleta (RF5, RF8, RF13).	Restringido	`clients-service`. Copias congeladas: `clienteNombre` en orders, payments y dispatch.	Lee: `cliente` (el propio), `administrador`; en las copias, el personal que atiende pedidos, ventas o despacho. Modifica: `cliente` (S25) y `administrador` (S28).	Perfil (P12, P16), detalle de pedido y boleta (P11, P14) y orden impresa (P15).	TLS. Las copias son solo lectura y no modifican al cliente. Fuera de los registros.	Perfil: como `run`. Copia en pedido: 12 meses desde la entrega. Copia en venta y boleta: 6 años. Copia en despacho: 90 días.
D03	`correo`	Iniciar sesión, validar que existe, enviar el enlace de verificación y la boleta (RF5, RF8, RF11).	Restringido	`auth-service` (cuenta). Copias: `clienteCorreo` en orders (solo para la venta) y en payments (para la boleta).	Lee: `cliente` (el propio) y `administrador`. Modifica: nadie desde el perfil (S25 no lo cambia). Se fija en el alta (S03, S26).	Perfil, administración de clientes (P16), formulario de inicio de sesión (P06). No aparece en listas de pedidos ni de ventas.	TLS. No se devuelve en S13, S14, S16, S18, S22 ni S23. Fuera de los registros. No va en la URL.	Cuenta: como `run`. Copia en pedido: 12 meses desde la entrega. Copia en venta: 6 años.

ID	Dato o credencial	Finalidad	Clasificación	Sistema custodio	Roles autorizados	Representación en el frontend	Protección	Retención
D04	`telefono`	Contactar para la entrega (RF1, RF13).	Restringido. Sensible según el contexto: junto con la dirección, ubica a una persona.	`clients-service`. Copias: `clienteTelefono` en orders y `telefono` en dispatch.	Lee: `cliente` (el propio), `administrador`, y `encargado_despachos` y `administrador` en la orden de despacho. Modifica: `cliente` (S25) y `administrador` (S28).	Perfil (P12, P16) y orden de despacho impresa (P15). No se devuelve en S16.	TLS. Fuera de los registros. En la cola ActiveMQ viaja solo dentro de la red privada.	Perfil: como `run`. Copia en pedido: 12 meses desde la entrega. Copia en despacho: 90 días.
D05	`fechaNacimiento` y `sexo`	Datos de registro que pide el Caso 19 (RF5), para las promociones y ofertas.	Restringido	`clients-service`	Lee: `cliente` (el propio) y `administrador`. Modifica: `cliente` (S25) y `administrador` (S28).	Solo en el perfil (P12) y en administración de clientes (P16).	TLS. No viajan a orders, payments ni dispatch (I07 no los incluye). Fuera de los registros.	Como `run`.
D06	`direccion` del perfil (`calle`, `comuna`, `provincia`, `region`)	Domicilio del cliente para precargar la entrega (RF5, RF1).	Restringido. Sensible según el contexto: es el domicilio de una persona.	`clients-service`	Lee: `cliente` (el propio) y `administrador`. Modifica: `cliente` (S25) y `administrador` (S28).	Perfil (P12) y precarga en el carrito (P03).	TLS. Se entrega a orders solo en I07, por cuenta. Fuera de los registros.	Como `run`.
D07	`direccionEntrega` del pedido (`calle`, `comuna`, `referencia`)	Entregar el pedido y validar los 3 km (RF1).	Restringido. Sensible según el contexto.	`orders-service`. Copia: `direccionEntrega` en dispatch.	Lee: `cliente` dueño del pedido; `administrador`, `cajero_virtual` y `encargado_despachos` (el DTO operativo, C3). `dueno`: nunca. Modifica: nadie después de crear el pedido.	Formulario de pedido (P03), detalle (P11, P14) y orden impresa (P15). No hay en el catálogo público.	TLS. No va en el carrito ni en el almacenamiento del navegador. Dispatch la recibe sin precios, RUN ni correo (I14).	Pedido: 12 meses desde la entrega, luego se borran `calle` y `referencia`. Despacho: 90 días.
D08	Pedido: ítems, total, estado, historial y `motivo` de anulación	Operar la venta (RF1 a RF3). El motivo puede contener texto libre del cliente.	Operativo. `motivo` es Restringido.	`orders-service`	Lee: `cliente` dueño (C1); personal según M4. Modifica: solo orders (S12, S15, S17, I12).	Mis pedidos y detalle (P11), administración de pedidos (P14).	TLS. El cliente solo ve lo suyo. El motivo no se registra.	Pagado: 6 años (respalda la venta). Vencido o anulado sin pago: 12 meses.
D09	Pago: `medio`, `monto`, `estado`, `urlPago`	Cobrar en la plataforma externa (RF2).	Operativo. `urlPago` es de un solo uso (ver 7.5).	`payments-service`	Lee: cliente dueño (solo el resultado en S20); payments. Modifica: solo payments (S20, S21).	Página de pago (P10): redirige a `urlPago`. No se guarda.	TLS. No hay datos de tarjeta en ningún sistema. `urlPago` no se registra.	Pago y avisos (`EventoPago`): 12 meses. Si el pago aprobado origina una venta: 6 años.
D10	Venta, líneas y boleta (`numeroBoleta`, total, ítems, `clienteNombre`, `clienteCorreo`)	Registro de la caja virtual y boleta digital (RF4, RF8).	Operativo. Nombre y correo: Restringido.	`payments-service`	Lee: `cajero_virtual` y `administrador` (S22, S23); `cliente` dueño (S23). `dueno`: solo cifras agregadas (S34).	Boleta (P11, P14) y ventas de la caja (P14).	TLS. S22 no devuelve el correo. S34 no devuelve datos de clientes individuales.	6 años (plazo referencial por obligación tributaria, a confirmar con contabilidad).
D11	Orden de despacho (`numeroOrden`, nombre, teléfono, dirección, ítems sin precio, `impresa`)	Armar y entregar el pedido en orden de llegada (RF13).	Restringido (nombre, teléfono, dirección). Operativo (el resto).	`dispatch-service`	Lee: `encargado_despachos` y `administrador`. Modifica: solo dispatch (I14, S19).	Cola de despacho y orden impresa (P15).	TLS. Sin precios, RUN ni correo. El usuario de dispatch en ActiveMQ es de solo lectura.	90 días desde la llegada. Se borra con un índice TTL de MongoDB.

ID	Dato o credencial	Finalidad	Clasificación	Sistema custodio	Roles autorizados	Representación en el frontend	Protección	Retención
D12	Mensaje de contacto (`nombre`, `correo`, `telefono`, `motivo`, `mensaje`)	Atender consultas del visitante (RF14).	Restringido. El `mensaje` es texto libre y puede traer datos de cualquier tipo.	`notifications-service`	Lee: nadie desde la API (es un extra del frontend, RF14, sin servicio de lectura). Escribe: el visitante (S35).	Formulario de contacto (P05). No se guarda en el navegador.	TLS. El `mensaje` no se registra. Límite de 10 solicitudes por minuto por IP.	12 meses. Se borra con un índice TTL de MongoDB.
D13	Catálogo: `nombre`, `categoria`, `descripcion`, `precio`, `imagen`, `destacado`, `disponible`	Mostrar la carta (RF1, RF7).	Público	`inventory-service`	Lee: cualquiera (S06). Modifica: `administrador` y `dueno` (S08 a S11).	Inicio, menú y carrito (P01 a P03).	Sin `stock` ni productos retirados en la respuesta pública (C12). Escritura solo con rol.	Sin límite. La baja es lógica (C13).
D14	`stock` y movimientos de stock	Controlar el inventario y las reservas.	Operativo	`inventory-service`	Lee: `administrador` y `dueno` (S07). Modifica: `administrador` y `dueno` (S11) e `orders-service` por reservas (I08 a I10).	Panel de administración de productos (P13).	No se devuelve en S06. Solo orders puede reservar.	Movimientos: 12 meses.
D15	`password` (entrada) y `passwordHash`	Autenticar al usuario (S01, S03, S04).	Credencial	`auth-service`	Lee: nadie. Ni el hash se devuelve (M4 4.8). Modifica: solo el propio usuario al activar la cuenta (S04).	Campo de contraseña en el inicio de sesión (P06) y en el registro (P08, P09). No se guarda en el navegador. El campo es de tipo contraseña.	Solo llega por TLS en el cuerpo. Se guarda un **hash adaptativo con sal** (bcrypt, costo 12, sal propia por contraseña), nunca la contraseña ni un cifrado reversible. No se devuelve, ni se registra, ni se envía a otro sistema.	Mientras exista la cuenta. Se elimina al anonimizar.
D16	`access_token` (Bearer) y `Sesion.tokenHash`	Identificar al usuario en cada solicitud (S01, S02).	Credencial	`auth-service`	Lo recibe quien inicia sesión. Auth guarda solo el hash. Lo valida el gateway (I01).	`sessionStorage`, nunca `localStorage`. Se envía en `Authorization`. Se borra al cerrar sesión o la pestaña.	Valor aleatorio de 256 bits. Auth guarda solo su SHA-256. Vence a los 900 s. No se registra.	Documento de sesión: se borra 24 horas después de vencer (TTL).
D17	Token de verificación del correo	Confirmar el correo o definir la contraseña (S04, S05).	Credencial	`auth-service`. Viaja en el correo (`enlace`).	Lo recibe el dueño del correo.	Llega en el enlace. La página lo lee, lo envía por POST (S04) y lo quita de la URL.	Aleatorio de 256 bits, un solo uso, vence en 24 h (C10). Se guarda solo su hash. El reenvío invalida el anterior.	Se borra 7 días después de vencer o usarse (TTL).
D18	`intentosFallidos`, `estado` y `rol` de la cuenta	Bloquear la fuerza bruta (C9) y decidir el acceso (C4).	Operativo	`auth-service`	Lee: `administrador` (estado y rol, S31). Modifica: solo auth.	Lista de usuarios (P17): `estado` y `rol`. `intentosFallidos` no se muestra.	5 intentos fallidos bloquean la cuenta 15 minutos. La respuesta de error no dice si el correo existe.	Mientras exista la cuenta.
D19	Secretos de integración (claves de proveedores, secretos del gateway, pares entre sistemas, bases de datos, tokens de Vault)	Autenticar a los componentes entre sí y con los proveedores (Módulo 6).	Secreto de integración	Vault	Ningún rol de la API. Cada componente lee solo lo suyo. Los administra el responsable de infraestructura.	Nunca. No existen en el código, el HTML ni las capturas del frontend.	Módulo 6 (6.9 y 6.10): referencias simbólicas, permisos mínimos, rotación y revocación.	Hasta la rotación (Módulo 6).

Lo que guarda el navegador, y nada más:

Dónde	Qué	Por qué
`localStorage`	El carrito: solo `id` y `cantidad` de cada producto	No tiene precio ni datos personales. El servidor vuelve a calcular todo al crear el pedido.

Dónde	Qué	Por qué
`sessionStorage`	El `access_token` y el rol y nombre que devuelve S01	Se borra al cerrar la pestaña. El token vence a los 900 s.
En ningún lado	Contraseñas, RUN, dirección, perfil, claves o secretos	Se piden o se muestran, y se descartan.

7.4 Decisión: ¿se custodian secretos del cliente?

No. Fundamento:

1. **No se guardan medios de pago.** El pago ocurre en la plataforma externa: el cliente es redirigido a `urlPago` y los datos de tarjeta o de cuenta no pasan por ningún sistema nuestro. No hay "guardar tarjeta".
2. **No hay claves, llaves de API ni credenciales de terceros del cliente.** Lo que el sistema le pide al cliente es su contraseña, y esa no se *custodia*: se *verifica*. Se guarda solo un hash que no permite recuperarla (D15).
3. **Los datos restringidos no son claves.** El RUN, la dirección o el teléfono se protegen controlando quién los ve y cuánto duran, no escondiéndolos como se hace con una clave.

Si en el futuro se quisiera recordar un medio de pago, la solución sería un token que entregue el proveedor, nunca los datos del medio.

El token de pago. `urlPago` identifica un solo cobro y vence en `expiraEn`. No permite operar la cuenta del comercio ni cobrar otros montos, así que se trata como dato de un solo uso y no como secreto: viaja al navegador y no se registra. La **clave del proveedor** (`payments-provider-key`) sí es un secreto de integración que permite crear cobros: está en Vault y solo la lee `payments-service`.

7.5 Qué devuelve cada servicio

El modelo interno no sale completo (M2 2.10). Cada servicio devuelve solo lo que su pantalla necesita.

Servicio	DTO	Datos que devuelve	Datos que quedan dentro
S01	`SesionDTO`	`access_token`, `token_type`, `expires_in`, `rol`, `nombre`	`passwordHash`, `intentosFallidos`, `tokenHash`
S06	`ProductoPublicoDTO`	Campos públicos del catálogo	`stock`, `retirado`
S12, S14	`PedidoDetalleDTO`	Pedido con ítems, total, `direccionEntrega`, historial y estado. Al cliente dueño o al personal	`clienteId`, `cuentaId`, `actorId`, banderas de reintento, correo, teléfono, RUN
S13	`PedidoResumenDTO`	`id`, `estado`, `total`, `creadoEn`	Todo lo demás
S16	`PedidoOperacionDTO`	`id`, `estado`, `total`, `creadoEn`, `clienteNombre`, `direccionEntrega`	`clienteTelefono`, `clienteCorreo`, `cuentaId`, RUN
S18, S19	`OrdenDespachoDTO` y documento imprimible	`numeroOrden`, `pedidoId`, `pagadoEn`, `clienteNombre`, `telefono`, `direccionEntrega`, ítems sin precio	Precios, total, RUN y correo (la orden nunca los recibe)
S20	`PagoIniciadoDTO`	`pagoId`, `estado`, `urlPago`, `expiraEn`	`pedidoId`, `monto`, banderas
S22	`VentaResumenDTO`	`ventaId`, `pedidoId`, `numeroBoleta`, `total`, `fecha`, `correoEnviado`	`clienteCorreo`, `cuentaId`, `intentosEnvio`
S23	`BoletaDTO`	`numeroBoleta`, `fechaEmision`, `pedidoId`, `clienteNombre`, `items`, `total`, `ivaIncluido`	`cuentaId`, `clienteCorreo`
S24, S25	`PerfilClienteDTO`	`id`, `run`, `nombreCompleto`, `correo`, `telefono`, `fechaNacimiento`, `sexo`, `direccion`	`cuentaId`, `passwordHash`, `rol`, `intentosFallidos`
S27, S28	`ClienteAdminDTO`	Los del perfil más `estado`	`cuentaId`, `passwordHash`, `intentosFallidos`
S31, S32	`UsuarioDTO`	`id`, `nombreCompleto`, `correo`, `rol`, `estado`	`passwordHash`, `cuentaId`, `intentosFallidos`
S34	`ReporteVentasDTO`	Totales, `porDia[]` y `topProductos[]`	Nombre, correo y cuenta de cualquier cliente

Se aplican las reglas de ámbito del Módulo 4 (C3): el personal recibe el DTO operativo y el cliente su propio DTO. El dueño no recibe datos de pedidos ni de clientes individuales.

7.6 Cómo se limita el acceso a lo propio

Regla	Cómo se cumple
La identidad sale del token, no de la solicitud.	`cuentaId` viene de `X-Authenticated-User`, que el gateway escribe a partir de I01. El cuerpo no puede traer `clienteId`, `cuentaId` ni `rol`.

Regla	Cómo se cumple
Cada consulta filtra por dueño.	`orders-service`, `payments-service` y `clients-service` buscan siempre por el identificador del recurso **y** `cuentaId` (C1).
Un recurso ajeno no se distingue de uno inexistente.	Respuesta 404 idéntica (T-DAT-04).
El perfil no lleva identificador en la ruta.	S24 y S25 usan `/clients/me`. No hay manera de pedir el de otro.
Las listas propias no aceptan un `cuentaId`.	S13 devuelve solo los pedidos de la cuenta autenticada.
Los identificadores de recurso son opacos.	Los genera el servidor. Adivinar uno no da acceso, porque la propiedad se comprueba igualmente.
Ocultar un botón no protege.	Toda regla se aplica en el backend aunque la pantalla sea accesible desde Internet.

7.7 Almacenamiento y transporte

Tema	Decisión
Dónde se guardan los datos	Cada sistema en su propia base de MongoDB, con su usuario (`db-<sistema>`). Ningún otro componente la lee.
Contraseñas	**Hash adaptativo con sal: bcrypt, costo 12**, con sal propia por contraseña. El límite de 72 caracteres del Módulo 1 es el del algoritmo. Ni cifrado reversible ni hash simple.
Tokens de sesión y de verificación	Valores aleatorios de 256 bits. Se guarda solo su SHA-256.
Datos personales en la base	En texto, sin cifrado por campo. Razón: el acceso a la base está limitado al sistema dueño y cifrar por campo exige gestionar otra clave sin cambiar quién puede leer el dato. Se declara como límite: no protege contra quien tenga acceso al disco del servidor.
Transporte	TLS entre navegador, gateway, sistemas y Vault (M6 6.12). Excepción declarada de MongoDB y ActiveMQ dentro de la red privada.
Datos en la URL	Ninguno restringido, salvo el filtro `q` de S27, que no se registra (7.8). El token de verificación llega en el enlace del correo: la página lo envía por POST, lo quita de la URL con `history.replaceState` y usa `Referrer-Policy: no-referrer`.
Copias de respaldo	Heredan las mismas reglas. No se incluyen en el repositorio.
Datos de ejemplo	Siempre ficticios (por ejemplo, `ClaveFicticia123` y `tok_ficticio_abc123`).

7.8 Qué no se registra

El registro de cada solicitud guarda `requestId`, verbo, ruta **sin la cadena de consulta**, estado, duración, `cuentaId` y la decisión (permitido o denegado, con el código de error). Nunca guarda:

Dato	Tipo
Contraseñas, tokens (Bearer y de verificación), hashes y cabecera `Authorization`	Credenciales
`X-Gateway-Secret`, `X-Service-Secret`, `X-Signature` y cualquier valor de Vault	Secretos de integración
RUN, correo, teléfono, fecha de nacimiento, sexo, dirección y nombre	Datos personales
Cuerpos de solicitudes y respuestas, `motivo` de anulación y `mensaje` de contacto	Texto libre del cliente
La cadena de consulta de la URL (por ejemplo, `?q=` en S27, que es texto de búsqueda)	Datos personales en la URL
`urlPago` y la clave del proveedor	Datos de pago

Los mensajes de error no repiten el valor recibido (dicen "correo inválido", no el correo). Las capturas del informe se toman con las cabeceras de credenciales ocultas.

7.9 Retención y eliminación

Dato	Retención	Mecanismo	Estado
Sesión (`Sesion`)	Token: 900 s. Documento: 1 h después de vencer	Purga periódica de `auth-service` (cada 5 minutos)	Implementado
Token de verificación	24 h de vigencia. Documento: 24 h después de vencer	Purga periódica de `auth-service` (cada 5 minutos)	Implementado
Orden de despacho	90 días desde la llegada	Purga periódica de `dispatch-service`	Implementado y probado (RET-01)
Mensaje de contacto	12 meses	Purga periódica de `notifications-service`	Implementado y probado (RET-02)
Aviso de pago (`EventoPago`) y pago sin venta	12 meses	Purga periódica de `payments-service`	Implementado y probado (RET-03). El pago con venta se conserva (RET-04)
Movimiento de stock	12 meses	Purga periódica de `inventory-service`	Implementado y probado (RET-05)
Pedido pagado, líneas e historial	6 años	Sin borrado automático. Respalda la venta	Política

Dato	Retención	Mecanismo	Estado
Pedido vencido o anulado sin pago	12 meses desde el cambio de estado	Purga periódica de `orders-service`	Implementado y probado (RET-06). Si el pedido llegó a pagarse se conserva (RET-07)
Dirección y teléfono copiados en el pedido	12 meses desde la entrega. Después se borran `calle`, `referencia` y `clienteTelefono`	Purga periódica de `orders-service`	Implementado y probado (RET-08 y RET-09)
Venta, líneas y boleta	6 años (plazo referencial, a confirmar con contabilidad)	Sin borrado automático	Política
Cliente dado de baja	24 meses desde la baja. Después se anonimiza `run`, `nombreCompleto`, `telefono`, `fechaNacimiento`, `sexo` y `direccion` en clients; en auth se reemplazan `correo` y `nombre` y se elimina el hash de la contraseña	Purga periódica de `clients-service` y de `auth-service`	Implementado y probado (RET-10 y RET-11)
Solicitud de eliminación de un cliente	Se atiende con la baja (S29) y la anonimización de su perfil. Se conserva lo que la obligación tributaria exige (venta y boleta)	Procedimiento del administrador y del responsable de infraestructura	Política
Secretos de integración	Hasta su rotación (Módulo 6)	Vault	Módulo 6

"Implementado" significa que cada sistema ejecuta una tarea periódica propia (se eligieron tareas del propio sistema en lugar de índices TTL), con los plazos en `common/fornace/retencion.py` y un intervalo de una hora. Las pruebas RET-01 a RET-12 (10.9.12) crean los datos, les retroceden las fechas en la base y comprueban qué se elimina, qué se anonimiza y qué se conserva; no se esperó el paso real del tiempo (10.11). Los pedidos pagados y las ventas siguen siendo política: se conservan 6 años sin borrado automático. Los plazos son decisiones de diseño del equipo. El de 6 años de ventas es referencial por obligación tributaria y debe confirmarse con la contabilidad del local.

7.10 Sesiones y tokens

Tema	Decisión
Tipo de token	Opaco y aleatorio de 256 bits, sin datos dentro. Por eso su validez se consulta a auth en cada solicitud (I01).
Expiración	900 s desde que se emite (M1). No hay token de renovación en esta entrega. Si vence, el frontend recibe 401 y vuelve al inicio de sesión. Consecuencia: quien tarde más de 15 minutos en la página de pago debe iniciar sesión otra vez al volver.
Revocación	Inmediata: S02 cierra la sesión, y la baja de una cuenta (S29, S33) y el cambio de rol (S32) invalidan sus sesiones. Como el gateway consulta a auth en cada solicitud, un token revocado deja de servir en la siguiente.
Almacenamiento en el servidor	Solo el hash del token. Si alguien lee la base de auth, no obtiene tokens válidos.
Almacenamiento en el navegador	`sessionStorage`. Riesgo: un script inyectado (XSS) podría leerlo. Se reduce con escape de todo texto mostrado y con una `Content-Security-Policy` en el servidor del frontend que solo deja conectarse al API Gateway (`connect-src`), bloquea plugins, base ajena, marcos y formularios hacia otros sitios. La política permite scripts en línea (`unsafe-inline`) porque las páginas los usan, así que no impide que un script inyectado se ejecute: solo limita adónde puede enviar el token (GW-CSP, 10.9.6). No se usan cookies, así que no hay riesgo de CSRF.
Fuerza bruta	5 intentos fallidos bloquean la cuenta 15 minutos (C9), más `RL-SENSIBLE` por IP. La respuesta de S01 es la misma si el correo no existe o la contraseña es incorrecta.
Token de verificación	Un solo uso, 24 horas, y el reenvío invalida el anterior (C10).
Claves en el navegador	El frontend **no contiene ninguna clave compartida**: un valor puesto en JavaScript lo puede leer cualquiera que abra la página. La única credencial del navegador es el token del usuario, que es personal, vence y se puede revocar. Los secretos de integración y la clave del proveedor de pagos solo existen en el backend (D19).

7.11 Recorrido de un dato restringido: la dirección de entrega

Etapa	Qué ocurre con el dato	Dónde	Quién puede verlo	Protección
1. Captura	El cliente la escribe en el pedido (P03) o se precarga desde su perfil (S24)	Navegador	El propio cliente	No se guarda en el navegador. El carrito solo tiene `id` y `cantidad`.
2. Transporte	`POST /api/v1/orders` (S12)	Navegador → gateway	Nadie en el camino	TLS. El gateway no registra el cuerpo.
3. Reenvío	El gateway agrega la identidad verificada y pasa el cuerpo	Gateway → orders	Solo orders	TLS y `gateway-to-orders`.
4. Validación y guardado	Orders comprueba que esté dentro de los 3 km y guarda una copia congelada en el pedido	`orders-service`	Cliente dueño y personal que atiende (C1, C3)	Base propia. Fuera de los registros.
5. Publicación	Tras el pago, orders la envía en la orden de despacho, sin precios, RUN ni correo (I14)	ActiveMQ → `dispatch-service`	Solo dispatch lee la cola	Usuarios de escritura y lectura separados (M6).

Etapa	Qué ocurre con el dato	Dónde	Quién puede verlo	Protección
6. Uso	El encargado la imprime (S19) y entrega la orden impresa a cocina y al repartidor	Panel (P15)	`encargado_despachos` y `administrador`	El rol se verifica en el gateway y en dispatch. La hoja impresa sale de la API.
7. Eliminación en despacho	La orden se borra a los 90 días	`dispatch-service`	—	Purga periódica (RET-01).
8. Conservación en el pedido	Se conserva con el pedido mientras sirve para atender reclamos	`orders-service`	Como en la etapa 4	12 meses desde la entrega.
9. Eliminación	Se borran `calle`, `referencia` y el teléfono del pedido. Se conserva el resto, necesario para la venta	`orders-service`	—	Purga periódica (RET-08).

7.12 Casos de validación

Cada caso se ejecuta contra el backend implementado en el Módulo 10. La columna de resultado se completa con la salida real de esa ejecución (estado HTTP y registro), no con lo esperado.

ID	Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Resultado registrado
T-DAT-01	Acceso anónimo al catálogo	`GET /api/v1/products` sin `Authorization`	Visitante	200 con `ProductoPublicoDTO`. Sin `stock` ni productos retirados.	API Gateway (ruta anónima y límite de tasa). `inventory-service` (C12)	OK. 200, 12 productos
T-DAT-02	Operación protegida sin sesión válida	`GET /api/v1/orders` (a) sin token, (b) con un token inventado, (c) con un token vencido o cerrado en S02	Ninguna	401 con `{ error: { codigo, mensaje, requestId } }` en los tres casos. No llega a orders.	API Gateway (I01)	OK. [401, 401, 401]
T-DAT-03	Administración desde un rol sin permiso	`POST /api/v1/products` y `GET /api/v1/admin/users` con la sesión de un `cliente`. `GET /api/v1/admin/users` también con `cajero_virtual`	`cliente` y `cajero_virtual`	403 en todos. Sin cambios en la base.	API Gateway (política de roles). `inventory-service` y `auth-service` (segunda barrera, probada llamando directamente con el secreto del gateway y el rol `cliente`)	OK. gateway=[403, 403, 403], directo=403
T-DAT-04	Consulta de un pedido ajeno	El cliente B pide `GET /api/v1/orders/{id}`, `GET /api/v1/orders/{id}/receipt` y `POST /api/v1/orders/{id}/payments` sobre un pedido del cliente A	`cliente` B	404 idéntico al de un pedido inexistente, sin dato alguno de A.	`orders-service` y `payments-service` (C1)	OK. [404, 404, 404]
T-DAT-05	Acceso directo al backend sin identidad autorizada	(a) Conectarse desde el host al puerto de `orders-service`. (b) Desde otro contenedor, `GET /orders/{id}` sin `X-Gateway-Secret`. (c) Con el secreto de `gateway-to-clients`. (d) Con el secreto correcto pero sin `X-Authenticated-User`. (e) Igual para `graphql-gateway`	Ninguna	(a) Conexión rechazada, no hay puerto publicado (Docker Compose: CMP-NET-04 probó 12 puertos y Compose no abrió ninguno, 10.9.11.1). (b), (c) y (d) 401 en Compose (CMP-RED-01, 10.9.11.2) y en modo local (T-DAT-05). Un par de secretos de otro sistema recibe 403 (CMP-RED-01).	Red de Docker. `InternalAccessGuard` de cada sistema	OK. [401, 401, 401, 401, 401] (modo local). CMP-RED-01: sin secreto=401, secreto de otro destino=401, secreto propio sin identidad=401, par de otro sistema=403 (Compose).

ID	Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Resultado registrado
T-DAT-06	Confirmar un pago con datos alterados del frontend	(a) POST /api/v1/payments/webhook con estado: aprobado y sin firma. (b) Con firma válida y marca de tiempo de hace más de 5 minutos. (c) Con firma válida y monto distinto del total. (d) El mismo eventoId dos veces. (e) Un cliente envía PATCH /api/v1/admin/orders/{id}/status con estado: pagado. (f) Un cliente envía POST /api/v1/orders con total y estado: pagado, y X-Authenticated-Roles: administrador	Plataforma de pago o cliente	(a) y (b) 401, el pedido sigue en pendiente_pago. (c) Pago observado, pedido sin cambios. (d) Un solo pagado, una sola venta. (e) 403. (f) Pedido creado con el precio del catálogo y pendiente_pago. El rol se toma del token.	payments-service (C8). orders-service (I12 y C7). API Gateway (descarte de cabeceras)	OK. (c) webhook=200, pago=observado, pedido=pendiente_pago; (d) segundo aviso=200, ventas=1; (e) 403; (f) 201 total=8990 (catalogo=8990) estado=pendiente_pago
T-DAT-07	La contraseña y su hash no salen	Iniciar sesión, registrar y consultar el perfil. Revisar las respuestas, la base y los registros de todos los contenedores	cliente	La contraseña no aparece en ninguna respuesta ni registro. En la base solo hay un hash bcrypt.	auth-service. Revisión con grep	OK. hash=12$... (bcrypt), campos=['_id', 'correo', 'creadoEn', 'estado', 'hash', 'id', 'nombre', 'rol']
T-DAT-08	Sesión revocada	Cerrar sesión (S02) y repetir GET /api/v1/orders con el mismo token. Lo mismo tras dar de baja la cuenta (S29 o S33) y tras cambiar el rol (S32)	cliente, administrador	401 de inmediato.	auth-service y API Gateway (I01)	OK. antes=200; logout=401, baja cliente=401, cambio rol=401, baja usuario=401
T-DAT-09	Registros sin secretos ni datos personales	Ejecutar el flujo completo (registro, pedido, pago, boleta, despacho) y buscar en los registros de todos los contenedores los valores usados: contraseña, token, correo, RUN, teléfono, dirección, secretos	—	Cero coincidencias.	AccessLogger del gateway y de cada sistema. Revisión con grep	OK. 15 valores buscados en 26 archivos; coincidencias=0
T-DAT-10	Campos mínimos por rol	GET /api/v1/admin/orders, GET /api/v1/admin/reports/sales y GET /api/v1/admin/dispatch-orders	cajero_virtual, dueno, encargado_despachos	S16 sin RUN, correo ni teléfono. S34 sin datos de clientes individuales. S18 sin precios, RUN ni correo.	orders-service, payments-service y dispatch-service (C3)	OK. S16=True, S34=True, S18=True
T-DAT-11	Sin secretos en el frontend	Buscar en el código HTML, JavaScript y CSS, en las capturas y en los ejemplos del informe cadenas como secret, api_key, Bearer seguido de un valor, y cualquier valor generado para Vault	—	Sin coincidencias. Los valores de ejemplo son ficticios.	Revisión con grep sobre el repositorio y el informe	OK. archivos revisados; coincidencias=0
T-DAT-12	Token vencido	Esperar los 900 s (en la prueba, con una vigencia corta configurable) y llamar	cliente	401.	auth-service (I01)	OK. antes=200, despues=401 TOKEN_INVALIDO
T-DAT-13	Fuerza bruta de contraseñas	Seis inicios de sesión fallidos seguidos para la misma cuenta	Visitante	Cinco intentos fallidos devuelven 401 y el sexto, 429 con Retry-After de 15 minutos. La respuesta no revela si el correo existe.	auth-service (C9) y API Gateway (RL-SENSIBLE)	OK. [401, 401, 401, 401, 401, 429], Retry-After=900

Los seis casos que pide la guía son T-DAT-01 a T-DAT-06. Los demás cubren contraseñas, sesiones, registros, campos mínimos y secretos en el frontend.

7.13 Riesgos aceptados

Riesgo	Por qué se acepta	Medida
Token en `sessionStorage` expuesto a XSS	El token vence en 15 minutos y se revoca. Guardarlo solo en memoria obligaría a iniciar sesión en cada recarga.	Escape de salida y CSP que limita adónde puede enviar datos un script inyectado (con `unsafe-inline`, no impide su ejecución). El riesgo queda declarado en 10.11.
S03 revela si un correo ya está registrado (409)	Es necesario para que el usuario sepa qué hacer	`RL-SENSIBLE` de 10 por minuto por IP. S01 no revela si el correo existe.
Datos personales sin cifrado por campo	Ver 7.7	Acceso limitado a la base del sistema dueño, retención corta y anonimización.
MongoDB y ActiveMQ sin TLS en la red privada	Ver M6 6.12	Red aislada, usuarios de mínimo permiso y consola sin publicar.
Sin token de renovación	Simplifica el diseño	Se documenta el efecto en la página de pago (7.10).

7.14 Control antes de entregar

Pregunta	Respuesta
¿Existe algún secreto en JavaScript, HTML, capturas o ejemplos?	No. El navegador solo guarda el token del usuario y el carrito. Los ejemplos del informe usan valores ficticios y el caso T-DAT-11 lo comprueba en el repositorio y el informe.
¿Los datos retornados son los mínimos necesarios?	Sí. 7.5 lista por servicio lo que sale y lo que queda dentro, y T-DAT-10 lo comprueba por rol.
¿La propuesta protege información del cliente aunque la pantalla sea accesible por Internet?	Sí. Cada regla se aplica en el backend (7.6) y el navegador no tiene secretos. Ocultar un botón no cambia lo que el servidor permite.

Módulo 8 — Separación privada y agregaciones GraphQL

Este módulo diseña el GraphQL Gateway privado y define lo que se prueba. El código y las evidencias de ejecución se entregan en el Módulo 10: las columnas de resultado de 8.13 se completan con la salida real de esas pruebas.

8.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Distinguir servicios transaccionales, de agregación, analítica y de alta demanda. Privados y Vault sin acceso público.	8.2 y 8.4.
2. Límites de inventory, orders, clients y payments por cohesión, acoplamiento, mantenibilidad, evolución y propiedad de datos.	8.3.
3. Al menos un GraphQL Gateway privado con una agregación de dos o más subsistemas, por llamadas a APIs internas.	8.4, 8.5 y 8.7.
4. Schema, tipos, consultas y resolvers. Las reglas transaccionales quedan en los servicios.	8.6 y 8.7.
5. Identidad verificada, autorización por recurso y campo, paginación, límites de profundidad y costo, plazos, errores parciales y batching.	8.8, 8.9 y 8.10.
6. Integración con una pantalla real. Demostrar que el navegador no alcanza los subsistemas ni los campos que su rol no permite.	8.11 y 8.13.
Mapa de clasificación, schema, resolvers, consulta de prueba, respuesta esperada, matriz de autorización y evidencia.	8.2, 8.6, 8.7, 8.10, 8.8 y 8.13.

8.2 Mapa de clasificación

Servicio	Naturaleza	Responsabilidad	Datos propios	Contrato	Dependencias	Justificación de separación
`api-gateway`	Borde	Autenticar, autorizar por ruta, limitar y enrutar. Único punto de entrada.	Ninguno de negocio	HTTPS y `/api/v1/...` hacia afuera. Cabeceras de identidad hacia adentro (M3 3.8, M6)	auth (I01), todos los sistemas (I02), Vault	Es el único componente con puerto publicado. Mantenerlo fino evita que una lógica de negocio quede expuesta a Internet.
`graphql-gateway`	Agregación	Combinar lecturas de varios sistemas para una pantalla, con autorización por operación y por campo.	Ninguno. Sin base de datos y sin caché entre solicitudes	`POST /graphql` (solo desde el API Gateway). Schema 8.6	orders, inventory y clients por sus APIs de lectura; Vault	Cambia con las pantallas, no con las reglas del negocio. Separarlo deja las reglas en los sistemas y evita agregar lógica al borde.
`auth-service`	Transaccional (identidad)	Cuentas, contraseñas con hash, sesiones, verificación de correo y personal.	Cuenta, Sesion, TokenVerificacion, Usuario	REST (8 servicios) e internos I01, I03 a I05	notifications (I06), Vault, `auth_db`	Las credenciales necesitan más aislamiento que los datos personales y cambian por razones de seguridad.

Servicio	Naturaleza	Responsabilidad	Datos propios	Contrato	Dependencias	Justificación de separación
`clients-service`	Transaccional (datos personales)	Perfil y domicilio del cliente y su alta.	Cliente, Direccion	REST (7 servicios) e interno I07	auth (I03 a I05), Vault, `clients_db`	Autoridad de los datos personales: un solo lugar para protegerlos y retenerlos.
`inventory-service`	Transaccional con lectura frecuente	Catálogo, precio, disponibilidad, stock y reservas.	Producto, MovimientoStock	REST (6 servicios) e internos I08 a I10	Vault, `inventory_db`	Autoridad de precio y stock. Se consulta mucho (catálogo público) y cambia poco. Es el sistema más independiente.
`orders-service`	Transaccional (flujo principal)	Ciclo de vida del pedido y la regla de que solo un pago confirmado lo pasa a `pagado`.	Pedido, DetallePedido, DireccionEntrega, CambioEstado	REST (6 servicios) e internos I11 e I12	clients (I07), inventory (I08 a I10), ActiveMQ (I14), Vault, `orders_db`	Dueño del estado oficial del pedido. Tiene otro ritmo de cambio que el precio y el stock.
`payments-service`	Transaccional (dependiente de un tercero)	Cobro, aviso firmado, venta, boleta y reporte de ventas.	Pago, EventoPago, Venta, LineaVenta, Boleta, CajaVirtual	REST (5 servicios) y aviso firmado I15	orders (I11, I12), notifications (I13), plataforma de pago (I16), Vault, `payments_db`	Depende de la plataforma externa (firmas, avisos repetidos, reintentos). Separarlo deja una sola autoridad del estado del pedido.
`dispatch-service`	Transaccional asíncrono	Orden de llegada e impresión para cocina.	OrdenDespacho, LineaDespacho	REST (2 servicios) y consumo de la cola I14	ActiveMQ, Vault, `dispatch_db`	No necesita precios ni datos de pago. Sigue funcionando con lo ya recibido si el broker cae.
`notifications-service`	Transaccional (salida de correo)	Validar y enviar correos, y guardar mensajes de contacto.	MensajeContacto	REST (1 servicio) e interno I06 e I13	Proveedor de correo (I17), Vault, `notifications_db`	Aísla al proveedor externo de correo.
Servicio de analítica	Analítica	No existe en esta propuesta.	—	—	—	El reporte de ventas (S34) es de un solo local y un máximo de 92 días. Se decide con medición en el ADR del Módulo 9 (M5 5.13).
Servicios de alta demanda	Alta demanda	No hay un servicio separado. La lectura frecuente es el catálogo público (S06).	—	—	—	Se mide en el Módulo 9. Si el catálogo no cumpliera el objetivo, se agregaría caché o réplica de lectura en inventory, no un servicio nuevo.
Vault, MongoDB y ActiveMQ	Infraestructura privada	Secretos, datos persistentes y cola de despacho.	Secretos; una base por sistema; `/queue/cola_pedidos`	Protocolos propios, solo dentro de la red privada	—	Sin puerto publicado ni acceso desde el navegador (M5 5.10).

Los 35 servicios del Módulo 1 se reparten así: auth 8, clients 7, inventory 6, orders 6, payments 5, dispatch 2, notifications 1. El GraphQL Gateway no suma servicios REST: tiene una sola ruta de entrada (R36 de M6).

8.3 Límites de inventory, orders, clients y payments

Sistema	Cohesión	Acoplamiento	Mantenibilidad	Evolución	Propiedad de datos
`inventory-service`	Reúne todo lo del producto: precio, disponibilidad, stock y reservas. Una reserva solo tiene sentido junto al stock.	No llama a ningún otro sistema. Lo llaman orders (reservas) y GraphQL (lectura).	Se puede cambiar el modelo de stock sin tocar pedidos. Su catálogo se prueba solo.	Puede crecer (variantes, promociones) o tener caché sin afectar al resto.	Producto y MovimientoStock. Nadie más cambia el precio ni el stock.
`orders-service`	Reúne el ciclo de vida del pedido y su regla de pago. Todas las transiciones de estado están en un lugar.	Depende de clients (datos del cliente), inventory (reservas) y, por cola, de dispatch. Payments depende de orders y no al revés.	Las reglas de estado se cambian y se prueban sin la plataforma de pago.	Nuevos estados o canales entran aquí. El contrato con payments (I11 e I12) no cambia.	Pedido, DetallePedido, DireccionEntrega y CambioEstado. Es la única autoridad de `pagado`.
`clients-service`	Reúne los datos personales y el domicilio. El RUN, la fecha de nacimiento y el sexo no salen de aquí.	Depende de auth para crear la cuenta. Orders lo lee por cuenta y sin datos de más (I07).	Un cambio en las reglas de datos personales afecta a un solo sistema.	Puede agregar datos del cliente sin tocar credenciales ni pedidos.	Cliente y Direccion. Los demás sistemas guardan copias congeladas solo para mostrarlas.
`payments-service`	Reúne todo lo que depende de la plataforma de pago: cobro, firma, venta, boleta y reporte.	Depende de orders (I11, I12) y notifications (I13). Orders no depende de él.	Cambiar de proveedor de pago afecta solo a este sistema.	Otro medio de pago o la integración real con la plataforma se reemplaza sin tocar el pedido.	Pago, EventoPago, Venta, LineaVenta, Boleta y CajaVirtual. Es la única autoridad del estado del cobro.

Cada sistema modifica solo sus datos. Las colaboraciones usan contratos explícitos (I01 a I18 del Módulo 3), nunca la base de otro. Las alternativas descartadas (unir orders con payments, inventory con orders, auth con clients, o dividir por tabla o pantalla) están en M3 3.2.

8.4 Separación privada

El GraphQL Gateway **no tiene puerto publicado**. Se llega a él solo por el API Gateway, y solo por una ruta:

Elemento	Cómo queda separado
Entrada	`POST /api/v1/graphql` (R36). El API Gateway autentica (I01), aplica `ROLES-TODOS`, límite por cuenta y tamaño, y reenvía con `gateway-to-graphql` más la identidad verificada.
Quién puede llamarlo	Solo con `X-Gateway-Secret` válido para `graphql-gateway`. Sin él, 401 (T-GQL-13).
Qué puede llamar	Solo GET de orders, inventory y clients, con su propio par por sistema (`graphql-to-orders`, `graphql-to-inventory`, `graphql-to-clients`). Sin acceso a bases de datos.
Qué aceptan los sistemas de él	Las rutas de negocio de lectura aceptan dos tipos de llamador: el API Gateway (`X-Gateway-Secret`) o el GraphQL Gateway (`X-Service-Id` y `X-Service-Secret`, solo GET). El resto de los verbos responde 403 para ese par (M4 4.10).
Identidad	El GraphQL Gateway reenvía `X-Authenticated-User`, `X-Authenticated-Email` y `X-Authenticated-Roles` que recibió ya verificados. No autentica por su cuenta.
Propiedad	Cada sistema aplica C1 aunque llame GraphQL (T-GQL-16). La agregación no es un atajo.
Vault, MongoDB, ActiveMQ	Sin puerto publicado. GraphQL solo lee de Vault los secretos de sus caminos y su certificado (M6 6.9).

8.5 Agregaciones y pantallas

Operación	Pantalla	Qué reemplaza	Subsistemas	Por qué es útil
`resumenPedido`	P11 "Ver detalle"	S14 más una consulta de catálogo (S06) por ítem	orders e inventory	El detalle muestra nombre, foto y descripción vigentes del producto sin que el frontend haga N+1 llamadas. El nombre y precio de la compra siguen congelados en orders.
`miCuenta`	P12 perfil, con el panel "Mis últimos pedidos"	S24 más S13	clients y orders	Una sola ida y vuelta para armar la pantalla de la cuenta.

S14 sigue existiendo: lo usa P10 al volver del pago, y el frontend lo usa si no hay respuesta de GraphQL.

8.6 Schema GraphQL

```
# Solo consultas. No hay mutaciones ni suscripciones.
type Query {
  "Detalle de un pedido con los datos vigentes de sus productos (orders + inventory)."
  resumenPedido(id: ID!): ResumenPedido
  "Perfil y últimos pedidos de la cuenta autenticada (clients + orders)."
  miCuenta(limitePedidos: Int = 5): MiCuenta
}

enum EstadoPedido {
  pendiente_pago
  pagado
  en_preparacion
  en_despacho
  entregado
  anulado
}

type ResumenPedido {
  id: ID!
  estado: EstadoPedido!
  estadoPago: String!
  total: Int!
  creadoEn: String!
  expiraEn: String
  boletaDisponible: Boolean!
  direccionEntrega: DireccionEntrega
  historialEstados: [CambioEstado!]!
  items: [ItemPedido!]!
  clienteNombre: String            # solo personal
}

type ItemPedido {
```

```
  productoId: ID!
  nombre: String!                  # congelado al comprar (orders)
  cantidad: Int!
  precioUnitario: Int!             # congelado al comprar (orders)
  producto: ProductoPublico        # vigente hoy (inventory). null si no hay respuesta o fue retirado
}

type ProductoPublico {
  id: ID!
  nombre: String!
  categoria: String!
  descripcion: String!
  precio: Int!
  imagen: String
  disponible: Boolean!
}

type CambioEstado {
  estado: EstadoPedido!
  fecha: String!
}

type DireccionEntrega {
  calle: String!
  comuna: String!
  referencia: String
}

type MiCuenta {
  perfil: PerfilCliente
  pedidos: [PedidoResumen!]!
}

type PerfilCliente {
  nombreCompleto: String!
  correo: String!
  telefono: String!
  direccion: Domicilio!
}

type Domicilio {
  calle: String!
  comuna: String!
  provincia: String!
  region: String!
}

type PedidoResumen {
  id: ID!
  estado: EstadoPedido!
  total: Int!
  creadoEn: String!
}
```

Decisiones del schema:

- **Un campo, una fuente.** `nombre` y `precioUnitario` del ítem vienen de orders (lo comprado). `producto.*` viene de inventory (lo vigente). Que los dos puedan diferir es justamente lo que aporta la agregación.
- **Anulable donde puede fallar.** `ResumenPedido.items.producto` y `MiCuenta.perfil` son opcionales para poder devolver resultados parciales. `resumenPedido` es opcional para poder responder `NO_ENCONTRADO` sin ocultar el error.
- **Mínimo necesario.** No están `run`, `fechaNacimiento`, `sexo`, `stock`, `cuentaId`, teléfono del pedido ni correos de ventas. Lo que no está definido no se puede pedir.
- **Sin mutaciones.** Las reglas de negocio y las escrituras quedan en los servicios REST.

8.7 Resolvers

Resolver	Operación	Qué hace	Subsistemas	Plazos	Errores
`resolverResumenPedido`	`resumenPedido`	1. Pide a orders `GET /orders/{id}` con el contexto del usuario (`graphql-to-orders`). 2. Reúne los `productoId` de los ítems y los pide a inventory en una sola llamada `GET /products?ids=...` (`graphql-to-inventory`). 3. Une cada ítem con su producto.	`orders-service` y `inventory-service`	Orders: 5 s. Inventory: 5 s.	Orders no responde: `resumenPedido` queda `null` y `SUBSISTEMA_NO_DISPONIBLE`. Orders responde 404: `NO_ENCONTRADO` y no se llama a inventory. Inventory no responde: `producto` queda `null` en cada ítem, con el nombre y precio congelados, y el error indica la ruta `items.producto`. Un producto retirado o ausente del catálogo: `producto` `null` sin error.
`resolverMiCuenta`	`miCuenta`	1. En paralelo: `GET /clients/me` a clients (`graphql-to-clients`) y `GET /orders?limit=N` a orders (`graphql-to-orders`). 2. Arma `perfil` y `pedidos`.	`clients-service` y `orders-service`	Clients: 5 s. Orders: 5 s.	Cada parte falla por separado: si clients no responde, `perfil` es `null` y `pedidos` sí llega, y al revés. No se inventan valores.

Sin llamadas por ítem. `resolverResumenPedido` usa un `BatchLoader` creado **por solicitud**: junta todos los `productoId` y hace una sola llamada. Se evita así una consulta de catálogo por cada ítem, y no se comparte caché entre usuarios.

Qué no hacen los resolvers: no calculan precios, no cambian estados, no deciden la propiedad y no leen bases de datos. Solo combinan lo que cada sistema entrega.

8.8 Matriz de autorización

Se decide en tres niveles. Los roles usan los nombres del Módulo 5 (5.6).

Por operación

Operación	Roles permitidos	Rechazo	Referencia
`resumenPedido`	`ROLES-PEDIDO-LECTURA`: `cliente`, `administrador`, `cajero_virtual`, `encargado_despachos`	`dueno`: operación no autorizada (C3). Un visitante no llega: el API Gateway responde 401.	Mismo criterio que S14 (M4).
`miCuenta`	`ROLES-CLIENTE`: `cliente`	El personal no tiene perfil de cliente. Los demás roles reciben operación no autorizada.	Mismo criterio que S24 (M4).

Por recurso

Recurso	Regla	Quién la verifica
Pedido (`resumenPedido`)	El `cliente` solo puede pedir un pedido de su cuenta. El personal puede pedir cualquiera.	`orders-service` con `cuentaId` propagado (C1). GraphQL no decide la propiedad: si orders responde 404, GraphQL devuelve `NO_ENCONTRADO` idéntico al de un pedido inexistente.
Perfil (`miCuenta`)	Solo el de la cuenta autenticada. No existe un argumento para pedir el de otra.	`clients-service` (`/clients/me`).
Pedidos recientes (`miCuenta`)	Solo los de la cuenta autenticada.	`orders-service` (C1).
Productos (ambas)	Solo el catálogo público.	`inventory-service` (C12). GraphQL no puede pedir `stock`: el campo no existe en el schema.

Por campo

Antes de llamar a ningún sistema, el GraphQL Gateway revisa **cada campo seleccionado** contra una `FieldPolicy`. Si el rol no puede leer alguno, rechaza la consulta completa con `CAMPO_NO_AUTORIZADO` (no devuelve el campo en blanco, para no sugerir que existe un valor).

Campo	Roles que pueden leerlo	Motivo
`ResumenPedido.id`, `estado`, `estadoPago`, `total`, `creadoEn`, `expiraEn`, `boletaDisponible`, `historialEstados`, `items`	Los roles de la operación	Datos del pedido que el rol ya puede ver en S14.
`ResumenPedido.direccionEntrega`	`cliente` (si es dueño), `administrador`, `cajero_virtual`, `encargado_despachos`	Dato restringido sensible según el contexto (M7 D07). Lo recibe quien atiende o recibe el pedido.
`ResumenPedido.clienteNombre`	`administrador`, `cajero_virtual`, `encargado_despachos`	Dato operativo del DTO del personal (C3). El cliente conoce su nombre y no se duplica en el pedido.

Campo	Roles que pueden leerlo	Motivo
`ItemPedido.producto` y sus campos	Los roles de la operación	Catálogo público (C12).
`MiCuenta.perfil.*`	`cliente` (la propia cuenta)	Datos restringidos del propio cliente.
`MiCuenta.pedidos.*`	`cliente` (los propios)	Pedidos propios (C1).
No existen en el schema	Nadie	`run`, `fechaNacimiento`, `sexo`, `stock`, `cuentaId`, `clienteTelefono`, `clienteCorreo`, hashes y secretos. Un campo que no está definido no se puede pedir.

8.9 Límites, plazos y errores

Control	Valor	Motivo
Una operación por solicitud	Sin lotes (arreglos de operaciones)	Evita multiplicar el costo con un solo cuerpo.
Campos de primer nivel	Máximo 3 por solicitud (incluye alias)	Impide repetir `resumenPedido` con alias para sortear el costo.
Profundidad	Máximo 5 niveles	El caso más profundo es `resumenPedido.items.producto.nombre` (4 niveles).
Costo	Máximo 600 puntos por solicitud	Cada campo cuesta 1. Una lista multiplica el costo de lo que contiene por su tamaño máximo (ítems: 30, historial: 10, pedidos: `limitePedidos`). Una consulta completa de `resumenPedido` (la de T-GQL-09) cuesta 432 y la de `miCuenta(limitePedidos: 20)` con `id`, `estado`, `total` y `creadoEn` de cada pedido, 103. Tres alias de `resumenPedido` costarían 1296 y se rechazan.
Paginación	`limitePedidos` de 1 a 20 (por defecto 5)	Fuera de rango, `CONSULTA_INVALIDA`.
Plazo	5 s por subsistema y 15 s por solicitud	El API Gateway espera 15 s (M6). Orders más inventory en secuencia suman a lo sumo 10 s.
Tamaño del cuerpo	16 KB	Lo limita ya el API Gateway.
Introspección y mensajes	Introspección desactivada. Errores sin trazas ni URLs	No se publica el schema completo. Los errores usan el formato `{ message, extensions: { code, requestId } }`.
Operaciones de escritura	No existen	Las escrituras siguen siendo los servicios REST con sus reglas.
Caché	Solo dentro de una solicitud (DataLoader por solicitud)	No se comparte entre usuarios, así que un dato de una cuenta nunca llega a otra.

Códigos de error (en `extensions.code`; el API Gateway no los transforma):

Código	Cuándo	Qué se llama
`CONSULTA_INVALIDA`	Sintaxis, argumentos fuera de rango o operación inexistente (incluye mutaciones)	No se llama a ningún sistema
`LIMITE_PROFUNDIDAD`, `LIMITE_COSTO`	La consulta excede los límites	No se llama a ningún sistema
`OPERACION_NO_AUTORIZADA`	El rol no puede usar esa operación	No se llama a ningún sistema
`CAMPO_NO_AUTORIZADO`	La selección incluye un campo que el rol no puede leer	No se llama a ningún sistema. La consulta completa se rechaza
`NO_ENCONTRADO`	Orders respondió 404 (pedido inexistente o ajeno)	`resumenPedido` es `null`. No se llama a inventory
`SUBSISTEMA_NO_DISPONIBLE`	Un sistema no respondió o respondió 5xx	Resultado parcial: lo resuelto llega, lo demás es `null`, y `path` indica qué falló
`PLAZO_EXCEDIDO`	Un sistema superó los 5 s	Igual que el anterior

Ejemplo: `resumenPedido(id)` con `id`, `items{productoId producto{id}}` e `historialEstados{estado fecha}` cuesta 152, porque el costo se calcula con el tamaño máximo de cada lista (30 ítems, 10 cambios de estado) y no con el tamaño real del pedido.

8.10 Consultas de prueba y respuestas esperadas

Los datos son los ficticios del Módulo 1 (`ped_0001`).

Consulta (la usa P11 "Ver detalle"):

```
query DetallePedido($id: ID!) {
  resumenPedido(id: $id) {
    id
    estado
    estadoPago
    total
    direccionEntrega { calle comuna referencia }
    historialEstados { estado fecha }
    items {
      cantidad
      precioUnitario
```

```
      producto { nombre descripcion imagen disponible }
    }
  }
}
# variables
{ "id": "ped_0001" }
```

R1. Respuesta esperada, cliente dueño

```
{
  "data": {
    "resumenPedido": {
      "id": "ped_0001",
      "estado": "pagado",
      "estadoPago": "aprobado",
      "total": 19970,
      "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "referencia": "Casa azul" },
      "historialEstados": [
        { "estado": "pendiente_pago", "fecha": "2026-11-07T18:30:00Z" },
        { "estado": "pagado", "fecha": "2026-11-07T18:33:00Z" }
      ],
      "items": [
        { "cantidad": 2, "precioUnitario": 8990,
          "producto": { "nombre": "Napolitana", "descripcion": "Pizza con tomate, mozzarella y orégano.",
                        "imagen": "https://ejemplo.cl/img/napolitana.jpg", "disponible": true } },
        { "cantidad": 1, "precioUnitario": 1990,
          "producto": { "nombre": "Bebida 500ml", "descripcion": "Bebida gaseosa.",
                        "imagen": "https://ejemplo.cl/img/bebida.jpg", "disponible": true } }
      ]
    }
  }
}
```

R2. Inventory no responde (error parcial)

```
{
  "data": {
    "resumenPedido": {
      "id": "ped_0001", "estado": "pagado", "estadoPago": "aprobado", "total": 19970,
      "direccionEntrega": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "referencia": "Casa azul" },
      "historialEstados": [ ... ],
      "items": [
        { "cantidad": 2, "precioUnitario": 8990, "producto": null },
        { "cantidad": 1, "precioUnitario": 1990, "producto": null }
      ]
    }
  },
  "errors": [
    { "message": "El catálogo no está disponible.", "path": ["resumenPedido", "items", 0, "producto"],
      "extensions": { "code": "SUBSISTEMA_NO_DISPONIBLE", "requestId": "req_ficticio_01" } },
    { "message": "El catálogo no está disponible.", "path": ["resumenPedido", "items", 1, "producto"],
      "extensions": { "code": "SUBSISTEMA_NO_DISPONIBLE", "requestId": "req_ficticio_01" } }
  ]
}
```

R3. Un cliente pide un campo del personal

```
// query: { resumenPedido(id: "ped_0001") { id clienteNombre } }   (sesión de un cliente)
{ "errors": [ { "message": "No tienes permiso para consultar el campo clienteNombre.",
                "extensions": { "code": "CAMPO_NO_AUTORIZADO", "requestId": "req_ficticio_02" } } ] }
```

R4. Pedido de otra cuenta

```
// query: { resumenPedido(id: "ped_0001") { id estado } }   (sesión de otro cliente)
{ "data": { "resumenPedido": null },
  "errors": [ { "message": "Pedido no encontrado.", "path": ["resumenPedido"],
                "extensions": { "code": "NO_ENCONTRADO", "requestId": "req_ficticio_03" } } ] }
```

R5. Encargado de despachos

```
// query: { resumenPedido(id: "ped_0001") { id clienteNombre } }   (sesión de un encargado de despachos)
{ "data": { "resumenPedido": { "id": "ped_0001", "clienteNombre": "Camila Rojas Pérez" } } }
```

Consulta de `miCuenta`

```
query MiCuenta {
  miCuenta(limitePedidos: 5) {
    perfil { nombreCompleto correo telefono direccion { calle comuna provincia region } }
    pedidos { id estado total creadoEn }
```

```
  }
}
```

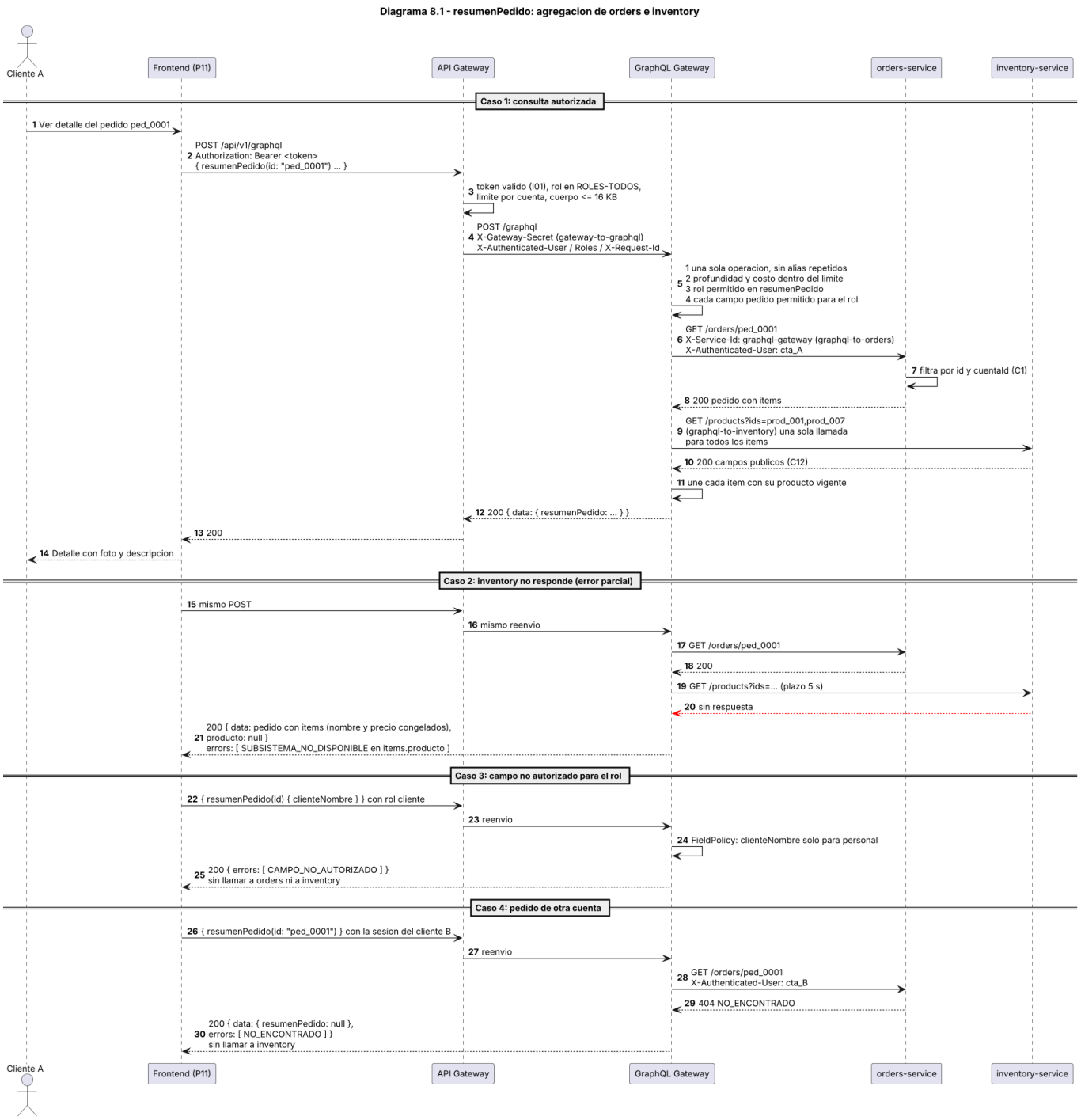
R6. Respuesta esperada

```
{
  "data": { "miCuenta": {
    "perfil": { "nombreCompleto": "Camila Rojas Pérez", "correo": "camila@ejemplo.cl", "telefono":
"+56911112222",
                "direccion": { "calle": "Av. Ejemplo 1234", "comuna": "Maipú", "provincia": "Santiago",
"region": "Metropolitana" } },
    "pedidos": [ { "id": "ped_0001", "estado": "pagado", "total": 19970, "creadoEn": "2026-11-
07T18:30:00Z" } ]
  } }
}
```

8.11 Integración con el frontend

Tema	Decisión
Llamada	`fetch` de `POST /api/v1/graphql` con `Authorization: Bearer` desde `sessionStorage`. La URL es la del API Gateway; el frontend no conoce otra.
Datos de la pantalla	P11 usa `resumenPedido`. P12 usa `miCuenta`.
Errores parciales	Si `producto` es `null`, la pantalla muestra el nombre y el precio congelados del pedido y oculta la foto. No muestra un error al cliente por eso.
Errores completos	`NO_ENCONTRADO`: "Pedido no encontrado". 401: vuelve al inicio de sesión. Otro error: mensaje genérico con el `requestId`.
Respaldo	Si el GraphQL Gateway no responde, P11 usa S14 (REST) y muestra el detalle sin las fotos.
Secretos	Ninguno en el frontend. El token es el del usuario (M7 7.10).

8.12 Diagrama 8.1 — Secuencia de `resumenPedido`



Fuente editable: `diagramas/diagrama-8-1-secuencia-graphql.puml`. Versión vectorial: `diagramas/diagrama-8-1-secuencia-graphql.svg`.

Muestra una consulta autorizada, un error parcial por inventory caído, un campo no autorizado rechazado antes de llamar a los sistemas, y un pedido de otra cuenta.

8.13 Pruebas y evidencia

Las pruebas se ejecutan contra el sistema implementado (Módulo 10). La columna de resultado se completa con la salida real (estado, cuerpo y registros con `requestId`), no con lo esperado. Para T-GQL-15 la evidencia es una captura de la pestaña de red del navegador en P11.

ID	Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Resultado registrado
T-GQL-01	Agregación autorizada	`resumenPedido(id: "ped_0001")` con la sesión del cliente dueño	`cliente` A	200 con el pedido y, en cada ítem, el producto vigente (R1 de 8.10). Los registros de orders e inventory muestran el mismo `requestId`.	GraphQL Gateway, orders e inventory	OK. 200, 2 items con producto; registros con el requestId en ['api-gateway.log', 'auth-service.log', 'graphql-gateway.log', 'inventory-service.log', 'orders-service.log']
T-GQL-02	Sin llamada por ítem (batching)	El pedido de T-GQL-01 con 2 ítems. Contar las llamadas a inventory en sus registros	`cliente` A	Una sola llamada `GET /products?ids=...`, no dos.	`BatchLoader` del GraphQL Gateway	OK. llamadas a inventory con el requestId = 1
T-GQL-03	Inventory no disponible	Detener `inventory-service` y repetir T-GQL-01	`cliente` A	200 con el pedido, `producto` `null` en cada ítem y errores `SUBSISTEMA_NO_DISPONIBLE` con su `path` (R2 de 8.10). El tiempo total no pasa de 15 s.	GraphQL Gateway (plazo de 5 s)	OK. 200, errores=['SUBSISTEMA_NO_DISPONIBLE', 'SUBSISTEMA_NO_DISPONIBLE'], path=['resumenPedido', 'items', 0, 'producto'], 0.0 s
T-GQL-04	Orders no disponible	Detener `orders-service` y repetir T-GQL-01	`cliente` A	`resumenPedido` `null` y `SUBSISTEMA_NO_DISPONIBLE`. No hay llamada a inventory.	GraphQL Gateway	OK. 200, errores=['SUBSISTEMA_NO_DISPONIBLE'], llamadas a inventory=0
T-GQL-05	Pedido de otra cuenta	`resumenPedido(id: "ped_0001")` con la sesión de otro cliente. Comparar con un id inexistente	`cliente` B	`NO_ENCONTRADO` en ambos casos, sin ningún dato de A (R4 de 8.10). No hay llamada a inventory.	`orders-service` (C1)	OK. ajeno=['NO_ENCONTRADO'], inexistente=['NO_ENCONTRADO']
T-GQL-06	Campo no autorizado	Pedir `clienteNombre` con la sesión de un cliente	`cliente` A	`CAMPO_NO_AUTORIZADO` (R3 de 8.10). No hay llamadas a orders ni a inventory. Con `encargado_despachos` el mismo campo sí llega (R5 de 8.10).	GraphQL Gateway (`FieldPolicy`)	OK. cliente=['CAMPO_NO_AUTORIZADO'], despachos nombre recibido=True
T-GQL-07	Operación no autorizada para el rol	`resumenPedido` con `dueno`, y `miCuenta` con `cajero_virtual`	`dueno` y `cajero_virtual`	`OPERACION_NO_AUTORIZADA` en ambos.	GraphQL Gateway	OK. ['OPERACION_NO_AUTORIZADA'], ['OPERACION_NO_AUTORIZADA']
T-GQL-08	Límite de profundidad	Una consulta anidada con más niveles que el límite (en la prueba el límite se configura en 2 con `GQL_PROFUNDIDAD`, porque el schema no admite consultas con más de 5 niveles)	`cliente` A	`LIMITE_PROFUNDIDAD`. No se llama a ningún sistema.	GraphQL Gateway	OK. 400 ['LIMITE_PROFUNDIDAD']; limite real de produccion = 5 (el schema no admite más de 5 niveles)
T-GQL-09	Límite de costo	Tres alias de `resumenPedido` completo (costo 1296)	`cliente` A	`LIMITE_COSTO`. No se llama a ningún sistema.	GraphQL Gateway	OK. 400 ['LIMITE_COSTO']
T-GQL-10	Paginación fuera de rango	`miCuenta(limitePedidos: 500)`	`cliente` A	`CONSULTA_INVALIDA`.	GraphQL Gateway	OK. 400 ['CONSULTA_INVALIDA']
T-GQL-11	Sin escrituras ni introspección	`mutation { ... }` y `{ __schema { types { name } } }`	`cliente` A	`CONSULTA_INVALIDA` en ambos.	GraphQL Gateway	OK. ['CONSULTA_INVALIDA'], ['CONSULTA_INVALIDA']
T-GQL-12	Sin sesión	`POST /api/v1/graphql` sin `Authorization`	Ninguna	401. No llega al GraphQL Gateway.	API Gateway (I01)	OK. 401, registros de graphql-gateway con ese requestId=0
T-GQL-13	Acceso directo al GraphQL Gateway	Desde el host, conectarse a su puerto. Desde otro contenedor, `POST /graphql` sin `X-Gateway-Secret`, y con el de otro destino	Ninguna	Conexión rechazada desde el host (graphql-gateway no publica ningún puerto en Docker Compose: CMP-NET-04, 10.9.11.1). 401 en los otros dos (CMP-RED-02 en Compose; T-GQL-13 en modo local).	Red de Docker y `InternalAccessGuard`	OK. Modo local: sin secreto=401, secreto de otro destino=401. Compose: CMP-RED-02 = 401, 401, 401 y CMP-NET-04 sin puertos abiertos por Compose. En modo local la conexión rechazada desde el host no es verificable (todos los puertos escuchan en loopback).

ID	Caso	Entrada	Identidad	Resultado esperado	Componente que verifica	Resultado registrado
T-GQL-14	Subsistemas privados desde el navegador	En la pantalla P11, intentar `fetch` a `orders-service`, `inventory-service` y `clients-service` por su nombre o puerto	`cliente` A	No se alcanzan: no hay ruta (404, T-GQL-14) ni nombre resoluble desde la red pública ni puerto publicado (Docker Compose: CMP-NET-06 y CMP-NET-04, 10.9.11.1).	Red de Docker	OK. Modo local: rutas por nombre de servicio=[404, 404, 404], rutas /internal=[404, 404]. Compose: CMP-NET-06 desde la red pública resuelve [] nombres privados, /internal 404, sin token 401.
T-GQL-15	Agregación desde el frontend	Abrir P11 "Ver detalle" y revisar la red del navegador	`cliente` A	Una sola llamada `POST /api/v1/graphql` trae el detalle completo. No hay llamadas a `/orders/{id}` ni a `/products` desde esa pantalla.	Frontend, API Gateway y GraphQL Gateway	OK. graphql=1, otras=0
T-GQL-16	Los sistemas siguen decidiendo	Llamar `GET /orders/{id}` directamente con el par `graphql-to-orders` y la identidad de otra cuenta	Par de GraphQL	404 de orders. GraphQL no puede saltarse la propiedad.	`orders-service` (C1 y C11)	OK. 404 NO_ENCONTRADO
T-GQL-17	GraphQL solo lee	Llamar `POST /orders` o `PATCH /admin/orders/{id}/status` con el par `graphql-to-orders`	Par de GraphQL	403. El par solo tiene GET.	`orders-service` (C11, M4 4.10)	OK. POST=403, PATCH=403

8.14 Control antes de avanzar

Pregunta	Respuesta
¿La consulta combina realmente subsistemas?	Sí. `resumenPedido` une orders (estado e ítems comprados) con inventory (producto vigente). `miCuenta` une clients y orders. T-GQL-01 y T-GQL-02 lo verifican en los registros.
¿Se conserva la autoridad de cada servicio?	Sí. GraphQL solo hace GET, no lee bases de datos y no decide propiedad ni estado. Orders sigue respondiendo 404 a un pedido ajeno (T-GQL-05 y T-GQL-16).
¿Las políticas impiden filtrar información restringida mediante una selección de campos?	Sí. Cada campo tiene una `FieldPolicy`, los campos sensibles no existen en el schema y una selección no autorizada se rechaza antes de llamar a ningún sistema (T-GQL-06).
¿La red privada y los permisos se verifican en ejecución?	Están definidos los casos T-GQL-12 a T-GQL-17. Su resultado se registra en el Módulo 10.

Módulo 9 — Analítica y comunicación gRPC cuando corresponden

Este módulo decide con una **proyección** de la carga si hacen falta un servicio de analítica y comunicación gRPC. La decisión es que **no**, y se compromete a demostrar con una prueba de carga reproducible que la alternativa implementada cumple los objetivos. Las mediciones se registran en el Módulo 10: la sección 9.9 queda con la tabla vacía hasta entonces y no contiene valores supuestos.

9.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Caracterizar volumen, concurrencia, tamaño, frecuencia, latencia y streaming, y declarar si es proyección o medición.	9.2 (proyección).
2. Decidir si hay analítica y gRPC, con un ADR de contexto, alternativas, elección y consecuencias.	9.3 a 9.6.
3. Si hay analítica: origen de datos, actualización, retraso, agregaciones, permisos y datos personales.	No aplica. 9.4 lo fundamenta y 9.7 define la alternativa implementada.
4. Si hay gRPC: contratos, autenticación, deadlines y errores.	No aplica. 9.5 lo fundamenta.
5. Si hay streaming: control de flujo, límites y recuperación.	No aplica. 9.6 lo fundamenta. Los fallos y desconexiones de lo que sí existe se prueban en 9.8.
6. Prueba de carga con entorno, datos, concurrencia, duración, p95, throughput, errores y consumo.	Plan en 9.8 y resultados en 9.9.
Matriz de necesidades y ADR.	9.3 y 9.4 a 9.6.
Justificar la ausencia de cada componente con los requisitos y la carga prevista.	9.4 a 9.6.

9.2 Caracterización de la carga

Esto es una proyección, no una medición. El Caso 19 describe un solo restaurante y no entrega volúmenes. Se declaran supuestos explícitos y se calcula a partir de ellos.

ID	Supuesto	Valor	Origen
S1	Pedidos por día	80 en un día normal y 150 en viernes y sábado	Proyección del equipo. El Caso 19 no entrega volúmenes: describe un solo restaurante.
S2	Concentración	50 % de los pedidos del día en 2 horas	Proyección. Almuerzo o cena.
S3	Ítems por pedido	3 en promedio (máximo permitido: 30)	Proyección. El máximo es del Módulo 1.
S4	Visitantes simultáneos del catálogo	100 durante una promoción, con una consulta cada 10 s	Proyección. Es un escenario pesimista para un solo local.
S5	Período máximo del reporte	92 días	Dato del diseño (Módulo 1, C6).
S6	Tamaño de una venta con sus líneas	0.6 KB	Estimación por el tamaño de los contratos del Módulo 1.

Magnitud derivada	Cálculo	Resultado
Pedidos por segundo en la hora punta	150 × 0.5 ÷ (2 × 3600)	0.0104 por segundo (unos 38 por hora)
Consultas de catálogo por segundo	100 ÷ 10	10 por segundo
Ventas por año en el escenario pico	150 × 365	54.750
Ventas dentro de un reporte de 92 días	150 × 92	13.800 ventas y 41.400 líneas
Tamaño de esos datos	13.800 × 0.6 KB	unos 8 MB

ID	Necesidad	Volumen y frecuencia	Concurrencia	Tamaño de los mensajes	Carácter	Latencia objetivo	Streaming
N1	Consultar el catálogo (S06)	10 consultas por segundo en promoción	Hasta 100 visitantes	Respuesta de 2 a 20 KB (hasta 100 productos)	Frecuente y de solo lectura	p95 de 300 ms	No
N2	Crear un pedido (S12)	0.01 por segundo en la hora punta	Pocos simultáneos	Menos de 2 KB	Rara	p95 de 800 ms	No
N3	Aviso de pago (S21)	Igual al de pedidos pagados	1 a 2 simultáneos	Menos de 1 KB	Rara, con repeticiones posibles	p95 de 500 ms	No
N4	Reporte de ventas (S34)	1 a 2 consultas por día. Hasta 13.800 ventas por consulta	1 a 2 usuarios (dueño y administrador)	Respuesta de menos de 5 KB	Baja	p95 de 1,5 s con los datos de 92 días	No
N5	Orden de despacho a cocina (I14)	0.01 por segundo	Un consumidor	Cerca de 1 KB por mensaje	Rara, y debe ser durable	Llega en menos de 5 s si el broker está activo	No. Es asíncrono
N6	Agregación GraphQL (R36)	Una consulta por detalle de pedido o por cuenta	Pocos simultáneos	Hasta 15 KB con 30 ítems	Baja	p95 de 600 ms	No
N7	Llamadas entre sistemas (I01, I03 a I13)	Como máximo el de pedidos más el de autenticación: una por solicitud autenticada	Igual que la concurrencia de cada flujo	Menos de 2 KB	Una por solicitud	Plazos de 3 s y 5 s (M3)	No
N8	Estado del pedido en el navegador	Una consulta cada 15 s por pantalla abierta	Hasta 100 pantallas	Menos de 2 KB	Baja	El cambio se ve en 15 s	No hace falta

Lectura de la tabla: lo más exigente es el catálogo con 10 consultas por segundo y el reporte de 92 días con 13.800 ventas (unos 8 MB). Un pedido nuevo llega cada 96 segundos en la hora punta. Los objetivos de latencia son decisiones del equipo, y la prueba de 9.8 los compara con lo que se mida.

9.3 Matriz de necesidades

Necesidad	¿Procesamiento intensivo?	¿Streaming?	¿Justifica gRPC o un servicio aparte?	Mecanismo elegido
N1 a N3, N6	No. Operaciones por solicitud con pocos datos	No	No: mensajes pequeños, tasa baja	REST (S06, S12, S21) y GraphQL (R36)
N4 Reporte de ventas	No: unos 14 mil documentos por consulta. Una agregación de la base lo resuelve	No	No: un solo sistema (payments) lo calcula	Agregación de MongoDB dentro de `payments-service` (9.7)
N5 Orden de despacho	No	No: un mensaje por pedido, sin flujo continuo	No: ya está resuelto con una cola durable y reintentos	ActiveMQ (I14)
N7 Entre sistemas	No	No	No: ver ADR-02	REST con JSON y plazos (M3)

Necesidad	¿Procesamiento intensivo?	¿Streaming?	¿Justifica gRPC o un servicio aparte?	Mecanismo elegido
N8 Estado en el navegador	No	No: no hay eventos continuos que justifiquen mantener una conexión	No: gRPC no lo habla el navegador	Consulta periódica de S14 (ADR-03)

9.4 ADR-01 — No crear un servicio de analítica

Estado	Aceptado. Se revisa con las condiciones de 9.10.
Contexto	El caso pide un reporte de ventas por período para el dueño (RF9, S34). Es un solo local, con a lo sumo 13.800 ventas por consulta en 92 días (proyección 9.2). El reporte tiene totales, ventas por día y los diez productos más vendidos, sin datos de clientes individuales (M7, C3).
Alternativas	**A.** Servicio de analítica privado con copia de las ventas y actualización periódica. **B.** Colección de totales por día mantenida al registrar cada venta. **C.** Agregación de MongoDB bajo demanda, dentro de `payments-service`, con un índice por fecha. **D.** Una solución de big data.
Decisión	**C.** `payments-service` calcula el reporte con sus propios datos (9.7). Ajuste hecho en la implementación (Módulo 10): el reporte lee resúmenes diarios que el mismo servicio mantiene a partir de las ventas (parte de la opción B, dentro de `payments_db`), no recorre las ventas en cada consulta.
Por qué	Las ventas ya están en `payments_db`, que es su autoridad (M3 3.4). La carga proyectada es de unos 8 MB por consulta y una o dos consultas por día. La opción A agrega retraso, otra base y una copia de datos de ventas sin que el volumen lo pida. La B agrega una segunda fuente de verdad que mantener. La D es desproporcionada para este volumen.
Consecuencias	Menos piezas que operar y ningún dato personal fuera de payments. El costo del reporte crece con el período, por eso el rango está limitado a 92 días (C6). La consulta no es en tiempo real para períodos muy largos: no se ofrecen. La medición C5 y C6 (9.8) verifica que la decisión alcanza. Resultado en el Módulo 10: con el reporte leyendo los resúmenes diarios, C5 (13.800 ventas) y C6 (138.000) cumplen con holgura sobre MongoDB 7 (p95 de 40,5 ms y 39,3 ms frente a objetivos de 1.500 y 5.000 ms). No se midió la agregación directa sobre las ventas en esa ejecución, así que no se afirma cuánto habría costado; el resumen diario se mantiene porque su costo no crece con el volumen de ventas. La fuente de verdad sigue siendo `ventas`; los resúmenes son derivados y se pueden reconstruir.
Qué se pierde	Análisis históricos de más de 92 días y comparaciones entre locales. No están en el caso.
Permisos y datos personales	S34 es solo para `dueno` y `administrador`, y no devuelve nombre, correo ni cuenta de ningún cliente (M4, M7 7.5).

9.5 ADR-02 — No usar gRPC entre sistemas

Estado	Aceptado. Se revisa con las condiciones de 9.10.
Contexto	Los sistemas se llaman entre sí en I01 y de I03 a I13 con mensajes de menos de 2 KB y no más de una llamada por solicitud autenticada (9.2, N7). La frecuencia total es de 0.01 pedidos por segundo en la hora punta. Ya existen plazos por llamada (3 s y 5 s), reintento controlado e idempotencia (M3).
Alternativas	**A.** gRPC unary para todas las llamadas entre sistemas. **B.** gRPC solo para la reserva de stock (I08 a I10). **C.** Mantener REST con JSON entre sistemas.
Decisión	**C.**
Por qué	gRPC aporta contratos tipados y modos de streaming, pero con mensajes pequeños y una tasa baja el costo de serializar JSON no se nota y no hay nada que medir. Cambiar a gRPC obliga a mantener archivos `.proto`, generar código en Python y en Node, definir TLS, autenticación y autorización por método, y probarlo, para resolver un problema que el caso no tiene. Los contratos ya están definidos y probados como REST (Módulos 1 y 3). El stack del curso es REST con FastAPI.
Consecuencias	Una sola tecnología de comunicación síncrona y la misma identidad de servicio (M6). No hay tipado generado por `.proto`: se compensa con validación de esquemas en cada sistema.
Qué cambiaría la decisión	Llamadas muy frecuentes o mensajes grandes entre dos sistemas, o una operación con flujo continuo (9.10).

9.6 ADR-03 — Despacho por cola y sin streaming hacia el navegador

Estado	Aceptado. Se revisa con las condiciones de 9.10.
Contexto	La cocina debe recibir las órdenes en el orden de llegada (RF13). El Módulo 3 ya envía cada orden por `/queue/cola_pedidos` de ActiveMQ (I14). El frontend necesita ver el cambio de estado de un pedido, sin que el caso pida actualización en vivo. La guía menciona como ejemplo un monitoreo de despacho con streaming de servidor.

Alternativas	**A.** gRPC con streaming de servidor desde dispatch hacia un consumidor privado. **B.** Mantener la cola ActiveMQ y consultar el estado con S14 (y la cola de despacho con S18) cada 15 s. **C.** SSE o WebSocket desde el API Gateway hacia el navegador.
Decisión	**B.**
Por qué	Hay un mensaje por pedido (0.01 por segundo), no un flujo continuo. Una cola durable ya tolera caídas del consumidor y del broker, que un stream gRPC hay que reconstruir: puntos de reanudación, deduplicación y control de flujo. gRPC tampoco lo habla el navegador sin un adaptador. Consultar cada 15 s con 100 pantallas abiertas son unas 7 consultas por segundo, dentro de lo que el sistema soporta (C1).
Consecuencias	El cambio de estado se ve con hasta 15 s de retraso. El límite por cuenta (`RL-USUARIO`, 120 por minuto) cubre la consulta periódica (4 por minuto).
Recuperación sin duplicar efectos	Dispatch ignora una orden repetida (por `pedidoId`) y reconecta tras una desconexión. Orders republica las órdenes pendientes cada 30 s (M3). Se prueba en F2 y F3 (9.8).

9.7 Alternativa implementada: el reporte de ventas en `payments-service`

Elemento	Decisión
Almacenamiento	Colección `ventas` en `payments_db`. Cada venta lleva sus líneas embebidas (`lineas[]`) para no unir colecciones al agregar.
Índice	`ventas.fecha` ascendente. Los resúmenes tienen índice único por `fecha` (`resumen_dia`) y por `fecha` y `productoId` (`resumen_producto`).
Agregación	Cada venta se suma una vez a `resumen_dia` (ventas y total por día) y a `resumen_producto` (unidades por día y producto) con un contador `enResumen` en la venta. Una tarea periódica de `payments-service` aplica lo pendiente. La consulta lee a lo más 92 documentos de `resumen_dia`, suma `totalVentas` y `cantidadVentas`, y ordena los productos por unidades para devolver los diez más vendidos. No usa `$facet` ni recorre las ventas.
Límites	Rango de máximo 92 días y `desde` menor o igual que `hasta` (C6). El cuerpo de la respuesta no lleva datos de clientes.
Semántica	`totalVentas` es lo recaudado en la caja virtual. No descuenta reembolsos pendientes: la venta de un pedido anulado después de pagar sigue contando (M1 S34 y M3 3.11).
Permisos	`dueno` y `administrador` (M4). Las filas por cliente nunca salen de payments.
Datos de la prueba	Un script de carga genera ventas ficticias con una semilla fija en `payments_db` de una instancia de prueba (9.8).

9.8 Plan de la prueba de carga

Es reproducible: mismo script, mismas semillas y mismos límites de recursos. Se ejecuta en el Módulo 10 contra el sistema completo y **lo medido reemplaza** a la proyección de 9.2.

Tema	Definición
Tipo de resultado	**Medición** sobre el sistema implementado. Las cifras de entrada son **proyección** (9.2).
Entorno	Todos los componentes en Docker Compose en una sola máquina del equipo. Se registran el procesador, la memoria, el sistema operativo y las versiones. Límites por contenedor: 1 CPU y 512 MB.
Datos	Ficticios, con semilla 42. Catálogo de 40 productos. Ventas generadas por script directamente en `payments_db` de la instancia de prueba, con el usuario administrador de la inicialización, y nunca por la API.
Herramienta	Un script de Python con `asyncio` y `httpx` (las bibliotecas del curso). Genera la carga, mide cada solicitud y escribe un CSV. Sin herramientas adicionales.
Etapas	Calentamiento de 15 s que no se cuenta, el nivel de carga durante el tiempo indicado, y enfriamiento. Cada escenario se repite 3 veces y se informa la mediana de las repeticiones.
Métricas	Concurrencia, duración, solicitudes, latencia p50, p95 y p99, solicitudes por segundo (throughput), errores por código, y CPU y memoria de cada contenedor muestreados cada 2 s con `docker stats`.
Criterio	Cumple si todas las métricas de 9.2 y de la tabla están dentro del objetivo en las 3 repeticiones. Si no, se informa el valor medido y se decide si se corrige o se revisa el ADR.
Límite de recursos	Ningún contenedor supera el 80 % de su memoria ni sostiene más del 90 % de su CPU durante el nivel objetivo.
Cómo se ejecuta	`carga/` con instrucciones de arranque, generación de datos y ejecución (Módulo 10).

Escenarios de carga

ID	Escenario	Carga	Objetivo	Margen sobre la proyección
C1	Catálogo (S06)	Subir de 10 a 50 y a 100 usuarios simultáneos, 60 s cada nivel, consulta sin sesión	p95 de 300 ms a 50 por segundo. Errores menores al 1 %	5 veces la proyección

ID	Escenario	Carga	Objetivo	Margen sobre la proyección
C2	Crear pedido (S12)	2 por segundo durante 120 s y una ráfaga de 10 simultáneos	p95 de 800 ms. Errores menores al 1 %	200 veces la proyección
C3	Sin sobreventa	30 pedidos simultáneos de un producto con stock 10	Exactamente 10 pedidos creados, 20 respuestas 409 y stock final 0 sin valores negativos	Corrección, no rendimiento
C4	Aviso de pago (S21)	5 por segundo durante 60 s, con 20 % de avisos repetidos	p95 de 500 ms. Una sola venta por `eventoId`	500 veces la proyección
C5	Reporte de ventas (S34), 92 días	13.800 ventas, 5 consultas simultáneas, 60 s	p95 de 1,5 s. Errores menores al 1 %	Escenario pico real
C6	Reporte de ventas (S34), 10 veces el pico	138.000 ventas, 5 consultas simultáneas, 60 s	p95 de 5 s. Sin superar el límite de memoria	Margen de crecimiento
C7	Detalle por GraphQL (R36)	10 por segundo durante 60 s con pedidos de 2 y de 30 ítems	p95 de 600 ms. Una llamada a inventory por consulta	Verifica también el batching
C8	Comparación REST y GraphQL	El detalle de un pedido de 2 ítems por S14 y S06, frente a una consulta `resumenPedido`	Se informa la diferencia medida de latencia p95 y de llamadas. No hay objetivo	No se afirma una mejora sin medirla
C9	Despacho (I14)	Ráfaga de 200 órdenes publicadas en 10 s	Las 200 llegan a dispatch en el orden de llegada, sin pérdida ni duplicados, y en menos de 30 s	Margen sobre la proyección

Fallos y desconexiones

ID	Fallo provocado	Comportamiento esperado	Qué se prueba
F1	Inventory detenido durante C2	S12 responde 503 de forma controlada, sin pedidos a medias. Al reiniciar, vuelve a responder 201 en menos de 30 s.	Inventory caído (M3 3.11)
F2	ActiveMQ detenido durante un pago y reiniciado	El pedido queda `pagado` y `despachoPublicado` en falso. En menos de 30 s tras reiniciar el broker, la orden llega a dispatch una sola vez.	Broker caído y desconexión del consumidor
F3	Dispatch reiniciado con mensajes pendientes	Reconecta y consume lo pendiente sin duplicar órdenes.	Desconexión del consumidor
F4	Orders lento (más de 5 s) en C4	El aviso se registra, el pago queda `aprobado` con `confirmadoEnPedidos` en falso y se repite I12. El pedido pasa a `pagado` al normalizarse.	Dependencia lenta
F5	Vault detenido con los sistemas en ejecución	Los sistemas siguen con sus secretos. El API Gateway sigue 15 minutos y luego responde 503 (M6 6.10).	Gestor de secretos caído
F6	Auth detenido durante C1 y C4	El catálogo y el aviso de pago siguen respondiendo. Las rutas con token responden 503.	Autenticación caída
F7	Proveedor de pago o de correo sin respuesta	S20 responde 502 o 503 sin guardar un pago pendiente. La boleta queda con `correoEnviado` en falso y se reintenta cada 5 min.	Dependencia externa caída (adaptador de prueba)

9.9 Resultados medidos

Salida real de `carga/carga.py` y `carga/fallos.py`, ejecutados en el Módulo 10 en Windows, en modo local sobre MongoDB 7.0.45, Vault 1.18.5 y ActiveMQ 6.2.0, a la escala descrita allí (10.6 y 10.11). Duraciones al 25 % del plan, sin límite de recursos por proceso. CPU y memoria son el máximo entre los procesos del sistema medidos. La CPU se cuenta por núcleo (como la entrega `psutil`): 100 % es un núcleo completo, así que un proceso con varios hilos puede pasar de 100 %. El detalle por proceso, los percentiles p50 y p99 y las variantes de C1 con 10 y 100 usuarios están en 10.9.9.

ID	Escenario	Objetivo	Concurrencia y duración	p95	Throughput	Errores 5xx	CPU máx.	Memoria máx.	¿Cumple?
C1	Catálogo (S06)	p95 de 300 ms a 50 por segundo. Errores menores al 1 %	50 usuarios, 16.2 s	203.8 ms	40.1 por s	0 (0.0 %)	46 %	199 MB	Sí
C2	Crear pedido (S12)	p95 de 800 ms. Errores menores al 1 %	2 por segundo, 30.3 s, más ráfaga de 10	154.5 ms	2.0 por s	0 (0.0 %)	12 %	199 MB	Sí

ID	Escenario	Objetivo	Concurrencia y duración	p95	Throughput	Errores 5xx	CPU máx.	Memoria máx.	¿Cumple?
C3	Sin sobreventa	10 pedidos creados, 20 respuestas 409 y stock final 0	30 simultáneas	267.4 ms	104.3 por s	0	—	—	Sí (creados 10, 409: 20, stock final 0)
C4	Aviso de pago (S21)	p95 de 500 ms. Una sola venta por `eventoId`	5 por segundo con 20 % repetidos, 15.2 s	305.5 ms	6.0 por s	0 (0.0 %)	26 %	204 MB	Sí
C5	Reporte de ventas (S34), 92 días	p95 de 1,5 s. Errores menores al 1 %	5 simultáneas, 15.1 s, 13.800 ventas	40.5 ms	135.5 por s	0 (0.0 %)	104 %	202 MB	Sí
C6	Reporte de ventas (S34), 10 veces el pico	p95 de 5 s. Sin superar el límite de memoria	5 simultáneas, 15.1 s, 138.000 ventas	39.3 ms	135.8 por s	0 (0.0 %)	107 %	329 MB	Sí
C7	Detalle por GraphQL (R36)	p95 de 600 ms. Una llamada a inventory por consulta	10 por segundo, 16.3 s	20.3 ms	9.2 por s	0 (0.0 %)	15 %	329 MB	Sí
C8	Comparación REST y GraphQL	Se informa la diferencia	100 repeticiones	REST 22.7 ms; GraphQL 19.3 ms	—	0	—	—	Sí (diferencia -3.4 ms; llamadas del navegador 2 frente a 1)
C9	Despacho (I14)	200 en orden, sin pérdida ni duplicados, en menos de 30 s	200 mensajes	—	—	—	—	—	Sí (recibidas 200, última a los 12.6 s)

ID	Fallo	Resultado observado	¿Cumple?
F1	Inventory detenido durante C2	{"detenido_en_s": 5.8, "reiniciado_en_s": 20.9, "respuestas_durante_la_caida": [[503, "INVENTARIO_NO_DISPONIBLE"]], "pedidos_sin_reserva": 0, "segundos_hasta_el_primer_201": 0.1}	Sí
F2	ActiveMQ detenido durante un pago y reiniciado	{"estado_con_broker_caido": "pagado", "despachoPublicado_con_broker_caido": false, "segundos_tras_reiniciar_hasta_dispatch": 1.0, "ordenes_en_dispatch": 1, "despachoPublicado_final": true, "objetivo_del_plan_s": 30}	Sí
F3	Dispatch reiniciado con mensajes pendientes	{"ordenes_antes_del_reinicio": 0, "ordenes_despues": 20, "duplicadas": 0, "segundos_hasta_consumir": 0.0}	Sí
F4	Orders lento (más de 5 s) en C4	{"respuesta_del_aviso": 200, "pago_durante_la_demora": "aprobado", "confirmadoEnPedidos_durante_la_demora": false, "segundos_hasta_pagado_tras_reanudar_orders": 0.0}	Sí
F5	Vault detenido con los sistemas en ejecución	{"antes_de_la_ventana": [200, 200], "dentro_de_la_ventana": [200], "despues_de_la_ventana": 503, "codigo": "SERVICIO_NO_DISPONIBLE", "catalogo_publico_despues": 503, "ventana_prueba_s": 25, "ventana_plan_s": 900}	Sí
F6	Auth detenido durante C1 y C4	{"catalogo": 200, "aviso_de_pago": 200, "ruta_con_token": 503, "codigo": "AUTENTICACION_NO_DISPONIBLE", "pedido_pagado_despues": true}	Sí
F7	Proveedor de pago o de correo sin respuesta	{"S20_con_pasarela_caida": 503, "codigo": "PASARELA_NO_DISPONIBLE", "pagos_guardados": 0, "correoEnviado_con_correo_caido": false, "segundos_hasta_reintento_exitoso": 0.0, "intervalo_de_reintento_de_la_prueba_s": 10, "intervalo_del_plan_s": 300}	Sí

Diferencias con el plan de 9.8 (ver 10.11): duraciones al 25 %, sin límite de 1 CPU y 512 MB por proceso, F5 con ventana de 25 s en lugar de 15 minutos y F7 con reintento de correo cada 10 s en lugar de 300 s. En C1 se corrió también con 10 y 100 usuarios (10.9.9).

9.10 Cuándo se revisan las decisiones

Condición	Qué se haría
El reporte de 92 días supera 1,5 s de p95 con los datos del pico, o 5 s con 10 veces el pico	Ya aplicado en el Módulo 10 (resúmenes diarios en payments). Siguiente paso: mover la lectura a una réplica o a un servicio de analítica. Si no alcanza, un servicio de analítica privado con la actualización definida (ADR-01).
El dueño pide comparar años, varios locales o análisis más allá de 92 días	Reabrir ADR-01: un servicio de analítica separado de las transacciones, con su origen de datos, retraso aceptado y permisos.
Aparecen más de 100 llamadas por segundo entre dos sistemas, mensajes de más de 100 KB, o una operación larga	Evaluar gRPC unary para ese camino y, para la operación larga, un trabajo asíncrono con identificador.
La cocina o los repartidores necesitan ver el estado en vivo	Evaluar eventos hacia el navegador (SSE o WebSocket) con un punto de adaptación en el API Gateway, sin exponer gRPC interno.
El catálogo supera su objetivo con 50 consultas por segundo	Caché de respuesta en inventory o réplica de lectura. No es una razón para crear un servicio nuevo.

9.11 Control antes de avanzar

Pregunta	Respuesta
¿La decisión responde a una necesidad medida o proyectada de forma explícita?	Sí. 9.2 declara que es una proyección, con los supuestos y los cálculos. Las mediciones reales se registran en 9.9.
¿Están implementados los componentes que la propuesta declara necesarios?	Se declaran necesarios el backend, el GraphQL Gateway y la agregación del reporte en payments (9.7). Analítica y gRPC se declaran no necesarios, con la ausencia justificada en los ADR. Su implementación se entrega en el Módulo 10.
¿Las pruebas muestran cómo se comportan ante carga, fallos y desconexiones?	Están definidas en 9.8 (C1 a C9 y F1 a F7). Sus resultados reales están en 9.9 y en 10.9.9 y 10.9.10.

Módulo 10 — Backend completo e integración verificable

Este módulo muestra el backend funcionando. Lo que entregamos es el **backend**: el código de los 7 sistemas, el API Gateway, el GraphQL Gateway, Vault, MongoDB, la cola de despacho en ActiveMQ y los adaptadores de prueba de pago y correo.

Como indicó el profesor, **la entrega de backend es con datos de frontend, no con el frontend integrado**. Por eso lo probamos por su API con los datos y los flujos de la Evaluación 1 (`frontend-la-fornace`, 5 páginas HTML con Bootstrap 5.3): los 12 productos de `js/data.js` están cargados tal cual en `inventory`, y las pruebas recorren las mismas operaciones de las pantallas (P01 en adelante). La carpeta `frontend/` es una demostración opcional que usa la misma API; el backend no depende de ella y las pruebas de la guía se hacen sin abrirla.

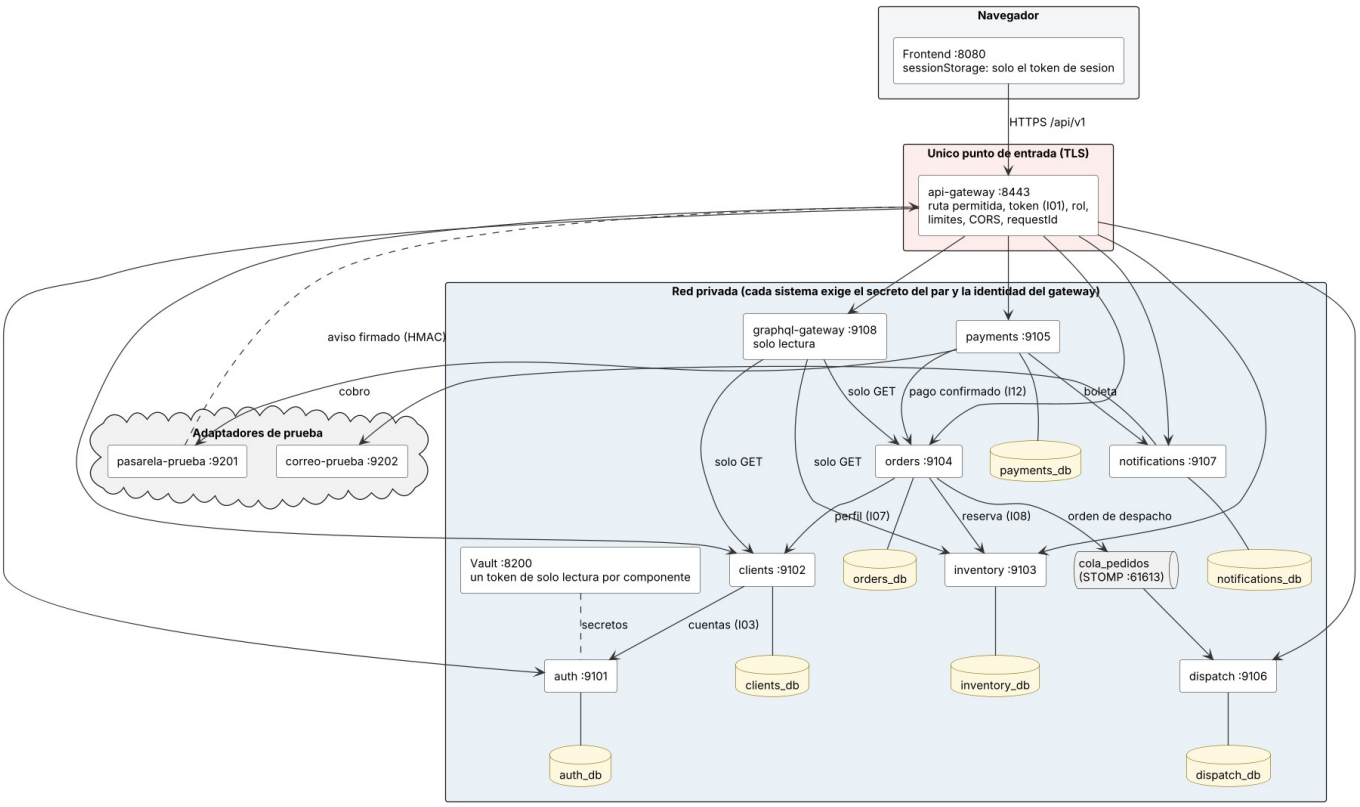
Las tablas de resultados de este módulo salen de ejecutar las pruebas (`impl/scripts/run_tests.py` y `impl/probar_todo.py`). No escribimos ningún valor a mano. La última ejecución completa dio **221 de 221 pruebas en OK** el 2026-11-08: 193 en modo local y 28 en Docker Compose, en Windows y con MongoDB, Vault y ActiveMQ reales (10.9.11).

10.1 Cumplimiento de los pasos a seguir de la guía

Paso de la guía	Dónde está
1. Implementar todos los requisitos funcionales y las operaciones de las pantallas.	10.2 (qué se implementó), 10.8 (matriz de pantallas, servicios y pruebas) y 10.9.4 (35 servicios con caso válido y caso inválido).
2. Persistencia real, esquema, scripts de creación, datos ficticios, validaciones, permisos, estados y errores.	10.3.
3. Integrar gateway, Auth, secretos, GraphQL y sistemas privados, con puertos limitados e identidad verificada.	10.4, diagrama 10.1 y 10.9.11 (redes pública y privada en Docker Compose).
4. Integrar el frontend responsivo con las APIs reales y usar adaptadores de prueba para pago y correo.	10.5 (los datos de la Evaluación 1 por API y el frontend integrado como demostración) y 10.9.7.
5. Pruebas de contratos, integración, reglas de negocio y seguridad, con fallos de dependencias, pagos duplicados y stock.	10.9.
6. Instrucciones de inicio y detención, secretos por referencia, datos de prueba, dependencias versionadas, variables sin valores reales, health checks y registros con correlación.	10.6, `impl/README.md` y `impl/GUIA-DE-PRUEBA.md`.
Matriz requisito, pantalla, servicio, prueba, resultado y evidencia.	10.8.
Demostración de los flujos principales y consulta de pedido autorizada y denegada.	10.7 y `impl/tests/evidencia/demo-flujos.md`.
Pruebas gRPC y analíticas aplicables.	No aplican (ADR de los puntos 9.4 y 9.5). La alternativa implementada se mide en 10.9.9 (C5 y C6).

10.2 Qué se entrega

Diagrama 10.1 - Implementacion entregada: procesos, puertos y accesos



Código fuente en `impl/`:

Carpeta	Contenido
`common/fornace/`	Librería compartida: acceso a Vault con caché, fábrica de aplicaciones con registro de accesos, guardia de identidad y roles, cliente entre sistemas con el secreto del par, conexión a la base y contadores, validaciones. `routes.json` es la tabla de rutas del Módulo 6.
`services/auth`, `clients`, `inventory`, `orders`, `payments`, `dispatch`, `notifications`	Los 7 sistemas (Python, FastAPI), cada uno con su base `<sistema>_db`.
`services/api_gateway`	Único punto de entrada. Enruta por la tabla de rutas, valida el token con Auth (I01), verifica el rol, limita la tasa, el tamaño y el origen, y quita cualquier cabecera de identidad que venga del navegador.
`services/graphql_gateway`	Apollo Server 5 de solo lectura. Políticas de operación, campo, profundidad, costo y paginación. `resumenPedido` y `miCuenta`.
`services/pasarela_prueba`, `correo_prueba`	Adaptadores de prueba de la plataforma de pago y del proveedor de correo. Conservan el contrato, la autenticación por clave y la firma del aviso.
`frontend/`	Demostración opcional: el frontend de la Evaluación 1 conectado al API Gateway, con páginas nuevas (registro, confirmar correo, retorno de pago, mis pedidos, mi cuenta y panel del personal). El backend no depende de esta carpeta.
`scripts/`	`stack.py` (iniciar, detener, estado en modo local), `compose.py` (Docker Compose), `vault_init.py`, `bootstrap_compose.py`, `certs.sh`, `seed.py`, `run_tests.py`, `serve_frontend.py`, `bandeja.py`.
`tests/` y `carga/`	Pruebas automáticas, resultados reales (`tests/resultados/*.json`), capturas y transcripción de la demostración (`tests/evidencia/`), pruebas de carga y de fallos.
`docker-compose.yml`, `Dockerfile.python`, `Dockerfile.node`, `Dockerfile.broker`, `.env.example`, `requirements.txt`	Despliegue en contenedores según el Módulo 6: redes `publica` y `privada` (esta con `internal: true`), solo tres puertos publicados, usuario sin privilegios, límites de memoria y CPU. Imágenes `mongo:7`, `hashicorp/vault:1.18` (modo desarrollo) y ActiveMQ 6.2.0 (`Dockerfile.broker`). Dependencias con versión fija.

Tecnologías: Python 3.13 en los contenedores (3.14 en la ejecución local de Windows) con FastAPI, httpx, motor y bcrypt; Node 22 con Apollo Server 5.5.1, GraphQL 16.11 y DataLoader (Node 26 en la ejecución local); MongoDB 7, Vault 1.18 y ActiveMQ 6.2.0 con STOMP para la cola; HTML, CSS y JavaScript con Bootstrap para el frontend de demostración. No se usaron tecnologías fuera de las del curso.

10.3 Persistencia, esquema y datos de prueba

Cada sistema escribe solo en su base. En Docker Compose MongoDB exige usuario y cada sistema tiene el suyo, con permisos solo sobre su base (`MONGO_AUTH=1`); en el modo local corre sin usuarios (`MONGO_AUTH=0`). Ver 10.11. El esquema se crea al iniciar cada sistema (colecciones e índices únicos o de búsqueda); no hay migraciones porque el sistema parte vacío. `scripts/seed.py` carga los datos de ejemplo.

Base	Colecciones con índices	Índices (campos)
`auth_db`	`cuentas`, `sesiones`, `tokens`, `intentos`	cuentas: id (único), correo (único); sesiones: hash (único); tokens: hash (único); intentos: correo
`clients_db`	`clientes`	clientes: id (único), run (único), cuentaId (único)
`inventory_db`	`productos`, `reservas`	productos: id (único), nombre (único); reservas: pedidoId (único), estado+expiraEn
`orders_db`	`pedidos`	pedidos: id (único), cuentaId+creadoEn, estado+expiraEn
`payments_db`	`pagos`, `eventos`, `ventas`, `resumen_dia`, `resumen_producto`	pagos: id (único), pedidoId; eventos: eventoId (único); ventas: pedidoId (único), numeroBoleta (único), fecha, enResumen; resumen_dia: fecha (único); resumen_producto: fecha+productoId (único)
`dispatch_db`	`mensajes`, `ordenes`	mensajes: pedidoId+tipo (único); ordenes: pedidoId (único), numeroOrden (único)
`notifications_db`	`mensajes`	mensajes: id (único)

Reglas de negocio y de datos que aplican los sistemas (todas con prueba en 10.9):

- El servidor fija el precio, el total y el estado del pedido; ignora los valores que envía el navegador (T-DAT-06f).
- La reserva de stock es todo o nada y no deja stock negativo (C3). Anular o vencer un pedido devuelve el stock (S15, FL-04).
- El pago solo se confirma con un aviso firmado, reciente y con el monto exacto. Un aviso repetido no duplica la venta (T-DAT-06, C4).
- La orden de despacho llega por la cola, una sola vez por pedido y en orden de llegada (FL-01, C9, F2, F3).
- Las anulaciones, bajas y ventas no se borran: cambian de estado y conservan el historial. Las contraseñas se guardan con bcrypt (T-DAT-07).
- Una sesión se revoca al cerrarla, al dar de baja la cuenta y al cambiar el rol (T-DAT-08).

Datos de prueba: 12 productos (clásicas, premium, acompañamientos, bebidas y 4 combos), 4 cuentas de personal (una por rol) y 2 clientes. Todos ficticios, con correos `@ejemplo.cl`. Las contraseñas se generan al azar al sembrar y quedan en `impl/.run/credenciales-demo.json`, que no forma parte de la entrega.

Vencimientos y retención: los plazos corren como tareas periódicas del propio sistema, no como índices TTL, para que el comportamiento sea el mismo en cualquier motor y quede probado por las suites: vencimiento de pedidos sin pago y liberación de su reserva (orders, cada 30 s), purga de sesiones, tokens de verificación e intentos de acceso vencidos (auth, cada 5 minutos) y de claves de idempotencia (orders, cada hora). Los plazos de retención de 7.9 para órdenes de despacho, mensajes de contacto, avisos de pago, movimientos de stock, direcciones de pedidos antiguos y clientes dados de baja no tienen tarea automática en esta entrega; se declara en 10.11.

10.4 Integración de la arquitectura

Pieza	Puerto	Quién puede llamarla
API Gateway	8443 (TLS)	Navegador y la plataforma de pago (aviso firmado). Es el único puerto de entrada.
auth, clients, inventory, orders, payments, dispatch, notifications	9101 a 9107 (TLS)	Solo con `X-Gateway-Secret` del destino y la identidad verificada, o con el par `X-Service-Id` y `X-Service-Secret` de un llamador autorizado en esa ruta.
GraphQL Gateway	9108 (TLS)	Solo el API Gateway. Hacia los sistemas usa el par `graphql-gateway` con permiso de lectura (GET).
Vault	8200	Cada componente con un token de solo lectura limitado a sus rutas.
Cola STOMP (ActiveMQ)	61613	orders publica; dispatch consume con confirmación manual. Un usuario por rol, con permiso de escritura (orders) o lectura (dispatch) sobre `/queue/cola_pedidos`.
Adaptadores de prueba	9201 y 9202	payments (cobro) y notifications (correo). La pasarela envía el aviso al gateway.

Cómo se cumple cada control (con la prueba que lo comprueba):

Control	Implementación	Prueba
Token validado en cada petición	El gateway llama a `POST /internal/introspect` de Auth. Si Auth no responde, devuelve 503 (falla cerrado).	T-DAT-02, F6
Identidad propagada	El gateway quita las cabeceras `X-Authenticated-*` del navegador y agrega las verificadas. Cada sistema confía solo en ellas y solo si llega el secreto del gateway.	T-AUT-10, T-AUT-11, T-DAT-05
Permisos por rol y por recurso	Roles por ruta en el gateway y de nuevo en cada sistema. La propiedad del recurso se decide en el sistema dueño (regla C1).	T-AUT-01 a T-AUT-14, T-DAT-03, T-DAT-04
Secretos fuera del código y del navegador	Todos los valores viven en Vault. El repositorio, el frontend y el informe solo tienen referencias simbólicas. El navegador solo guarda el token de sesión.	T-DAT-09, T-DAT-11, E2E-07
Aviso de pago confiable	HMAC-SHA256 sobre `marca de tiempo + "." + cuerpo`, ventana de 5 minutos, monto exacto y `eventoId` único.	T-AUT-08, T-DAT-06, C4
GraphQL de solo lectura con límites	Políticas antes de resolver: operación, campo, primer nivel (máx. 3), profundidad, costo (máx. 600) y paginación. Sin mutaciones ni introspección.	T-GQL-01 a T-GQL-17
Límites del gateway	Tasa por IP y por cuenta, tamaño de cuerpo, CORS solo para el frontend, cabeceras de seguridad y TLS.	GW-RL-*, GW-BODY, GW-CORS, GW-CABECERAS, GW-CSP, GW-TLS
Registros con correlación	Una línea JSON por evento con el mismo `requestId` en gateway, GraphQL y sistemas. Sin cuerpos, cabeceras, contraseñas, tokens ni datos personales.	T-DAT-09, GW-LOG, demo de 10.7

10.5 Integración con el frontend

Backend con los datos de la Evaluación 1. `inventory` carga los 12 productos de `js/data.js` del frontend de la Evaluación 1 (ids `prod_001` a `prod_012`, mismos nombres, categorías y precios: de $1.990 la bebida a $34.990 el Combo Noche Fornace). Toda la compra se puede recorrer por API con esos datos, sin abrir ningún frontend: `GUIA-DE-PRUEBA.md` da los comandos `curl` de registro, catálogo, pedido, pago con aviso firmado, boleta y consulta de pedido propio y ajeno.

Frontend integrado (demostración opcional). `impl/frontend/` es ese mismo frontend sin datos simulados: cada pantalla llama al API Gateway (`js/config.js`). El token de sesión vive en `sessionStorage` (limitación declarada en 10.11); el carrito solo guarda `id` y `cantidad` y el precio lo calcula el servidor al crear el pedido. Los textos del API se escapan antes de mostrarlos. El detalle del pedido (P11) y "Mi cuenta" (P12) usan una sola llamada GraphQL; si el GraphQL no responde, P11 muestra el detalle por REST. La tabla de 10.8 enlaza cada pantalla con su servicio y su prueba.

La vista de teléfono (390 px) se prueba con un navegador real (Chromium de Playwright): sin desplazamiento horizontal y con el menú plegable (E2E-M1 a E2E-M5). Capturas en `impl/tests/evidencia/`.

10.6 Cómo iniciar, detener, sembrar y probar

El profesor indicó que basta con ejecutar el sistema en local, y que si se usa Docker hay que compartir el `docker-compose` para poder replicarlo. Entregamos las dos cosas. Hay dos formas de ejecutar el mismo código contra los mismos motores reales (detalle en `impl/README.md`). Lo que aquí se llama modo local aparece como «modo nativo» en el texto de algunas pruebas y resultados.

A. Docker Compose (redes `publica` y `privada`; es el despliegue del Módulo 6). Requiere Docker Desktop abierto y Python 3:

```
python scripts/compose.py up         # crea .env con un token de desarrollo al azar, construye e inicia
los contenedores
python scripts/compose.py semilla     # datos ficticios; las contraseñas quedan en .run/credenciales-
demo.json
python scripts/compose.py estado      # contenedores y puertos publicados (solo 8443, 8080 y 9201)
python scripts/compose.py bandeja <correo>   # correos de prueba enviados (por ejemplo, el enlace de
confirmación)
python scripts/compose.py down        # detener (con --borrar también los datos)
```

B. Local, sin Docker (Windows; MongoDB 7 como servicio, Vault 1.18.5 y ActiveMQ 6.2.0 descomprimidos, Java 17, Python y Node). Una vez: `py -m venv .venv`, activar, `pip install -r requirements.txt -r requirements-test.txt`, `pip install -e common`, `python -m playwright install chromium` y `npm ci --omit=dev` en `services/graphql_gateway`. En cada sesión se definen `VAULT_BIN` y `ACTIVEMQ_HOME`:

```
python scripts\stack.py up            # inicia Vault, ActiveMQ y los 11 procesos (certificados y secretos
se generan solos)
python scripts\seed.py                # datos ficticios
```

```
python scripts\stack.py status        # estado de cada puerto
python scripts\stack.py down --todo   # detener todo y borrar las bases del proyecto (*_db)
python scripts\run_tests.py           # reinicia desde cero y ejecuta las 193 pruebas (unos 25 minutos;
--sin-carga las acorta)
```

El paso a paso por API está en `impl/GUIA-DE-PRUEBA.md`.

- **Secretos por referencia.** `vault_init.py` genera los valores al azar en el momento y los guarda solo en Vault. Si Vault se reinicia, los tokens viejos se detectan y se crean de nuevo. Cada componente tiene un token de solo lectura que `stack.py` entrega por archivo `600` en `.run/tokens`.
- **Configuración sin valores reales.** `.env.example` solo trae valores no secretos.
- **Dependencias con versión fija.** `requirements.txt` y `package-lock.json` del GraphQL Gateway.
- **Health checks.** `GET /health` en cada pieza, por TLS (prueba GW-HEALTH).
- **Escala de carga completa.** `python carga/carga.py --escala=completa` usa las duraciones del plan del Módulo 9.

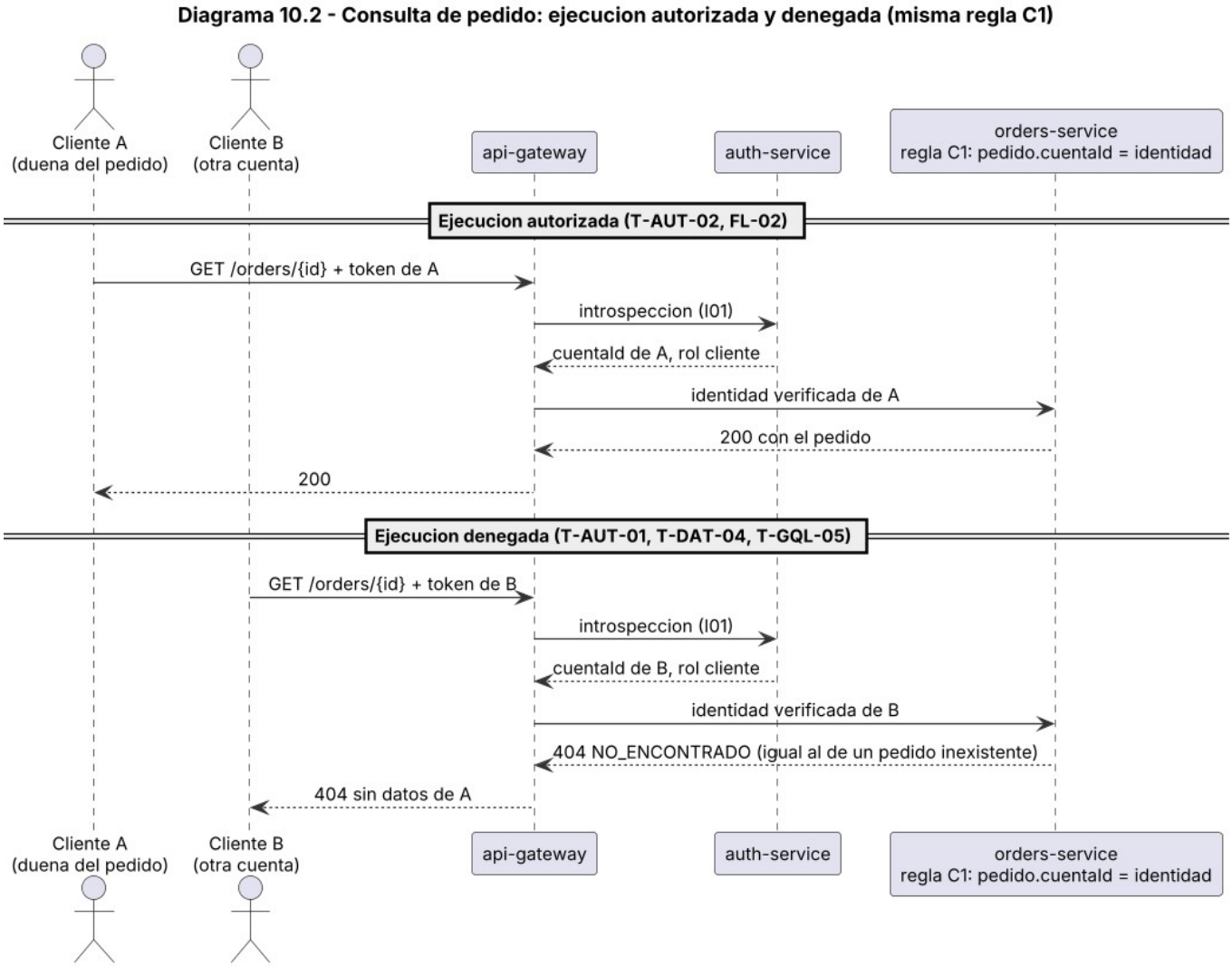
Entorno donde se ejecutaron las pruebas (las cifras de CPU y memoria del equipo no se registraron):

Dato	Valor
Sistema operativo	Windows 11
Python	3.14.10
Node	v26.13.0
Java (para ActiveMQ)	Eclipse Temurin 17.0.21
MongoDB	7.0.45 Community, servicio de Windows
Vault	1.18.5 (modo desarrollo)
ActiveMQ Classic	6.2.0
Navegador de la prueba E2E	Chromium de Playwright
Docker Compose (10.9.11)	Docker 29.9.2 y Compose v5.6.1 con Docker Desktop; imágenes mongo:7, hashicorp/vault:1.18 y apache/activemq-classic:6.2.0 (JRE 17)

10.7 Demostración de los flujos principales

`impl/tests/demo_flujos.py` ejecuta los flujos principales contra el sistema real y escribe `impl/tests/evidencia/demo-flujos.md`: registro con correo y verificación, catálogo, pedido con precio del servidor, pago por la plataforma de prueba con aviso firmado, boleta con correo, cola de despacho, impresión, avance de estados, reporte del dueño y detalle por GraphQL. Cada llamada muestra la respuesta y las líneas de registro de cada sistema con el mismo `requestId`.

Consulta protegida de un pedido: autorizada y denegada



Salida real de la demostración (`tests/evidencia/demo-flujos.md`):

Las dos ejecuciones aplican la misma regla en `orders-service`: el pedido solo se devuelve si `pedido.cuentaId` es la cuenta de la identidad verificada por el gateway. La cuenta ajena recibe el mismo 404 que un pedido que no existe.

Autorizada: la dueña del pedido

`GET /api/v1/orders/ped_0020` con requestId `demo-99952522` · C1 permite

Respuesta 200: `{"id": "ped_0020", "estado": "pendiente_pago", "estadoPago": "pendiente", "items": [{"productoId": "prod_002", "nombre": "Margarita", "precioUnitario": 7990, "cantidad": 2}], "total": 15980, "direccionEntrega": {"calle": "Calle Ficticia 321", "comuna": "Maipú", "referencia": ""}, "historialEstados": [{"estado": "pendiente_pago", "fecha": "2026-11-08T21:59:10Z"}], "boletaDisponible": false, "creadoEn": "2026-11-08T21: ...`

Registros con el mismo requestId:

```
api-gateway        GET /api/v1/orders/ped_0020 -> 200 (15 ms) cuentaId=cta_0018
auth-service       POST /internal/introspect -> 200 (4 ms) cuentaId=None
orders-service     GET /orders/ped_0020 -> 200 (2 ms) cuentaId=cta_0018
```

Denegada: otra cuenta con sesión válida

`GET /api/v1/orders/ped_0020` con requestId `demo-91561857` · C1 deniega

Respuesta 404: `{"error": {"codigo": "NO_ENCONTRADO", "mensaje": "Pedido no encontrado.", "requestId": "demo-91561857"}}`

Registros con el mismo requestId:

```
api-gateway         GET /api/v1/orders/ped_0020 -> 404 (14 ms) cuentaId=cta_0019
auth-service        POST /internal/introspect -> 200 (3 ms) cuentaId=None
orders-service      GET /orders/ped_0020 -> 404 (2 ms) cuentaId=cta_0019
```

Referencia: pedido que no existe

`GET /api/v1/orders/ped_9999` con requestId `demo-15107439` · C1

Respuesta 404: `{"error": {"codigo": "NO_ENCONTRADO", "mensaje": "Pedido no encontrado.", "requestId": "demo-15107439"}}`

Registros con el mismo requestId:

```
api-gateway         GET /api/v1/orders/ped_9999 -> 404 (15 ms) cuentaId=cta_0019
auth-service        POST /internal/introspect -> 200 (4 ms) cuentaId=None
orders-service      GET /orders/ped_9999 -> 404 (2 ms) cuentaId=cta_0019
```

Prueba	Qué comprueba	Resultado
T-AUT-01	Cliente B consulta pedido de A (S14): 404 identico a un pedido inexistente	OK. ajeno=404 NO_ENCONTRADO; inexistente=404
T-AUT-02	Cliente A consulta su pedido (S14): 200	OK. 200 estado=pagado
FL-02	Consulta de pedido: el dueno recibe 200 y otra cuenta 404 identico al inexistente (misma regla C1 en S14)	OK. dueno=200, otra cuenta=404 NO_ENCONTRADO, inexistente=404
T-DAT-04	Pedido ajeno por S14, S23 y S20: 404 identico al inexistente	OK. [404, 404, 404]
T-GQL-05	Pedido de otra cuenta e inexistente: NO_ENCONTRADO identico, sin datos de A ni llamada a inventory	OK. ajeno=['NO_ENCONTRADO'], inexistente=['NO_ENCONTRADO']
T-GQL-16	Orders sigue decidiendo: par graphql-to-orders con la identidad de otra cuenta recibe 404	OK. 404 NO_ENCONTRADO

10.8 Matriz requisito, pantalla, servicio, prueba, resultado y evidencia

Parte de la trazabilidad del punto 1.8. En la columna **Prueba**, `CT-Sxx` son los casos válido (P) e inválido (N) del servicio (10.9.4) y `E2E` son las pruebas con navegador (10.9.7). El resultado es OK solo si todas las pruebas de la fila están en OK.

Requisito	Pantalla y acción	Servicio	Prueba	Resultado	Evidencia
RF1, RF7	P01 Inicio: Cargar combos	S06 (`categoria=combo`)	CT-S06, E2E-01, E2E-M1	OK	tests/resultados/negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-01-inicio.png
RF1, RF7	P01 Inicio: Cargar "más pedidas"	S06 (`destacado=true`)	CT-S06, E2E-01, E2E-M1	OK	tests/resultados/negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-01-inicio.png
—	P01 Inicio: "Pedir combo" y "Agregar"; carrusel; "¿Cómo pedir?"	Sin servicio (carrito local o contenido fijo)	E2E-01, E2E-M1	OK	tests/resultados/e2e_frontend.json; tests/evidencia/e2e-01-inicio.png
RF1, RF7	P02 Menú: Cargar el catálogo y filtrar por categoría	S06	CT-S06, E2E-02, E2E-M2	OK	tests/resultados/negocio.json, tests/resultados/e2e_frontend.json
—	P02 Menú: "Agregar"	Sin servicio (carrito local)	E2E-02, E2E-M2	OK	tests/resultados/e2e_frontend.json
RF1	P03 Carrito: Mostrar el detalle de los ítems	S06 (`ids`)	CT-S06, E2E-08, E2E-09	OK	tests/resultados/negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-08-carrito.png
RF5	P03 Carrito: Precargar la dirección de entrega	S24	CT-S24, E2E-08, E2E-09	OK	tests/resultados/negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-08-carrito.png

Requisito	Pantalla y acción	Servicio	Prueba	Resultado	Evidencia
RF1	P03 Carrito: "Confirmar pedido"	S12	CT-S12, E2E-08, E2E-09	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-08-carrito.png
—	P03 Carrito: Cambiar cantidad y "Quitar"	Sin servicio (carrito local)	E2E-08, E2E-09	OK	tests/resultados/ e2e_frontend.json; tests/evidencia/e2e-08-carrito.png
—	P04 Sucursales: Ver ubicación, horario y mapa	Sin servicio (contenido informativo)	E2E-M5	OK	tests/resultados/ e2e_frontend.json
RF14	P05 Contacto: "Enviar mensaje"	S35	CT-S35, E2E-14, E2E-M4	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF12	P06 Modal de sesión (y navbar): "Entrar"	S01	CT-S01, E2E-04, E2E-07	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF12	P06 Modal de sesión (y navbar): "Cerrar sesión"	S02	CT-S02, E2E-04, E2E-07	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF1	P07 Modal "Pedido confirmado": Mostrar el resultado	Respuesta de S12	CT-S12, E2E-09	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF5, RF11	P08 Crear cuenta: "Crear cuenta"	S03	CT-S03, E2E-03, E2E-M3	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-03-registro.png
RF11, RF6	P09 Confirmar correo: Abrir el enlace y confirmar (o definir contraseña)	S04	CT-S04, E2E-05, E2E-06	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF11	P09 Confirmar correo: "Reenviar correo"	S05	CT-S05, E2E-05, E2E-06	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF2	P10 Pago y retorno: "Pagar"	S20	CT-S20, E2E-09, E2E-10	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-09-pasarela.png, e2e-10-retorno.png
RF2	P10 Pago y retorno: Consultar el estado tras volver del proveedor	S14	CT-S14, E2E-09, E2E-10	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-09-pasarela.png, e2e-10-retorno.png
RF1	P11 Mis pedidos: Cargar la lista	S13	CT-S13, T-GQL-15, E2E-12, E2E-15	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-11-detalle-graphql.png
RF1, RF2	P11 Mis pedidos: "Ver detalle"	S14	CT-S14, T-GQL-15, E2E-12, E2E-15	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-11-detalle-graphql.png
RF3	P11 Mis pedidos: "Anular pedido"	S15	CT-S15, T-GQL-15, E2E-12, E2E-15	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-11-detalle-graphql.png
RF8	P11 Mis pedidos: "Ver boleta"	S23	CT-S23, T-GQL-15, E2E-12, E2E-15	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-11-detalle-graphql.png

Requisito	Pantalla y acción	Servicio	Prueba	Resultado	Evidencia
RF5	P12 Mi cuenta: Cargar y "Guardar" el perfil	S24, S25	CT-S24, CT-S25, E2E-13	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-13-mi-cuenta.png
RF7, RF10	P13 Panel · Productos: Listar con stock	S07	CT-S07, E2E-19	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF10	P13 Panel · Productos: "Nuevo producto", "Guardar cambios", "Retirar"	S08, S09, S10	CT-S08, CT-S09, CT-S10, E2E-19	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF7	P13 Panel · Productos: "Disponibilidad y stock"	S11	CT-S11, E2E-19	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF2, RF4	P14 Panel · Pedidos y ventas: Listar pedidos y ver detalle	S16, S14	CT-S14, CT-S16, E2E-21	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF3	P14 Panel · Pedidos y ventas: "Anular" (administrador)	S17	CT-S17, E2E-21	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF4, RF8	P14 Panel · Pedidos y ventas: Listar ventas de la caja virtual y ver boleta	S22, S23	CT-S22, CT-S23, E2E-21	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF13	P15 Panel · Despacho: Ver la cola de órdenes de despacho	S18	CT-S18, E2E-16, E2E-17, E2E-18	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-17-despacho.png
RF13	P15 Panel · Despacho: "Imprimir orden"	S19	CT-S19, E2E-16, E2E-17, E2E-18	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-17-despacho.png
RF2	P15 Panel · Despacho: "Avanzar estado"	S17	CT-S17, E2E-16, E2E-17, E2E-18	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-17-despacho.png
RF6, RF10, RF11	P16 Panel · Clientes: "Registrar cliente"	S26	CT-S26, E2E-21	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF10	P16 Panel · Clientes: Listar, "Guardar cambios" y "Dar de baja"	S27, S28, S29	CT-S27, CT-S28, CT-S29, E2E-21	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF10, RF12	P17 Panel · Usuarios: "Nuevo usuario", listar, "Guardar" y "Dar de baja"	S30, S31, S32, S33	CT-S30, CT-S31, CT-S32, CT-S33, E2E-21, E2E-22	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json
RF9	P18 Panel · Reporte: "Generar reporte"	S34	CT-S34, E2E-20	OK	tests/resultados/ negocio.json, tests/resultados/e2e_frontend.json; tests/evidencia/e2e-20-reporte.png
RF2, RF4, RF8	Sin pantalla: La plataforma de pago avisa el resultado	S21	T-AUT-08, T-DAT-06, C4	OK	tests/resultados/ seguridad.json, tests/resultados/carga.json

Requisitos del Caso 19 resumidos (cada uno con las pruebas que lo cubren):

Requisito	Descripción	Servicios	Pruebas	Resultado
RF1	Armar un pedido con los productos disponibles	S06, S12, S13, S14	CT-S06-P, CT-S12-P, CT-S13-P, CT-S14-P, C3, FL-03	OK
RF2	Preparar la orden tras el pago por plataforma externa	S20, S21, S17, I12	CT-S20-P, T-AUT-08b, FL-01, CT-S17-P	OK
RF3	Anular una compra con motivo	S15, S17	CT-S15-P, CT-S15-N, FL-04, T-AUT-03	OK

Requisito	Descripción	Servicios	Pruebas	Resultado
RF4	Ventas solo a clientes registrados, en la caja del cajero virtual	S12, S22	CT-S12-N, CT-S22-P, FL-01	OK
RF5	Registro de clientes con todos los datos	S03, S24, S25	CT-S03-P, CT-S03-N, CT-S24-P, CT-S25-P	OK
RF6	Registro por la web o por el administrador, con validación del correo	S03, S26, S04	CT-S26-P, CT-S04-P, E2E-03, E2E-05	OK
RF7	Consultar y actualizar la disponibilidad	S06, S07, S11	CT-S07-P, CT-S11-P, CT-S11-N	OK
RF8	Boleta digital por correo	S23, I13	CT-S23-P, FL-01, F7	OK
RF9	Reporte de ventas por período para el dueño	S34	CT-S34-P, CT-S34-N, C5, C6, E2E-20	OK
RF10	Mantenedores de productos, usuarios y clientes	S08 a S11, S26 a S33	CT-S08-P, CT-S09-P, CT-S10-P, CT-S28-P, CT-S29-P, CT-S30-P, CT-S32-P, CT-S33-P	OK
RF11	Validar que el correo existe	S03, S04, S05, I06	CT-S03-P, CT-S04-P, CT-S05-P, E2E-03	OK
RF12	Autenticación y acceso según el perfil	S01, S02, S30 a S33	CT-S01-P, CT-S02-P, T-AUT-05, T-AUT-09, T-DAT-08	OK
RF13	Órdenes de despacho en orden de llegada, impresas por el encargado	S18, S19, cola	CT-S18-P, CT-S19-P, C9, F2, F3	OK
RF14	Formulario de contacto (extra del frontend)	S35	CT-S35-P, CT-S35-N, E2E-14	OK

10.9 Resultados de las pruebas

Suite	Archivo con la salida	Pruebas	OK
Carga (M9)	`tests/resultados/carga.json`	11	11
Docker Compose: acceso directo, bases de datos y compra completa en la red privada	`tests/resultados/compose_interna.json`	18	18
Docker Compose: redes y puertos (M6)	`tests/resultados/compose_red.json`	10	10
Frontend con navegador real	`tests/resultados/e2e_frontend.json`	27	27
Fallos y desconexiones F1 a F4, F6 y F7 (M9)	`tests/resultados/fallos_F1_F2_F3_F4_F6_F7.json`	6	6
Fallo F5, Vault detenido (M9)	`tests/resultados/fallos_F5.json`	1	1
API Gateway	`tests/resultados/gateway.json`	13	13
GraphQL (M8)	`tests/resultados/graphql.json`	16	16
Contratos y flujos de negocio	`tests/resultados/negocio.json`	73	73
Retención y eliminación de datos (M7, 7.9)	`tests/resultados/retencion.json`	14	14
Autorización y protección de datos (M4 y M7)	`tests/resultados/seguridad.json`	32	32
Total		**221**	**221**

Todas las pruebas salen de una sola ejecución de `python probar_todo.py` en Windows el 8 de noviembre de 2026 (unos 10 minutos): primero la batería local y después Docker Compose. Antes hubo corridas con dos fallas de las propias pruebas, que corregimos (10.10).

10.9.1 Autorización por recurso y rol (Módulo 4)

Prueba	Qué comprueba	Resultado	Observado
T-AUT-01	Cliente B consulta pedido de A (S14): 404 identico a un pedido inexistente	OK	ajeno=404 NO_ENCONTRADO; inexistente=404
T-AUT-02	Cliente A consulta su pedido (S14): 200	OK	200 estado=pagado
T-AUT-03	Anular pedido ajeno (404) y propio en_preparacion (409)	OK	ajeno=404, en_preparacion=409 TRANSICION_INVALIDA
T-AUT-04	Cliente A... B pide boleta de pedido de A (S23): 404	OK	404 NO_ENCONTRADO
T-AUT-05	Cliente intenta DELETE y POST de producto: 403	OK	DELETE=403, POST=403
T-AUT-06	Visitante: catalogo 200 sin stock; pedido 401	OK	catalogo=200, pedido=401
T-AUT-07	encargado_despachos no anula (403); nadie pasa a pagado por S17 (409)	OK	anular=403, a pagado=[409, 409]

Prueba	Qué comprueba	Resultado	Observado
T-AUT-08a	Webhook sin firma, con firma invalida y con marca de tiempo vieja: 401 y el pedido no cambia	OK	401,401,401; estado=pendiente_pago
T-AUT-08b	Webhook valido: 200 y el pedido pasa a pagado	OK	webhook=200, pagado=True
T-AUT-09	Dueno: lista de pedidos 403 y reporte 200	OK	S16=403, S34=200
T-AUT-10	Cabecera X-Authenticated-Roles del navegador se descarta: 403 con el rol real	OK	403 SIN_PERMISO
T-AUT-11	Llamada directa a orders sin secreto (401) y con secreto sin identidad (401)	OK	sin secreto=401, sin identidad=401
T-AUT-12	clients-service llama a payment-confirmation de orders: 403	OK	403 LLAMADOR_NO_PERMITIDO
T-AUT-13	Introspeccion no se expone por el gateway: 404	OK	404
T-AUT-14	Administrador no se da de baja (409) ni se cambia el rol del ultimo admin (409)	OK	baja=409 NO_PUEDE_ELIMINARSE, rol=409 ULTIMO_ADMINISTRADOR

10.9.2 Protección de datos y sesiones (Módulo 7)

Prueba	Qué comprueba	Resultado	Observado
T-DAT-01	Catalogo anonimo: 200, campos publicos, sin stock ni retirados	OK	200, 12 productos
T-DAT-02	Sin token, token inventado y token cerrado (S02): 401 con formato de error	OK	[401, 401, 401]
T-DAT-03	Rutas de administracion con cliente y cajero: 403; segunda barrera en inventory: 403	OK	gateway=[403, 403, 403], directo=403
T-DAT-04	Pedido ajeno por S14, S23 y S20: 404 identico al inexistente	OK	[404, 404, 404]
T-DAT-05	Acceso directo sin identidad autorizada: (b) sin secreto, (c) secreto de otro destino, (d) sin usuario, (e) graphql: 401	OK	[401, 401, 401, 401, 401]
T-DAT-06a-b	Aviso sin firma y con marca de tiempo de hace mas de 5 minutos: 401	OK	401,401
T-DAT-06c	Firma valida con monto distinto: 200, pago observado y pedido sin cambios	OK	webhook=200, pago=observado, pedido=pendiente_pago
T-DAT-06d	Mismo eventoId dos veces: un solo pagado y una sola venta	OK	segundo aviso=200, ventas=1
T-DAT-06e	Cliente intenta PATCH estado=pagado: 403	OK	403
T-DAT-06f	POST /orders con total, estado y rol falsos: precio del catalogo y pendiente_pago	OK	201 total=8990 (catalogo=8990) estado=pendiente_pago
T-DAT-07	Contrasena y hash no salen en respuestas; en la base solo hay hash bcrypt	OK	hash=12$... (bcrypt), campos=['_id', 'correo', 'creadoEn', 'estado', 'hash', 'id', 'nombre', 'rol']
T-DAT-08	Sesion revocada de inmediato: logout, baja de cliente (S29), cambio de rol (S32) y baja de usuario (S33)	OK	antes=200; logout=401, baja cliente=401, cambio rol=401, baja usuario=401
T-DAT-09	Registros de todos los componentes sin contrasena, token, correo, RUN, telefono, direccion ni secretos	OK	15 valores buscados en 25 archivos; coincidencias=0
T-DAT-10	Campos minimos por rol: S16 sin RUN/correo/telefono, S34 sin datos de clientes, S18 sin precios/RUN/correo	OK	S16=True, S34=True, S18=True
T-DAT-11	Sin secretos ni valores de Vault en frontend, codigo e informe	OK	archivos revisados; coincidencias=0
T-DAT-12	Token vencido (vigencia de prueba de 3 s): 401	OK	antes=200, despues=401 TOKEN_INVALIDO
T-DAT-13	Cinco fallos devuelven 401 y el sexto 429 con Retry-After de 15 minutos; correo inexistente igual que contrasena mala	OK	[401, 401, 401, 401, 401, 429], Retry-After=900

10.9.3 GraphQL Gateway (Módulo 8)

Prueba	Qué comprueba	Resultado	Observado
T-GQL-01	Agregacion autorizada: pedido con el producto vigente en cada item; mismo requestId en los registros de los tres sistemas	OK	200, 2 items con producto; registros con el requestId en ['api-gateway.log', 'auth-service.log', 'graphql-gateway.log', 'inventory-service.log', 'orders-service.log']

Prueba	Qué comprueba	Resultado	Observado
T-GQL-02	Una sola llamada a inventory (GET /products?ids=) para los 2 items	OK	llamadas a inventory con el requestId = 1
T-GQL-03	Inventory detenido: pedido con producto null y SUBSISTEMA_NO_DISPONIBLE con path; menos de 15 s	OK	200, errores=['SUBSISTEMA_NO_DISPONIBLE', 'SUBSISTEMA_NO_DISPONIBLE'], path=['resumenPedido', 'items', 0, 'producto'], 0.0 s
T-GQL-04	Orders detenido: resumenPedido null, SUBSISTEMA_NO_DISPONIBLE y ninguna llamada a inventory	OK	200, errores=['SUBSISTEMA_NO_DISPONIBLE'], llamadas a inventory=0
T-GQL-05	Pedido de otra cuenta e inexistente: NO_ENCONTRADO identico, sin datos de A ni llamada a inventory	OK	ajeno=['NO_ENCONTRADO'], inexistente=['NO_ENCONTRADO']
T-GQL-06	clienteNombre con cliente: CAMPO_NO_AUTORIZADO sin llamadas a sistemas; con encargado_despachos si llega	OK	cliente=['CAMPO_NO_AUTORIZADO'] , despachos nombre recibido=True
T-GQL-07	resumenPedido con dueno y miCuenta con cajero_virtual: OPERACION_NO_AUTORIZADA	OK	['OPERACION_NO_AUTORIZADA'], ['OPERACION_NO_AUTORIZADA']
T-GQL-08	Consulta mas profunda que el limite (configurado en 2 para la prueba): LIMITE_PROFUNDIDAD sin llamar a sistemas	OK	400 ['LIMITE_PROFUNDIDAD']; limite real de produccion = 5 (el schema no admite más de 5 niveles)
T-GQL-09	Tres alias de resumenPedido completo superan el costo maximo: LIMITE_COSTO sin llamar a sistemas	OK	400 ['LIMITE_COSTO']
T-GQL-10	limitePedidos: 500 fuera de rango: CONSULTA_INVALIDA	OK	400 ['CONSULTA_INVALIDA']
T-GQL-11	Mutation e introspeccion: CONSULTA_INVALIDA	OK	['CONSULTA_INVALIDA'], ['CONSULTA_INVALIDA']
T-GQL-12	Sin Authorization: 401 y la peticion no llega al GraphQL Gateway	OK	401, registros de graphql-gateway con ese requestId=0
T-GQL-13	Acceso directo al GraphQL Gateway sin secreto y con el de otro destino: 401	OK	sin secreto=401, secreto de otro destino=401. La parte "conexion rechazada desde el host" NO se pudo verificar: en modo nativo todos los puertos escuchan en loopback del mismo equipo.
T-GQL-14	Subsistemas desde el navegador: el API Gateway no tiene ruta hacia orders-service, inventory-service ni clients-service ni hacia /internal (404)	OK	rutas por nombre de servicio=[404, 404, 404], rutas /internal=[404, 404]. La parte "puerto no publicado" NO se pudo verificar en modo nativo (puertos en loopback).
T-GQL-15	P11 usa una sola llamada POST /api/v1/graphql y ninguna a /orders/{id} ni /products	OK	graphql=1, otras=0
T-GQL-16	Orders sigue decidiendo: par graphql-to-orders con la identidad de otra cuenta recibe 404	OK	404 NO_ENCONTRADO
T-GQL-17	GraphQL solo lee: POST /orders y PATCH de estado con el par graphql-to-orders: 403	OK	POST=403, PATCH=403

10.9.4 Contratos de los 35 servicios

Cada servicio tiene un caso válido (P) y uno inválido (N). Los datos de la columna "Observado" son los reales.

Servicio	Caso válido	Resultado	Caso inválido	Resultado
S01	login valido: 200 con token — 200 claves=['access_token', 'expires_in', 'nombre', 'rol', 'token_type']	OK	contrasena incorrecta: 401 CREDENCIALES_INVALIDAS — 401 CREDENCIALES_INVALIDAS	OK
S02	logout: 204 — 204	OK	el token cerrado ya no sirve: 401 — 401	OK
S03	registro valido: 201 pendiente_verificacion — 201 pendiente_verificacion	OK	RUN repetido 409 y RUN invalido 400 — repetido=409 RUN_DUPLICADO, invalido=400	OK
S04	verificacion con el enlace del correo: 200 — 200	OK	el mismo enlace por segunda vez: 410 — 410 TOKEN_VENCIDO	OK
S05	correo existente o no: siempre 202 (no revela cuentas) — inexistente=202, existente(ya verificado)=202	OK	correo mal formado: 400 — 400	OK

Servicio	Caso válido	Resultado	Caso inválido	Resultado
S06	catalogo por categoria: 200 solo premium — 200 3 productos	OK	categoria invalida: 400 — 400	OK
S07	administrador ve stock y retirados: 200 — 200 12 productos	OK	cajero_virtual: 403 — 403	OK
S08	dueno crea producto: 201 — 201 prod_013	OK	nombre repetido 409 y precio negativo 400 — 409, 400	OK
S09	modificar precio: 200 — 200 precio=10990	OK	producto inexistente: 404 — 404	OK
S10	retirar producto: 204 y desaparece del catalogo publico — 204, en catalogo=False	OK	producto inexistente: 404 — 404	OK
S11	stock 0 deja el producto no disponible — 200 {'id': 'prod_013', 'stock': 0, 'disponible': False}	OK	stock negativo: 400 — 400	OK
S12	pedido valido: 201 pendiente_pago; misma Idempotency-Key devuelve el mismo pedido — 201 ped_0007; repetido -> ped_0007	OK	fuera de cobertura 422 y cantidad invalida 400 — 422 FUERA_DE_COBERTURA, 400	OK
S13	mis pedidos: 200 y contiene el pedido — 200 1 pedidos	OK	filtro de estado invalido: 400 — 400	OK
S14	detalle del pedido propio: 200 — 200	OK	pedido inexistente: 404 — 404	OK
S15	anular pedido pendiente: 200 anulado y el stock se libera — 200; stock prod_004 inicial=30, reservado=28, liberado=30	OK	motivo corto 400; anular dos veces 409 — 400, 409 TRANSICION_INVALIDA	OK
S16	cajero lista pedidos pagados: 200 con el pedido — 200 3	OK	fecha invalida: 400 — 400	OK
S17	pagado -> en_preparacion -> en_despacho -> entregado; el administrador anula con motivo — [200, 200, 200]; anulacion=200	OK	retroceder el estado: 409 TRANSICION_INVALIDA — 409 TRANSICION_INVALIDA	OK
S18	la orden llega a la cola de despacho por el broker — orden OD-000004	OK	filtro invalido 400 — 400	OK
S19	imprimir orden: HTML con el pedido y queda impresa — 200 html=432 bytes, impresa=True	OK	orden inexistente: 404 — 404	OK
S20	iniciar pago: 201 con urlPago de la pasarela de prueba — 201 estado=pendiente	OK	medio invalido 400; pedido ya pagado 409 — 400, 409 PEDIDO_NO_PAGABLE	OK
S21	Aviso válido firmado — webhook=200, pagado=True (T-AUT-08b)	OK	Sin firma, firma inválida y marca vieja — 401,401,401; estado=pendiente_pago (T-AUT-08a)	OK
S22	cajero consulta ventas: 200 con la venta del pedido — 200 5 ventas	OK	fecha invalida 400 — 400	OK
S23	cliente obtiene su boleta: 200 con numero B- — 200 B-000004 total=19480 iva=3110	OK	pedido no pagado 409; encargado_despachos 403 — 409 PEDIDO_NO_PAGADO, 403	OK
S24	perfil propio: 200 con RUN y direccion — 200	OK	un administrador no tiene perfil de cliente: 403 — 403	OK
S25	cambiar telefono: 200 — 200	OK	telefono invalido: 400 — 400	OK
S26	administrador crea cliente: 201 — 201 cli_0012	OK	RUN repetido 409; cajero 403 — 409, 403	OK
S27	buscar cliente por nombre: 200 — 200 1 resultados	OK	limit fuera de rango: 400 — 400	OK
S28	modificar cliente: 200 — 200	OK	cliente inexistente: 404 — 404	OK
S29	baja de cliente: 204 — 204	OK	cliente inexistente: 404 — 404	OK
S30	crear usuario de personal: 201 pendiente y llega el correo de activacion — 201 usr_0006 rol=cajero_virtual estado=pendiente_verificacion	OK	correo repetido 409; rol cliente 400 — 409, 400	OK
S31	listar por rol: 200 con el usuario — 200 2	OK	rol invalido: 400 — 400	OK
S32	cambiar rol: 200 — 200 rol=encargado_despachos	OK	usuario inexistente: 404 — 404	OK

Servicio	Caso válido	Resultado	Caso inválido	Resultado
S33	baja de usuario: 204 — 204	OK	usuario inexistente: 404 — 404	OK
S34	reporte de ventas del dueno: 200 con totales y maximo 10 productos — 200 ventas=5 total=51440	OK	rango de mas de 92 dias: 400 — 400 RANGO_INVALIDO	OK
S35	mensaje de contacto: 201 con id msg_ — 201 msg_0001	OK	motivo fuera de la lista: 400 — 400	OK

10.9.5 Flujos de negocio de extremo a extremo

Prueba	Qué comprueba	Resultado	Observado
FL-01	Compra completa: pedido -> pago por la pasarela de prueba -> pagado -> venta y boleta -> correo con boleta -> orden de despacho -> stock descontado	OK	estado pagado=True, correo de boleta=True, orden=OD-000006, stock prod_007 200 -> 198
FL-02	Consulta de pedido: el dueno recibe 200 y otra cuenta 404 identico al inexistente (misma regla C1 en S14)	OK	dueno=200, otra cuenta=404 NO_ENCONTRADO, inexistente=404
FL-03	Stock insuficiente: pedir 2 con stock 1 da 409 STOCK_INSUFICIENTE; el ultimo se reserva una sola vez	OK	409 SIN_STOCK, 201, 409 SIN_STOCK
FL-04	Pedido sin pago vencido (vigencia de prueba de 4 s): pasa a anulado, libera el stock y ya no se puede pagar	OK	anulado=True, stock reservado=49 -> liberado=50, pago=409 PEDIDO_NO_PAGABLE
FL-05	Servidor de correo de prueba detenido al pagar: la venta existe, el pedido queda pagado y la boleta queda pendiente de reintento	OK	venta=True, correoEnviado=False, intentos=0, pedido=pagado

10.9.6 API Gateway: límites, CORS, TLS, salud y registros

Prueba	Qué comprueba	Resultado	Observado
GW-BODY	Cuerpo de mas de 64 KB en login: 413	OK	413 CUERPO_DEMASIADO_GRANDE
GW-CABECERAS	Cabeceras de seguridad en las respuestas del API (incluida Content-Security-Policy)	OK	{'x-content-type-options': 'nosniff', 'cache-control': 'no-store', 'strict-transport-security': 'max-age=31536000', 'x-frame-options': 'DENY', 'referrer-policy': 'no-referrer', 'content-security-policy': "default-src 'none'; frame-ancestors 'none'"}
GW-CORS	CORS: solo el origen del frontend; un origen ajeno no recibe Access-Control-Allow-Origin	OK	frontend=http://127.0.0.1:8080, ajeno=None/None
GW-CSP	El frontend envia Content-Security-Policy: conexiones solo al API Gateway, sin plugins, sin base ajena, sin ser incrustado y formularios solo al propio sitio	OK	default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src 'self' data: https://images.unsplash.com https://maps.googleapis.com https://maps.gstatic.com; connect-src 'self' https://127.0.0.1:8443; frame-src https://maps.google.com https://www.google.com; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'
GW-HEALTH	Las 11 piezas responden /health 200 por TLS	OK	{'auth': 200, 'clients': 200, 'inventory': 200, 'orders': 200, 'payments': 200, 'dispatch': 200, 'notifications': 200, 'graphql': 200, 'pasarela': 200, 'correo': 200, 'gateway': 200}
GW-LOG	Los registros son JSON de una linea con requestId y sin cuerpos ni cabeceras	OK	2196 lineas revisadas
GW-RL-CATALOGO	Catalogo publico: 120 por minuto por IP; la peticion 121 recibe 429	OK	200=120, 429=10, primer 429 en la peticion 121
GW-RL-RETRY	La respuesta 429 trae Retry-After y el formato de error con requestId	OK	429 Retry-After=59 codigo=DEMASIADAS_SOLICITUDES
GW-RL-SENSIBLE	Rutas sensibles (login, registro): 10 por minuto por IP; desde la 11 hay 429	OK	[401, 401, 401, 401, 401, 401, 401, 401, 401, 429, 429, 429, 429]

Prueba	Qué comprueba	Resultado	Observado
GW-RL-USUARIO	Rutas autenticadas: 120 por minuto por cuenta; luego 429	OK	200=120, 429=10
GW-RL-WEBHOOK	Aviso de pago: 60 por minuto; los no firmados dan 401 y desde el 61 hay 429	OK	401=60, 429=6
GW-RUTAS	Ruta inexistente 404, metodo no permitido 404/405 y JSON invalido 400, todos con formato de error	OK	404, 405, 400
GW-TLS	El gateway solo atiende por TLS 1.2+ y el certificado se verifica con la CA del proyecto	OK	HTTP plano -> b''; version=TLSv1.3

10.9.7 Frontend con navegador real (Chrome)

Prueba	Qué comprueba	Resultado	Observado
E2E-01	Inicio muestra combos y destacados desde S06	OK	4 combos, 3 destacados
E2E-02	Menu filtra por categoria con datos del servidor	OK	todas=8, premium=3
E2E-03	Registro de cliente (S03) deja la cuenta pendiente y envia correo	OK	mensaje de confirmacion visible
E2E-04	Login con correo sin confirmar se rechaza (403)	OK	Confirma tu correo para activar la cuenta. (código de soporte: req_3633c0680ae5)
E2E-05	Confirmar correo con el enlace de un solo uso (S04)	OK	cuenta activa
E2E-06	El enlace de confirmacion no se puede reutilizar (410)	OK	410 TOKEN_VENCIDO
E2E-07	Login (S01): el token queda solo en sessionStorage, no en localStorage	OK	token en sessionStorage; localStorage solo tiene el carrito
E2E-08	Carrito precarga la direccion del perfil (S24) y no cobra despacho	OK	calle=Calle Ficticia 100, total=$10.980
E2E-09	Confirmar pedido (S12) + iniciar pago (S20) redirige a la plataforma de pago	OK	https://127.0.0.1:9201
E2E-10	Retorno de pago: el estado "pagado" viene del servidor (S14) tras el aviso firmado	OK	Pago confirmado. Tu pedido está en cola de preparación.
E2E-12	Boleta digital del pedido pagado (S23)	OK	Boleta B-000009 · 2026-11-08T21:58:44Z Napolitana × 1 — $8.990 Bebida 500ml × 1 — $1.990 Total $10.980 (IVA incluido $1.
E2E-13	Mi cuenta (miCuenta GraphQL + S24) muestra perfil y ultimos pedidos	OK	ped_0018 · pagado · $10.980
E2E-14	Formulario de contacto guarda el mensaje (S35)	OK	confirmacion visible
E2E-15	Anular un pedido pendiente propio (S15)	OK	estado Anulado visible
E2E-16	Panel del encargado de despachos muestra solo sus secciones	OK	['Pedidos y ventas', 'Despacho']
E2E-17	Cola de despacho (S18) muestra ordenes pagadas	OK	5 ordenes en cola
E2E-18	Imprimir orden (S19) abre el documento HTML con numero de orden	OK	Orden OD-000005 Pedido ped_0009 - pagado 2026-11-08T21:58:11Z Cliente: Cliente na 800931 - Tel. +5
E2E-19	Panel del dueno: productos y reporte	OK	['Productos', 'Reporte de ventas']
E2E-20	Reporte de ventas (S34) con total y productos mas vendidos	OK	Total recaudado: $104.360 · Ventas: 9 El total no descuenta reembolsos pendientes. Por día Fecha Ventas Total 2026-11-
E2E-21	Panel del administrador muestra las seis secciones	OK	['Productos', 'Pedidos y ventas', 'Despacho', 'Clientes', 'Usuarios', 'Reporte de ventas']
E2E-22	Usuarios del personal (S31) sin contrasenas ni hashes	OK	6 usuarios
E2E-M1	Telefono 390 px: index.html sin desplazamiento horizontal y con menu plegable	OK	scrollWidth=398, innerWidth=398, menu plegable=True
E2E-M2	Telefono 390 px: menu.html sin desplazamiento horizontal y con menu plegable	OK	scrollWidth=390, innerWidth=390, menu plegable=True
E2E-M3	Telefono 390 px: registro.html sin desplazamiento horizontal y con menu plegable	OK	scrollWidth=390, innerWidth=390, menu plegable=True
E2E-M4	Telefono 390 px: contacto.html sin desplazamiento horizontal y con menu plegable	OK	scrollWidth=390, innerWidth=390, menu plegable=True

Prueba	Qué comprueba	Resultado	Observado
E2E-M5	Telefono 390 px: sucursales.html sin desplazamiento horizontal y con menu plegable	OK	scrollWidth=390, innerWidth=390, menu plegable=True

Capturas: `impl/tests/evidencia/e2e-*.png`. Esta suite recorre el frontend de demostración; no interviene en las demás suites, que prueban el backend por su API.

10.9.8 Qué se ejecutó y a qué escala

Las pruebas de carga y de fallos del Módulo 9 se ejecutaron en el modo local de Windows descrito en 10.6, contra MongoDB 7.0.45, Vault 1.18.5 y ActiveMQ 6.2.0 reales, con el generador de carga en la misma máquina que el sistema. Datos: pedidos y pagos creados por el propio script, y 13.800 y 138.000 ventas generadas con la semilla 42. Diferencias con el plan de 9.8 (se repiten en 10.11): duraciones al 25 % (15 s en lugar de 60 s y 30 s en lugar de 120 s), sin límite de 1 CPU y 512 MB por proceso, y el reporte de 92 días lee resúmenes diarios (9.7). La columna de CPU y memoria muestra los tres procesos con más CPU de cada escenario; la CPU es porcentaje de un núcleo y puede pasar de 100 cuando un proceso usa varios hilos. En C6 el 99 % de `payments` corresponde a la siembra de las 138.000 ventas, previa a la medición. En C1-10u el registro trae una lectura de 18.646 % de CPU para ActiveMQ en la primera muestra, que no es plausible (el resto de sus lecturas fue 0 %); se descarta de la tabla y no se pudo establecer su causa.

10.9.9 Carga (C1 a C9)

ID	Escenario	Objetivo	Concurrencia y duración	p50 / p95 / p99 (ms)	Throughput (por s)	Errores 5xx	CPU máx. y memoria máx.	¿Cumple?
C1-10u	C1 Catálogo (S06), 10 usuarios	p95 ≤ 300 ms hasta 50 por segundo; errores < 1 %	10 usuarios, 1 consulta por segundo cada uno, 16.2 s	19.7 / 108.9 / 114.2	8.0	0 (0.0 %)	inventory 6 % / 85 MB; vault 6 % / 184 MB; api_gateway 5 % / 70 MB	OK
C1-50u	C1 Catálogo (S06), 50 usuarios	p95 ≤ 300 ms hasta 50 por segundo; errores < 1 %	50 usuarios, 1 consulta por segundo cada uno, 16.2 s	23.3 / 203.8 / 373.0	40.1	0 (0.0 %)	api_gateway 46 % / 73 MB; inventory 25 % / 85 MB; mongod 11 % / 74 MB	OK
C1-100u	C1 Catálogo (S06), 100 usuarios	p95 ≤ 300 ms hasta 50 por segundo; errores < 1 %	100 usuarios, 1 consulta por segundo cada uno, 16.2 s	23.7 / 361.0 / 735.0	78.4	0 (0.0 %)	api_gateway 65 % / 70 MB; inventory 31 % / 84 MB; mongod 6 % / 74 MB	OK
C2	Crear pedido (S12)	p95 ≤ 800 ms; errores < 1 %	2 por segundo, 30.3 s, más ráfaga de 10	30.7 / 154.5 / 157.8	2.0	0 (0.0 %)	api_gateway 12 % / 70 MB; inventory 5 % / 85 MB; clients 5 % / 84 MB	OK
C3	Sin sobreventa	10 creados, 20 con 409, stock final 0	30 simultáneas	203.2 / 267.4 / 270.2	104.3	0	—	OK (creados 10, 409: 20, stock final 0)
C4	Aviso de pago (S21)	p95 ≤ 500 ms; una venta por eventoId	5 por segundo, 20 % repetidos, 15.2 s	12.6 / 305.5 / 475.0	6.0	0 (0.0 %)	payments 26 % / 87 MB; mongod 17 % / 77 MB; api_gateway 14 % / 72 MB	OK
C5	Reporte S34, 13.800 ventas	p95 ≤ 1500 ms; errores < 1 %	5 consultas simultáneas, 15.1 s	30.3 / 40.5 / 56.4	135.5	0 (0.0 %)	payments 104 % / 88 MB; api_gateway 82 % / 69 MB; mongod 70 % / 104 MB	OK
C6	Reporte S34, 138.000 ventas	p95 ≤ 5000 ms; sin superar la memoria	5 consultas simultáneas, 15.1 s	30.4 / 39.3 / 57.3	135.8	0 (0.0 %)	payments 107 % / 87 MB; mongod 90 % / 329 MB; api_gateway 84 % / 70 MB	OK
C7	Detalle GraphQL (R36)	p95 ≤ 600 ms; una llamada a inventory	10 por segundo, 16.3 s	19.3 / 20.3 / 20.8	9.2	0 (0.0 %)	api_gateway 15 % / 70 MB; auth 14 % / 85 MB; inventory 11 % / 84 MB	OK
C8	REST (S14 + S06) frente a GraphQL	Se informa la diferencia	100 repeticiones secuenciales	REST p50 19.3 / p95 22.7; GraphQL p50 15.8 / p95 19.3	—	0	—	OK. Diferencia de p95: -3.4 ms; llamadas del navegador: 2 frente a 1
C9	Despacho (I14)	200 en orden, sin pérdida ni duplicados, en menos de 30 s	200 mensajes en 12.6 s	—	—	—	—	OK (recibidas 200, en orden sí, última a los 12.6 s)

Lectura de los resultados: C5 y C6 miden 40.5 ms y 39.3 ms de p95 porque el reporte lee los resúmenes diarios (a lo más 92 documentos) y no recorre las ventas; con 10 veces más ventas la latencia casi no cambia. La decisión de diseño es de 9.7. No se midió en esta ejecución una agregación directa sobre las ventas, por lo que no se afirma cuánto habría costado. En C8 la diferencia de p95 entre REST y GraphQL fue de 3,4 ms (GraphQL más rápido); la ventaja que interesa es una llamada del navegador en lugar de dos, no una latencia menor de forma sistemática. C1-100u superó 300 ms de p95 (361.0 ms): su objetivo cubre hasta 50 consultas por segundo y con 100 usuarios la prueba generó 78.4 por segundo con el generador y el sistema en el mismo equipo; se informa, no se corrige. Estas cifras valen para este equipo y esta escala; no se extrapolan a otro entorno.

10.9.10 Fallos y desconexiones (F1 a F7)

ID	Fallo	Resultado observado	¿Cumple?
F1	Inventory detenido durante la creacion de pedidos: 503 controlado, sin pedidos a medias, vuelve a 201	{"detenido_en_s": 5.8, "reiniciado_en_s": 20.9, "respuestas_durante_la_caida": [[503, "INVENTARIO_NO_DISPONIBLE"]], "pedidos_sin_reserva": 0, "segundos_hasta_el_primer_201": 0.1}	OK
F2	Broker detenido durante un pago: pedido pagado con despachoPublicado en falso; al reiniciar la orden llega una sola vez	{"estado_con_broker_caido": "pagado", "despachoPublicado_con_broker_caido": false, "segundos_tras_reiniciar_hasta_dispatch": 1.0, "ordenes_en_dispatch": 1, "despachoPublicado_final": true, "objetivo_del_plan_s": 30}	OK
F3	Dispatch reiniciado con 21 mensajes pendientes (uno repetido): reconecta, consume y no duplica ordenes	{"ordenes_antes_del_reinicio": 0, "ordenes_despues": 20, "duplicadas": 0, "segundos_hasta_consumir": 0.0}	OK
F4	Orders sin responder mas de 5 s durante el aviso: el aviso se registra, el pago queda aprobado sin confirmar y el pedido pasa a pagado al normalizarse	{"respuesta_del_aviso": 200, "pago_durante_la_demora": "aprobado", "confirmadoEnPedidos_durante_la_demora": false, "segundos_hasta_pagado_tras_reanudar_orders": 0.0}	OK
F5	Vault detenido: el gateway sigue con el secreto en memoria durante la ventana (25 s en la prueba, 15 min en produccion) y luego responde 503	{"antes_de_la_ventana": [200, 200], "dentro_de_la_ventana": [200], "despues_de_la_ventana": 503, "codigo": "SERVICIO_NO_DISPONIBLE", "catalogo_publico_despues": 503, "ventana_prueba_s": 25, "ventana_plan_s": 900}	OK
F6	Auth detenido: el catalogo y el aviso de pago siguen; las rutas con token responden 503	{"catalogo": 200, "aviso_de_pago": 200, "ruta_con_token": 503, "codigo": "AUTENTICACION_NO_DISPONIBLE", "pedido_pagado_despues": true}	OK
F7	Pasarela sin respuesta: S20 502/503 sin guardar pago. Correo sin respuesta: boleta con correoEnviado en falso y reintento	{"S20_con_pasarela_caida": 503, "codigo": "PASARELA_NO_DISPONIBLE", "pagos_guardados": 0, "correoEnviado_con_correo_caido": false, "segundos_hasta_reintento_exitoso": 0.0, "intervalo_de_reintento_de_la_prueba_s": 10, "intervalo_del_plan_s": 300}	OK

10.9.11 Despliegue en Docker Compose

Se ejecutó el `docker-compose.yml` del Módulo 6 en Windows con Docker Desktop (`python scripts/compose.py up`): imágenes `mongo:7`, `hashicorp/vault:1.18` (modo desarrollo) y ActiveMQ 6.2.0 construida con `Dockerfile.broker`, más los 7 sistemas, el API Gateway, el GraphQL Gateway, los adaptadores de pago y correo y el frontend de demostración. El contenedor `preparacion` termina después de crear certificados, secretos y tokens. `up` termina con `todos los servicios estan sanos`. La red `privada` es `internal: true` y solo se publican `127.0.0.1:8443`, `127.0.0.1:8080` y `127.0.0.1:9201`.

Salida real de `python scripts/compose.py estado` (`tests/evidencia/compose-estado.txt`):

```
SERVICE                STATE     <no value>   PORTS
api-gateway            running   healthy      127.0.0.1:8443->8000/tcp
auth-service           running   healthy      8000/tcp
broker                 running                1099/tcp, 1883/tcp, 5672/tcp, 8161/tcp, 61613-61614/tcp,
61616/tcp
clients-service        running   healthy      8000/tcp
correo-prueba          running   healthy      8000/tcp
dispatch-service       running   healthy      8000/tcp
frontend               running                127.0.0.1:8080->8080/tcp
graphql-gateway        running   healthy      8000/tcp
inventory-service      running   healthy      8000/tcp
```

```
mongo                   running                 27017/tcp
notifications-service   running   healthy       8000/tcp
orders-service          running   healthy       8000/tcp
pasarela-prueba         running   healthy       127.0.0.1:9201->8000/tcp
payments-service        running   healthy       8000/tcp
vault                   running                 8200/tcp
```

Solo el API Gateway, el frontend y la pasarela de prueba tienen un puerto publicado hacia el equipo; los demás muestran únicamente el puerto interno de la imagen. Los puertos que aparecen sin `127.0.0.1` (`8000/tcp`, `61613/tcp`, `27017/tcp`, `8200/tcp`) son los que declara cada imagen con `EXPOSE`: no están publicados y solo se alcanzan desde la red `privada`. La columna de salud de `broker`, `vault`, `mongo` y `frontend` está en blanco porque esos contenedores no definen comprobación de salud; su estado es `running`.

Pruebas automáticas de Compose (`python scripts/compose.py probar`, 28 de 28 en OK). Se ejecutan desde el equipo, desde un contenedor de la red pública, desde uno de la red privada y dentro de la red privada con el contenedor `herramientas`. Salida completa en `tests/evidencia/compose-probar-salida.txt`.

10.9.11.1 Redes y puertos publicados

Prueba	Qué comprueba	Resultado	Observado
CMP-NET-01	Solo publican puerto el API Gateway (8443), el frontend (8080) y el simulador del proveedor de pago (9201), todos en 127.0.0.1	OK	{"api-gateway": [["127.0.0.1", 8443]], "frontend": [["127.0.0.1", 8080]], "pasarela-prueba": [["127.0.0.1", 9201]]}
CMP-NET-02	Todos los contenedores del sistema estan en ejecucion y sanos	OK	15 contenedores en ejecucion
CMP-NET-03	La red privada es interna (sin salida a Internet); solo el API Gateway, el frontend y el simulador de pago estan en la red publica	OK	privada.internal=True; publica=['api-gateway', 'frontend', 'pasarela-prueba']
CMP-NET-04	Desde el equipo anfitrion, Compose no publica ningun puerto de los 7 sistemas, GraphQL, correo, base de datos, Vault ni broker (T-DAT-05a, T-GQL-13a)	OK	12 puertos probados; abiertos por Compose: ninguno; abiertos en el equipo por otro programa (no es Compose): ['mongo']
CMP-NET-05	Los puertos publicados si responden	OK	gateway:8443, frontend:8080, pasarela:9201
CMP-NET-06	Desde la red publica no existen los nombres de orders, inventory, clients, payments, auth, dispatch, notifications, GraphQL, base de datos, Vault ni broker (T-GQL-14)	OK	{"resuelve": [], "gateway_health": 200, "internal": 404, "sin_token": 401}
CMP-NET-07	Desde la red publica solo se llega al API Gateway: responde /health, no expone /internal y exige token en las rutas protegidas	OK	{"gateway_health": 200, "internal": 404, "sin_token": 401}
CMP-NET-08	Un contenedor de la red privada no tiene salida a Internet	OK	sin salida
CMP-NET-09	Las pruebas internas (acceso directo, GraphQL directo y compra completa) terminan sin fallas	OK	18/18 OK

CMP-NET-04 informa por separado los puertos abiertos en el equipo por otro programa: en esta ejecución, el 27017 del servicio MongoDB instalado para el modo local. Compose no lo publica (CMP-NET-01).

10.9.11.2 Acceso directo, bases de datos y compra completa dentro de la red privada

Prueba	Qué comprueba	Resultado	Observado
CMP-RED-01	orders-service llamado desde otro contenedor sin secreto, con el secreto de otro destino, con el suyo pero sin identidad, y con un par de otro sistema: siempre rechazado	OK	sin secreto=401, secreto de otro destino=401, secreto propio sin identidad=401, par de otro sistema=403
CMP-RED-02	GraphQL Gateway llamado directamente sin secreto, con el de otro destino y con el de orders: 401 en los tres	OK	401, 401, 401
CMP-RED-03	El API Gateway no publica /internal, /docs ni la introspeccion de Auth	OK	404, 404, 404
CMP-DB-01	MongoDB rechaza una conexion sin credenciales: no puede leer ni listar las colecciones de orders_db	OK	leer: rechazado (codigo 13); listar: rechazado (codigo 13)
CMP-DB-02	Cada sistema, con su usuario de MongoDB, escribe y lee en su base y es rechazado (Unauthorized) en las de los otros seis	OK	bases propias accesibles=7/7; bases ajenas rechazadas=42/42

Prueba	Qué comprueba	Resultado	Observado
CMP-DB-03	MongoDB tiene exactamente un usuario por sistema, definido en la base del sistema y con el unico rol readWrite sobre esa base	OK	auth_user: auth_db ['readWrite@auth_db']; clients_user: clients_db ['readWrite@clients_db']; dispatch_user: dispatch_db ['readWrite@dispatch_db']; inventory_user: inventory_db ['readWrite@inventory_db']; notifications_user: notifications_db ['readWrite@notifications_db']; orders_user: orders_db ['readWrite@orders_db']; payments_user: payments_db ['readWrite@payments_db']
CMP-DB-04	Los 7 sistemas se ejecutan con MONGO_AUTH=1: se conectan con su usuario de MongoDB cuya clave sale de Vault	OK	{'auth': '1', 'clients': '1', 'inventory': '1', 'orders': '1', 'payments': '1', 'dispatch': '1', 'notifications': '1'}
CMP-01	Catalogo publico sin sesion: 12 productos, sin stock	OK	200, 12 productos
CMP-02	Operacion protegida sin token: 401	OK	401
CMP-03	Registro (S03), correo con enlace en la bandeja de prueba, confirmacion (S04) e inicio de sesion (S01) de dos clientes	OK	clientes cli_0003 y cli_0004
CMP-04	Pedido con precio fijado por el servidor, pago por la pasarela de prueba con aviso firmado y pedido en estado pagado	OK	ped_0001 estado=pagado total=3980
CMP-05	Stock descontado en inventory por el pedido pagado	OK	200 -> 198
CMP-06	Boleta enviada por correo (proveedor de prueba) y disponible por S23	OK	1 correo(s) con la boleta
CMP-07	La orden de despacho llega por la cola STOMP y el encargado la ve sin precios	OK	1 orden(es)
CMP-08	Consulta autorizada (cuenta dueña) 200; cuenta distinta 404 identico al de un pedido inexistente (regla C1 en orders-service)	OK	dueña=200, otra cuenta=404 NO_ENCONTRADO, inexistente=404
CMP-09	GraphQL: resumenPedido combina orders e inventory para la dueña; otra cuenta recibe NO_ENCONTRADO; el requestId aparece en los registros de cada sistema	OK	archivos con el requestId: ['api-gateway.log', 'auth-service.log', 'graphql-gateway.log', 'inventory-service.log', 'orders-service.log']
CMP-10	Aviso de pago con firma invalida (como lo haria un navegador alterado): 401	OK	401
CMP-11	Reporte de ventas: 200 para el dueño y 403 para un cliente	OK	200, 403
CMP-12	Los registros compartidos no contienen contraseñas, correos ni direcciones de las pruebas	OK	4 valores buscados en /logs; coincidencias=0

Las pruebas CMP-DB comprueban que, en Compose, MongoDB corre con autenticación (`mongod --auth`): rechaza una conexión sin credenciales, existe un solo usuario por sistema con el rol `readWrite` solo sobre su base (la clave sale de Vault) y cada sistema es rechazado con el código 13 en las otras seis bases (42 intentos).

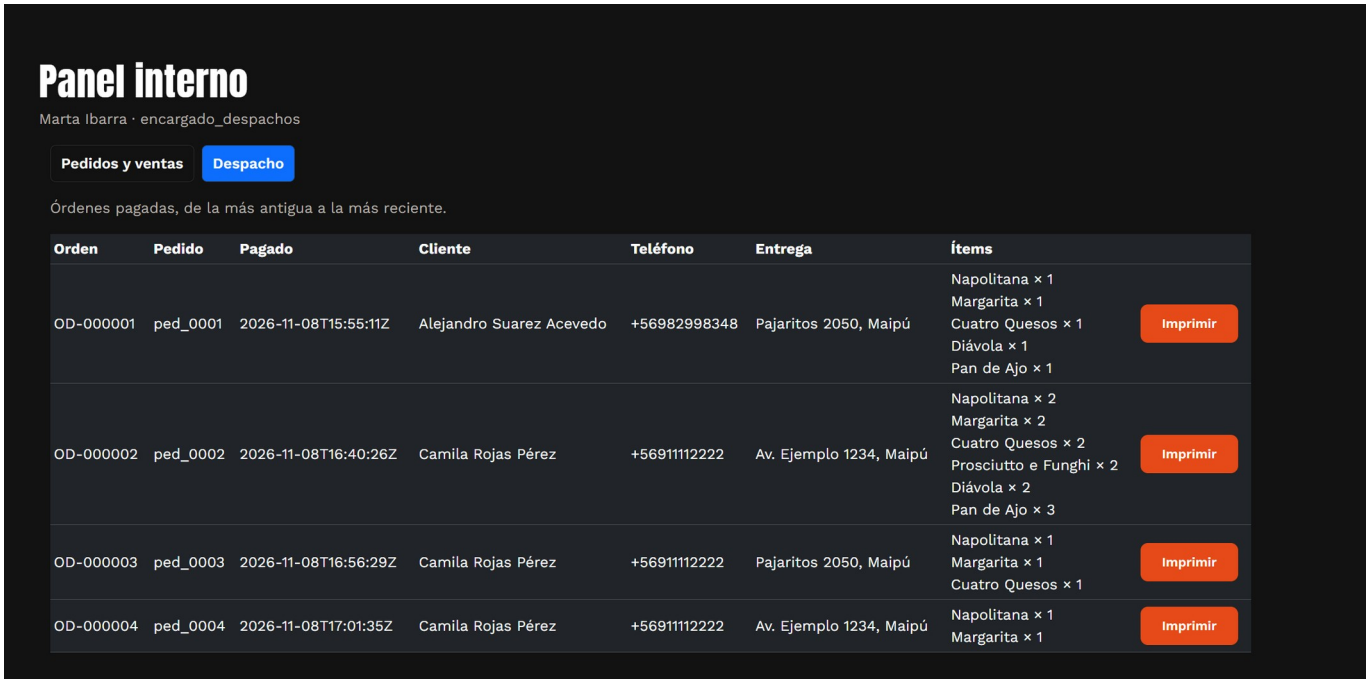
Resiliencia comprobada a mano en Compose (dispatch caído y broker caído). Se detuvo cada pieza con `docker compose -p fornace stop dispatch-service` y `docker compose -p fornace stop broker`, se hizo una compra completa con el pago por la pasarela de prueba y se volvió a iniciar la pieza. La secuencia se identifica por pedido y orden, no por hora (capturas del panel interno, en `impl/tests/evidencia/`):

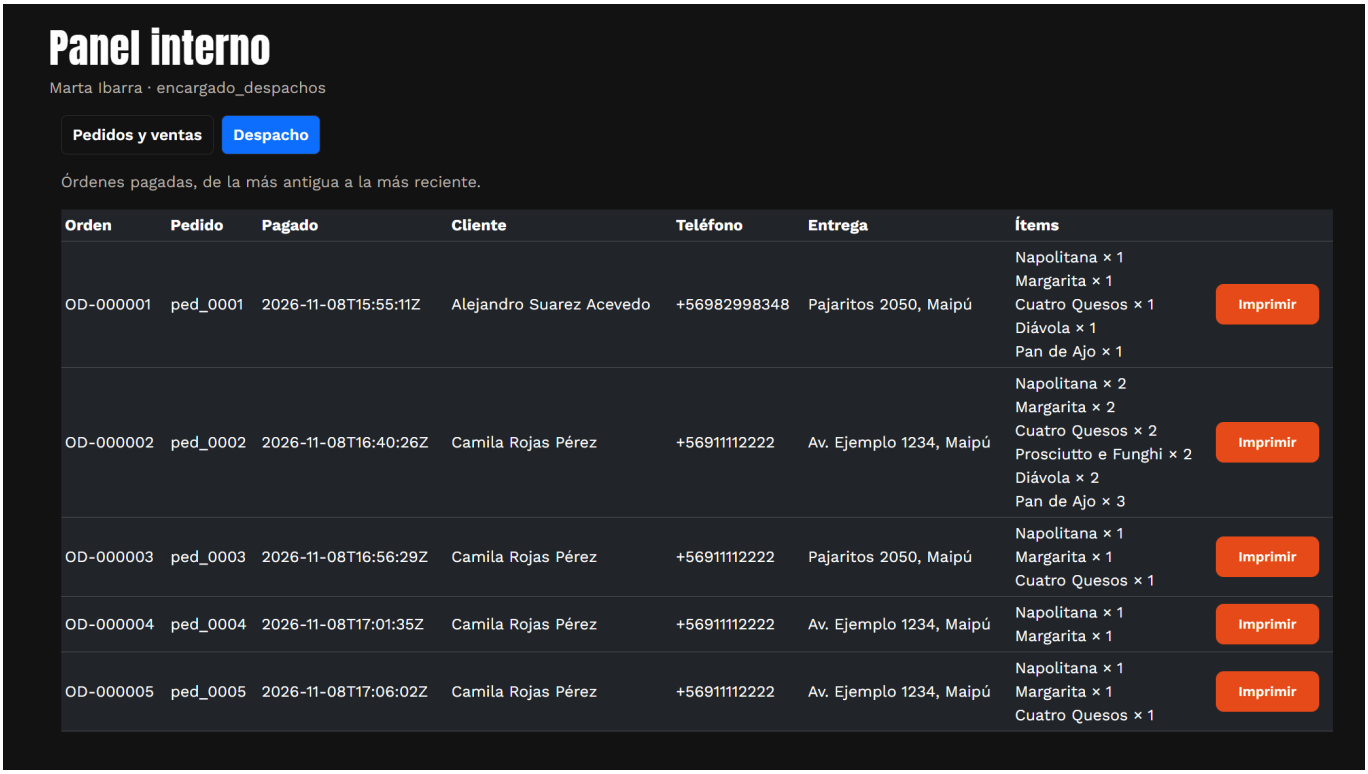
Paso	Qué se hizo	Qué se observó	Captura
0	Estado inicial, todo en marcha	Pedidos `ped_0001` a `ped_0003` pagados o entregados; órdenes `OD-000001` a `OD-000003` en despacho	`compose-00-pedidos-antes.png`, `compose-00-despacho-antes.png`
1	`dispatch-service` detenido; el encargado abre la pestaña Despacho	Mensaje controlado "Servicio no disponible" con código de soporte; sin pantalla de error ni datos parciales	`compose-01-despacho-caido-503.png`
2	Con dispatch detenido se crea y se paga `ped_0004`	El pedido queda `Pagado`: la venta no depende de que despacho esté arriba	`compose-02-pedido-pagado-con-despacho-caido.png`
3	Se reinicia `dispatch-service`	La orden `OD-000004` del pedido `ped_0004` aparece en despacho, sin duplicados	`compose-03-despacho-recuperado.png`
4	`broker` detenido; se crea y se paga `ped_0005`	El pedido queda `Pagado` aunque la cola no está disponible	`compose-04-pedido-pagado-con-broker-caido.png`
5	Con el broker aún detenido se consulta Despacho	Solo hay `OD-000001` a `OD-000004`: no existe `OD-000005` todavía	`compose-05-despacho-sin-orden-con-broker-caido.png`

Paso	Qué se hizo	Qué se observó	Captura
6	Se reinicia `broker`	`orders` republica el pendiente y llega `OD-000005` del pedido `ped_0005`, en orden de llegada	`compose-06-orden-llega-al-reiniciar-broker.png`

Esto es el mismo comportamiento que miden por API F2 (broker detenido: pedido `pagado` con `despachoPublicado` en falso, y la orden llega al reiniciar) y F3 (dispatch reiniciado con mensajes pendientes) en la batería de 10.9.10.







Qué cubre y qué no. La batería de 193 pruebas se ejecutó en modo local (10.9.1 a 10.9.10 y 10.9.12); en ese modo todos los procesos escuchan en `127.0.0.1`, de modo que T-DAT-05a, T-GQL-13a y T-GQL-14 verifican allí el rechazo por secreto y la ausencia de rutas (401 y 404). La parte de red de esos casos (puerto no publicado, nombre no resoluble desde la red pública, sin salida a Internet) la comprueban las 28 pruebas de Compose. No se ejecutó dentro de los contenedores la batería completa de 193 pruebas ni la carga con límites de 1 CPU y 512 MB.

10.9.12 Retención y eliminación de datos (7.9)

Cada sistema tiene una tarea periódica que aplica los plazos de 7.9 (`common/fornace/retencion.py`; por defecto cada 3600 s). La prueba `suite_retencion.py` crea los datos con el sistema en marcha (compra, pago, entrega, anulación, bajas de clientes, mensajes de contacto), les retrocede las fechas directamente en la base para simular el paso del tiempo, reinicia los 7 sistemas con la tarea cada 2 s y comprueba qué se eliminó, qué se anonimizó y qué se conservó. No se esperó el paso real de 90 días, 12 ni 24 meses.

10.9.12.1 Plazos de retención aplicados

Prueba	Qué comprueba	Resultado	Observado
RET-00	Datos de partida creados por el sistema: pedido pagado con venta y orden de despacho, pedido anulado sin pago con un pago pendiente, pedido entregado y pedido pagado y anulado	OK	anular=200 entrega=[200, 200, 200] anular_pagado=200
RET-00b	Dos clientes dados de baja (S29) guardan la fecha de baja en clients y en auth	OK	bajas=[204, 204]
RET-01	dispatch-service: la orden y el mensaje con mas de 90 dias se eliminan; los de menos de 90 dias se conservan	OK	a(100 d): orden=0 mensajes=0; c(120 d): orden=0; e(10 d): orden=1 mensajes=1
RET-02	notifications-service: el mensaje de contacto de hace 13 meses se elimina y el reciente se conserva	OK	viejo=0 reciente=1
RET-03	payments-service: los avisos de pago de hace 13 meses y el pago antiguo sin venta se eliminan; el reciente se conserva	OK	avisos antiguos=0, pago antiguo sin venta=0, avisos recientes=1, pago reciente=1
RET-04	payments-service: el pago antiguo con venta y la venta con su boleta se conservan (6 anios)	OK	pago con venta=1 venta=1
RET-05	inventory-service: los movimientos de stock de hace 13 meses se eliminan y los recientes se conservan	OK	antiguos=0 recientes=1
RET-06	orders-service: el pedido anulado sin pago de hace 13 meses se elimina	OK	pedido anulado sin pago: 0 documento(s)

Prueba	Qué comprueba	Resultado	Observado
RET-07	orders-service: el pedido que fue pagado y despues anulado se conserva aunque sea antiguo	OK	anulado
RET-08	orders-service: a los 12 meses de la entrega se borran calle, referencia y telefono, y se conservan comuna, lineas y total	OK	calle='' telefono=None comuna='Maipú' total=7990
RET-09	orders-service: el pedido entregado hace pocos dias conserva calle y telefono	OK	calle='Calle Ficticia 681456'
RET-10	clients-service: el cliente dado de baja hace 25 meses queda anonimizado; el de hace 100 dias y el activo conservan sus datos	OK	baja 25 m: nombre='[eliminado]' run='ANON-cli_0021'; baja 100 d: nombre='Cliente retr 119063'; activo: nombre='Cliente reta 440900'
RET-11	auth-service: la cuenta del cliente dado de baja hace 25 meses se anonimiza (correo, nombre y hash) y ya no permite iniciar sesion; la de hace 100 dias no cambia	OK	correo=eliminado-cta_0021@anonimo.invalid hash=None login con el correo anterior=401
RET-12	El listado de clientes responde con clientes anonimizados y los eventos de retencion del registro no contienen datos personales	OK	listado=200 eventos de retencion=7 datos personales en ellos=[]

10.10 Defectos que encontraron las pruebas y se corrigieron

Estas correcciones salieron de ejecutar las pruebas durante el desarrollo y en la ejecución en Windows; el código entregado ya las incluye.

Hallazgo	Corrección
La función de registro `evento()` fallaba cuando se le pasaba el campo `tipo`. El consumidor de despacho perdía la conexión y se perdían mensajes.	El primer parámetro pasó a ser solo posicional. El consumidor ahora confirma cada mensaje después de guardar la orden (F3).
Los avisos de pago tardaban en responder porque confirmaban el pedido, la venta y el correo antes de contestar a la plataforma de pago.	El aviso responde al validar la firma y guardar el pago; la confirmación sigue en segundo plano y, si falla, la repite la conciliación (C4, F4).
Con Vault detenido, el gateway devolvía 500 y no tenía todos los secretos en memoria.	El gateway carga todos sus secretos al iniciar y responde 503 controlado cuando vence la ventana (F5).
Un token de una ejecución anterior se reutilizaba aunque Vault se hubiera reiniciado.	`vault_init.py` valida el token y lo crea de nuevo si no sirve.
Al ejecutar otra vez el contenedor `preparacion`, se borraban los certificados y tokens de los contenedores en marcha y los sistemas respondían 503 por secreto no disponible.	`bootstrap_compose.py` actualiza los archivos en su lugar, sin borrar las carpetas, y conserva la CA en el volumen (`/pki/_trabajo`).
El costo de la lista `pedidos` de `miCuenta` ignoraba el argumento `limitePedidos` (que se escribe en `miCuenta`) porque leía los argumentos del propio campo `pedidos`; una consulta que pedía una lista grande se costeaba como la lista por defecto.	`policy.js` pasa los argumentos del campo padre al calcular el multiplicador y lo limita a `limitePedidosMax`.
`stack.py restart` esperaba 1 s tras la señal de término. Un servidor que cerraba más lento seguía ocupando el puerto, el nuevo proceso no podía iniciar y las pruebas siguientes corrían contra el proceso viejo (fallaban CT-S17-P y T-GQL-04).	`stack.py` espera a que el proceso termine (con terminación forzada si no lo hace), libera el puerto y comprueba que el proceso nuevo siga vivo.
En Windows, F4 falló: con `orders` "suspendido" el aviso quedó confirmado en pedidos. El ejecutable del entorno virtual lanza un intérprete hijo y la prueba suspendía solo al proceso padre, de modo que el hijo seguía atendiendo.	`carga/fallos.py` suspende y reanuda el árbol completo de procesos de `orders`. `stack.py` mata el árbol completo al detener (con `psutil`).
La prueba CMP-NET-04 de Compose dio un falso fallo en Windows: contó como publicado el puerto 27017, abierto por el servicio MongoDB instalado para el modo local, no por Compose.	La prueba compara los puertos abiertos con los que Compose publica y separa los abiertos por otro programa. El resultado indica ambos grupos.
Con el volumen `fornace_mongo-datos` conservado al cambiar de carpeta, `seed.py` (idempotente) no cambiaba las contraseñas de cuentas existentes y CMP-NET-09 fallaba con credenciales obsoletas.	`compose.py down --borrar` elimina el volumen; `README.md` lo documenta para cuando se cambia de carpeta.
La versión del paquete sin `probar` no podía ejecutar la suite de Compose: el contenedor `herramientas` no incluía `tests/` ni montaba `tests/resultados`.	Se agregó la orden `compose.py probar`, `COPY tests` en `Dockerfile.herramientas` y el montaje de `tests/resultados`.
Con la Content-Security-Policy activa, la prueba con navegador se cayó en la página Mi cuenta: `page.wait_for_function` de Playwright usa `eval()` dentro de la página y la política no permite `unsafe-eval`.	No se relajó la política. `tests/e2e_frontend.py` espera con sondeos de `page.evaluate`, que sí funciona bajo la política, de modo que las 27 pruebas con navegador corren con la CSP activa.
RET-10 falló aunque el cliente dado de baja hace 25 meses sí quedó anonimizado: la prueba comparaba el RUN con el que se registró el cliente de 100 días (sin puntos) con el guardado por el servicio (normalizado, con puntos).	La prueba compara con `run_normal()`; el servicio no cambió.
La prueba con navegador tenía fija la ruta de Google Chrome de Linux y no corría en Windows.	`tests/e2e_frontend.py` usa `CHROME_PATH` o, si no existe, el Chromium que instala Playwright.
Los servicios fallaban al iniciar con `No module named fornace` si no se instalaba la librería común: el modo `-I` de Python ignora `PYTHONPATH`.	Se agregó `pip install -e common` a los requisitos de `README.md` y `GUIA-DE-PRUEBA.md`.

10.11 Diferencias con el diseño y limitaciones

Tema	Qué se hizo	Efecto
Dos formas de ejecución	La batería de 193 pruebas se ejecutó en modo local (sin Docker) sobre MongoDB 7.0.45, Vault 1.18.5 y ActiveMQ 6.2.0 instalados en Windows. El despliegue en Docker Compose usa los mismos motores en contenedores y se verificó con `estado`, las 28 pruebas de `compose.py probar` y las pruebas manuales de dispatch y broker caídos (10.9.11).	No se ejecutó la batería de 193 pruebas dentro de los contenedores. En modo local no existe aislamiento de red; la separación pública y privada se comprueba en Compose (10.9.11).
Un solo modo a la vez	Docker Compose y el modo local usan los mismos puertos del equipo (8443, 8080 y 9201). El modo local usa el servicio MongoDB del equipo (27017); Compose usa su propio contenedor de MongoDB, sin publicar puerto.	Hay que detener un modo (`compose.py down` o `stack.py down --todo`) antes de levantar el otro. Un puerto 27017 abierto en el equipo es del MongoDB local, no de Compose (`compose.py probar` lo informa aparte).
Datos persistentes en Compose	Los datos viven en el volumen `fornace_mongo-datos`, que sobrevive a `compose.py down`. `seed.py` es idempotente: no cambia las contraseñas de cuentas que ya existen.	Al copiar el proyecto a otra carpeta o perder `.run/credenciales-demo.json`, las contraseñas de las cuentas de ejemplo dejan de coincidir. Se corrige con `compose.py down --borrar` antes de `up` y `semilla` (visto al probar en otra carpeta; indicado en el README).
Vault en modo desarrollo	Vault 1.18 se ejecuta con `-dev`: los secretos viven en memoria, el sello está abierto y el token raíz es de laboratorio.	Al reiniciar Vault se pierden los secretos; `vault_init.py` los vuelve a generar y entrega tokens nuevos. Una instalación real usaría almacenamiento persistente y sellado.
MongoDB con autenticación solo en Compose	En Compose, MongoDB corre con `--auth` y cada sistema usa su propio usuario, con `readWrite` solo sobre su base y la clave en Vault (CMP-DB-01 a CMP-DB-04). En el modo local se usa `MONGO_AUTH=0`: MongoDB corre sin usuarios.	En modo local cada sistema escribe solo en su base por el código, no porque el motor lo imponga. La 7.4 y la 6.4 se cumplen en Compose. Los usuarios de Mongo se crean con el usuario raíz de laboratorio (`MONGO_ROOT_PASSWORD` en `.env`).
Cola de despacho (ActiveMQ)	ActiveMQ 6.2.0 con un usuario por rol (orders escribe, dispatch lee). En Compose la imagen se construye con `Dockerfile.broker` sin privilegios y con límite de memoria de 1 GB; la JVM arranca con un máximo de 1 GB de heap (`-Xmx1G`) y no se logró reducirlo. En modo local ActiveMQ escucha en `0.0.0.0:61613`.	El heap de 1 GB no se pudo reducir y el puerto 61613 queda abierto a la red local en modo local (el firewall de Windows pregunta). En Compose el puerto no se publica.
Metadatos EXPOSE	La salida de `compose.py estado` muestra `8000/tcp`, `61613/tcp`, `27017/tcp` y `8200/tcp` sin `127.0.0.1`.	Son metadatos de las imágenes base, no puertos publicados; solo 8443, 8080 y 9201 llegan al equipo.
Comprobaciones de salud	`mongo`, `broker`, `vault` y `frontend` no definen `healthcheck` en Compose.	Aparecen `running` sin la columna de salud en `estado`. Los 11 sistemas con `/health` sí la muestran.
Token de sesión en el navegador	El frontend de demostración guarda el token en `sessionStorage`, no en una cookie `HttpOnly`.	Un XSS podría leerlo. Se mitiga escapando los textos del API y con una Content-Security-Policy en el servidor del frontend y cabeceras de seguridad en el gateway, pero no se elimina: la política permite scripts en línea (`unsafe-inline`) porque las páginas los usan, así que limita adónde puede enviar datos un script inyectado (`connect-src`, `form-action`) y no impide que se ejecute. El gateway responde con `default-src 'none'; frame-ancestors 'none'`. No afecta al backend, que valida cada token con Auth.
Certificados	El proyecto crea su propia autoridad certificadora. Al regenerarla (por ejemplo con `down --todo` y un nuevo `up`) el navegador debe volver a aceptar los certificados de 8443 y 9201.	Es un paso manual solo de la demostración con navegador.
Vencimientos	Tareas periódicas del sistema en lugar de índices TTL.	Cambia 7.9. Se comprobaron el vencimiento de pedidos sin pago con devolución de stock (FL-04), la sesión vencida (T-DAT-12) y la sesión revocada (T-DAT-08).
Retención 7.9 probada con fechas simuladas	La tarea de retención existe en los 7 sistemas y las pruebas RET-01 a RET-12 la ejecutan sobre datos con fechas retrocedidas y un intervalo de 2 s.	No se esperó el paso real de los plazos. En una instalación real la tarea corre cada hora y el borrado, de 200 documentos por pasada y sistema como máximo, se completa en varias pasadas.
Reporte S34	Lee resúmenes diarios (por día y por producto) que se actualizan al registrar cada venta, en lugar de recorrer las ventas con una agregación.	Es la decisión de 9.7. C5 y C6 miden el resultado (10.9.9). No se midió la alternativa de agregar sobre las ventas.
Serialización de stock y contadores	inventory serializa dentro del proceso los cambios de stock y las bases serializan los contadores, además de las actualizaciones condicionales de MongoDB (`stock >= cantidad`).	Es una defensa adicional que supone una sola instancia de inventory. Con varias instancias se dependería solo de la atomicidad de cada actualización en MongoDB. C3: 30 pedidos simultáneos con stock 10 dieron 10 creados y 20 rechazados con 409.
Escala de carga	Duraciones al 25 % del plan, en modo local sin límite de recursos por proceso, con el generador y el sistema en el mismo equipo.	Los números de 10.9.9 no son comparables con una ejecución en contenedores de 1 CPU y 512 MB. `--escala=completa` repite con las duraciones del plan.

Tema	Qué se hizo	Efecto
Cambios de contrato respecto a módulos anteriores	S14 devuelve además `creadoEn` y `expiraEn` (M1). El gateway agrega la cabecera `X-Authenticated-Session` (M6). El gateway carga todos sus secretos de Vault al iniciar y, si Vault no responde, usa el valor en memoria durante una ventana configurable (por defecto 900 s) y luego responde 503 `SERVICIO_NO_DISPONIBLE` (M6 6.10). En la lista de costos de GraphQL (M8) los valores medidos son 432 para el detalle completo y 1296 para tres alias.	Los módulos 1, 6 y 8 se corrigieron para coincidir con el código entregado.
Pago y correo	Usamos adaptadores de prueba propios (`pasarela_prueba` y `correo_prueba`) que conservan el contrato, la autenticación por clave, la firma del aviso y las transiciones del pago (pendiente, aprobado, rechazado). El caso pide Servipag o depósito bancario: Servipag no ofrece un entorno de prueba público (exige convenio comercial) y el depósito bancario no tiene API. Para el correo preferimos un adaptador local para que las pruebas no dependan de una cuenta externa ni de Internet (la red privada de Compose no tiene salida).	No hay cobros ni envíos reales. Para usar un proveedor real basta cambiar la URL (`URL_PASARELA`, `URL_CORREO`) y la clave en Vault (`secret/payments/provider` y `secret/notifications/mail`); payments y notifications no cambian. Los adaptadores no reemplazan a los sistemas propios, que están implementados y probados.
Ventana de Vault	La prueba F5 usó una ventana de 25 s en lugar de 15 minutos.	La lógica es la misma; solo cambia el valor configurado (`VAULT_GRACIA_SEGUNDOS`).
Pruebas de carga y datos	C5 y C6 siembran ventas y sus resúmenes y, al terminar, vacían las colecciones de ventas y resúmenes (borrar decenas de miles de documentos una por una tardaba decenas de minutos).	Por eso `run_tests.py` ejecuta la carga al final.

10.12 Control antes de cerrar

Pregunta	Respuesta
¿El sistema funciona como aplicación completa y no devuelve respuestas fijas?	Sí. 221 pruebas ejecutadas contra procesos reales con persistencia (resultado 221 de 221: 193 en modo local y 28 en Docker Compose).
¿Los flujos del caso funcionan con los datos del frontend de la Evaluación 1?	Sí. El catálogo de 12 productos de `js/data.js` está cargado en `inventory`; las suites de contratos, negocio, seguridad y GraphQL recorren la compra completa por API. Además, 22 recorridos con navegador real y 5 pruebas de teléfono sobre el frontend de demostración (10.9.7).
¿Se comprobó la propiedad de recursos, los roles, el acceso directo, los límites GraphQL, la caída de dependencias, los pagos duplicados y la recuperación de stock?	Sí: T-AUT, T-DAT-03 a T-DAT-06, T-GQL, F1 a F7, C3 y S15.
¿Los secretos están fuera del código y del navegador?	Sí. T-DAT-09 y T-DAT-11 buscan los valores reales de Vault, los datos usados y los patrones de secretos en los registros, el frontend, el código y este informe, y no encuentran coincidencias.
¿Las diferencias con el diseño están declaradas?	Sí, en 10.10 y 10.11.
¿Se probó el despliegue con redes separadas?	Sí, en Docker Compose: redes `publica` y `privada` (esta con `internal: true`), tres puertos publicados, nombres privados no resolubles desde la red pública, acceso directo rechazado y sin salida a Internet desde la red privada (28 de 28 pruebas, 10.9.11), más dispatch caído y broker caído comprobados a mano. En modo local no hay aislamiento de red.
¿Qué queda sin verificar?	La batería de 193 pruebas y la carga dentro de contenedores con límites de 1 CPU y 512 MB, la carga a escala completa, el paso real del tiempo en la retención de 7.9 (se probó con fechas simuladas), MongoDB con autenticación en el modo local y Vault con almacenamiento persistente.